

现代 C++ 数据结构教程

基于张铭、王腾蛟、赵海燕《数据结构与算法》重编
高等教育出版社 2008 年版教学内容的现代化讲义

目录

写给学生	i
如何使用本教程	i
各章一览	i
阅读约定	ii
验证与边界	ii
第 1 章 概论	1
1.1 问题求解	1
1.1.1 问题描述：股市的传言	1
1.1.2 问题分析和抽象	1
为什么要取“最慢的一项”	2
1.1.3 数据结构和算法设计	2
实现导读	4
现代实现	6
1.2 数据结构	8
1.2.1 数据的逻辑结构	8
1.2.2 数据的存储结构	9
1.2.3 抽象数据类型	9
1.3 算法	11
1.3.1 算法的概念	11
1.3.2 算法设计	11
1.4 算法分析	12
1.4.1 渐进分析方法	12
1.4.2 最佳、最差和平均情况	14
1.4.3 时间和空间的折衷	14
1.4.4 求解问题时数据结构的选择和评价	14
本章小结	14
习题	15
上机题	16
第 2 章 线性表	17
先跑一遍	17
2.1 线性表的概念	18
2.2 顺序表	19
2.2.1 存储与所有权	20
2.2.2 顺序表的检索	20
2.2.3 顺序表的插入	21
2.2.4 顺序表的删除	22

与原书的对照	23
2.3 链表	24
2.3.1 单链结点与头结点	24
2.3.2 构析与循链定位	25
2.3.3 插入与删除	26
2.3.4 双链结点	27
2.4 线性表实现方法的比较	27
本章小结	27
习题	27
上机题	28
第 3 章 栈与队列	29
先跑一遍	29
3.1 栈	30
3.1.1 栈的抽象数据类型	30
3.1.2 顺序栈	31
存储与所有权	31
出栈：用 optional 取代双通道返回	32
一个容易踩空的地方：move_if_noexcept 在这里是错的	35
3.1.3 链式栈	36
3.1.4 表达式求值	38
3.1.5 栈与递归	41
运行栈有多大：三个档位，三种结果	41
反直觉的那一条：-O2 为什么不崩	41
递归吃运行栈，显式栈吃堆	42
原书这三个版本共同的问题：不查溢出	43
一个更复杂的例子：背包问题	44
与原书的对照	48
3.2 队列	49
3.2.1 队列的抽象数据类型	49
3.2.2 顺序队列	49
3.2.3 链式队列	50
3.3 栈与队列的比较	51
3.3.1 顺序栈与链式栈	51
3.3.2 顺序队列与链式队列	51
3.3.3 限制存取点的表	51
本章小结	51
习题	52
上机题	52
第 4 章 字符串	53

先跑一遍	53
4.1 字符串的基本概念	54
4.2 字符串的存储结构和实现	55
4.2.1 构造与所有权	55
4.2.2 追加与拼接	57
4.2.3 抽取子串	57
4.2.4 查找与比较	58
4.3 字符串的模式匹配	59
4.3.1 朴素的模式匹配算法	59
4.3.2 字符串的特征向量	61
4.3.3 KMP 模式匹配算法	62
与原书的对照	63
本章小结	64
习题	64
上机题	65
第 5 章 二叉树	67
5.1 二叉树的概念	67
5.1.1 定义和基本术语	67
5.1.2 满二叉树、完全二叉树、扩充二叉树	67
5.1.3 主要性质	68
5.2 二叉树的周游	68
5.2.1 先跑一遍	68
5.2.2 深度优先周游	69
5.2.3 广度优先周游	69
5.3 二叉树的存储结构	69
5.4 二叉搜索树	74
5.5 堆与优先队列	76
5.6 Huffman 树及其应用	79
本章小结	80
习题	81
上机题	81
第 6 章 树	83
6.1 树的定义和基本术语	83
6.1.1 树和森林	83
6.1.2 森林与二叉树的等价转换	84
6.1.3 树的抽象数据类型	84
6.1.4 树的周游	84
6.2 树的链式存储结构	84
6.2.1 「子结点表」表示方法	85

6.2.2 静态「左子/右兄」表示法	85
6.2.3 动态表示法	85
6.2.4 动态「左子/右兄」表示	85
6.2.5 父指针表示法和并查集	90
6.3 树的顺序存储结构	91
6.3.1 带右链的先根次序表示	91
6.3.2 带双标记的先根次序表示	91
6.3.3 带度数的后根次序表示	91
6.3.4 带双标记的层次次序表示	92
6.4 K 叉树	92
本章小结	92
习题	92
上机题	92
第 7 章 图	93
7.1 图的定义和基本术语	93
7.2 图的抽象数据类型	94
7.3 图的存储结构	94
7.3.1 相邻矩阵	94
7.4 图的周游	94
7.4.1 先跑一遍	94
7.4.2 深度优先与广度优先	95
7.4.3 拓扑排序	96
7.5 最短路径	97
7.5.1 单源最短路径	97
7.5.2 每对顶点之间的最短路径	98
7.6 最小生成树	98
7.6.1 Prim 算法	98
7.6.2 Kruskal 算法	99
本章小结	100
习题	100
上机题	100
第 8 章 内部排序	101
8.1 排序问题的基本概念	101
8.2 插入排序	101
8.2.1 先跑一遍	101
8.2.2 Shell 排序	103
8.3 选择排序	103
8.3.2 堆排序	103
8.4 交换排序	104

8.4.2 快速排序	104
8.5 归并排序	104
8.6 分配排序和索引排序	105
8.6.2 基数排序	105
8.7 排序算法的时间代价	105
本章小结	105
习题	105
上机题	106
第 9 章 文件管理与外部排序	107
9.1 主存储器和外存储器	107
9.2 文件的组织和管理	107
9.3 外排序	108
9.3.1 置换选择排序	108
9.3.2 二路外排序	110
9.3.3 多路归并——选择树	111
本章小结	113
习题	113
上机题	114
第 10 章 检索	115
10.1 基于线性表的检索	115
10.1.1 顺序检索	115
10.1.2 二分检索	115
10.1.3 分块检索	115
10.2 集合的检索	116
10.2.1 集合的数学特性	116
10.2.2 计算机中的集合	116
10.3 散列方法	117
10.3.1 散列函数	117
10.3.2 开散列方法（拉链法）	118
10.3.3 闭散列方法（开地址法）	118
10.3.4 闭散列表的算法	119
10.3.5 散列方法的效率分析	122
10.3.6 散列方法的应用	122
本章小结	122
习题	122
上机题	122
第 11 章 索引技术	123
11.1 线性索引	123
11.2 静态多分树	123

11.3 倒排索引	123
11.4 B 树与 B+ 树	124
11.5 位图、签名和红黑树	125
11.6 练习路径	125
本章小结	125
习题	125
上机题	126
第 12 章 高级数据结构	127
12.1 多维数组	127
12.1.1 多维数组的存储	127
12.1.2 特殊矩阵	127
12.1.3 稀疏矩阵	128
12.2 广义表和存储管理	128
12.2.1 广义表的定义和存储结构	128
12.2.2 可利用空间表	128
12.2.3 存储的动态分配和回收	130
12.2.4 失败处理策略和无用单元回收	130
12.3 Trie 结构和 Patricia 树	131
12.4 改进的二叉搜索树	131
12.4.1 最佳二叉搜索树	131
12.4.2 平衡的二叉搜索树	132
12.4.3 伸展树	133
本章小结	133
习题	134
上机题	134
原书插图	135
前言	135
第 1 章 概论	135
第 2 章 线性表	136
第 3 章 栈与队列	139
第 4 章 字符串	145
第 5 章 二叉树	147
第 6 章 树	163
第 7 章 图	178
第 8 章 内排序	200
第 9 章 文件管理与外排序	207
第 10 章 检索	210
第 11 章 索引技术	213
第 12 章 高级数据结构	221

原书勘误	247
怎么读	247
编译不过	247
能编译、跑起来是错的	248
书内自相矛盾	249
反复出现、但单独一条通常还能「跑两下」	249
曾经误判、已撤回	249
还没收进本表的	250

写给学生

基于《数据结构与算法》（张铭、王腾蛟、赵海燕编著，高等教育出版社，2008）重编。

保留原书的章节脉络、数据结构与算法思想；全部示例统一为可编译、可测试的 C++17。

如何使用本教程

每一段印在书上的 C++ 都与配套源码逐字一致。建议按第 1 至第 12 章顺序阅读：先理解问题，再运行该章的示例程序，最后对照实现。

建议的学习顺序是第 1 至第 10 章；第 11 章给出索引技术导读；第 12 章收束内存复用与动态规划。每章先理解抽象模型，再阅读实现，最后运行对应单元的测试。所有提取操作（例如 pop、dequeue）都用 `std::optional` 表达正常的空状态；参数、容量或权重不合法则抛标准异常。

```
# 编译并运行全书的现代实现与测试
python3 tools/check_code.py --allow-degraded

# 检查书稿中的代码与源码是否仍逐字一致
python3 tools/check_doc.py
```

单章可以先跑那个单元的 `demo.cpp`。第 1 章的编译命令在 `ch01-adt.md` 里逐项解释过；后面各章的命令形状相同，只改 `-I` 路径和源文件。

各章一览

1. 概论：1.1 问题求解，1.2 数据结构，1.3 算法，1.4 算法分析。
2. 线性表：2.1 概念，2.2 顺序表，2.3 链表，2.4 比较。
3. 栈与队列：3.1 栈，3.2 队列，3.3 比较。
4. 字符串：4.1 概念，4.2 存储与实现，4.3 模式匹配。
5. 二叉树：5.1 概念，5.2 周游，5.3 存储，5.4 BST，5.5 堆，5.6 Huffman。
6. 树：6.1 定义，6.2 链式存储，6.3 顺序存储，6.4 K 叉树。
7. 图：7.1–7.3 概念与存储，7.4 周游，7.5 最短路，7.6 最小生成树。
8. 内部排序：8.1 概念，8.2–8.6 各类排序，8.7 时间代价。
9. 文件管理与外部排序：9.1–9.2 外存与文件，9.3 置换选择与选择树。
10. 检索：10.1 线性表检索，10.2 集合，10.3 散列。
11. 索引技术：11.1–11.6 线性/倒排/B 树/位图/红黑树。
12. 高级数据结构：12.1 多维数组，12.2 广义表与存储，12.3 Trie，12.4 改进 BST。

对照 2008 年纸书时，编译级硬伤与算法错见书末「原书勘误」。

阅读约定

- 算法思想优先。没有用标准库容器或排序算法替代本章要教的核心结构或算法。
- 资源所有权显式。手写数组和结点链使用 RAII、析构、深复制和移动语义管理寿命。
- 测试是正文的一部分。每个实现目录的 `test.cpp` 都覆盖其声明的原书清单；运行测试比 阅读一段未经执行的伪代码更可靠。
- 风险如实保留。递归树周游与析构仍适合教学，但极深退化结构存在栈溢出风险；详见 `collab/UNVERIFIED-RISKS.md`。

验证与边界

本教程覆盖原书 105 条算法/代码清单。`code/` 下的每个单元都有对应的 `unit.json`、现代实现、原书问题证据和可执行测试；书稿的 C++ 代码由工具逐字同步。完整的边界条件、未覆盖故障路径与 递归深度风险见 `collab/UNVERIFIED-RISKS.md`。

本教程采用 C++17，侧重数据结构的实现与算法语义。它不提供文件系统、数据库页缓存、并发控制 或跨平台性能承诺；这些属于使用数据结构的系统层设计。

第 1 章 概论

数据结构刻画信息及其关系，算法描述求解过程。二者互相依赖：结构选错了，算法再巧也补不回来。本章先用股市传言把问题说到能写程序，再补回逻辑结构、存储映射、抽象数据类型和渐进分析——这些原书没错，是后面十一章共用的词汇。

1.1 问题求解

程序是指令的组合，算法是求解过程的逻辑抽象，数据结构是数据及其关系在计算机里的反映。Wirth 的「程序 = 数据结构 + 算法」说的就是这件事。求解路径通常是：把现实问题抽象成模型，选定逻辑结构与存储方式，再设计算法并写成可运行的程序。

1.1.1 问题描述：股市的传言

本章把原书 1.1 的“股市传言”问题写成一个可直接调用的程序。它不是在找“边最多”的经纪人，而是在找一个起点：从这个人出发能把消息传给所有人，并且最慢的那条最短传播路径尽可能短。

1.1.2 问题分析和抽象

把 B1 到 B5 看成五个顶点。一条 $B_i \rightarrow B_j$ 边表示消息能从 B_i 直接传给 B_j ；边权是传播时间。没有路径就记为 ∞ ，表示消息永远传不到那里。B3 是唯一可以到达全部经纪人的起点。

图 1.1 经纪人间的消息传递（有向边，时间为权）：

起点	终点	时间
B1	B4	4
B1	B5	3
B2	B5	8
B3	B1	6
B3	B2	7
B3	B4	10
B3	B5	2
B5	B1	5
B5	B2	5

B3 指出四条边，是唯一能到达其余所有人的起点。

原书算法 1.1 的工作可以拆成三步：

- 1. Floyd 算法算出任意两人间的最短传播时间。
- 2. 对每一个候选起点，找出“从它出发到每个人的最短时间”中最慢的一项。
- 3. 从这些最大值中选最小的一个；如果某行仍有不可达的 ∞ ，该起点不能胜任。

原书中的 D 对应现代代码里的 `shortest` 矩阵，`max[i]` 对应每一行的 `largest_distance`，`pos` 对应 `best_source()` 的返回值。下标从 0 开始，因此 B3 的返回值是 2。

为什么要取“最慢的一项”

假设从 B3 开始，Floyd 算出的最短传播时间为：到 B1 是 6，到 B2 是 7，到自身是 0，到 B4 是 10，到 B5 是 2。消息要“传遍所有人”，必须等最慢的 B4 收到消息，因此 B3 的完成时间是这五个数里的最大值 10，而不是它们的和，也不是最小值。

把每个人都当一次起点，结果如下。 ∞ 表示至少有一人永远收不到消息，所以这一行没有可用的完成时间。

起点	到 B1	到 B2	到 B3	到 B4	到 B5	最慢的最短时间	是否可选
B1	0	∞	∞	4	3	∞	否
B2	13	0	∞	17	8	∞	否
B3	6	7	0	10	2	10	是
B4	∞	∞	∞	0	∞	∞	否
B5	5	5	∞	9	0	∞	否

因此答案是 B3。这个目标常称为“最小化最大距离”：不是选离某一个人最近的起点，而是选让最后一个收到消息的人也尽可能早收到消息的起点。

1.1.3 数据结构和算法设计

下面是完整可运行的程序。

下面是完整可运行的程序。`RumorNetwork(5)` 创建 B1 至 B5；`add_route(from, to, cost)` 添加一条有向传播路径，三个参数均从 0 开始编号。`best_source()` 返回 `std::optional<std::size_t>`：有解时保存起点下标，无解时是 `std::nullopt`。

源码文件在这里：可运行示例、数据结构实现、测试用例。`demo.cpp` 负责输入/输出；`modern.hpp` 只负责数据和计算，这就是把“怎么展示”与“怎么算”分开。

```
#include "modern.hpp"

#include <iostream>

int main() {
    // B1 ... B5 对应下标 0 ... 4。
    dsa::adt::RumorNetwork network(5);
    network.add_route(0, 3, 4);    // B1 -> B4, 耗时 4
    network.add_route(0, 4, 3);    // B1 -> B5, 耗时 3
    network.add_route(1, 4, 8);    // B2 -> B5, 耗时 8
```

```

network.add_route(2, 0, 6); // B3 -> B1, 耗时 6
network.add_route(2, 1, 7); // B3 -> B2, 耗时 7
network.add_route(2, 3, 10); // B3 -> B4, 耗时 10
network.add_route(2, 4, 2); // B3 -> B5, 耗时 2
network.add_route(4, 0, 5); // B5 -> B1, 耗时 5
network.add_route(4, 1, 5); // B5 -> B2, 耗时 5

const auto source = network.best_source();
if (source) {
    std::cout << " 最佳传播起点是 B" << *source + 1 << '\n';
} else {
    std::cout << " 不存在能到达全部经纪人的起点\n";
}
}

```

在仓库根目录运行：

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch01/adt code/ch01/adt/demo.cpp -o /
↳ tmp/rumor-demo
/tmp/rumor-demo

```

第一行是“编译”，第二行是“运行刚刚编译出的程序”。逐项解释如下：

片段	含义
c++	C++ 编译器命令。在 macOS 上通常指 Clang；Linux 上也可能指 GCC。
-std=c++17	按 C++17 语言规则编译；本教程的 <code>std::optional</code> 需要 C++17。
-Wall -Wextra	打开常见和额外的编译器警告，例如未使用变量或可疑转换。
-Werror	把警告当作错误；用于学习时可及早发现问题。初学调试时可暂时去掉它。
-Icode/ch01/adt	加一个头文件搜索目录，因此 <code>#include "modern.hpp"</code> 能找到 <code>code/ch01/adt/modern.hpp</code> 。
code/ch01/adt/demo.cpp	要编译的源文件。它包含 <code>main()</code> ，所以可以成为可执行程序。
-o /tmp/rumor-demo	指定输出文件名为 <code>/tmp/rumor-demo</code> 。/ tmp 是临时目录，重启后文件可能消失。
/tmp/rumor-demo	运行该可执行文件，打印程序结果。

如果系统提示 `c++: command not found`，需要先安装 C++ 编译器；如果想把

程序留在当前目录，可以把 `-o /tmp/rumor-demo` 改成 `-o rumor-demo`，再运行 `./rumor-demo`。

输出是：

```
最佳传播起点是 B3
```

如果删除 B3 的某条出边，例如 `network.add_route(2, 1, 7)`，B3 将无法到达 B2；当不存在任何能覆盖全部顶点的起点时，`best_source()` 返回空值，程序会输出“不存在能到达全部经纪人的起点”。

可以先只改边和权值，再运行同一条命令观察结果。例如把 `B3 -> B4` 的时间从 10 改为 1，答案仍是 B3，只是它的最慢传播时间从 10 缩短为 7。

实现导读

先只关注三个公开操作：构造函数建立距离矩阵，`add_route` 填入直接边，`best_source` 计算答案。`best_source` 内部复制一份矩阵，因而多次调用不会改变原始网络。下面按代码出现顺序解释。

预处理、头文件与命名空间

`#pragma once` 告诉编译器：同一个头文件即使被间接包含多次，也只处理一次，避免类定义重复。

头文件	本程序用它做什么
<code><cstdlib></code>	提供 <code>std::size_t</code> ，表示非负的大小或下标。
<code><limits></code>	提供 <code>std::numeric_limits<int>::max()</code> ，取得 <code>int</code> 可表示的最大值。
<code><optional></code>	提供 <code>std::optional</code> ，表达“可能没有答案”。
<code><stdexcept></code>	提供 <code>std::invalid_argument</code> ，用于报告非法参数。
<code><vector></code>	提供动态数组 <code>std::vector</code> ，这里用它保存二维距离矩阵。

`namespace dsa::adt { ... }` 把名字放入 `dsa::adt` 命名空间。完整类名因此是 `dsa::adt::RumorNetwork`，可避免项目中另一个人也定义 `RumorNetwork` 时发生重名。代码块中的 `// >>> adt` 与 `// <<< adt` 只是本书稿的同步标记，不参与 C++ 逻辑。

`class RumorNetwork` 定义一种新类型。`public:` 以下是调用者可以使用的接口；`private:` 以下是实现细节，调用者不能直接改写。

```
static constexpr int infinity = std::numeric_limits<int>::max() / 4;
```

`static` 表示 `infinity` 属于类本身，而不是每个对象各存一份；`constexpr` 表示编译期常量。取最大值的四分之一而非最大值，是为了计算 `a + b` 时仍留有余量，避免整数溢出。

```
explicit RumorNetwork(std::size_t people)
    : distance_(people, std::vector<int>(people, infinity)) { ... }
```

这是构造函数：`RumorNetwork network(5)` 会调用它。冒号后的部分叫成员初始化列表，先创建 `distance_`，再进入花括号。它构造一个 5 行、每行 5 列的整数矩阵，初始全为 `infinity`。随后循环把对角线改为 0，因为一个人到自己不需要传播时间。矩阵的第 `from` 行、第 `to` 列总是表示从 `from` 到 `to` 的当前已知最短时间。

添加一条边

```
void add_route(std::size_t from, std::size_t to, int cost)
```

这三个参数分别是起点编号、终点编号和直接传播时间。第一段 `if` 检查编号是否越界、时间是否为负；不满足题目定义时抛出 `std::invalid_argument`，调用者可用 `try/catch` 处理。第二段 `if` 只在新边更短时更新矩阵，所以即使重复添加同一方向的边，也保留较小的时间。

Floyd 三重循环

`auto shortest = distance_;` 复制输入矩阵。`auto` 让编译器自动推断类型，这里实际是 `std::vector<std::vector<int>>`。

三层循环的含义不是“随便循环三次”，而是按中转站逐步扩大可用路径：

```
for each via:      允许路径经过 via
  for each from:   固定起点 from
    for each to:   固定终点 to
      比较 原来的 from->to 与 from->via->to
```

条件 `shortest[from][via] != infinity && shortest[via][to] != infinity` 先确认两段都可达。若 `from -> via -> to` 的总时间更小，就更新 `shortest[from][to]`。例如 `B2` 到 `B4` 没有直接边，但 `B2→B5` 是 8、`B5→B1` 是 5、`B1→B4` 是 4，所以 Floyd 最终得到 `B2→B4` 是 $8 + 5 + 4 = 17$ 。

从最短路径矩阵选答案

```
std::optional<std::size_t> result;  
int smallest_eccentricity = infinity;
```

默认构造的 `result` 是空值, 表示“还没有找到合格起点”。`smallest_eccentricity` 保存目前见过的 最小完成时间。这里的 `eccentricity` (离心率) 就是某起点到全部顶点的最短距离中的最大值。

外层循环每次处理一行, 即一个候选起点; 内层循环扫描该行。三目表达式 `distance > largest_distance ? distance : largest_distance` 等价于“若新值更大就采用新值, 否则保留旧值”, 最终得到这行的最大值。若它比当前最佳值小, 就同时更新最佳时间与 `result`。

任何一行含 `infinity` 时, 该行最大值也是 `infinity`, 不会优于有限答案; 若所有行都是 `infinity`, `result` 保持为空, 函数返回 `std::nullopt`。[[nodiscard]] 提醒编译器: 调用者不应 无意丢弃这个可能为空的重要返回值。

私有数据成员

```
std::vector<std::vector<int>>> distance_;
```

外层 `vector` 是行, 内层 `vector<int>` 是一行中的列, 合起来是二维矩阵。末尾下划线是本教程的 约定, 表示私有成员; 它与函数参数 `distance` 或局部变量 `shortest` 不会混淆。

时间复杂度为 $O(V^3)$, 空间复杂度为 $O(V^2)$ 。这适合顶点数较少、需要比较所有起点的示例; 大图 通常应根据图的稀疏度和查询需求选择其他最短路径算法。

现代实现

```
#pragma once  
  
#include <cstddef>  
#include <limits>  
#include <optional>  
#include <stdexcept>  
#include <vector>  
  
namespace dsa::adt {  
  
// >>> adt  
class RumorNetwork {  
public:  
    static constexpr int infinity = std::numeric_limits<int>::max() / 4;
```

```

explicit RumorNetwork(std::size_t people)
    : distance_(people, std::vector<int>(people, infinity)) {
    for (std::size_t person = 0; person < people; ++person) {
        distance_[person][person] = 0;
    }
}

void add_route(std::size_t from, std::size_t to, int cost) {
    if (from >= distance_.size() || to >= distance_.size() || cost < 0) {
        throw std::invalid_argument("route");
    }
    if (cost < distance_[from][to]) {
        distance_[from][to] = cost;
    }
}

[[nodiscard]] std::optional<std::size_t> best_source() const {
    auto shortest = distance_;
    for (std::size_t via = 0; via < shortest.size(); ++via) {
        for (std::size_t from = 0; from < shortest.size(); ++from) {
            for (std::size_t to = 0; to < shortest.size(); ++to) {
                if (shortest[from][via] != infinity &&
                    shortest[via][to] != infinity &&
                    shortest[from][to] > shortest[from][via] + shortest[via][to])
↪ {
                    shortest[from][to] = shortest[from][via] + shortest[via][to];
                }
            }
        }
    }

    std::optional<std::size_t> result;
    int smallest_eccentricity = infinity;
    for (std::size_t from = 0; from < shortest.size(); ++from) {
        int largest_distance = 0;
        for (int distance : shortest[from]) {
            largest_distance = distance > largest_distance ? distance :
↪ largest_distance;
        }
        if (largest_distance < smallest_eccentricity) {
            smallest_eccentricity = largest_distance;
            result = from;
        }
    }
    return result;
}

```

```
private:
    std::vector<std::vector<int>> distance_;
};
// <<< adt

} // namespace dsa::adt
```

1.2 数据结构

数据结构描述的是：按一定逻辑关系组织起来的数据，它们在计算机里怎么存放，以及定义在其上的运算。三件事要分开看——逻辑结构、存储结构、运算——后面各章都在这三层之间做选择。

1.2.1 数据的逻辑结构

逻辑结构是从具体问题里抽出来的模型，只谈「有哪些结点、结点之间是什么关系」，不谈它们占几个字节。1.1 节经纪人之间的传播路径就是一个有向图：结点是人，关系是「消息能直接传给谁」。

用集合的语言写，逻辑结构是一个二元组 $B = (K, R)$ 。 K 是结点的有穷集合；关系集 R 是定义在 K 上的一组二元关系。若 $\langle k, k' \rangle \in r$ ，则称 k 是 k' 在关系 r 上的前驱， k' 是 k 的后继。没有前驱的是开始结点，没有后继的是终止结点。

教学计划是另一个常见例子。每门课是一个结点，全部必修课组成 K ；「必须先修」是 K 上的一个关系。 $\langle \text{程序设计}, \text{数据结构} \rangle$ 表示必须先上程序设计，再上数据结构。没有先修约束的课可以并行开设。

结点本身的类型可以很简单，也可以很复杂。整数、布尔、字符是基本类型；数组、结构、对象是复合类型。数据结构课关心的是结点之间的关系，所以通常把结点当作黑盒。

本书讨论的结构， R 里通常只有一个关系 r 。按 r 的形态可以分成三类：

1. 线性结构。每个结点在关系上最多一个前驱、一个后继。有唯一的开始结点（无前驱）和唯一的终止结点（无后继），其余内部结点恰好一前一后。数组、链表、栈、队列都是线性结构。
2. 树形结构。有且仅有一个没有前驱的根；其余每个结点恰好一个前驱，后继数目不限。从根到任意结点都存在唯一的一条由关系元组串起来的路径。文件系统的目录就是树。
3. 图结构。前驱和后继的数目都不加限制。交通网、1.1 节的传言网都是图。

树和图的分界是「每个结点是否至多一个直接前驱」；线和树的分界是「每个结点是否至多一个直接后继」。后面各章都建立在这三种关系上。

1.2.2 数据的存储结构

存储结构回答的是：同一套逻辑关系，在计算机的存储器里怎么落下来。主存按字节编址，相邻单元的地址连续，并且可以按地址随机访问、访问时间基本相同。

要把 (K, r) 存进去，需要两层映射：每个结点 k 对应存储器里一块连续区域；每个关系元组 $\langle k_i, k_j \rangle$ 对应两块区域地址之间的某种关系——或者是地址相邻，或者是指针相指。同一种逻辑结构可以有多种映射。线性表既可以顺序存放，也可以链接存放，得到的就是顺序表和链表。

常用的四种基本方法如下。

顺序方法把一组结点放进一片地址相邻的单元，逻辑上的先后用地址的先后表示。按下标访问因此是 $O(1)$ 。程序语言里的数组就是顺序方法。它通常也是紧凑的：除了数据本身，几乎不存附加信息。存储密度定义为「真正的数据」所占空间与整块结构所占空间之比。非线性结构有时也能顺序存放，但要额外记下一些关系信息，例如完全二叉树按层编号后，孩子下标可以用 $2i + 1$ 、 $2i + 2$ 算出来。

链接方法在每个结点里附一个（或多个）指针域，专门存放后继的地址。结点因此分成数据域和指针域。需要一个结点同时挂多个后继时，就放多个指针。链接适合经常增删、长度事先不知道的结构。代价是：不知道指针时，只能从链头一个一个比下去；按位置访问不再是 $O(1)$ 。

索引方法是顺序存储的推广。另造一张从整数下标到存储地址的表，每个表项是指向一条记录的指针。主文件可以无序、记录可以变长；检索时先在较小的索引里定位，再按地址读记录。数据量大、一次磁盘读写很贵时，索引的意义就是少读盘。图 1.4 画的就是「主文件无序、旁边一张按关键码排序的索引」：



散列方法是索引的延伸：不先造一张排好序的表，而是用散列函数从关键码直接算出地址，再把结点放进对应槽位。散列函数应当算得快，并且尽量把地址均匀铺开。冲突怎么处理、装载因子多高会退化，是第 10 章的内容。

具体应用里这四种方法经常组合。树的「子结点表」就是顺序加链接；选哪一种，还要看定义在结构上的运算——要不要随机访问、会不会频繁插入、数据在内存还是外存。

1.2.3 抽象数据类型

抽象意味着同一个概念可以有多种实现。一旦抽象问题被解决，许多同类问题就不必从头再来。用户自定义的类型，通常就是对「多种可能的存储和实现」做一次抽

象。

抽象数据类型 (abstract data type, ADT) 这个词是逐步形成的。二十世纪六七十年代，人们把用户自己定义的类型叫做抽象数据类型。更早的算法语言往往不提供这种机制。六十年代后期，为了把算法细节和数据的内部结构藏起来，Simula 67 引入了类，随后出现了模块：模块式给出外部看得见的运算接口，模块体存放对外不可见的私有数据和实现。接口与实现分离之后，用户才能真正自定义类型，达到数据抽象和信息隐蔽。在这个意义下，模块所定义的数据类型就叫做抽象数据类型。

作为描述数据结构的理论工具，ADT 可以看成「定义了一组操作的抽象模型」。它把数据和操作封在一起，使用方只能通过这些操作访问数据，不能直接去翻内部表示。这样，讨论一种结构时可以先不管它是数组还是链表，只问它提供哪些运算、这些运算有什么约定。整数连同加、减、乘、除，就是一个最简单的 ADT。

一般写成三元组 (D, R, P) ：

```
ADT 名字 {  
    数据对象 D: 结点是什么  
    数据关系 R: 结点之间有什么关系  
    基本操作 P: 对外提供哪些运算  
}
```

D 和 R 给出数据模型，是对数据的抽象； P 给出这个模型能做什么，是对操作的抽象。

在面向对象语言里，一组操作常常先写成接口，不透露实现。C++ 用类（以及类模板）的声明表示 ADT，用类的定义去实现它。因此，一个写好的 C++ 类同时承担两件事：它是某种存储结构，也是这组运算在该存储上的实现。

这两层不要混：概念层上的 ADT 描述「问题需要什么运算」；实现层上的类描述「这些运算在内存里怎么做」。类本身还是一种普通类型，可以拿来定义变量，也就是对象。所以 ADT 也可以说成：数据的逻辑结构，加上定义在这个逻辑结构上的一组抽象运算。

原书【代码 1.2】用下面这个空模板来提醒读者「先写接口」：

```
template <class Type>  
class ClassName {  
private:  
    Type data;          // 取值类型和存储  
public:  
    void method();      // 对外运算  
};
```

本书各章不再另设一个空基类——那种基类给不了多态，析构函数如果不是虚的，通过基类指针删除派生对象还是未定义行为。运算直接写在实现类上。对元素类型的要求，正文里用一句话说清即可：例如顺序栈、顺序表底层是 `new T[n]`，会把整块槽位都默认构造出来，所以 `T` 必须能默认构造。

1.3 算法

算法研究可以追溯得很远。中文「算法」出自《周髀算经》；英文 algorithm 来自九世纪波斯数学家花拉子米 (al-Khwarizmi)。Ada Lovelace 在 1842 年为巴贝奇分析机写过求解伯努利方程的程序，因此常被看作第一位程序员；那台机器没有建成，程序也没有真正跑过。二十世纪图灵提出图灵机，给「什么叫做可计算」一个精确模型，多数算法才可以稳定地写成交给计算机执行的程序。Knuth 在七十年代说，计算机科学就是研究算法的学问。

按应用范围，算法分成数值的和非数值的；按工作方式，分成串行的和并行的。本书后面各章几乎都是非数值、串行的算法。

1.3.1 算法的概念

算法是对特定问题求解过程的描述：为解决该问题而采取的、具体而有限的操作步骤。程序是算法的一种实现；计算机按程序一步一步做，就把问题解出来。

描述一个算法，通常要说清输入是什么类型、输出应满足什么性质。辗转相除法的输入是两个正整数 n 、 m ，输出是它们的最大公因子。高斯消元的输入是系数矩阵和右端常数，输出是使方程成立的一组变元取值。

算法一般要满足四条性质。

1. 通用。凡是符合声明类型的输入，都能按同一套步骤求解，并且结果正确。不是只对书上那一组样例成立。
2. 有效。每一步都是人或机器能确切执行的动作，指令集是有限、明确的。辗转相除只用「整除取余、取商、判断余数是否为零」。每一步的结果也要有确定的类型，不能算完不知道得到的是什么。
3. 确定。做完这一步，下一步是什么必须说死：顺序往下、按条件分支、或者结束。不能停在「被锁住」的状态，也不能只写一句含糊的话。
4. 有穷。必须在有限步内停机，不能含死循环。注意：指令条数有限，并不自动保证执行步数有限——循环条件写错，有限条指令也可以转个没完。设计时要单独检查结束条件。

程序是算法落在某种语言里的样子。同一算法可以有许多程序；一个程序如果对某类输入会永远循环，它就不是该问题的算法。

1.3.2 算法设计

算法设计要回答：面对这一类问题，步骤从哪来。常用方法有穷举、回溯、分治、递归、贪心和动态规划。

穷举（枚举）把问题空间里的对象按某种规则一一列出来，逐个检查是否满足条件。对象必须有限，并且有明确的列举顺序。全局穷举往往太慢，但在局部很有用。switch 按离散取值分支，就是一种小范围穷举。

回溯（试探）按某种顺序枚举候选解。发现当前候选不可能成功，就放弃它、退回上一步换下一个，这个动作叫回溯；当前候选还不够大、但局部约束都满足，就把它扩大再试，这个动作叫向前试探。第 5、6、7 章的深度优先周游，以及第 3 章

的背包，都是回溯。

分治把一个大问题剖成若干较小的、同类的子问题，分别求解，再把子问题的解合并成原问题的解。这是自顶向下的。如果子问题仍是原问题的缩小版，反复剖下去就会自然地写成递归。分治和递归经常成对出现。第 8 章的快排、归并，第 10 章的二分检索，都是分治。

贪心从初始状态出发，每一步按某个局部最优标准做一个不可撤回的选择，希望这些局部最优能拼成全局最优。度量标准选得对不对，是贪心能不能用的核心。动态规划也处理最优性问题，但子问题往往互相重叠：如果直接按分治递归，同一子问题会被算很多次。办法是把已经算过的答案存进一张表，以后直接查。第 7 章的 Prim、Kruskal、Dijkstra 是贪心；Floyd 是比较典型的动态规划。第 12 章的最优二叉搜索树也是动态规划。

1.1 节的传言问题要任意两点最短路，并且顶点数很小，所以选 Floyd，而不是对每个起点跑一遍 Dijkstra。

这几种方法可以组合、变形，也可以只拿来处理困难问题的特殊情形。后面各章会反复回到它们。

1.4 算法分析

同一问题往往有多种算法，各有适用场合。算法分析用数学工具讨论每种算法的代价，目的是：在已有算法里选出比较合适的一种，或者看清该往哪个方向改进。

Hartmanis 和 Stearns 把这种代价叫做计算复杂性。复杂性高，意思是跑这个算法要消耗的计算机资源多。最要紧的两种资源是时间（处理器）和空间（存储器）。本书后文说的「时间代价」「空间代价」，指的就是这两项。

不能用墙上的秒数来比较算法。同一段程序在巨型机和 PC 上可以差几个数量级；换一种语言、换一个编译器，数字也会变。C 写的程序曾经比同一算法的 LISP 实现快近二十倍——那是实现环境的差别，不是算法的差别。所以分析时看的是：输入规模为 n 时，算法做了多少次与具体机器无关的基本操作。规模一般指输入量的个数，排序里就是待排元素的个数；一次基本操作耗时应当与操作数的具体数值无关，例如两个机器字整数相加。

若时间与规模成线性， $t = cn$ ，规模乘 5，时间也乘 5。若 $t = \log_2 n$ ，规模加倍，时间只加 1。后面各章的「 $O(n)$ 」「 $O(n \log n)$ 」都按这个口径读。

1.4.1 渐进分析方法

精确计数往往是一个很复杂的函数。 n 很大时，不能显著改变量级的项都可以丢掉，剩下的近似在规模足够大时会贴近原值。这种方法叫渐进分析。例如

$$f(n) = n^2 + 100n + \log_{10} n + 1000$$

$n = 1$ 时常数项 1000 最大； $n = 10$ 时 $100n$ 与 1000 相当； $n = 100$ 时前两项相当； n 再大，二次项就压倒其余所有项。所以这个函数在 n 很大时就是 $\Theta(n^2)$ 。

渐进分析是一种故意简化的模型，用来把注意力放在最重要的部分。并非任何时候都能忽略常数：只排 10 个数的算法，和排上万个数的算法，适用的实现可以完全不同。

大 O。 设 f 、 g 是从自然数到非负实数的函数。若存在正数 c 和 N ，使得所有 $n \geq N$ 都有 $f(n) \leq c g(n)$ ，则称 $f(n)$ 是 $O(g(n))$ 的。 g 是 f 增长的一个上限：这个算法最多会差到什么程度。上限可以有很多， $O(n)$ 里的函数一定也在 $O(n^2)$ 里；习惯上取那个最小的、仍然成立的上限。

几条常用性质： f 是 $O(g)$ 、 g 是 $O(h)$ ，则 f 是 $O(h)$ ；两个 $O(h)$ 相加仍是 $O(h)$ ； an^k 是 $O(n^k)$ ；常数倍不改变阶；不同底的对数同阶，本书把 $\log_2 n$ 简写为 $\log n$ 。

常见的上限有：

$g(n)$	名称	直观
1	常数	与 n 无关
$\log n$	对数	比线性慢，二分检索
n	线性	扫一遍
$n \log n$	线性对数	低于平方，快排 / 归并 / 许多树算法
n^2	平方	$1 + 2 + \dots + n$
a^n	指数	比任何多项式都快，递归穷举时要小心

令时间单位为 $1 \mu s$ ， $n = 1000$ 时： $T(n) = n$ 是 1 ms； $T(n) = n^2$ 是 1 s； $T(n) = n^3$ 约 16 分钟； $T(n) = 2^n$ 大约 10^{286} 年，而地球年龄不超过 10^{10} 年。指数爆炸型算法能不用就不用。

图 1.5 用同一组 n 把这些阶摆在一起：

阶	$n = 16$	$n = 256$	直觉
1	1	1	常数
$\log n$	4	8	二分
n	16	256	扫一遍
$n \log n$	64	2048	快排 / 归并
n^2	256	65536	双重循环
2^n	65536	—	穷举子集

Ω 。 把大 O 定义里的不等式反过来：存在 c 、 N ，使 $n \geq N$ 时 $f(n) \geq c g(n)$ ，则 f 是 $\Omega(g)$ 的。这是增长的下限，习惯上取那个最大的、仍然成立的下限。

Θ 。 若 f 既是 $O(g)$ 又是 $\Omega(g)$ ，则称 f 是 $\Theta(g)$ ：存在正常数 c_1 、 c_2 和正整数 N ，使 $n > N$ 时

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

例如 $f(n) = 100n^2 + 5n + 500$ 是 $\Theta(n^2)$ ，可取 $c_1 = 100$ 、 $c_2 = 105$ 、 $N = 10$ 。
数一数基本操作。对数组求和：

```
for (i = sum = 0; i < n; i++)  
    sum += a[i];
```

循环前两次赋值，循环 n 次、每次两次赋值，一共 $2 + 2n$ 次，即 $O(n)$ 。

两层循环求所有前缀和：外层 n 次，内层第 i 趟做约 $2i$ 次赋值，总和是 $1 + 3n + n(n-1) = O(n^2)$ 。循环嵌套通常会提高阶，但不是必然：若内层每次只处理固定的 4 个元素，总赋值次数仍是 $O(n)$ 。

排序算法的时间主要花在比较和移动上，具体见第 8 章。

1.4.2 最佳、最差和平均情况

即使规模相同，输入不同，有些算法的操作次数也会差很多。因为实际走哪条分支，往往取决于数据本身。所以分析时要说清是哪一种输入。

顺序检索：目标在第一个位置，比较 1 次（最好）；目标不在表里，比较 n 次（最坏）；等概率时平均 $(n+1)/2$ 。快速排序平均 $O(n \log n)$ ，已经有序时退化到 $O(n^2)$ 。只报一个数字、不说针对哪种输入，这个数字没有意义。

1.4.3 时间和空间的折衷

预排序、建索引、开散列表，都是先花时间或空间，再换检索时的时间。没有免费的加速。空间紧、时间松，和空间松、时间紧，选的结构会不一样。

1.4.4 求解问题时数据结构的选择和评价

先看运算集合：要不要按下标访问，会不会在中间频繁插入删除，要不要按关键词直接找到记录。再看规模和存储层次：数据在内存还是外存，一次磁盘 I/O 往往比一次比较贵几个数量级。

1.1 节要任意两点最短路，并且只有五个经纪人，邻接矩阵加 Floyd 是合适的：实现简单，三重循环在 $n = 5$ 时可以忽略。顶点很多、边很少、只问单源最短路时，就应该换成邻接表加 Dijkstra。后面每一章都会再做一次这样的取舍。

本章小结

本章从求解问题入手，引入数据结构、算法和衡量效率的方法。用计算机解题，是问题抽象、数据抽象和算法抽象：先把需求说清楚，再建成模型、选定结构和算法，最后用某种语言实现。

数据结构管三件事：逻辑结构（结点之间是什么关系）、存储结构（这套关系在存储器里怎么落）、运算。逻辑结构按关系形态分成线性、树、图三类。同一种逻辑结构可以顺序、链接、索引或散列存放。ADT 把数据和运算封在一起，使用方只看见接口。

算法是有限、确定、有效、有穷的指令序列。设计常用穷举、回溯、分治、递归、

贪心和动态规划。分析不看墙上的秒数，而看基本操作次数随规模 n 怎么长，用大 O 、 Ω 、 Θ 说话，并分清最好、最坏和平均。时间和空间往往可以互换。

习题

1. 设计一个算法，从大到小依次输出顺序读入的三个整数 X 、 Y 、 Z 。
2. 举例说明什么是数据结构。
3. 数据有哪些存储方式？各有什么特点。
4. 分析下面程序段中循环体执行多少次。

```
int i = 0, s = 0, n = 100;
do {
    i = i + 1;
    s = s + 10 * i;
} while ((i < n) && (s < n));
```

5. 指出算法 A 的功能和时间复杂度。 h 、 g 是同一个单循环链表上的两个结点指针。

```
void B(Node* s, Node* q) {
    Node* p = s;
    while (p->next != q) p = p->next;
    p->next = s;
}
void A(Node* h, Node* g) {
    B(h, g);
    B(g, h);
}
```

6. 设 n 是偶数，计算下列程序段结束后 m 的值，并给出时间复杂度。

```
m = 0;
for (i = 1; i <= n; i++)
    for (j = 2 * i; j <= n; j++)
        m = m + 1;
```

7. 把下列运行时间写成大 O : $T_1(n) = 1000$, $T_2(n) = n^2 + 1000n$, $T_3(n) = 3n^3 + 100n^2 + n + 1$ 。
8. 给出下面两个算法的时间复杂度：(1) 一层循环 $i = 1..n$ 做 $x = x + 1$ ；(2) 两层循环 $i, j = 1..n$ 做 $x = x + 1$ 。
9. 给出求两个整数最大公因数的算法，并分析时间和空间复杂度。
10. 计算机每秒 10^{10} 次操作，要求一小时内算完。下列复杂度各能处理多大的 n : n^2 , n^3 , $100n^2$, $n \log_{10} n$, 2^n , 2^{2^n} 。
11. 已知 $f(n)$ 是 $O(g(n))$ 。判断并证明或举反例：(1) $\log_2 f(n)$ 是 $O(\log_2 g(n))$ ；(2) $2^{f(n)}$ 是 $O(2^{g(n)})$ ；(3) $f(n)^2$ 是 $O(g(n)^2)$ 。

12. 计算下列递推的复杂度: (1) $T(n) = T(\lceil n/2 \rceil) + 1$; (2) $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$; (3) $T(n) = 2T(\lfloor n/2 \rfloor) + n$ 。

上机题

1. m 、 n 是自然数。 m 可以写成若干个不超过 n 的自然数之和, 编写 $f(m, n)$ 计算有多少种写法。例如 $f(5, 3) = 5$: $3 + 2$, $3 + 1 + 1$, $2 + 2 + 1$, $2 + 1 + 1 + 1$, $1 + 1 + 1 + 1 + 1$ 。
2. 用 C++ 类定义复数 ADT: 内部用浮点数存实部和虚部; 三个构造函数 (默认、只给实部、同时给实部和虚部); 获取/修改实虚部, 以及加减乘除; 重载输出流。

第 2 章 线性表

线性表把一串同类型元素排成唯一的先后次序。顺序表把元素连续放在数组里，能 $O(1)$ 按下标访问；链表用链接保存相邻关系，能在已知结点位置 $O(1)$ 插入或删除。关键是区分「按位置找元素」和「已知结点后改链接」这两类操作。

源码：顺序表、顺序表示例、链表、链表示例。

先跑一遍

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::ArrayList<int> values;
    values.append(10);
    values.append(30);
    values.insert(1, 20);
    std::cout << " 顺序表:";
    for (std::size_t index = 0; index < values.size(); ++index) {
        std::cout << ' ' << values.at(index);
    }
    std::cout << "\n查找 20 的下标: " << *values.find(20) << '\n';
    std::cout << " 删除位置 1 得到 " << values.remove(1) << ", 剩余:";
    for (std::size_t index = 0; index < values.size(); ++index) {
        std::cout << ' ' << values.at(index);
    }
    std::cout << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch02/array_list \
    code/ch02/array_list/demo.cpp -o /tmp/list-demo
/tmp/list-demo
```

```
顺序表: 10 20 30
查找 20 的下标: 1
删除位置 1 得到 20, 剩余: 10 30
```

`insert(1, 20)` 要把后面的元素右移一位，这是顺序表的固有代价。链表同一组操作只改两条链接，但按位置找前驱仍是 $O(n)$ ：

```
#include "modern.hpp"
```

```
#include <iostream>

int main() {
    dsa::LinkedList<int> values;
    values.append(10);
    values.append(30);
    values.insert(1, 20);
    std::cout << " 链表:";
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << "\n删除位置 0 得到 " << values.remove(0) << ", 剩余:";
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << "\nappend 之后尾元素是 " << values.at(values.size() - 1) << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch02/linked_list \
    code/ch02/linked_list/demo.cpp -o /tmp/link-demo
/tmp/link-demo
```

```
链表: 10 20 30
删除位置 0 得到 10, 剩余: 20 30
append 之后尾元素是 30
```

append 经尾指针 $O(1)$ 接链，不必再从头走到尾。

2.1 线性表的概念

线性表 (linear list) 是由 $n(n \geq 0)$ 个类型相同的数据元素组成的有限序列。除第一个元素外，每个元素有且仅有一个直接前驱；除最后一个元素外，每个元素有且仅有一个直接后继。

线性表的抽象数据类型给出的是一组运算：置空、判空、在表尾追加、在指定位置插入、删除指定位置的元素、按位置取值与改值、按值查找位置。

原书【代码 2.1】用一个类模板 `List<T>` 来写这组运算。那份清单有两处今天必须改：

1. 它声明了 `bool delete(const int p);`——**delete** 是 C++ 关键字，不能作函数名。整章的删除操作都建立在这个编译不过的名字上。
2. 它写的是 `class List { void clear(); ... };`，通篇没有 **public:**。class 的默认访问权限是 `private`，于是这个抽象数据类型的每一个运算都调不到。（同一本书第 3 章的代码 3.1 是写了 `public:` 的。）

第 2 点尤其值得停一下：抽象数据类型的意义就是对外给出一组运算。一个所有运算都私有的 ADT，在语法上成立、在语义上是空的。

本书把这组运算直接定义在 `ArrayList<T>` 上，不再另设一个基类——理由与第 3 章相同：那样的空基类给不了多态，却会带来非虚析构的未定义行为。底层 `new T[n]` 会默认构造整块槽位，所以 `T` 必须能默认构造。这句话写在正文里即可。

```
/// 顺序表（按顺序方式存储的线性表，又称向量）。
///
/// 与原书 arrList 的差别：容量不足时自动翻倍，而不是打印 "The list is overflow"
/// 然后返回 false；位置非法抛 std::out_of_range，而不是打印一行再返回 false；
/// 查找返回 std::optional<size_type>，而不是「出参 + bool」双通道。
template <typename T>
class ArrayList {
public:
    using value_type = T;
    using size_type = std::size_t;
```

2.2 顺序表

按顺序方式存储的线性表称为顺序表 (array-based list)，又称向量 (vector)，通过数组建立。

假设每个元素占用 L 个存储单元，顺序表的开始结点 k_0 的存储位置记为 $b = \text{loc}(k_0)$ ，称为首地址；则下标为 i 的元素 k_i 的存储位置为

$$\text{loc}(k_i) = b + i \times L$$

每个元素的存储位置都与起始位置相差一个与位序成正比的常数。只要确定了基地址，表中任一元素的地址都能直接算出——顺序表因此是一种随机存取的存储结构，按下标取值的时间代价为 $O(1)$ 。物理相邻表示了逻辑相邻。

(a) 线性表的逻辑结构

数据元素	k_0	k_1	...	k_i	...	k_{n-1}	...
逻辑地址	0	1	...	i	...	$n-1$	... $\text{maxSize}-1$

(b) 线性表的顺序存储结构

数据元素	k_0	k_1	...	k_i	...	k_{n-1}	...
存储地址	b	$b+L$	...	$b+i \cdot L$	...	$b+(n-1) \cdot L$	...

图 2.1 顺序表的示意图

2.2.1 存储与所有权

原书【代码 2.2】用四个成员表示顺序表：`T* aList`、`int maxSize`、`int curLen`、`int position`。前三个对应缓冲区、容量、当前长度，现代实现一一对应（改用 `std::size_t`，因为「负下标」不该在类型层面存在）。

第四个 `position` 是个当前处理位置游标，配合正文提到的 `setPos / setStart / next / prev` 用来「依次处理元素」。这个设计今天不能要：遍历状态一旦住进容器，`const` 对象就没法遍历（游标要改），两处代码不能同时遍历，嵌套遍历直接互相踩。本书删掉这个成员，改为提供 `begin()/end()`，把遍历状态交回调用方——`range-for` 因此可以直接用在顺序表上。（顺带一提：原书那个 `position`，在书中展示的所有算法里一次都没被用到。）

与第 3 章一样，缓冲区是裸指针加显式五法则：顺序表的存储管理正是本节的教学内容，换成 `std::unique_ptr<T[]>` 会让五法则退化成走过场。原书 `arrList` 有析构函数却没有拷贝构造与拷贝赋值，一次 `arrList<int> b = a;` 就二次释放——与第 3 章 `arrStack` 是同一个错误，同一份 ASan 报告可以复现。

另外原书的 `clear()` 是这样写的：

```
void clear() { delete [] aList; curLen = position = 0; aList = new T[maxSize]; }
```

释放整块再重新分配。既没必要（把长度归零即可，容量留着复用），也不是异常安全的：若 `new` 抛异常，对象就停在「指针已释放、长度已归零」的破碎状态，之后析构还会再 `delete[]` 一次。本书的 `clear()` 是 `noexcept` 的，只把长度归零。

2.2.2 顺序表的检索

顺序表上的检索分按位置和按内容两类。按位置的检索直接由地址公式算出， $O(1)$ ：

```
/// 按下标读取， $O(1)$ 。越界抛 std::out_of_range。
/// 原书 getValue 用「出参 + bool」，越界时打印一行再返回 false。
const T& at(size_type index) const {
    check_index(index, "ArrayList::at");
    return data_[index];
}

T& at(size_type index) {
    check_index(index, "ArrayList::at");
    return data_[index];
}

/// 修改指定位置的值。越界抛 std::out_of_range。
void set(size_type index, const T& value) {
    check_index(index, "ArrayList::set");
    data_[index] = value;
}
```


按内容的检索是把待查值与表中元素依次比较, $O(n)$ 。原书【算法 2.3】写作 `bool getPos(int& p, const T value)`——位置由出参带出、成败由返回值带出。这个形状的问题是: 调用方忘了检查返回值, 读到的就是没被写过的 `p`。

(顺带一提, 算法 2.3 那段按印刷原样是编译不过的: 循环写的是 `for (i = 0; i < n; i++)`, 而 `n` 从未声明, 按上下文应为 `curLen`。)

```
/// 按内容查找, 返回第一次出现的下标; 没有则返回 std::nullopt。  $O(n)$ 。
///
/// 原书【算法 2.3】是 'bool getPos(int& p, const T value)': 出参带位置、
/// 返回值带成败。调用方忘了检查返回值, 读到的就是没被写过的 p。
/// 这里让「找没找到」进入类型系统, 忽略返回值还会被 -Wunused-result 拦下。
std::optional<size_type> find(const T& value) const {
    for (size_type i = 0; i < size_; ++i) {
        if (data_[i] == value) {
            return i;
        }
    }
    return std::nullopt;
}
```

检索的时间代价体现在比较次数上。最好情况是第 1 个元素即为所求, 比较 1 次; 最差情况是表中没有该元素, 比较 n 次。等概率假设下平均比较次数为

$$\sum_{i=1}^n p \times i = \frac{1}{n}(1 + 2 + \cdots + n) = \frac{n+1}{2}$$

即平均需要检查表中一半的元素, 时间开销为 $O(n)$ 。

2.2.3 顺序表的插入

插入要在指定位置腾出一个空位: 从表尾起, 把 `pos` 之后的元素逐个右移一位。

```
/// 在位置 pos 插入元素, pos 可以等于 size() (即追加到表尾)。
/// 位置非法抛 std::out_of_range; 容量不足自动翻倍。
///
/// 时间代价仍是  $O(n)$ ——pos 之后的元素都要右移一位。这是顺序表的固有代价,
/// 也是第 2.3 节要拿它和链表对比的地方, 没有被"优化"掉。
void insert(size_type pos, const T& value) {
    make_gap(pos);
    data_[pos] = value;
    ++size_;
}

void insert(size_type pos, T&& value) {
    make_gap(pos);
    data_[pos] = std::move(value);
}
```

```

    ++size_;
}

void append(const T& value) { insert(size_, value); }
void append(T&& value) { insert(size_, std::move(value)); }

```

两处与原书不同：

- 容量不足时自动翻倍，而不是打印 "The list is overflow" 然后返回 false。扩容策略与第 3 章算法 3.3 相同，搬迁判据见 3.1.3 节末（移动赋值是否 noexcept）。
- 位置非法抛 `std::out_of_range`，而不是打印一行再返回 false。按公约，可预期的空状态用 optional，真正的错误抛异常——插到表外是错误。

插入位置 p 处腾空位的过程如下（图 2.2）： p 及其之后的元素整体右移一位，空出的 p 位置写入新元素。

aList	k0	k1	...	value	kp	...	kn-1	...
下标	0	1	...	p	p+1	...	n	...
								maxSize-1

图 2.2 顺序表元素插入示意图

时间代价仍然是 $O(n)$ 。最好情况是插在表尾，移动 0 次；最差情况是插在表头， n 个元素全要移动；等概率假设下平均移动次数为

$$\sum_{i=0}^n p \times (n-i) = \frac{1}{n+1} \sum_{i=0}^n (n-i) = \frac{n}{2}$$

扩容没有改变这个结论：翻倍带来的搬迁摊还到每次插入是 $O(1)$ ，而元素右移本身仍是 $O(n)$ 。这一点是 2.3 节拿顺序表与链表对比的依据，不能被优化掉。

2.2.4 顺序表的删除

删除是插入的镜像：把 pos 之后的元素逐个左移一位。

```

/// 删除位置 pos 上的元素并返回它。位置非法抛 std::out_of_range。
///
/// 空表上删除必然越界，所以不需要原书那句单独的空表检查——
/// 「表空」在这里不是一种可预期状态，而就是下标非法的一个特例。
T remove(size_type pos) {
    check_index(pos, "ArrayList::remove");
    T removed = std::move(data_[pos]);
    for (size_type i = pos; i + 1 < size_; ++i) {
        data_[i] = std::move(data_[i + 1]); // 左移一位, O(n)
    }
    --size_;
    return removed;
}

```

```
}
```

删除位置 p 上的元素后，其后的元素整体左移一位（图 2.3）：

删除前：

aList	k0	k1	k2	...	kp	...	kn-1	...
下标	0	1	2	...	p	...	n-1	...

删除后：

aList	k0	k1	k2	...	kp+1	...	kn-1	...
下标	0	1	2	...	p	...	n-2	...

图 2.3 顺序表元素删除示意图

原书【算法 2.5】的函数名是 `delete`——C++ 关键字，编译不过；本书叫 `remove`。另外它返回 `bool`，被删掉的值就此丢失；这里把它返回给调用方，「删除并取用」不必先 `getValue` 再 `delete` 两步走。

删除的时间代价分析与插入相同，平均为 $O(n)$ 。

与原书的对照

原书	现在	为什么
<code>bool delete(const int p)</code>	<code>T remove(size_type pos)</code>	<code>delete</code> 是关键字，原样编译不过；顺带把被删元素还给调用方
<code>class List { ... } 无 public:</code>	不设基类，要求写成 <code>static_assert</code>	所有运算私有的 ADT 在语义上是空的；空基类给不了多态
<code>for (i = 0; i < n; i++)</code>	<code>i < size_</code>	原书 <code>n</code> 从未声明，编译不过
<code>bool getPos(int& p, T v)</code>	<code>std::optional<size_type> find(const T&)</code>	「找没找到」交给类型系统，忽略返回值会告警
<code>bool getValue(int p, T& v)</code>	<code>const T& at(size_type)</code> ，越界抛异常	同上；越界是错误，不是可预期状态
<code>int maxSize / curLen, 满了拒绝</code>	<code>size_t capacity_ / size_</code> ，自动翻倍	<code>int</code> 溢出是未定义行为；「负下标」不该存在；容量不该是使用者的负担

原书	现在	为什么
<code>int position</code> 游标 + <code>next/prev</code>	<code>begin()/end()</code>	遍历状态住在容器里， <code>const</code> 不能遍历、不能嵌套遍历
<code>cout << "The list is overflow"</code>	不做任何 I/O	数据结构不该和标准输出耦合
<code>clear()</code> 释放后重新分配	<code>clear() noexcept</code> 只归零 长度	原写法没必要，且 <code>new</code> 抛 异常会留下破碎对象

刻意没改的：连续存储、按下标 $O(1)$ 随机存取、插入与删除搬动 $O(n)$ 个元素。这三条正是 2.3 节拿顺序表和链表做对比的全部依据，一条都不能优化掉。

完整实现见 `code/ch02/array_list/modern.hpp`，测试见同目录 `test.cpp` (47 项断言，覆盖上表每一行；用 `python3 tools/check_code.py` 在 `-Werror + ASan/UBSan` 与 `-O2` 两种构建下各跑一遍)。

2.3 链表

顺序表用物理相邻表示逻辑相邻，所以按下标读取是 $O(1)$ ，但中间插入和删除需要搬动后续元素。链表把逻辑相邻写进结点的链接域：结点可以散落在内存中；给定一个前驱结点后，插入或删除只改常数条指针。代价也必须如实保留：要按位置找到那个前驱，仍须从表头循链，所以按位置访问、查找、插入和删除的总时间仍是 $O(n)$ 。

2.3.1 单链结点与头结点

【代码 2.6】单链表的结点定义。

```
/// 原书【代码 2.6】的单链结点：数据域与指向后继的链接域。
///
/// 实际容器把它藏在私有实现里，避免调用方任意改坏链；这里保留独立类型，
/// 因为后续栈、队列可复用同一种结点形状。
template <typename T>
struct SinglyLink {
    T data;
    SinglyLink* next{nullptr};

    template <typename U>
    explicit SinglyLink(U&& value, SinglyLink* successor = nullptr)
        : data(std::forward<U>(value)), next(successor) {}
};

/// 原书【代码 2.12】的双链结点：额外保存前驱链接。
template <typename T>
struct DoublyLink {
```

```

T data;
DoublyLink* next{nullptr};
DoublyLink* prev{nullptr};

template <typename U>
explicit DoublyLink(U&& value, DoublyLink* predecessor = nullptr, DoublyLink*
↳ successor = nullptr)
    : data(std::forward<U>(value)), next(successor), prev(predecessor) {}
};

```

【代码 2.6 结束】

LinkedList<T> 在对象内嵌一个不承载 T 的头结点。它等价于原书的“第 -1 个结点”，从而表头插入和删除都变成“修改某个前驱的 next”；空表不必另写一套分支。尾指针指向最后一个实际结点，空表时回指头结点，故 append 不需要从头寻找末尾。

【代码 2.7】单链表的类型定义。

```

/// 带头结点、尾指针的单链表。
///
/// 原书的 `setPos(-1)` 返回头结点；本实现把这个实现细节留在 predecessor_at,
/// 对外位置统一为 [0, size()]。按值查找返回 optional，位置错误抛 out_of_range。
template <typename T>
class LinkedList {
    struct NodeBase {
        NodeBase* next{nullptr};
    };
    struct Node final : NodeBase {
        T value;

        template <typename U>
        explicit Node(U&& item, NodeBase* successor = nullptr)
            : NodeBase{successor}, value(std::forward<U>(item)) {}
    };

public:
    using value_type = T;
    using size_type = std::size_t;

```

【代码 2.7 结束】

2.3.2 构析与循链定位

【算法 2.8】带有头结点的单链表构造函数与析构函数。

本实现的头结点嵌入对象本身，不需要动态分配；clear() 沿 next 逐结点释放实际结点，析构函数调用它。复制构造采用“先逐个接入新链，失败即清理”的规则，

避免半成品对象在元素复制抛异常时泄漏。

【算法 2.8 结束】

【算法 2.9】寻找链表的第 i 个结点。

定位从头结点开始逐步走 `next`，不新建任何结点。原书的 `setPos(-1)` 是处理头结点的技巧；现代接口不暴露 `-1` 这个特殊位置，内部 `predecessor_at(0)` 直接返回头结点。

【算法 2.9 结束】

2.3.3 插入与删除

【算法 2.10】插入单链表的第 i 个结点。

```
/// 尾插直接经 tail_ 接链, O(1)。不能转调 insert(size_), 后者必须循链定位前驱。
void append(const T& value) { append_impl(value); }
void append(T&& value) { append_impl(std::move(value)); }

void insert(size_type pos, const T& value) { insert_impl(pos, value); }
void insert(size_type pos, T&& value) { insert_impl(pos, std::move(value)); }
```

【算法 2.10 结束】

插入先定位前驱、再构造新结点、最后接入链接；结点构造或分配失败时，链接和长度尚未变化。若前驱恰是尾结点，同步让 `tail_` 指向新结点。给定前驱后的接链是 $O(1)$ ，定位仍是 $O(n)$ 。

【算法 2.11】单链表的删除算法。

```
T remove(size_type pos) {
    NodeBase* predecessor = predecessor_at(pos);
    NodeBase* removed = predecessor->next;
    if (removed == nullptr) {
        throw std::out_of_range("LinkedList::remove: 下标越界");
    }
    // 先移动出值；若 T 的移动构造抛，链接尚未改变，容器仍完整。
    T value = std::move(static_cast<Node*>(removed)->value);
    predecessor->next = removed->next;
    if (removed == tail_) {
        tail_ = predecessor;
    }
    delete static_cast<Node*>(removed);
    --size_;
    return value;
}
```

【算法 2.11 结束】

删除不能只摘除链接：被删结点必须释放，且删除尾结点后 `tail_` 必须回退到前

驱。否则下一次 append 会写入已释放内存。位置不合法统一抛 `std::out_of_range`，容器不打印提示。

2.3.4 双链结点

【代码 2.12】双链表的结点定义和实现。

上面的 `DoublyLink<T>` 已保留 `prev` 与 `next` 两个链接域。双链表删除某一结点时，需要同时维护前驱的 `next` 和后继的 `prev`；这能高效向前走，但每个结点多一个指针且不变量更多。原书在此只给出结点定义，没有给出完整双链表算法，因此本轮不假装已经现代化完整双链表。

【代码 2.12 结束】

完整可运行实现见 `code/ch02/linked_list/modern.hpp`，测试覆盖头/中/尾插入、删尾后的尾指针修复、深拷贝、移动、元素构造异常和 `move-only` 元素；变异自检还确认“删尾不回退 `tail`”会在后续尾插崩溃，“复制构造失败不清理”会留下元素对象。原书逐条证据见 `code/ch02/linked_list/legacy.md`。

2.4 线性表实现方法的比较

顺序表按下标随机访问是 $O(1)$ ，插入删除要搬动后续元素，平均 $O(n)$ ；链表在已知前驱时插入删除是 $O(1)$ ，但按位置找前驱仍是 $O(n)$ 。顺序表局部性好、无指针开销；链表按需分配、没有预留空洞。选择取决于「按位置读」多，还是「在已知结点旁改链接」多。

不要用顺序表的场合：经常在表中间插入删除，或事先不知道长度上限。不要用链表的场合：按位置读远比插入删除多，或者指针本身比结点内容还占空间。

本章小结

线性结构里元素满足一对一的先后关系。线性表有顺序和链式两种主要存法。顺序表易用、可随机访问，适合静态或按位读取多的数据；链表适合长度常变、或经常在已知结点旁增删的场合。具体选哪一种，看访问统计和操作特点。

习题

1. 顺序表 a 中元素递增有序。设计算法把 x 插到适当位置，保持有序。
2. 顺序表 $A = (a_1, \dots, a_m)$ 、 $B = (b_1, \dots, b_n)$ 。去掉最长公共前缀后比较剩余子表，写出比较 A 、 B 大小的算法。
3. 递增单链表中删除所有大于 $\min$ 且小于 $\max$ 的元素，释放结点，并分析时间。
4. 两个递增单链表归并成一个递减表，要求占用原结点空间。
5. 含字母、数字和其他字符的单链表，拆成三个循环链表，每表只含一类字符。
6. 删除顺序表中从第 i 个起的 k 个元素。
7. 递增单链表中删掉值相同的多余元素，释放结点。
8. 两个值递增、表内无重复的顺序表 A 、 B ，构造它们的交集 C ，仍递增。
9. 第 8 题在单链表上重做。

10. 表达式已存入字符数组并以 # 结束，判断括号是否配对。

上机题

1. 设计非递归算法，在 $O(n)$ 时间和常数辅助空间内把含 n 个元素的单链表逆置。

第 3 章 栈与队列

栈是「最后放入、最先取出」，适合递归调用、括号匹配和表达式计算；队列是「先放入、先取出」，适合排队服务和广度优先搜索。空栈、空队列是正常状态，现代接口以 `std::optional` 返回。

源码：顺序栈、栈示例、链式栈、表达式求值、背包、队列、队列示例。

先跑一遍

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::ArrayStack<int> stack;
    stack.push(1);
    stack.push(2);
    stack.push(3);
    std::cout << " 栈顶是 " << *stack.top() << '\n';
    std::cout << " 依次弹出:";
    while (auto value = stack.pop()) {
        std::cout << ' ' << *value;
    }
    std::cout << "\n空栈再弹? " << (stack.pop() ? " 有值" : " 空") << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch03/array_stack \
    code/ch03/array_stack/demo.cpp -o /tmp/stack-demo
/tmp/stack-demo
```

```
栈顶是 3
依次弹出: 3 2 1
空栈再弹? 空
```

后进先出：最后压入的 3 最先出来。空栈上 `pop()` 返回空 `optional`，不打印、不崩溃。

循环队列牺牲一个槽位区分空与满，所以逻辑容量 3 实际申请 4 个槽：

```
#include "modern.hpp"

#include <iostream>
```

```
int main() {
    dsa::ArrayQueue<int> queue(3);
    if (!queue.enqueue(1) || !queue.enqueue(2) || !queue.enqueue(3)) {
        std::cout << " 入队失败\n";
        return 1;
    }
    std::cout << " 逻辑容量 3 时再入队? " << (queue.enqueue(4) ? " 成功" : " 已满") <<
        '\n';
    std::cout << " 依次出队:";
    while (auto value = queue.dequeue()) {
        std::cout << ' ' << *value;
    }
    std::cout << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch03/queue \
    code/ch03/queue/demo.cpp -o /tmp/queue-demo
/tmp/queue-demo
```

```
逻辑容量 3 时再入队? 已满
依次出队: 1 2 3
```

3.1 栈

3.1.1 栈的抽象数据类型

栈 (stack) 是限定仅在一端进行插入和删除运算的线性表，该端称为栈顶 (top)，另一端称为栈底 (bottom)。栈的元素按后进先出 (LIFO, last in first out) 的次序访问：最后压入的元素最先弹出。

基于栈的特性，栈的抽象数据类型包含进栈 push、出栈 pop、读栈顶 top 等常用操作，以及判断栈是否为空的边界操作。根据抽象和封装的原则，只可通过抽象数据类型 定义的运算来操作栈。

原书【代码 3.1】用一个成员函数既非纯虚、析构函数也非 virtual 的空基类 Stack<T> 来表达这层抽象。那样的基类给不了任何多态能力：成员函数既不是纯虚的、也没有定义，派生类“实现”它们并不构成覆盖；反而埋下了「通过 Stack<T>* 删除派生对象」这一未定义行为。

抽象数据类型描述的是「一组运算」，不是「一个基类」。在 C++ 里，模板本身已经承担了这层抽象——ArrayStack<T> 提供哪些运算，由它的接口决定，不需要继承任何东西。真正需要写下来的是对元素类型 T 的要求：底层 new T[n] 会把整块槽位都默认构造出来，所以 T 必须能默认构造。这句话写在正文里即可，不必把四条工程契约印进类头。

```

/// 基于数组的栈（后进先出）。
///
/// 与原书 arrStack 的差别：容量耗尽时自动翻倍（原书要么溢出报错，要么靠算法 3.3
/// 手工换成扩容版本）；不打印任何东西；空栈上的 pop/top 返回空 optional，
/// 而不是靠「出参 + bool」双通道返回。
template <typename T>
class ArrayStack {
public:
    using value_type = T;
    using size_type = std::size_t;

```

底层的 `new T[n]` 会把整块槽位都默认构造出来，所以 `T` 必须可默认构造——这是原书 `new T[mSize]` 就有的限制。真正的容器做法是申请未初始化存储再逐个 `placement new`，本书还没有走到那一步。扩容时的异常安全见 3.1.2 节末。

3.1.2 顺序栈

采用顺序存储结构的栈称为顺序栈 (array-based stack)，需要一块连续的区域存储栈中元素。

对元素数目为 n 的栈，首先要确定数组的哪一端表示栈顶。如果把数组的第 0 个位置作为栈顶，所有的插入和删除都在第 0 个位置进行，每次 `push` 或 `pop` 都要把栈中所有元素后移或前移一个位置，时间代价为 $O(n)$ 。反之，把最后一个元素的位置作为栈顶，新元素添加在表尾、出栈也只删除表尾元素，每次操作的时间代价仅为 $O(1)$ 。图 3.2 所示为按后一种方案实现的栈。

图 3.2 顺序栈：下标 0 是栈底，`top_index_` 是元素个数，也是下一个空位。

下标	0	1	2	3	4	...
内容	[A]	[B]	[C]	?	?	...

▲

`top_index_ = 3` (栈顶是 C，下一个 `push` 写到 3)

存储与所有权

原书【代码 3.2】用三个成员表示一个顺序栈：`int mSize`（容量）、`int top`（栈顶位置）、`T * st`（裸指针数组）。这套写法有三处在今天必须改：

1. `int top` 与成员函数 `bool top(T&)` 重名，导致这个类按印刷原样根本编译不过；
2. 无参构造只设了 `top = -1`，`mSize` 与 `st` 都没初始化，析构时 `delete[]` 一个不确定的指针；
3. 有析构函数却没有拷贝构造与拷贝赋值，一次 `arrStack<int> b = a;` 就会二次释放。

第 2、3 条是未定义行为，编译器开到 `-Wall -Wextra -Wpedantic` 也一句警告都不给。

现代实现保留裸指针 `T* data_`——顺序栈的存储管理正是本节要教的内容，把

它换成 `std::unique_ptr<T[]>` 固然更省事，但五法则也就随之退化成走过场（编译器生成的版本就够用了）。留着裸指针，这五个函数才是承重的：少写一个，AddressSanitizer 立刻能复现出崩溃。

```
// 三/五法则：原书只写了析构函数，没有拷贝构造与拷贝赋值，
// 于是 `arrStack<int> b = a;` 之后两个对象持有同一根指针，各析构一次 → 二次释放。
//
// 这里用的是 ** 裸指针 **，所以这五个函数是承重的，不是仪式——
// 少写一个，ASan 立刻能复现出崩溃（见 legacy.md 缺陷 4 的实测输出）。
ArrayStack(const ArrayStack& other)
    : capacity_(other.capacity_),
      top_index_(other.top_index_),
      data_(other.capacity_ ? new T[other.capacity_] : nullptr) {
    for (size_type i = 0; i < top_index_; ++i) {
        data_[i] = other.data_[i];
    }
}

/// 拷贝并交换：先构造完整副本再与自身交换，天然自赋值安全，
/// 且拷贝失败时原对象不受影响。
ArrayStack& operator=(const ArrayStack& other) {
    if (this != &other) {
        ArrayStack copy(other);
        swap(copy);
    }
    return *this;
}

ArrayStack(ArrayStack&& other) noexcept { swap(other); }

ArrayStack& operator=(ArrayStack&& other) noexcept {
    if (this != &other) {
        ArrayStack moved(std::move(other)); // other 交出所有权
        swap(moved);                        // 自己原来的缓冲区随 moved 析构释放
    }
    return *this;
}

~ArrayStack() { delete[] data_; }
```

拷贝赋值用的是拷贝并交换 (copy-and-swap)：先构造一个完整副本，再与自身交换。它自然地做到了自赋值安全，也在拷贝失败时保证原对象不受影响。

出栈：用 **optional** 取代双通道返回

原书的 `bool pop(T & item)` 用出参带值、用返回值带成败。调用方一旦忘了检查返回值，读到的就是没被修改过的 `item`。现代写法让「有没有值」这件事进入类型系

统：空栈不是错误，而是一种可预期的状态，用 `std::optional` 表达它；真正的错误（下标越界、容量溢出）才抛异常。

```
/// 出栈。空栈返回 std::nullopt——原书是「返回 false + 往 cout 打一行中文」，
/// 调用方既没法在库里复用，也容易忽略返回值。
std::optional<T> pop() {
    if (empty()) {
        return std::nullopt;
    }
    --top_index_;
    return std::optional<T>(std::move(data_[top_index_]));
}

/// 读栈顶但不弹出，返回 ** 副本 **。空栈返回 std::nullopt，不是未定义行为。
/// 要求 T 可拷贝；move-only 元素请用 peek()。
std::optional<T> top() const {
    if (empty()) {
        return std::nullopt;
    }
    return std::optional<T>(data_[top_index_ - 1]);
}

/// 只读观望栈顶：不拷贝、不弹出。空栈返回 nullptr。
///
/// 与 top() 的分工——top() 给你一份可以带走的副本（安全，但要求 T 可拷贝，
/// 而且确实拷贝了一次）；peek() 零拷贝，move-only 元素也能用，代价是
/// ** 返回的指针在下次 push / pop / clear 之后即失效 **（扩容会换掉整块缓冲区）。
/// 生命周期由调用方负责，这一点必须写在文档里，不能靠使用者猜。
const T* peek() const noexcept {
    return empty() ? nullptr : &data_[top_index_ - 1];
}
```

返回值是 `optional`，空栈是可预期状态，不是错误。同样重要的是这里没有 `std::cout`：原书在空栈出栈时直接打印中文提示，把一个数据结构与标准输出焊死——它没法在库里复用，提示无法本地化，失败路径也无从测试。

「读栈顶」有两种正当需求，因此这里给了两个接口，不要把它们合并成一个：

- `top()` 返回 `std::optional<T>`，是一份可以带走的副本。安全，但要求 `T` 可拷贝，而且确实拷贝了一次。
- `peek()` 返回 `const T*`，空栈为 `nullptr`，零拷贝。`std::unique_ptr` 这类只能移动的元素只能用它来观望。

代价也是一对：`optional` 的安全靠拷贝换来，`peek()` 的零拷贝则把生命周期交给了调用方——它返回的指针在下次 `push` / `pop` / `clear` 之后即失效，因为扩容会换掉整块缓冲区。这条失效契约必须写在文档里，不能靠使用者猜。

两种代价都真实存在。把任何一种藏起来，都是替读者做了本该由读者做的选

择。

顺序栈的扩容

栈中元素动态变化,当栈满时继续进栈会产生上溢出(overflow)。原书【算法 3.3】给出的改进办法是:申请一个扩大一倍的新数组,把原有内容顺序移动过去,再执行进栈。这个策略本身是对的——每个元素在均摊意义下只被搬运常数次数, push 的摊还时间代价 仍是 $O(1)$ ——但那段代码有两个问题: 循环变量 `i` 从未声明(编译不过), 以及先 `delete[] st` 再赋新指针, 搬运中途若抛出异常, 栈就停在半新半旧的状态。

```
static constexpr size_type kInitialCapacity = 4;

void ensure_capacity() {
    if (top_index_ < capacity_) {
        return;
    }
    constexpr size_type kMax = std::numeric_limits<size_type>::max();
    if (capacity_ > kMax / 2) {
        throw std::overflow_error("ArrayStack: 容量翻倍会溢出");
    }
    const size_type next = capacity_ == 0 ? kInitialCapacity : capacity_ * 2;
    T* fresh = new T[next];
    try {
        for (size_type i = 0; i < top_index_; ++i) {
            // 这里是 ** 赋值 ** 而不是构造, 所以不能用 std::move_if_noexcept:
            // 它检查的是移动 ** 构造 ** 是否 noexcept, 而可抛的移动赋值会在搬迁
            // 中途把原栈的元素掏空——红队 T-002 实测复现过 (见 legacy.md 缺陷 11)。
            //
            // 判据必须落在「移动赋值抛不抛」这一个维度上:
            // 移动赋值 noexcept → 移动。不可能抛, 强异常保证不受影响, 也不白白深拷贝。
            // 否则 → 复制。拷贝赋值取 const&, 抛了也动不了原栈;
            // 上面的 static_assert 保证走到这里的 T 一定可复制赋值。
            if constexpr (std::is_nothrow_move_assignable<T::value>) {
                fresh[i] = std::move(data_[i]);
            } else {
                fresh[i] = data_[i];
            }
        }
    } catch (...) {
        // 裸指针的代价: RAII 版本不用写这一段。搬迁失败要自己收拾新缓冲区,
        // 且此时还没动 data_/capacity_, 所以原栈完好——这就是强异常保证。
        delete[] fresh;
        throw;
    }
    delete[] data_;
    data_ = fresh;
}
```

```
capacity_ = next;  
}
```

四处值得注意：

- 先建后换。新缓冲区搬完才动 `data_`，中途抛异常则原栈原封不动，这就是强异常保证。原书先 `delete[] st` 再赋新指针，一旦搬运途中抛出，栈就停在一个既不是旧状态也不是新状态的地方。
- **try/catch** 是裸指针的代价。搬迁失败要自己把新缓冲区收掉再重新抛出；如果底层换成 `std::unique_ptr<T[]>`，这一段可以整个删掉。取舍在这里是显式的：本节要教存储管理，就得连这份代价一起看见。
- 搬迁用移动还是拷贝，判据必须选对。这一条值得单独讲，见下一小节。
- 容量用 `std::size_t`。原书用 `int`，`mSize * 2` 溢出是未定义行为；这里在翻倍前先判界并抛 `std::overflow_error`。

这不是纸面推理。测试里有一个「第 3 次拷贝赋值必定抛异常」的元素类型，用它撑满栈再触发扩容，然后逐项断言：长度不变、容量不变、原有元素逐个完好、栈之后仍能继续使用；另一个类型让 `new T[]` 本身抛 `std::bad_alloc`，同样逐项断言。把「先建后换」改回原书的「先释放旧的」，Debug 构建下 UBSan 当场报空指针引用，Release 构建直接段错误。

一个容易踩空的地方：`move_if_noexcept` 在这里是错的

搬迁元素时，一个几乎条件反射的写法是 `std::move_if_noexcept(data_[i])`——「移动可能抛就退回拷贝」，标准库容器扩容正是这么做的。但在这段代码里它是错的。

`std::move_if_noexcept` 的判据是 `T` 的移动构造是否 `noexcept`。而这里 `fresh` 已经由 `new T[next]` 默认构造好了，搬迁执行的是移动赋值。两者是不同的函数，可以有不同的异常规格：一个类完全可以移动构造 `noexcept`、移动赋值却会抛。这时 `move_if_noexcept` 会放行移动，而抛出发生在搬到一半时——原栈里已经被移动走的那些元素，已经被掏空了，强异常保证就此破裂。

这个洞不是推演出来的：本书的测试里有一个正是这种形状的类型（移动构造 `noexcept`，第 3 次移动赋值抛异常），它让原来的实现真的失败了。

判据要落在实际执行的那个操作上：

- 移动赋值 `noexcept` → 移动。不可能抛，强异常保证不受影响，也不必白白深拷贝。
- 否则 → 拷贝。拷贝赋值取 `const&`，抛了也动不了原栈。

第二条要求 `T` 可拷贝赋值，所以类头处那条 `static_assert` 写明：不可拷贝的 `T` 必须满足 `is_nothrow_move_assignable`。这是本容器对元素类型的一条真实约束，写成编译期断言，好过让它变成运行期某次扩容失败后的谜案。

反过来也要小心：判据若写成「可拷贝就拷贝」，`std::string`（移动赋值本就是 `noexcept`）每次扩容都会退化成深拷贝，算法 3.3 摊还 $O(1)$ 的分析就打了折扣。测试里因此有一条用例专门数拷贝次数——这两种写法都能通过其他所有断言，只有这条能把

它们分开。

进栈本身随之简化为「保证容量、写入、长度加一」：

```
/// 入栈。容量不足时按算法 3.3 的策略翻倍。
/// 强异常保证：搬迁在新缓冲区上完成，中途抛异常则原栈原封不动。
void push(const T& item) {
    ensure_capacity();
    data_[top_index_] = item;
    ++top_index_;
}

void push(T&& item) {
    ensure_capacity();
    data_[top_index_] = std::move(item);
    ++top_index_;
}
```

两个重载分别处理左值和右值。原书的 `bool push(const T item)` 按值传参，顶层 `const` 对调用方没有意义，却强制了一次拷贝，而且 `std::unique_ptr` 这类只能移动的类型根本传不进去。

3.1.3 链式栈

采用链式存储结构的栈称为链式栈。结点分散在堆上，压栈只是接一个新结点，不需要连续空间，也没有“栈满”这回事——这正是它与顺序栈的核心差别。

```
/// 链式栈：结点分散在堆上，压栈只是接一个新结点，不需要连续空间、也不需要扩容。
///
/// 与原书 lnkStack 的差别：`top` 这个名字只留给成员函数（原书 `Link<T>* top`
/// 与 `bool top(T&)` 重名，导致整个类编译不过）；补齐五法则；不做任何 I/O；
/// 出栈返回 `std::optional<T>`。
template <typename T>
class LinkedStack {
    struct Node {
        T value;
        Node* next;

        template <typename U>
        Node(U&& item, Node* successor) : value(std::forward<U>(item)),
        ↪ next(successor) {}
    };

public:
    using value_type = T;
    using size_type = std::size_t;
```

原书【代码 3.4】的 `lnkStack` 有一处与顺序栈完全相同的错误：成员 `Link<T>* top`

与成员函数 `bool top(T&)` 重名，整个类编译不过。同一本书在两个存储结构上犯了同一个命名错误，两处都没有被编译器验证过。

另有两处值得指出：

- 构造函数是 `lnkStack(int defSize)`，而那个参数从未被使用。链式栈不需要预设容量，这个参数是从 `arrStack(int size)` 照抄来的；更麻烦的是它没有默认构造函数，使用者被迫为一个无意义的参数编个数出来。
- 有 `~lnkStack(){ clear(); }` 却没有拷贝构造与拷贝赋值。这是本书第五次遇到同一个错误（顺序栈、顺序表、链表、字符串、链式栈）。

```
// 原书有 `~lnkStack(){ clear(); }` 却没有拷贝构造与拷贝赋值：
// 一次 `lnkStack<int> b = a;` 之后两个栈共享同一串结点，各自析构一次 → 二次释放。
// 与顺序栈、顺序表、链表、字符串是同一个错误，本书第五次遇到它。
lnkStack(const lnkStack& other) {
    // 先按原序收集，再逆序压回，避免递归拷贝（深链会爆栈，见 UNVERIFIED-RISKS.md）
    Node* source = other.top_;
    Node** tail = &top_;
    try {
        while (source != nullptr) {
            *tail = new Node(source->value, nullptr);
            tail = &(*tail)->next;
            ++size_;
            source = source->next;
        }
    } catch (...) {
        clear(); // 半截链必须自己收拾：构造函数抛出时析构函数不会运行
        throw;
    }
}

lnkStack& operator=(const lnkStack& other) {
    if (this != &other) {
        lnkStack copy(other);
        swap(copy);
    }
    return *this;
}

lnkStack(lnkStack&& other) noexcept
    : top_(std::exchange(other.top_, nullptr)), size_(std::exchange(other.size_,
    ↪ 0)) {}

lnkStack& operator=(lnkStack&& other) noexcept {
    if (this != &other) {
        clear();
        top_ = std::exchange(other.top_, nullptr);
        size_ = std::exchange(other.size_, 0);
    }
}
```

```

    }
    return *this;
}

~LinkedList() { clear(); }

```

压栈与出栈：

```

/// 入栈：接一个新结点。** 没有" 栈满" 这回事 **——这正是链式栈相对顺序栈的差别，
/// 原书顺序栈那边要判 `top == mSize - 1` 并打印" 栈满溢出"。
void push(const T& item) { top_ = new Node(item, top_); ++size_; }
void push(T&& item) { top_ = new Node(std::move(item), top_); ++size_; }

/// 出栈。空栈返回 std::nullopt (D-001 §3c：空是可预期状态，不抛异常)。
[[nodiscard]] std::optional<T> pop() {
    if (top_ == nullptr) {
        return std::nullopt;
    }
    Node* dying = top_;
    std::optional<T> result(std::move(dying->value));
    top_ = dying->next;
    delete dying;
    --size_;
    return result;
}

/// 取栈顶副本；零拷贝的观望用 peek() (D-001 §3b)。
[[nodiscard]] std::optional<T> top() const {
    return top_ == nullptr ? std::nullopt : std::optional<T>(top_->value);
}

[[nodiscard]] const T* peek() const noexcept {
    return top_ == nullptr ? nullptr : &top_->value;
}

```

一处实现上的取舍：clear() 与析构都写成迭代而非递归。链式结构最自然的写法是递归释放，但深链会耗尽调用栈——第 5 章有实测数字。本书的测试用 20 万个结点压栈再全部弹出，正是为这条兜底。

3.1.4 表达式求值

后缀（逆波兰）表达式不需要括号，也不需要优先级规则：求值时遇操作数压栈，遇操作符弹出两个算完再压回，读到末尾时栈里剩下的唯一元素就是结果。这条主线本书一字未改，用的还是本章自己的 ArrayStack<double>。

```

/// 对后缀（逆波兰）表达式求值。记号之间用空白分隔，例如 "3 4 + 2 *" → 14。
///
/// 与原书 `class Calculator` 的差别，逐条对应它的三个问题：
///
/// 1. ** 原书 `Run()` 直接从 `cin` 读、往 `cout` 写 **，算法与终端焊死：没法写测试、
/// 没法在库里复用、没法处理来自别处的表达式。这里接受一个 `string_view`、返回结果。
/// 2. ** 原书 `GetTwoOperands` 在第二个操作数缺失时已经把第一个弹掉了 **，
/// 返回 `false` 时栈已被破坏（legacy.md 缺陷 1）。
/// 但要说准确：真正让这个 bug 无害的，** 不是 ** 下面那句“先查够不够”，
/// 而是 ** 出错即抛出、整次求值随即作废 **——栈根本没有机会被下一步用到。
/// 预检查只是让错误信息更早、更准，属于锦上添花。
/// 原书的 bug 之所以要命，是因为它 `cerr` 一行之后 ** 继续跑 **。
/// 3. ** 原书出错时 `cerr` 打一行然后 `s.clear()` 继续跑 **，调用方拿不到任何信号。
/// 这里抛 `std::invalid_argument`（表达式不合法）或 `std::domain_error`（除零）。
[[nodiscard]] inline double evaluate_postfix(std::string_view expression) {
    ArrayStack<double> operands;
    std::size_t i = 0;

    const auto pop_two = [&operands] {
        // 先确认有两个再弹。注意这不是“修复”原书 bug 的关键（见函数注释第 2 条），
        // 而是让报错更早、更准。
        if (operands.size() < 2) {
            throw std::invalid_argument(" 后缀表达式：操作数不足");
        }
        right = *operands.pop(); // 先弹出的是右操作数
        left = *operands.pop();
    };

    while (i < expression.size()) {
        const char c = expression[i];
        if (c == ' ' || c == '\t' || c == '\n') {
            ++i;
            continue;
        }

        // 操作符：+ - * / 各弹两个。注意 '-' 也可能是负号，靠后面是不是数字来区分。
        const bool is_sign = (c == '-' || c == '+') && i + 1 < expression.size()
            && (std::isdigit(static_cast<unsigned char>(expression[i
                ↪ + 1]))
                || expression[i + 1] == '.');
        if (!is_sign && (c == '+' || c == '-' || c == '*' || c == '/')) {
            double left = 0.0;
            double right = 0.0;
            pop_two(left, right);
            switch (c) {
                case '+': operands.push(left + right); break;

```

```

        case '-': operands.push(left - right); break;
        case '*': operands.push(left * right); break;
        default:
            // 原书写 `if (operand1 == 0.0)`, 正文又说这样比较浮点数不对、
            // 该用阈值。两者都值得推敲：除以 1e-300 是 ** 合法 ** 的,
            // 结果是个很大的数或 inf, 用阈值把它当成错误是另一个决定。
            // 这里只拦精确的 0.0 (含 -0.0), 并把这个取舍写在明面上。
            if (right == 0.0) {
                throw std::domain_error(" 后缀表达式: 除数为零");
            }
            operands.push(left / right);
            break;
    }
    ++i;
    continue;
}

// 操作数: 交给标准库解析, 顺便拿到它吃掉了多少字符
std::size_t consumed = 0;
double value = 0.0;
try {
    value = std::stod(std::string(expression.substr(i)), &consumed);
} catch (const std::exception&) {
    throw std::invalid_argument(std::string(" 后缀表达式: 无法识别的记号 '" +
        ↵ c + "'"));
}
operands.push(value);
i += consumed;
}

if (operands.size() != 1) {
    // 空表达式、操作数多余、操作符不足, 都落在这里
    throw std::invalid_argument(" 后缀表达式: 不是一个完整的表达式");
}
return *operands.pop();
}

```

原书【算法 3.5】把它写成一个 class Calculator, 有三处今天必须改。

一、算法与终端焊死。Run() 直接 cin >> c 读、cout << res 写, 出错时 cerr 打一行。后果是这个算法没法写测试、没法在库里复用、也没法处理来自别处的表达式。本书改为接受一个字符串、返回结果、出错抛异常。

二、GetTwoOperands 在第二个操作数缺失时, 第一个已经被弹掉了。栈就此少一个元素。但要说准确——真正让这件事变成 **bug** 的是调用方继续跑: Compute 拿到 false 之后清空栈然后接着读下一个记号, Run() 的循环照转不误。

这一点改变了” 怎样才算修好”: 本书的实现出错即抛出、整次求值作废, 栈根本没有

机会被下一步用到。（写这一节时验证过：把实现改回”先弹一个再查”，全部断言依然通过——因为在抛异常即作废的设计里，栈被破坏与否观测不到。于是补了一条测试，钉住”上一次失败不给下一次留残留”这条真正可测的性质。）

三、除零判断。原书正文自己写道：

在实际编写程序时，浮点数是否为 0 不能直接与 0 进行相等比较，而是要通过某个很小的阈值来判断，例如采用 `if(abs(operand1) < 1E-7)`

但印出来的代码仍然是 `if (operand1 == 0.0)`。书知道更好的写法，却没有印它。

不过原书这条建议本身也值得推敲：除以 **1e-300** 是合法的，结果是一个极大的有限值或无穷，把它当作错误是另一个决定，不是”更正确”。本书只拦精确的零，并把这个取舍写在代码注释里；测试中有一条断言专门钉住“除以 1e-300 得到极大值而非报错”。

3.1.5 栈与递归

本节开篇先摆一组实测数字。原书正文说递归「需要在内存中开辟一个称为 运行栈 (runtime stack) 的足够大的动态区」。多大算足够大？这是可以量的，而量出来的结果里有一条相当反直觉。

运行栈有多大：三个档位，三种结果

同一份递归源码（每层做一次加法后返回），在一台 Linux 机器上（gcc 13.3, ulimit -s 为默认的 8 MB）逐档加深度：

构建档	20 万层	50 万层	100 万层
-O0（不优化）	通过	段错误	段错误
-O1 + ASan/UBSan	通过	stack-overflow	stack-overflow
-O2	通过	通过	通过

把同样的计算改成显式栈（数据压进本章的 ArrayStack，也就是放到堆上）：三个档位在 **1000** 万层都通过。

反直觉的那一条：**-O2** 为什么不崩

不是因为它栈更大，而是因为编译器把递归消掉了。查汇编可以确认：

```
-O0: recursive_sum 函数体内调用自己的次数 = 2
-O2: recursive_sum 函数体内调用自己的次数 = 0
```

-O2 下这个函数根本没有自调用——它被转成了循环。所以：

「这段递归会不会爆栈」不是源码单独决定的，是源码 × 编译器 × 优化档共同决定的。

这件事对学习者的实际含义是：在 -O2 下跑通的递归程序，换成 -O0 调试、或者换个编译器、或者递归形状稍微复杂到无法被转换，就可能在同样的输入上崩掉。

而崩掉时你看到什么，也取决于构建方式——开了 sanitizer 的构建会明确告诉你 `stack-overflow` 并给出递归回溯，`-O0` 的构建只有一个段错误，一行解释都没有。

递归吃运行栈，显式栈吃堆

这正是本节的主旨。原书用阶乘作例子，给了三个版本：

【算法 3.6】 递归——每一层的返回地址与局部变量都压在运行栈上，深度由进程栈上限决定：

```
/// 【算法 3.6】 递归实现。保留原书的递归形状——那正是本节要教的东西。
///
/// 加了原书没有的两道检查：负数是定义域错误，溢出是真错误 (D-001 §3)。
/// 原书 `if (n <= 0) return 1;` 把负数静默当成 0 处理，返回 1。
[[nodiscard]] inline factorial_type factorial_recursive(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围 (20! 是上限)");
    }
    if (n <= 1) {
        return 1; // 递归出口
    }
    return static_cast<factorial_type>(n) * factorial_recursive(n - 1);
}
```

【算法 3.8】 迭代——不用栈，也不占深度：

```
/// 【算法 3.8】 迭代实现。不用栈，也不占运行栈深度。
[[nodiscard]] inline factorial_type factorial_iterative(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围 (20! 是上限)");
    }
    factorial_type m = 1;
    for (long long i = 2; i <= n; ++i) {
        m *= static_cast<factorial_type>(i);
    }
    return m;
}
```

【算法 3.9】 显式栈——把待处理的数据压进一个自己管理的栈，用原书的话说是「模拟编译系统处理递归的机制，使用栈等数据结构保存回溯点」：

```

/// 【算法 3.9】用显式栈模拟递归。
///
/// 这一版存在的意义不是“更快”——它比迭代版慢——而是 ** 演示编译系统处理递归的机制 **：
/// 遇到递归规则就压栈，遇到递归出口就出栈返回。原书的话是
/// 「模拟编译系统处理递归的机制，使用栈等数据结构保存回溯点」。
///
/// 关键差别在 ** 数据放在哪 **：递归版把每层的返回地址与局部变量放在 ** 运行栈 ** 上，
/// 大小由进程栈上限决定；这一版把待处理的数据压进 ArrayStack，** 在堆上 **，
/// 只受内存限制。实测数字见书稿 3.1.5 节。
///
/// 原书写的是 while (s.pop(&tmp))——传的是 ** 指针 **，而同书代码 3.1 的栈 ADT
/// 声明的是 bool pop(T& item)（引用）。两处对不上（legacy.md 缺陷 3）。
[[nodiscard]] inline factorial_type factorial_with_explicit_stack(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围 (20! 是上限)");
    }
    ArrayStack<factorial_type> pending;
    for (long long i = n; i > 1; --i) { // 按递归规则压栈
        pending.push(static_cast<factorial_type>(i));
    }
    factorial_type m = 1; // 递归出口的返回值
    while (auto top = pending.pop()) { // 出栈即“递归返回”
        m *= *top;
    }
    return m;
}

```

三者的差别不在快慢（显式栈版最慢），而在数据放在哪：前者在运行栈上，受进程栈上限约束；后者在堆上，只受内存约束。上面那张表量的就是这个差别。

原书这三个版本共同的问题：不查溢出

三个版本都是 `long factorial(long n)`，既不检查负数也不检查溢出。64 位 `long` 装得下的最大阶乘是 $20!$ ，从 21 开始：

```

factorial(20) = 2432902008176640000
factorial(21) = -4249290049419214848    ← 负数
factorial(66) = 0                        ← 零
factorial(-5) = 1                       ← 负数静默当成 0 处理

```

而且这不只是“答案错”——有符号整数溢出在 C++ 里是未定义行为：

```
runtime error: signed integer overflow: 21 * 2432902008176640000
cannot be represented in type 'long int'
```

本书改用 `std::uint64_t` 并在入口显式判界：超过 20 抛 `std::overflow_error`，负数抛 `std::invalid_argument`。三个版本在边界上的行为完全一致——测试要求它们对同一输入给出相同答案，包括同样地抛出异常。

（另有一处书内不一致：算法 3.9 写 `s.pop(&tmp)` 传指针，而同章代码 3.1 的栈 ADT 声明的是 `bool pop(T& item)` 传引用，两处配不上。详见 `code/ch03/recursion_and_stack/legacy.md`。）

一个更复杂的例子：背包问题

阶乘只有一条递归规则，改写成循环几乎是显然的。原书接着给了一个有两条递归规则的例子——背包问题（更准确地说是子集和判定：能否从若干物品中选出一部分，使重量之和恰好等于背包承重）。

```
/// 【算法 3.10】递归解法。两条递归规则、两个递归出口，原书的结构一字未改：
/// 出口 1：承重恰为 0 → 有解（什么都不再选）
/// 出口 2：承重为负，或承重为正但已无物品可选 → 无解
/// 规则 1：选第  $n-1$  件 → 求解  $\text{knap}(s - w[n-1], n-1)$ 
/// 规则 2：不选第  $n-1$  件 → 求解  $\text{knap}(s, n-1)$ 
///
/// 与原书的差别只有两处：
/// 1. ** 原书直接 `cout << w[n-1]` 把选中的物品打印出来——算法与终端焊死，
///    调用方拿不到结果，也没法测试。这里把下标收集进 `chosen` 返回。
/// 2. ** 原书的 `w[]` 是一个从未声明的全局数组 **（legacy.md 缺陷 3）。这里作参数传入。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_recursive(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);
    knapsack_solution chosen;

    // 返回 true 表示  $\text{weights}[0..n)$  中存在一个子集，其和恰为  $s$ 
    const auto solve = [&weights, &chosen -> bool {
        if (s == 0) {
            return true; // 递归出口 1
        }
        if (s < 0 || n == 0) {
            return false; // 递归出口 2
        }
        if (self(self, s - weights[n - 1], n - 1)) { // 规则 1：选它
            chosen.push_back(n - 1);
            return true;
        }
        return self(self, s, n - 1); // 规则 2：不选它
    };
};
```



```

return solve(solve, capacity, weights.size())
    ? std::optional<knapsack_solution>(std::move(chosen))
    : std::nullopt;
}

```

两个递归出口、两条递归规则，本书一字未改。改的是两处：原书 `w[]` 是一个未声明的全局数组，这里作参数传入；原书在选中物品时 `cout << w[n-1]` 直接打印，调用方拿不到解、正确性也无从检验，这里把下标收集起来返回——测试因此可以独立验算：把返回的下标对应的重量加起来，必须恰好等于承重。

把它机械地改写成循环

原书的做法是引入一个显式栈，每帧保存四个域：参数 `s` 与 `n`、返回地址 `rd`、结果单元 `k`。返回地址是关键——它记的是“这一层算完之后该回到哪一步继续”，正是编译器为你做的那件事。原书用 `goto label0/1/2/3` 表达它。

本书保留这套机制，但把“返回地址”写成栈帧里的一个 `stage` 字段：

```

/// 【算法 3.11】把上面的递归机械地改写成显式栈驱动。
///
/// 原书用 `goto label0/1/2/3` 表示“执行到哪一步”，本书把同一件事写成栈帧里的
/// 一个 `stage` 字段——** 语义完全对应 **，只是不用 goto: goto 跳进跳出会让编译器
/// 无法保证局部对象的构造与析构配对，在有 RAII 的 C++ 里不能这么写。
///
/// `Enter`      ↔ label0，递归调用入口：判出口，否则按规则 1 展开
/// `AfterRule1` ↔ label1，规则 1（选第 n-1 件）返回后的处理
/// `AfterRule2` ↔ label2，规则 2（不选第 n-1 件）返回后的处理
///
/// 每一帧存原书说的四个域：参数 s 与 n、返回地址（这里是 stage）、结果单元 k。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_with_explicit_stack(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);

    enum class Stage { Enter, AfterRule1, AfterRule2 };
    struct Frame {
        int s = 0;
        std::size_t n = 0;
        Stage stage = Stage::Enter;
    };

    ArrayStack<Frame> stack;
    stack.push(Frame{capacity, weights.size(), Stage::Enter});
    knapsack_solution chosen;
    bool child_result = false;    // 下层刚刚返回的结果单元 k

```

```

while (!stack.empty()) {
    Frame frame = *stack.pop();

    if (frame.stage == Stage::Enter) {
        if (frame.s == 0) {           // 递归出口 1
            child_result = true;
            continue;                // 相当于 goto label3: 直接向上返回
        }
        if (frame.s < 0 || frame.n == 0) { // 递归出口 2
            child_result = false;
            continue;
        }
        frame.stage = Stage::AfterRule1;    // 记下" 回来时该走哪一步"
        stack.push(frame);
        stack.push(Frame{frame.s - weights[frame.n - 1], frame.n - 1,
↪ Stage::Enter});
        continue;
    }

    if (frame.stage == Stage::AfterRule1) {
        if (child_result) {           // 规则 1 成功: 第 n-1 件被选中
            chosen.push_back(frame.n - 1);
            continue;                // k 已是 true, 继续上传
        }
        frame.stage = Stage::AfterRule2;    // 回溯, 改用规则 2
        stack.push(frame);
        stack.push(Frame{frame.s, frame.n - 1, Stage::Enter});
        continue;
    }

    // Stage::AfterRule2: 规则 2 的结果就是本层的结果, 原样上传
}

return child_result ? std::optional<knapsack_solution>(std::move(chosen)) :
↪ std::nullopt;
}

```

Enter / AfterRule1 / AfterRule2 与原书的 label0 / label1 / label2 一一对应。不用 **goto** 的理由是硬的: goto 跳进跳出会让编译器无法保证局部对象的构造与析构配对, 在有 RAII 的 C++ 里不能这么写。

优化: 让每层要记的东西更少

原书接着指出两点可以省:

1. 结果单元 **k** 可以提到栈外——一旦某层为真就逐层上传且不再变化, 一个函数级变量就够了;

2. 参数 n 可以由栈深推出——每递归一层 n 减 1、栈深加 1，所以 $n = n_0 - \text{栈深}$ 。
于是栈帧从四个域缩到两个：

```
/// 【算法 3.12】原书" 优化版" 的两点观察，本书照单实现：
///
/// 1. ** 结果单元  $k$  可以提到栈外 **——一旦某层为 true 就逐层上传且不再变化，
///    因此一个函数级变量即可，栈帧里的 ` $k$ ` 域连同它的反复赋值、进出栈都省掉。
///    （上面那版其实已经这么做了：`child_result` 就在栈外。）
/// 2. ** 参数  $n$  可以由栈深推出 **——每递归一层  $n$  减 1、栈深加 1，
///    所以 ` $n = n_0 - \text{栈深}$ `，栈帧里的 ` $n$ ` 域也能省掉。
///
/// 于是栈帧从四个域缩到两个 ( $s$  与 stage)。这是本节真正的" 优化"：
/// 不是让它更快，而是 ** 让每层要记的东西更少 **——这正是手工模拟递归的意义。
///
/// 原书这一版另有一处致命问题：它同时把 `stack.top` 当 ** 数据成员 ** 用
/// (`t = stack.top;`) 又当 ** 成员函数 ** 用 (`stack.top(&tmp);`)。
/// 这在任何一种解释下都编译不过，而且它恰恰依赖代码 3.2/3.4 那个 `top` 重名缺陷。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_optimized(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);

    enum class Stage { Enter, AfterRule1, AfterRule2 };
    struct Frame {
        int s = 0; // 只剩两个域
        Stage stage = Stage::Enter;
    };

    const std::size_t n0 = weights.size();
    ArrayStack<Frame> stack;
    knapsack_solution chosen;
    bool child_result = false;

    stack.push(Frame{capacity, Stage::Enter});
    std::size_t depth = 1; // 栈中帧数；当前帧的  $n = n_0 - (\text{depth} - 1)$ 

    while (!stack.empty()) {
        Frame frame = *stack.pop();
        --depth;
        const std::size_t n = n0 - depth; // 观察 2:  $n$  由栈深推出，不再入栈

        if (frame.stage == Stage::Enter) {
            if (frame.s == 0) { child_result = true; continue; }
            if (frame.s < 0 || n == 0) { child_result = false; continue; }
            frame.stage = Stage::AfterRule1;
            stack.push(frame);
            stack.push(Frame{frame.s - weights[n - 1], Stage::Enter});
            depth += 2;
        }
    }
}
```

```

        continue;
    }

    if (frame.stage == Stage::AfterRule1) {
        if (child_result) { chosen.push_back(n - 1); continue; }
        frame.stage = Stage::AfterRule2;
        stack.push(frame);
        stack.push(Frame{frame.s, Stage::Enter});
        depth += 2;
        continue;
    }
    // AfterRule2: 结果原样上传
}

return child_result ? std::optional<knapsack_solution>(std::move(chosen)) :
    ↪ std::nullopt;
}

```

这才是本节真正的”优化”：不是让它跑得更快，而是让每层要记的东西更少。手工模拟递归的全部意义就在这里——你必须想清楚”每一层到底需要记住什么”，而这正是编译器替你想过的那个问题。

一句实话：写这个单元时，第一版显式栈实现我把”返回地址”塞进了位标志里，结果死循环、撞上构建超时；改写成与原书 label 一一对应的状态机之后才一次通过。手工模拟递归确实容易写错——而原书那份用 goto 的版本从未被编译器验证过。算法 3.12 里 stack.top 同时被当成数据成员 (t = stack.top;) 和成员函数 (stack.top(&tmp;))，任何一种解释下都编译不过。

与原书的对照

原书	现在	为什么
class Stack<T> 空基类	三条 static_assert + <type_traits>	空基类给不了多态，还带来非虚析构的未定义行为；对 T 的要求应当是编译期约束
int mSize / int top / T* st	size_t capacity_ / size_t top_index_ / T* data_	int 溢出是未定义行为；top_index_ 避开与成员函数 top() 重名
只有 ~arrStack()	五个特殊成员函数补齐	否则一次拷贝就二次释放
bool pop(T& item)	[[nodiscard]] std::optional<T> pop()	「有没有值」交给类型系统，忽略返回值会告警

原书	现在	为什么
<code>bool top(T& item)</code>	<code>optional<T> top()</code> 取副本; <code>const T* peek()</code> 零拷贝	两种正当需求, 两种代价, 都摆在明面上
<code>cout << " 栈满溢出"</code>	不做任何 I/O; 越界抛 <code>std::out_of_range</code>	数据结构不该和标准输出耦合; 可预期的空状态用 <code>optional</code> , 真错误才抛异常
<code>delete[] st; st = newSt;</code>	先建后换 + <code>try/catch</code> , 按移动赋值是否 <code>noexcept</code> 决定移动还是拷贝	搬迁中途抛异常不破坏原栈 (强异常保证); 判据落在实际执行的操作上, 不是 <code>move_if_noexcept</code> 看的移动构造

刻意没改的: 它仍然是一个手写的、自己管缓冲区的数组栈, 缓冲区仍是裸的 `T*` `data_`, 扩容仍是算法 3.3 的翻倍策略, `clear()` 仍然只把长度归零、保留已分配容量。把它换成 `std::stack` 的薄封装固然更短, 但这一节要教的正是这些实现细节本身。

完整实现见 `code/ch03/array_stack/modern.hpp`, 测试见同目录 `test.cpp` (58 项断言, 覆盖上表每一行; 用 `python3 tools/check_code.py` 在 `-Werror + ASan/UBSan` 与 `-O2` 两种构建下各跑一遍)。

3.2 队列

队列只允许在一端插入、在另一端删除, 元素按先进先出 (FIFO) 访问。插入端是队尾, 删除端是队头。空队列上的提取是可预期状态, 返回 `std::nullopt`。

3.2.1 队列的抽象数据类型

常用运算是入队 `enqueue`、出队 `dequeue`、读队头, 以及判空。顺序实现还需要判满。原书【代码 3.13】同样把这些运算写成一个假抽象基类; 本书直接定义在 `ArrayQueue` 与 `LinkedQueue` 上。

3.2.2 顺序队列

循环队列牺牲一个槽位区分空与满: 逻辑容量为 `n` 时实际申请 `n+1` 个槽。`front_ == rear_` 为空, `(rear_ + 1) % slots_ == front_` 为满。这正是章首 demo 里「容量 3 时第 4 个入不进去」的原因。

```
template <typename T>
class ArrayQueue {
public:
    explicit ArrayQueue(std::size_t capacity) : slots_(capacity + 1), data_(slots_ ?
        ↪ new T[slots_] : nullptr) {}
```

```

    ArrayQueue(const ArrayQueue& other) : slots_(other.slots_),
    ↪ front_(other.front_), rear_(other.rear_), data_(slots_ ? new T[slots_] :
    ↪ nullptr) { for (std::size_t i = front_; i != rear_; i = (i + 1) % slots_)
    ↪ data_[i] = other.data_[i]; }
    ArrayQueue& operator=(const ArrayQueue& other) { if (this != &other) {
    ↪ ArrayQueue copy(other); swap(copy); } return *this; }
    ArrayQueue(ArrayQueue&& other) noexcept { swap(other); }
    ArrayQueue& operator=(ArrayQueue&& other) noexcept { if (this != &other) {
    ↪ ArrayQueue moved(std::move(other)); swap(moved); } return *this; }
    ~ArrayQueue() { delete[] data_; }
    void swap(ArrayQueue& other) noexcept { using std::swap; swap(slots_,
    ↪ other.slots_); swap(front_, other.front_); swap(rear_, other.rear_);
    ↪ swap(data_, other.data_); }
    [[nodiscard]] bool empty() const noexcept { return front_ == rear_; }
    [[nodiscard]] bool full() const noexcept { return slots_ != 0 && (rear_ + 1) %
    ↪ slots_ == front_; }
    [[nodiscard]] std::size_t size() const noexcept { return rear_ >= front_ ? rear_
    ↪ - front_ : slots_ - front_ + rear_; }
    [[nodiscard]] bool enqueue(const T& value) { if (full()) return false;
    ↪ data_[rear_] = value; rear_ = (rear_ + 1) % slots_; return true; }
    [[nodiscard]] bool enqueue(T&& value) { if (full()) return false; data_[rear_] =
    ↪ std::move(value); rear_ = (rear_ + 1) % slots_; return true; }
    [[nodiscard]] std::optional<T> dequeue() { if (empty()) return std::nullopt; T
    ↪ value = std::move(data_[front_]); front_ = (front_ + 1) % slots_; return
    ↪ value; }
    [[nodiscard]] const T* front() const noexcept { return empty() ? nullptr :
    ↪ &data_[front_]; }
    void clear() noexcept { front_ = rear_ = 0; }
private: std::size_t slots_{0}, front_{0}, rear_{0}; T* data_{nullptr};
};

```

3.2.3 链式队列

链式队列用首尾指针维持 FIFO，入队接在尾、出队摘下头，并具备独立复制所有权。

```

template <typename T>
class LinkedQueue {
    struct Node { T value; Node* next{nullptr}; template <typename U> explicit
    ↪ Node(U&& value) : value(std::forward<U>(value)) {} };
public:
    LinkedQueue() = default;
    LinkedQueue(const LinkedQueue& other) { for (Node* n = other.front_; n !=
    ↪ nullptr; n = n->next) enqueue(n->value); }
    LinkedQueue& operator=(const LinkedQueue& other) { if (this != &other) {
    ↪ LinkedQueue copy(other); swap(copy); } return *this; }
    LinkedQueue(LinkedQueue&& other) noexcept { swap(other); }

```

```

LinkedQueue& operator=(LinkedQueue&& other) noexcept { if (this != &other) {
↳ LinkedQueue moved(std::move(other)); swap(moved); } return *this; }
~LinkedQueue() { clear(); }
void swap(LinkedQueue& other) noexcept { using std::swap; swap(front_,
↳ other.front_); swap(rear_, other.rear_); swap(size_, other.size_); }
[[nodiscard]] bool empty() const noexcept { return front_ == nullptr; }
[[nodiscard]] std::size_t size() const noexcept { return size_; }
void enqueue(const T& value) { append(new Node(value)); }
void enqueue(T&& value) { append(new Node(std::move(value))); }
[[nodiscard]] std::optional<T> dequeue() { if (front_ == nullptr) return
↳ std::nullopt; Node* old = front_; front_ = old->next; if (front_ == nullptr)
↳ rear_ = nullptr; --size_; T value = std::move(old->value); delete old;
↳ return value; }
[[nodiscard]] const T* front() const noexcept { return front_ == nullptr ?
↳ nullptr : &front_->value; }
void clear() noexcept { while (front_ != nullptr) { Node* old = front_; front_ =
↳ old->next; delete old; } rear_ = nullptr; size_ = 0; }
private: void append(Node* node) noexcept { if (rear_ == nullptr) front_ = rear_ =
↳ node; else { rear_->next = node; rear_ = node; } ++size_; } Node*
↳ front_{nullptr}; Node* rear_{nullptr}; std::size_t size_{0};
};

```

3.3 栈与队列的比较

3.3.1 顺序栈与链式栈

顺序栈预分配连续缓冲区，扩容要搬迁；链式栈按需分配，没有「栈满」，但每个结点多一个指针。两者的 push/pop 都是 $O(1)$ 。

3.3.2 顺序队列与链式队列

两种队列的端点操作都是常数时间。循环数组适合容量上界已知、希望局部性好的场合；链表适合长度变化大、不愿预留空洞槽位的场合。

3.3.3 限制存取点的表

栈和队列都是限制存取点的线性表：栈只开一端，队列开两端但方向相反。双端队列同时开放两端，不在本章实现范围内。

本章小结

栈只在一端插入删除，后进先出；队列在一端插、另一端删，先进先出。二者都可以顺序或链式实现，端点运算都是常数时间。顺序实现要处理满与空（循环队列牺牲一个槽）；链式实现按需分配，没有预先的容量上限。栈用于递归、括号和表达式；队列用于层次周游和缓冲。

习题

1. 循环队列两端都允许插入删除。给出类型定义，以及「从队尾删除」「从队头插入」的算法。
2. 用循环数组表示队列，只设头指针 `front` 和计数器 `count`，不设尾指针。实现判空、入队、出队。
3. 把栈 S 中的元素逆置：分别使用两个额外栈；一个额外队列；一个额外栈加若干非数组变量。
4. 用栈定义队列。
5. 带头结点的循环链表表示队列，只设一个指向队尾的指针。写出初始化、入队、出队。
6. 在长度为 n 的数组里实现两个栈，元素总数不到 n 时都不溢出，且 `push/pop` 为 $O(1)$ 。
7. 编号 1 到 5 的列车顺序进栈式站台，出站顺序有多少种？
8. 证明：用一个栈把 $1, 2, \dots, n$ 变成 $p_1, \dots, p_n$ 的充要条件是不存在 $i < j < k$ 使 $p_j < p_k < p_i$ 。
9. 用栈计算后缀表达式 $1289 * +$ ，写出每一步的栈。
10. 用栈把中缀 $a * (b * c - d) + e$ 转成后缀，写出每一步的栈。
11. 按 $GCD(n, m)$ 的递归定义写算法。
12. 写递归和非递归算法，求 $1 + 1/2 - 1/3 + 1/4 - \dots$ 的前 n 项和。

上机题

1. 用两个栈模拟队列，实现 `enqueue`、`dequeue`、`queue_empty`。
2. 用数组实现双端队列，两端插入删除都是常数时间。
3. 用辅助队列（两个或一个）使队列元素有序。
4. 把栈 s_1 的元素转到 s_2 并保持原顺序：分别用一个辅助栈；只用非数组变量。
5. 写计算 $fib(n)$ 的递归过程，再用栈改成非递归。
6. 按定义计算 $Ack(m, n)$ ：写出 $Ack(2, 1)$ 的过程，以及非递归算法。

第 4 章 字符串

字符串是字符的有限序列。先区分「保存字符串」的类与「在文本中找模式」的算法：前者处理容量、复制和下标边界，后者处理比较顺序。朴素匹配在失配后移动模式；KMP 预先计算 next 信息，避免重复比较已经知道相等的前缀。

源码：字符串类、字符串示例、模式匹配、匹配示例。

先跑一遍

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::String text = "Hello";
    text.append(' ').append('C').append('+').append('+');
    std::cout << " 拼接后: " << text.c_str() << '\n';
    const auto slice = text.substr(6, 3);
    std::cout << " 子串: " << slice.c_str() << '\n';
    const auto found = text.find('C');
    std::cout << " 首次出现 C 的下标: ";
    if (found) {
        std::cout << *found << '\n';
    } else {
        std::cout << " 无\n";
    }
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch04/string_class \
    code/ch04/string_class/demo.cpp -o /tmp/str-demo
/tmp/str-demo
```

拼接后: Hello C++
子串: C++
首次出现 C 的下标: 6

缓冲区仍是手写的 `char*`，换成 `std::string` 这一节就没了。`find` 找不到时返回空 optional，不用 `-1` 和位置 0 抢同一个数字。

图 4.12 自己的那对串，原书两个匹配算法都返回 11，正确答案是 10:

```
#include "modern.hpp"
```

```
#include <iostream>

int main() {
    const char* text = "abcddabcbabcbdaabcbabcbdaabcbabaa";
    const char* pattern = "abcdaabcbab";
    const auto naive = dsa::naive_search(text, pattern);
    const auto kmp = dsa::kmp_search(text, pattern);
    std::cout << " 图 4.12 的串, 正确起始下标是 10\n";
    std::cout << " 朴素: " << (naive ? static_cast<long>(*naive) : -1) << '\n';
    std::cout << "KMP: " << (kmp ? static_cast<long>(*kmp) : -1) << '\n';
    std::cout << " 原书返回 11, 一律差 1\n";
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch04/pattern_matching \
    code/ch04/pattern_matching/demo.cpp -o /tmp/kmp-demo
/tmp/kmp-demo
```

图 4.12 的串, 正确起始下标是 10
 朴素: 10
 KMP: 10
 原书返回 11, 一律差 1

4.1 字符串的基本概念

字符串 (string) 是由零个或多个字符组成的有限序列, 是一种特殊的线性表——它的每个元素都是字符。串中字符的数目称为串的长度, 长度为零的串称为空串。

原书【代码 4.1】给出字符串的抽象数据类型。那份声明有三处今天必须改:

1. 类名是小写的 **string**。在任何 `using namespace std;` 的翻译单元里, 它与 `std::string` 构成歧义。原书正文随后又改用大写 `String`, 同一章里两个名字混用。
2. `int isEmpty();` 用 `int` 表达布尔, `int find(const char c, const int start);` 用 `-1` 表示“没找到”——与“位置 0”只差一个符号, 调用方漏判就会把“没找到”当成“匹配在开头”。
3. 修改器按值返回: `string append(const char c);`、`string concatenate(const char* s);` 都返回 `string` 而非引用。代码 4.1 只有声明没有函数体, 所以不能断定原书会丢结果; 能断定的是签名含混——调用方无从判断 `s.append('x')`; 是改了 `s`, 还是返回了一个新串而 `s` 原封不动。

本书的 `String` 把这三处分别改为: 类名大写、`bool empty()`、`std::optional<size_type> find(...)`、修改器返回 `String&`。

```

/// 变长字符串。内部保存一块以 '\0' 结尾的字符数组和当前长度。
///
/// 与原书 String 的差别：构造函数取 const char* (原书取 char*,
/// 使书中自己的例子 `String s1 = "Hello";` 在 C++11 起就是非法转换);
/// 越界抛 std::out_of_range, 而不是 `return NULL` 让调用方拿到一个必然崩溃的对象;
/// 补齐五法则; 不做任何 I/O。
class String {
public:
    using size_type = std::size_t;

    /// 空串。注意它仍然持有一块 1 字节的缓冲区, 于是 c_str() 永远可用、永不为空指针。
    String() : data_(new char[1]{'\0'}), size_(0) {}

    /// 从 C 字符串构造。故意 ** 不加 explicit **: 原书 `String s1 = "Hello";` 这种写法
    /// 是本节的教学用例, 保留它; 代价是隐式转换, 值得知道但这里可以接受。
    String(const char* s) { // NOLINT(google-explicit-constructor)
        if (s == nullptr) {
            // 原书的 Substr 在越界时 `return NULL`, 随后 strlen(nullptr) 当场 SEGV。
            // 这里把它挡在门口, 并且说清楚是什么问题。
            throw std::invalid_argument("String: 不能用空指针构造字符串");
        }
        size_ = std::strlen(s);
        data_ = new char[size_ + 1];
        std::memcpy(data_, s, size_ + 1);
    }
}

```

4.2 字符串的存储结构和实现

String 采用动态变长的存储结构：内部持有一块以 `'\0'` 结尾的字符数组和当前长度, 构造时按初值长度分配, 赋值时按新长度重新分配。这正是本节要教的内容, 所以缓冲区是裸 *char**——换成 *std::string* 这一节就没了。

4.2.1 构造与所有权

原书【算法 4.4】的构造函数有两处问题。

第一处, `assert(str != '\0')` 本身就编译不过: `'\0'` 是 *char* 而不是空指针常量, 拿指针与它比较是 ill-formed (error: ISO C++ forbids comparison between pointer and integer)。而且即便改写成 `assert(str != nullptr)` 也是无效断言——*new* 失败时抛 `std::bad_alloc`, 从不返回空指针; *assert* 更会在 *NDEBUG* 构建里整个消失。

第二处, 参数类型是 *char**。原书 4.2.2 节自己写下:

```
String s1 = "Hello"; 隐含地调用构造函数 String::String(char* s)
```

而字符串字面量的类型是 `const char[6]`, 转成 *char** 在 C++11 起已被移除。GCC 默认降级为警告, 在本书的 `-Werror` 构建下即是错误。

还有一处原书没有做的检查: 构造函数对 `s == nullptr` 毫无防备——而 4.2.3

节的 Substr 恰好会喂给它一个空指针（见 4.2.3 节）。

原书的类里有 ~string(), 却从未把拷贝构造与拷贝赋值作为清单给出（正文描述过赋值时”必须释放 s1 的原有空间”，但没有代码）。只要有析构而没有这两个，一次 String b = a; 就是二次释放——与第 2、3 章是同一个错误。

```
// 原书只在正文里描述了赋值时" 必须释放 s1 的原有空间 (delete [] s1.str)",
// 却没有把拷贝构造和拷贝赋值作为清单给出。只要有析构函数而没有这两个,
// 一次 `String b = a;` 就是二次释放——与第 2、3 章是同一个错误。
String(const String& other) : data_(new char[other.size_ + 1]), size_(other.size_) {
    // 注意读的是 other.raw() 不是 other.data_: 源可能是被移动过的对象 (data_ 为空),
    // 从空指针 memcpy 即便长度为 0 也是未定义行为。
    std::memcpy(data_, other.raw(), size_ + 1);
}

String& operator=(const String& other) {
    if (this != &other) {
        String copy(other); // 拷贝并交换: 自赋值安全, 且拷贝失败时原对象不受影响
        swap(copy);
    }
    return *this;
}

/// 移动 ** 不分配 **: 直接接管缓冲区, 把对方置为 nullptr。
/// 被移动方仍是一个可用的空串——读取路径统一走 raw(), 它在 data_ 为空时返回 ""。
/// (若在这里 new 一块空缓冲区来" 修复" 对方, 移动就不再是 noexcept 的了。)
String(String&& other) noexcept : data_(other.data_), size_(other.size_) {
    other.data_ = nullptr;
    other.size_ = 0;
}

String& operator=(String&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    return *this;
}

~String() { delete[] data_; }
```

移动操作声明为 noexcept, 因此不能在里面分配。被移动方的指针置空, 读取路径统一走一个私有的 raw(): 指针为空时返回静态空串。于是”被移动之后仍是可用的空串”这个保证不花任何分配就成立, c_str() 也永远不会是空指针。

4.2.2 追加与拼接

```
/// 在串尾添加一个字符，返回自身引用。
///
/// 原书【代码 4.1】声明的是 string append(const char c);——** 按值返回 **。
/// 代码 4.1 只有声明没有函数体，所以「它到底改不改本串」在书里无从查证；
/// 而这正是问题所在：一个修改器按值返回，调用方无法从签名判断
/// s.append('x'); 是改了 s 还是返回了一个新串而 s 原封不动。
/// 返回自身引用把这件事说死，同时支持链式调用。
String& append(char c) {
    char* fresh = new char[size_ + 2];
    std::memcpy(fresh, raw(), size_);
    fresh[size_] = c;
    fresh[size_ + 1] = '\0';
    delete[] data_;
    data_ = fresh;
    ++size_;
    return *this;
}

/// 把 s 连接在本串后面。s 为空指针时抛 std::invalid_argument。
String& concatenate(const char* s) {
    if (s == nullptr) {
        throw std::invalid_argument("String::concatenate: 空指针");
    }
    const size_type extra = std::strlen(s);
    char* fresh = new char[size_ + extra + 1];
    std::memcpy(fresh, raw(), size_);
    std::memcpy(fresh + size_, s, extra + 1);
    delete[] data_;
    data_ = fresh;
    size_ += extra;
    return *this;
}

String& operator+=(char c) { return append(c); }
String& operator+=(const String& other) { return concatenate(other.c_str()); }
```

每次追加都重新分配一块、拷贝、释放旧块——这就是变长串管理的代价，也是后面章节讨论“预留容量”的动机。

4.2.3 抽取子串

原书【算法 4.5】在起始位置越界时写 `return NULL;`。这里的返回类型是 `String`，所以 `NULL` 不是“空串”：它先转成 `char*`，再走 `String(char*)` 构造函数，于是 `strlen(nullptr)`。这段能编译，崩在运行期：

```
$ ./s7
准备调用 s.Substr(99, 1)——原书会走到 return NULL 那一支
runtime error: null pointer passed as argument 1, which is declared to never be null
AddressSanitizer:DEADLYSIGNAL
ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000
```

本书越界就抛 `std::out_of_range`，让错误停在发生的地方，而不是变成调用方某处的段错误。

```
/// 从 pos 开始抽取长度至多为 len 的子串。
///
/// 原书【算法 4.5】在 `pos >= size` 时 `return NULL;`——那不是“返回空串”，
/// 而是拿 NULL 走 String(char*) 构造函数，接着 strlen(nullptr) 当场崩溃
/// （证据见 legacy.md 缺陷 3）。这里越界就抛 std::out_of_range，
/// 让错误停在发生的地方，而不是变成调用方某处的段错误。
///
/// pos == size() 是合法的，得到空串——与“从末尾取 0 个字符”的直觉一致。
[[nodiscard]] String substr(size_type pos, size_type len) const {
    if (pos > size_) {
        throw std::out_of_range("String::substr: 起始位置越界");
    }
    const size_type available = size_ - pos;
    const size_type take = len < available ? len : available; // 原书的 if (n >
    ↪ left) n = left
    String result;
    char* fresh = new char[take + 1];
    std::memcpy(fresh, raw() + pos, take);
    fresh[take] = '\0';
    delete[] result.data_;
    result.data_ = fresh;
    result.size_ = take;
    return result;
}
```

`len` 超出剩余长度时截断而不报错，与原书的 `if (n > left) n = left;` 语义一致。

4.2.4 查找与比较

```
/// 从 start 开始查找字符 c，返回下标；没有则 std::nullopt。
/// 原书 `int find(const char c, const int start)` 用 -1 表示没找到，
/// 与“位置 0”只差一个符号。
[[nodiscard]] std::optional<size_type> find(char c, size_type start = 0) const {
    for (size_type i = start; i < size_; ++i) {
        if (raw()[i] == c) {
```

```

        return i;
    }
}
return std::nullopt;
}

/// 三路比较，负/零/正 表示 小于/等于/大于。
///
/// 原书【算法 4.3】自己实现了一个 strcmp，返回值固定为 -1/0/1，
/// 并在正文里指出“这与 C/C++ 语言中通常的大小比较习惯（0 和非 0）不一致”——
/// 其实不一致的是原书自己：标准 strcmp 返回的就是差值的符号，
/// 调用方只该看符号，不该看具体数值。这里保持标准语义。
[[nodiscard]] int compare(const String& other) const noexcept {
    return std::strcmp(raw(), other.raw());
}

```

关于【算法 4.3】：原书自己实现了一个 `strcmp`，固定返回 `-1/0/1`，并在正文里说“这与 C/C++ 语言中通常的大小比较习惯（0 和非 0）不一致”。其实标准 `strcmp` 返回的就是差值的符号，调用方本来就只该看符号——不一致的是原书固定返回 `±1` 的写法。本书保持标准语义，并据此提供关系运算符。

（另外补一句实测结论：原书那个与标准库同名同签名的 `strcmp` 定义，既能编译也能链接，不构成冲突——这是我们查过之后否掉的一个猜测，记在 `legacy.md` 第五节。）

4.3 字符串的模式匹配

给定目标串 `T` 和模式串 `P`，模式匹配要回答：`P` 是否作为 `T` 的连续子串出现，若出现，首次出现在哪个位置。

本节的教学内容是算法本身，因此下面的实现用 `std::string_view` 接收输入：不拷贝、不拥有，把注意力留在匹配过程上。字符串容器的实现是 4.2 节的事。

4.3.1 朴素的模式匹配算法

朴素算法的思路直白：把模式的首字符对齐目标的每一个位置，逐字符比较；一旦失配，就把模式整体右移一位，从头再来。

```

/// 朴素模式匹配：返回 pattern 在 text 中首次出现的 ** 起始下标 **；没有则 std::nullopt。
///
/// 与原书【算法 4.6】的关键差别是返回值：原书写的是 `return (j - pLen + 1);`，
/// 而在 0 起始的下标体系里正确的是 `j - pLen`——** 原书这里差了 1**。


```

/// 用书中自己的例子可以当场看出来：T="abcddabcb..."、P="abcdaabcb" 匹配始于下标 10，
/// 原书返回 11（证据见 legacy.md 缺陷 1）。
///
/// 空模式约定返回 0（与 std::string::find("") 一致）；原书用 assert(m>0) 把它挡在门外，
/// 而 assert 在 NDEBUG 下会被整个编译掉。

```


```

```
[[nodiscard]] inline std::optional<std::size_t> naive_search(std::string_view text,
                                                            std::string_view pattern) {
    const std::size_t n = text.size();
    const std::size_t m = pattern.size();
    if (m == 0) {
        return std::size_t{0};
    }
    if (n < m) {
        return std::nullopt;
    }
    std::size_t i = 0; // 模式下标
    std::size_t j = 0; // 目标下标
    while (i < m && j < n) {
        if (text[j] == pattern[i]) {
            ++i;
            ++j;
        } else {
            j = j - i + 1; // 回退到本趟起点的下一个位置
            i = 0;
        }
    }
    return i >= m ? std::optional<std::size_t>(j - m) : std::nullopt;
}
```

失配时 $j = j - i + 1$ 把目标下标退回本趟起点的下一个位置——这一步的“+1”是对的，它表示“换一个起点重来”。最坏情况下，每个起点都要比较到模式末尾才失配（例如 $T = \text{“aaa...a”}$ 、 $P = \text{“aa...ab”}$ ），总比较次数为 $O(|T| \cdot |P|)$ 。

一处必须指出的错误：原书的返回值差 **1**

原书【算法 4.6】与【算法 4.8】在匹配成功时都写：

```
if (i >= pLen) return (j - pLen + 1);
```

匹配成功时 `j` 已经走到匹配段的末尾之后，所以起始位置是 `j - pLen`。那个 `+1` 是多余的。把原书两段代码照抄进程序，拿标准库的 `find` 做参照：

T=abc	P=abc	原书朴素	=	1	正确答案	=	0
T=xabc	P=abc	原书朴素	=	2	正确答案	=	1
T=aaab	P=ab	原书朴素	=	3	正确答案	=	2
T=abcdabcababcdaabcababcdaabcabaa	P=abcdaabcab	原书朴素	=	11	正确答案	=	10

每一组都恰好多 1，最后一组正是书中图 4.12 自己用来演示 KMP 的那对串。原书用它逐趟画了匹配过程，却没有给出返回值，这个错误因此在书里没有暴露。

这不是排版或 OCR 造成的，有三重佐证： $j - pLen + 1$ 在两个算法里独立印

出、写法一致；同一段代码里的 $j = j - i + 1$ 说明作者用的就是 0 起始下标；而 4.3 节开头的约定更是把这一点写死了——

P 和 T 的第一个字符都从位置 0 开始。

本书的实现返回 $j - m$ ，并且每一条匹配用例都拿标准库的 **find** 逐个对拍，再加 3000 组随机对拍。只断言“找到了”的测试，在原书那份实现下同样全绿——那样的测试等于没写。

4.3.2 字符串的特征向量

KMP 的想法是：失配时不必把模式退回从头开始，因为已经匹配上的那一段 本身携带了信息——它的最长相同前后缀长度决定了模式可以直接右移多少。把这个信息对模式的每个位置预先算出来，就是特征向量（next 数组）。

```
/// 计算模式的特征向量（next 数组），原书【算法 4.7】的“优化版”。
///
/// 与原书的差别只有所有权：原书 `int* findNext(String P)` 用 `new int[m]` 返回裸数组，
/// 而书中 ** 从未展示过与之配对的 `delete[]`——每调用一次泄漏一个数组。
/// 计算过程一字未改，包括 `next[i] = next[k]` 这一步优化。
///
/// 空模式返回空向量；原书是 `assert(m > 0)`，而 `assert` 在 `NDEBUG` 下会被编译掉，
/// 于是 `release` 构建里 `new int[0]` 加 `next[0] = -1` 就是一次越界写。
[[nodiscard]] inline std::vector<next_type> build_next(std::string_view pattern) {
    const std::size_t m = pattern.size();
    std::vector<next_type> next(m);
    if (m == 0) {
        return next;
    }
    next_type i = 0;
    next_type k = -1;
    next[0] = -1;
    while (i < static_cast<next_type>(m)) {
        while (k >= 0 && pattern[static_cast<std::size_t>(i)] !=
            ↪ pattern[static_cast<std::size_t>(k)]) {
            k = next[static_cast<std::size_t>(k)]; // 沿已算好的特征值回退
        }
        ++i;
        ++k;
        if (i == static_cast<next_type>(m)) {
            break;
        }
        const auto ui = static_cast<std::size_t>(i);
        const auto uk = static_cast<std::size_t>(k);
        // P[i] 与 P[k] 相等时可以直接借用 next[k]，省掉一次注定失败的比较——这就是“优化
        ↪ 版”。
        next[ui] = (pattern[ui] == pattern[uk]) ? next[uk] : k;
    }
}
```

```
return next;
}
```

`next[i] = next[k]` 那一步是原书所说的”优化”：如果 `P[i]` 与 `P[k]` 相同，那么用 `k` 作为回退目标必然会再失配一次，不如直接借用 `next[k]` 一步到位。

对模式 "abcdaabcb", 算法产生：

i	0	1	2	3	4	5	6	7	8	9
P[i]	a	b	c	d	a	a	b	c	a	b
next[i]	-1	0	0	0	-1	1	0	0	3	0

这与原书图 4.11 最后一行完全一致。

注意原书正文与图 4.11 不一致。正文写的是 `next = {-1,0,0,0,0,-1,1,0,0,3,0}`，数一下是 11 个值，而模式只有 10 个字符。图 4.11 给的是 10 个值，与算法实算的结果相符。正文里多出来的那个 0 是错的。本书的测试逐个比对图 4.11 的十个值，并单独断言”模式只有 10 个字符”，把这处矛盾钉住。

关于所有权：原书 `findNext` 用 `new int[m]` 返回裸数组，而书中调用它的地方——算法 4.8 的 `N` 参数、正文的示例——一次都没有出现配对的 `delete[]`。每匹配一个模式就漏一个数组。本书返回一个拥有所有权的容器，计算过程一字未改。

4.3.3 KMP 模式匹配算法

有了特征向量，匹配过程与朴素算法几乎一样，只有失配那一步不同：不再把模式退回开头，而是退到 `next[i]`。

```
/// KMP 模式匹配。失配时不再把模式右移一位，而是按特征值决定右移多少。
///
/// 返回值与 naive_search 一致，也修正了原书【算法 4.8】同样的差一错误。
/// next 由调用方传入：同一个模式可以只算一次、多次匹配复用——
/// 这正是原书强调的性质，接口把它显式表达出来。
[[nodiscard]] inline std::optional<std::size_t> kmp_search(std::string_view text,
                                                           std::string_view pattern,
                                                           const std::vector<next_type>&
                                                           ↪ next) {

    const std::size_t n = text.size();
    const std::size_t m = pattern.size();
    if (m == 0) {
        return std::size_t{0};
    }
    if (next.size() != m) {
        throw std::invalid_argument("kmp_search: next 数组长度与模式不符");
    }
}
```

```

    if (n < m) {
        return std::nullopt;
    }
    next_type i = 0;    // 模式下标，可以退到 -1
    std::size_t j = 0;  // 目标下标，只增不减
    while (i < static_cast<next_type>(m) && j < n) {
        if (i == -1 || text[j] == pattern[static_cast<std::size_t>(i)]) {
            ++i;
            ++j;
        } else {
            i = next[static_cast<std::size_t>(i)];
        }
    }
    return i >= static_cast<next_type>(m) ? std::optional<std::size_t>(j - m) :
        ↪ std::nullopt;
}

/// 便利重载：模式只用一次时，自己把 next 算掉。
[[nodiscard]] inline std::optional<std::size_t> kmp_search(std::string_view text,
                                                           std::string_view pattern) {
    return kmp_search(text, pattern, build_next(pattern));
}

```

时间代价的关键在于目标下标 j 只增不减。循环体里 $++j$ 至多执行 $|T|$ 次，与它同处一条语句的 $++i$ 也不超过 $|T|$ 次；能让 i 减少的只有 $i = \text{next}[i]$ ，而 $\text{next}[i] < i$ ，所以每执行一次 i 至少减 1；一旦减到 -1 ，下一步必然进入 $++i, ++j$ 分支。因此 $i = \text{next}[i]$ 的执行次数不超过 $++i, ++j$ 的次数加 1，整个循环体至多执行 $2|T| + 1$ 次——与目标串长度成线性关系。

加上计算 next 的 $O(|P|)$ ，KMP 整体为 $O(|P| + |T|)$ 。而同一个模式的特征向量只需算一次即可跨多个目标复用，这一点接口里用「 next 由调用方传入」显式表达。

本书的测试用朴素算法的最坏情况（ $T = 20$ 万个 ‘a’， $P = 1999$ 个 ‘a’ 加一个 ‘b’）验证这条线性性质：KMP 在其上瞬间完成，而失配后不按特征值回退的实现会退化成 $O(|T| \cdot |P|)$ ，直接撞上构建闸门的超时。

与原书的对照

原书	现在	为什么
<code>return (j - pLen + 1)</code>	<code>return j - m</code>	原书差 1 ，四组数据逐个对拍证实
<code>int</code> 返回值， <code>-1</code> 表示没找到	<code>std::optional<std::size_t></code> 与 “位置 0” 只差一个符号，漏判就把未匹配当成匹配在开头	

原书	现在	为什么
<code>int* findNext()</code> 返回 <code>new int[]</code>	返回拥有所有权的容器	书中从未展示配对的 <code>delete[]</code> ，每次调用漏一个数组
<code>assert(m > 0)</code>	空模式返回 0	<code>assert</code> 在 NDEBUD 下整个消失， <code>release</code> 构建里是越界写
<code>assert(next != 0)</code>	删除	<code>new</code> 失败抛 <code>bad_alloc</code> ，从不返回空指针，该断言永远为真

刻意没改的：朴素匹配仍是回溯式的；`next` 仍是原书的优化版（图 4.11 对的是这一版）；KMP 的 `next` 仍由调用方传入并可跨目标复用。这三条是本节全部的教学内容。

完整实现见 `code/ch04/pattern_matching/modern.hpp`，测试见同目录 `test.cpp`（56 项断言，含 3000 组随机对拍；用 `python3 tools/check_code.py` 在 `-Werror + ASan/UBSan` 与 `-O2` 两种构建下各跑一遍）。

本章小结

字符串是元素为字符的线性表。存储要考虑变长：定长数组访问方便，插入删除要搬字符；动态缓冲按长度分配。常用运算是复制、求长、比较、拼接、抽子串、查找字符。模式匹配是子串在主串中的定位：朴素算法直观但会回溯，时间与 $|P||T|$ 成正比；KMP 用模式自身的特征向量避免回溯，时间为 $O(|P| + |T|)$ 。本书还修正了原书返回值一律差 1 的错误。

习题

1. 已知 $s = (xyz)^+*$ ， $t = (x+z)^*y$ 。用连接、抽子串和置换把 s 变成 t 。
2. 给出使 $s_1 + s_2 = s_2 + s_1$ 成立的所有可能条件（+ 为连接）。
3. 统计输入串中各合法字符（A-Z、0-9）的频度。
4. 把长为 n 的串 S 改造成：偶数位字符按原下标从大到小放后半，奇数位从小到大放前半。例如 `ABCDEFGHIJKL` 变成 `ACEGIKLJHFDB`。
5. 判断串是否对称，对称返回 1 否则 0。
6. 求包含在 s 中但不在 t 中的字符构成的新串，以及每个字符在 s 中第一次出现的位置。
7. 线性时间判断 T 是否是 T' 的循环反转（如 `arc` 与 `car`）。
8. 从 s 中删除所有与 t 相同的子串。
9. 求 `BAAABBBAA` 的特征向量，并与目标 `BAAABBBCCCCCHHHHBBBAAABBBAAADD` 匹配，画出过程。

上机题

1. 实现 `atoi(x)`: 把由数字和可选负号组成的串转成整数。
2. 统计输入串中的整数个数并输出它们（连续数字看成一个整数）。
3. 返回串中最长重复子串及其下标。例如 `abcdacdac` 的最长重复子串是 `cdac`，下标 2。
4. 实现一个简单行编辑器：行插入/删除、当前行指针、分页显示，以及用 KMP（不得调用环境自带查找）做查找替换。

第 5 章 二叉树

二叉树的每个结点最多有左、右两个孩子。周游回答「以什么顺序访问全部结点」；二叉搜索树加上左小右大；堆把最小（或最大）值放在根；Huffman 树不断合并两个最小权值。

源码：二叉树与二叉搜索树、最小堆与 Huffman 树、树的示例、堆与 Huffman 的示例。

5.1 二叉树的概念

二叉树由结点的有限集合构成：或者为空，或者由一个根和两棵互不相交的左、右子树组成。左右次序不能颠倒。根没有父结点；其余每个结点恰有一个父结点，至多两个孩子。没有孩子的是叶，其余是内部结点。从根到某结点的边数是该结点的层数，根在第 0 层。

5.1.1 定义和基本术语

下面这棵树：

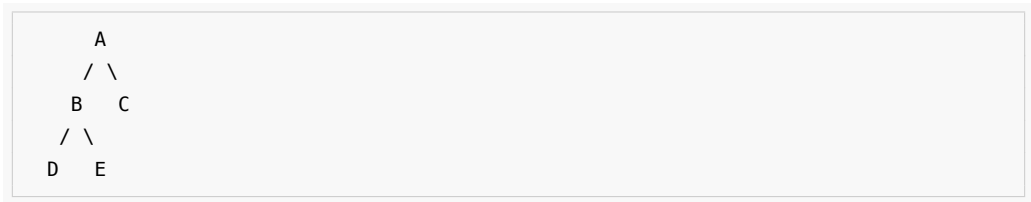


图 5.5 就是上面这棵树。

四种周游访问的是同一组结点，次序不同：

周游	顺序	本例
先序	根，左，右	A B D E C
中序	左，根，右	D B E A C
后序	左，右，根	D E B C A
层次	按离根的距离	A B C D E

递归版最贴合定义，也是 5.2 节要教的东西。极深的退化树会耗尽调用栈：Release 档大约在百万层段错误且没有诊断，ASan 档会打印 stack-overflow 并指到具体行。析构和拷贝走的也是递归，调用方看不见。数字和复现程序见仓库中的未验证风险说明。

5.1.2 满二叉树、完全二叉树、扩充二叉树

任何结点或者是叶，或者左右子树都非空，叫做满二叉树。叶只出现在最下两层、且最下层靠左对齐，叫做完全二叉树。在空子树位置补上空树叶，得到扩充二叉树；外

部路径长度 E 与内部路径长度 I 满足 $E = I + 2n$ 。

5.1.3 主要性质

第 i 层至多 2^i 个结点；深度为 k 的二叉树至多 $2^{k+1} - 1$ 个结点；叶结点数 $n_0 = n_2 + 1$ 。 n 个结点的完全二叉树高度为 $\lceil \log_2(n+1) \rceil$ 。按层从 0 编号时，结点 i 的父是 $\lfloor (i-1)/2 \rfloor$ ，左右孩子是 $2i+1$ 与 $2i+2$ 。这些性质没错，原样保留。

5.2 二叉树的周游

5.2.1 先跑一遍

先建树并打印四种周游，再插一棵 BST：

```
#include "modern.hpp"

#include <iostream>
#include <utility>

int main() {
    dsa::BinaryTree<char> left_leaf;
    dsa::BinaryTree<char> right_leaf;
    dsa::BinaryTree<char> left;
    dsa::BinaryTree<char> right;
    dsa::BinaryTree<char> root;
    left_leaf.create_tree('D');
    right_leaf.create_tree('E');
    left.create_tree('B', std::move(left_leaf), std::move(right_leaf));
    right.create_tree('C');
    root.create_tree('A', std::move(left), std::move(right));

    std::cout << " 先序: ";
    root.preorder([](char value) { std::cout << value; });
    std::cout << "\n中序: ";
    root.inorder([](char value) { std::cout << value; });
    std::cout << "\n后序: ";
    root.postorder([](char value) { std::cout << value; });
    std::cout << "\n层次: ";
    root.level_order([](char value) { std::cout << value; });
    std::cout << '\n';

    dsa::BinarySearchTree<int> tree;
    for (int key : {8, 3, 10, 1, 6, 14, 4, 7}) {
        (void)tree.insert(key);
    }
    std::cout << "BST 中序:";
    tree.inorder([](int key) { std::cout << ' ' << key; });
    std::cout << "\n含 6? " << (tree.contains(6) ? " 是" : " 否")
```



```

        << " 删 3 后含 3? ";
    (void)tree.remove(3);
    std::cout << (tree.contains(3) ? " 是" : " 否") << '\n';
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch05/binary_tree \
    code/ch05/binary_tree/demo.cpp -o /tmp/tree-demo
/tmp/tree-demo

```

```

先序: ABDEC
中序: DBEAC
后序: DEBCA
层次: ABCDE
BST 中序: 1 3 4 6 7 8 10 14
含 6? 是 删 3 后含 3? 否

```

`create_tree(value, left, right)` 接管两棵子树。参数是右值，表示所有权被移走；`left_leaf` 在 `std::move` 之后变空，不会和 `left` 抢着析构同一结点。

5.2.2 深度优先周游

先序 / 中序 / 后序的递归版就是三行：访问自己与走进左右孩子的次序不同。迭代版用一棵手写链式栈模拟调用栈，按 D-001 §3d 只作补充，不替换递归主实现。

5.2.3 广度优先周游

层次周游用手写链式 FIFO，不用 `std::queue` 替代本节要教的队列用法。

5.3 二叉树的存储结构

`create_tree` 先 `new` 出新根，再把两棵子树的根指针挪过来，最后才清空自己原来的树。这样即使 `new` 抛异常，调用方的两棵子树也不动。链式存储是本节主实现；完全二叉树还可以按 5.1.3 的编号放进数组，堆就是这种用法。

```

template <typename T>
class BinaryTree {
private:
    template <typename U>
    class LinkedStack {
        struct Entry {
            U value;
            Entry* next;
        };

    public:

```

```

LinkedStack() = default;
LinkedStack(const LinkedStack&) = delete;
LinkedStack& operator=(const LinkedStack&) = delete;
~LinkedStack() { clear(); }

[[nodiscard]] bool empty() const noexcept { return top_ == nullptr; }
void push(const U& value) { top_ = new Entry{value, top_}; }
void push(U&& value) { top_ = new Entry{std::move(value), top_}; }
[[nodiscard]] std::optional<U> pop() {
    if (top_ == nullptr) return std::nullopt;
    Entry* entry = top_;
    top_ = entry->next;
    U value = std::move(entry->value);
    delete entry;
    return value;
}

private:
void clear() noexcept {
    while (top_ != nullptr) {
        Entry* entry = top_;
        top_ = entry->next;
        delete entry;
    }
}
Entry* top_{nullptr};
};

public:
struct Node {
    T value;
    Node* left{nullptr};
    Node* right{nullptr};

    template <typename U>
    explicit Node(U&& item) : value(std::forward<U>(item)) {}
};

BinaryTree() noexcept = default;
BinaryTree(const BinaryTree& other) : root_(clone(other.root_)) {}
BinaryTree& operator=(const BinaryTree& other) {
    if (this != &other) {
        BinaryTree copy(other);
        swap(copy);
    }
    return *this;
}

```

```

BinaryTree(BinaryTree&& other) noexcept : root_(other.release_root()) {}
BinaryTree& operator=(BinaryTree&& other) noexcept {
    if (this != &other) {
        make_empty();
        root_ = other.release_root();
    }
    return *this;
}
~BinaryTree() { make_empty(); }

void swap(BinaryTree& other) noexcept {
    using std::swap;
    swap(root_, other.root_);
}

[[nodiscard]] bool empty() const noexcept { return root_ == nullptr; }
[[nodiscard]] const Node* root() const noexcept { return root_; }
[[nodiscard]] Node* root() noexcept { return root_; }

/// 构造一棵新树并接管两个子树，强异常保证：根结点创建失败时两棵子树不动。
template <typename U>
void create_tree(U&& value, BinaryTree&& left = {}, BinaryTree&& right = {}) {
    Node* fresh = new Node(std::forward<U>(value));
    fresh->left = left.release_root();
    fresh->right = right.release_root();
    make_empty();
    root_ = fresh;
}

/// 后序释放整个树。递归删除与递归周游同样受树高限制。
void make_empty() noexcept {
    destroy(root_);
    root_ = nullptr;
}

// Stack Overflow Risk: 以下递归周游忠实保留原书算法 5.3；病态深树可能耗尽调用栈。
template <typename Visitor>
void preorder(Visitor&& visit) const { preorder_impl(root_, visit); }
template <typename Visitor>
void inorder(Visitor&& visit) const { inorder_impl(root_, visit); }
template <typename Visitor>
void postorder(Visitor&& visit) const { postorder_impl(root_, visit); }

// 算法 5.4 至 5.6：作为补充保留原书的手写栈式非递归周游。
template <typename Visitor>
void preorder_iterative(Visitor&& visit) const {
    LinkedStack<const Node*> pending;

```

```

    if (root_ != nullptr) pending.push(root_);
    while (auto node = pending.pop()) {
        visit((*node)->value);
        if ((*node)->right != nullptr) pending.push((*node)->right);
        if ((*node)->left != nullptr) pending.push((*node)->left);
    }
}

template <typename Visitor>
void inorder_iterative(Visitor&& visit) const {
    LinkedStack<const Node*> pending;
    const Node* current = root_;
    while (current != nullptr || !pending.empty()) {
        while (current != nullptr) {
            pending.push(current);
            current = current->left;
        }
        current = *pending.pop();
        visit(current->value);
        current = current->right;
    }
}

template <typename Visitor>
void postorder_iterative(Visitor&& visit) const {
    struct Frame { const Node* node; bool expanded; };
    LinkedStack<Frame> pending;
    if (root_ != nullptr) pending.push(Frame{root_, false});
    while (auto frame = pending.pop()) {
        if (frame->expanded) {
            visit(frame->node->value);
        } else {
            pending.push(Frame{frame->node, true});
            if (frame->node->right != nullptr)
                pending.push(Frame{frame->node->right, false});
            if (frame->node->left != nullptr)
                pending.push(Frame{frame->node->left, false});
        }
    }
}
}

```

/// 算法 5.7 的层次周游。内部手写链式 *FIFO*，不以 *STL queue* 替代本章内容。

```

template <typename Visitor>
void level_order(Visitor&& visit) const {
    struct Pending { const Node* node; Pending* next; };
    Pending* front = nullptr;
    Pending* back = nullptr;
    auto enqueue = & {
        Pending* item = new Pending{node, nullptr};
    }
}

```

```

        if (back == nullptr) front = item; else back->next = item;
        back = item;
    };
    try {
        if (root_ != nullptr) enqueue(root_);
        while (front != nullptr) {
            Pending* item = front;
            front = front->next;
            if (front == nullptr) back = nullptr;
            const Node* node = item->node;
            delete item;
            visit(node->value);
            if (node->left != nullptr) enqueue(node->left);
            if (node->right != nullptr) enqueue(node->right);
        }
    } catch (...) {
        while (front != nullptr) { Pending* item = front; front = front->next;
            ↪ delete item; }
        throw;
    }
}

[[nodiscard]] const Node* parent_of(const Node* wanted) const noexcept {
    return parent_of_impl(root_, wanted);
}

private:
    static void destroy(Node* node) noexcept {
        if (node != nullptr) { destroy(node->left); destroy(node->right); delete
            ↪ node; }
    }
    static Node* clone(const Node* node) {
        if (node == nullptr) return nullptr;
        Node* copy = new Node(node->value);
        try { copy->left = clone(node->left); copy->right = clone(node->right); }
        catch (...) { destroy(copy); throw; }
        return copy;
    }
    static const Node* parent_of_impl(const Node* node, const Node* wanted) noexcept
        ↪ {
        if (node == nullptr || wanted == nullptr) return nullptr;
        if (node->left == wanted || node->right == wanted) return node;
        if (const Node* left = parent_of_impl(node->left, wanted)) return left;
        return parent_of_impl(node->right, wanted);
    }
    template <typename Visitor> static void preorder_impl(const Node* node, Visitor&
        ↪ visit) {

```

```

        if (node != nullptr) { visit(node->value); preorder_impl(node->left, visit);
        ↪ preorder_impl(node->right, visit); }
    }
    template <typename Visitor> static void inorder_impl(const Node* node, Visitor&
    ↪ visit) {
        if (node != nullptr) { inorder_impl(node->left, visit); visit(node->value);
        ↪ inorder_impl(node->right, visit); }
    }
    template <typename Visitor> static void postorder_impl(const Node* node,
    ↪ Visitor& visit) {
        if (node != nullptr) { postorder_impl(node->left, visit);
        ↪ postorder_impl(node->right, visit); visit(node->value); }
    }
    Node* release_root() noexcept { Node* result = root_; root_ = nullptr; return
    ↪ result; }
    Node* root_{nullptr};
};

```

5.4 二叉搜索树

二叉搜索树要求左子树的键都小于根、右子树都大于根。中序周游因此正好是排序。插入重复键、删除不存在的键都是可预期状态，返回 false，不抛异常。删除有左右孩子的结点时，用左子树里最右的前驱替换它：先把前驱从原位置摘下，再让它继承被删结点的两棵子树，最后只 delete 被删结点一次。漏掉「先脱离原父」会形成环或二次释放。

```

template <typename T, typename Compare = std::less<T>>
class BinarySearchTree {
    struct Node {
        T value;
        Node* left{nullptr};
        Node* right{nullptr};
        template <typename U> explicit Node(U&& item) : value(std::forward<U>(item))
        ↪ {}
    };

public:
    BinarySearchTree() = default;
    BinarySearchTree(const BinarySearchTree& other) : root_(clone(other.root_)),
    ↪ compare_(other.compare_) {}
    BinarySearchTree& operator=(const BinarySearchTree& other) {
        if (this != &other) { BinarySearchTree copy(other); swap(copy); }
        return *this;
    }
    BinarySearchTree(BinarySearchTree&& other) : root_(other.root_),
    ↪ compare_(std::move(other.compare_)) { other.root_ = nullptr; }

```

```

BinarySearchTree& operator=(BinarySearchTree&& other) {
    if (this != &other) { clear(); root_ = other.root_; other.root_ = nullptr;
        ↪ compare_ = std::move(other.compare_); }
    return *this;
}

~BinarySearchTree() { clear(); }

void swap(BinarySearchTree& other) {
    using std::swap; swap(root_, other.root_); swap(compare_, other.compare_);
}

[[nodiscard]] bool empty() const noexcept { return root_ == nullptr; }

/// 算法 5.9: 插入唯一键。重复键是可预期状态, 返回 false。
bool insert(const T& value) { return insert_impl(value); }
bool insert(T&& value) { return insert_impl(std::move(value)); }

[[nodiscard]] bool contains(const T& value) const {
    const Node* current = root_;
    while (current != nullptr) {
        if (equivalent(value, current->value)) return true;
        current = compare_(value, current->value) ? current->left :
        ↪ current->right;
    }
    return false;
}

/// 算法 5.10: 删除不存在的键是幂等的可预期状态, 返回 false (D-001 §3c)。
bool remove(const T& value) { return remove_impl(root_, value); }
void clear() noexcept { destroy(root_); root_ = nullptr; }

template <typename Visitor>
void inorder(Visitor&& visit) const { inorder_impl(root_, visit); }

private:
[[nodiscard]] bool equivalent(const T& left, const T& right) const {
    return !compare_(left, right) && !compare_(right, left);
}

template <typename U> bool insert_impl(U&& value) {
    Node** link = &root_;
    while (*link != nullptr) {
        if (equivalent(value, (*link)->value)) return false;
        link = compare_(value, (*link)->value) ? &(*link)->left :
        ↪ &(*link)->right;
    }
    *link = new Node(std::forward<U>(value));
    return true;
}

```

```

bool remove_impl(Node*& link, const T& value) {
    if (link == nullptr) return false;
    if (compare_(value, link->value)) return remove_impl(link->left, value);
    if (compare_(link->value, value)) return remove_impl(link->right, value);
    Node* removed = link;
    if (removed->left == nullptr) { link = removed->right; delete removed;
        ↪ return true; }
    Node** predecessor_link = &removed->left;
    while ((*predecessor_link)->right != nullptr) predecessor_link =
        ↪ &(*predecessor_link)->right;
    Node* replacement = *predecessor_link;
    *predecessor_link = replacement->left;
    replacement->left = removed->left;
    replacement->right = removed->right;
    link = replacement;
    delete removed;
    return true;
}

static void destroy(Node* node) noexcept { if (node != nullptr) {
    ↪ destroy(node->left); destroy(node->right); delete node; } }
static Node* clone(const Node* node) {
    if (node == nullptr) return nullptr;
    Node* copy = new Node(node->value);
    try { copy->left = clone(node->left); copy->right = clone(node->right); }
    catch (...) { destroy(copy); throw; }
    return copy;
}

template <typename Visitor> static void inorder_impl(const Node* node, Visitor&
    ↪ visit) {
    if (node != nullptr) { inorder_impl(node->left, visit); visit(node->value);
        ↪ inorder_impl(node->right, visit); }
}

Node* root_{nullptr};
Compare compare_{};
};

```

5.5 堆与优先队列

最小堆是一棵完全二叉树，父结点不大于孩子；用数组存时，下标 i 的孩子是 $2i+1$ 和 $2i+2$ 。sift_down 必须比较左右两个孩子。空堆上 remove_min() 返回 nullptr。

```

#include "modern.hpp"

#include <iostream>

int main() {

```



```

dsa::MinHeap<int> heap;
for (int value : {5, 1, 4, 2}) {
    heap.insert(value);
}
std::cout << " 依次取出最小元:";
while (auto value = heap.remove_min()) {
    std::cout << ' ' << *value;
}
std::cout << '\n';

const int weights[] = {2, 3, 4, 7};
const dsa::HuffmanTree tree(weights, 4);
std::cout << " 权 2,3,4,7 的 Huffman 树根权 = " << tree.total_weight() << '\n';
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch05/heap_huffman \
    code/ch05/heap_huffman/demo.cpp -o /tmp/heap-demo
/tmp/heap-demo

```

依次取出最小元: 1 2 4 5
权 2,3,4,7 的 Huffman 树根权 = 16

```

template <typename T>
class MinHeap {
public:
    static_assert(std::is_nothrow_move_constructible<T>::value &&
        ↪ std::is_nothrow_move_assignable<T>::value,
        "MinHeap growth relies on non-throwing moves; use a
        ↪ noexcept-movable element type.");

    MinHeap() = default;
    MinHeap(const MinHeap& other) : data_(other.capacity_ ? new T[other.capacity_] :
    ↪ nullptr), size_(other.size_), capacity_(other.capacity_) {
        try { for (std::size_t i = 0; i < size_; ++i) data_[i] = other.data_[i]; }
        catch (...) { delete[] data_; throw; }
    }
    MinHeap& operator=(const MinHeap& other) { if (this != &other) { MinHeap
    ↪ copy(other); swap(copy); } return *this; }
    MinHeap(MinHeap&& other) noexcept { swap(other); }
    MinHeap& operator=(MinHeap&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            data_ = other.data_;
            size_ = other.size_;

```

```

        capacity_ = other.capacity_;
        other.data_ = nullptr;
        other.size_ = other.capacity_ = 0;
    }
    return *this;
}

~MinHeap() { delete[] data_; }

void swap(MinHeap& other) noexcept { using std::swap; swap(data_, other.data_);
    ↪ swap(size_, other.size_); swap(capacity_, other.capacity_); }

[[nodiscard]] bool empty() const noexcept { return size_ == 0; }

[[nodiscard]] std::size_t size() const noexcept { return size_; }

void insert(const T& value) { ensure_capacity(); data_[size_] = value;
    ↪ sift_up(size_++); }

void insert(T&& value) { ensure_capacity(); data_[size_] = std::move(value);
    ↪ sift_up(size_++); }

[[nodiscard]] std::optional<T> remove_min() {
    if (empty()) return std::nullopt;
    T value = std::move(data_[0]);
    --size_;
    if (size_ == 0) return value;
    data_[0] = std::move(data_[size_]);
    sift_down(0);
    return value;
}

private:
void ensure_capacity() {
    if (size_ < capacity_) return;
    const std::size_t next = capacity_ == 0 ? 4 : capacity_ * 2;
    T* fresh = new T[next];
    // The class contract requires non-throwing move assignment, so the
    // migration loop cannot fail. Allocation failure is thrown before fresh
    ↪ exists.
    for (std::size_t i = 0; i < size_; ++i) fresh[i] = std::move(data_[i]);
    delete[] data_;
    data_ = fresh;
    capacity_ = next;
}

void sift_up(std::size_t index) {
    while (index != 0 && data_[index] < data_[(index - 1) / 2]) {
        using std::swap;
        swap(data_[index], data_[(index - 1) / 2]);
        index = (index - 1) / 2;
    }
}

void sift_down(std::size_t index) {
    for (;;) {
        const std::size_t left = index * 2 + 1;

```

```

        const std::size_t right = left + 1;
        std::size_t smallest = index;
        if (left < size_ && data_[left] < data_[smallest]) smallest = left;
        if (right < size_ && data_[right] < data_[smallest]) smallest = right;
        if (smallest == index) return;
        using std::swap;
        swap(data_[index], data_[smallest]);
        index = smallest;
    }
}
T* data_{nullptr};
std::size_t size_{0};
std::size_t capacity_{0};
};

```

5.6 Huffman 树及其应用

Huffman 树反复取出两个最小权，合成它们的和，直到只剩一棵——这就是前缀编码的那棵树。根权等于全部叶子权之和。合并时若 new 父结点失败，会拆开并销毁已经取出的两棵子树。

```

class HuffmanTree {
    struct Node { int weight; Node* left{nullptr}; Node* right{nullptr}; explicit
        ↪ Node(int w):weight(w){} };
    struct ByWeight { Node* node{nullptr}; bool operator<(const ByWeight& other)
        ↪ const noexcept { return node->weight < other.node->weight; } };
public:
    HuffmanTree()=default;
    explicit HuffmanTree(const int* weights, std::size_t count) {
        if (count == 0) return;
        if (weights == nullptr) throw std::invalid_argument("non-empty Huffman input
            ↪ requires weights");
        MinHeap<ByWeight> heap;
        try {
            for (std::size_t i = 0; i < count; ++i) {
                if (weights[i] < 0) throw std::invalid_argument("Huffman weights
                    ↪ must be non-negative");
                Node* leaf = new Node(weights[i]);
                try { heap.insert(ByWeight{leaf}); }
                catch (...) { delete leaf; throw; }
            }
            while (heap.size() > 1) {
                Node* left = heap.remove_min()->node;
                Node* right = heap.remove_min()->node;
                Node* parent = nullptr;

```

```

        try {
            if (left->weight > std::numeric_limits<int>::max() -
                right->weight) {
                throw std::overflow_error("Huffman weight sum overflows int");
            }
            parent = new Node(left->weight + right->weight);
            parent->left = left;
            parent->right = right;
            heap.insert(ByWeight{parent});
        } catch (...) {
            if (parent != nullptr) { parent->left = parent->right = nullptr;
                delete parent; }
            destroy(left);
            destroy(right);
            throw;
        }
        root_ = heap.remove_min()->node;
    } catch (...) {
        while (auto item = heap.remove_min()) destroy(item->node);
        throw;
    }
}

HuffmanTree(const HuffmanTree&)=delete; HuffmanTree& operator=(const
HuffmanTree&)=delete;
HuffmanTree(HuffmanTree&& other)noexcept:root_(other.root_)
HuffmanTree& operator=(HuffmanTree&&
other)noexcept{if(this!=&other)
{destroy(root_);root_=other.root_;other.root_=nullptr;}return *this;}
~HuffmanTree(){destroy(root_);} [[nodiscard]] int total_weight()const
noexcept{return root_?root_->weight:0;}
private: static void destroy(Node*n)noexcept{if(n)
{destroy(n->left);destroy(n->right);delete n;}} Node* root_{nullptr};
};

```

本章小结

二叉树每个结点至多两个有左右之分的孩子。满二叉树、完全二叉树和扩充二叉树是三种常用特殊形态；第 i 层至多 2^i 个结点， $n_0 = n_2 + 1$ 。周游有先序、中序、后序和层次。链式存储用左右指针；完全二叉树还可以按层编号放进数组，堆就是这种用法。二叉搜索树加上左小右大，中序即排序。堆把最值放在根；Huffman 树反复合并两个最小权，得到带权路径长度最小的树。

习题

1. 分别画出含 3 个结点的二叉树的所有不同形态。
2. 一棵二叉树的先序是 ABCDEFG、中序是 CBDAFEG，画出这棵树并写出后序。
3. 证明：有 n 个结点的二叉树，空指针域的个数是 $n + 1$ 。
4. 完全二叉树按层编号，写出结点 i 的父、左孩子、右孩子公式，并证明。
5. 写出层次周游的算法，并说明为什么需要队列。
6. 在二叉搜索树中插入序列 50, 30, 70, 20, 40, 60, 80，画出结果；再删除 50，画出删除后的树。
7. 对权 2, 3, 4, 7, 8 构造一棵 Huffman 树，计算带权路径长度，并给出一组前缀编码。

上机题

1. 由先序和中序序列重建二叉树，再输出后序。
2. 实现二叉搜索树的插入、按键删除和中序输出，并用有序插入验证会退化成链。
3. 用最小堆对 n 个整数排序，并与直接插入比较运行时间。
4. 根据一组权值建 Huffman 树，输出每个权的编码。

第 6 章 树

一般树允许一个结点有任意多个孩子。本章用「左孩子、右兄弟」表示它：child 指向第一个孩子，sibling 指向下一个兄弟。并查集则回答另一类问题：若干元素分属哪些互不相交的集合。

源码：一般树和并查集、可运行示例、测试。

6.1 树的定义和基本术语

树形结构用分支关系定义层次：一个结点至多一个前驱，但可以有任意多个后继。家谱、机关编制、文件系统目录、编译器的句法树，都是树。和第 5 章的二叉树不同，一般树的结点不限制孩子个数。

6.1.1 树和森林

树是 n ($n \geq 1$) 个结点的有穷集合 T ，满足：有且仅有一个称为根的结点；其余结点被分成 m ($m \geq 0$) 个互不相交的集合，每个集合又是一棵树，叫做根的子树。这个定义是递归的。

也可以用二元关系来写。结点集 K 上有一个关系 r ：恰好一个结点没有前驱，它是根；其余每个结点恰好一个前驱；从根到任意结点都存在一条由关系元组串起来的路径。

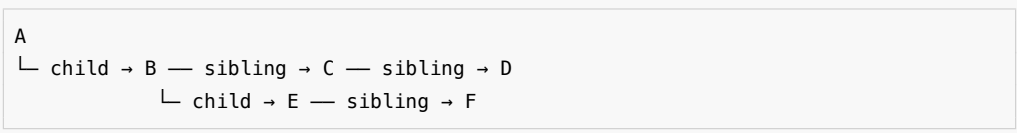
父、子、兄弟、根、叶、度、层、路径这些词，含义与二叉树相同。结点的度是孩子个数，树的度是各结点度的最大值。根在第 0 层，非根结点的层数是父结点的层数加 1。

自然界里孩子的左右次序往往无所谓，这种树叫无序树。计算机存储是有序的，所以实现时通常把孩子从左到右编上号，当作有序树。注意：度为 2 的有序树还不是二叉树——删掉第一个孩子后，第二个会顶上来占第一的位置；二叉树必须能表示「左空、右不空」这种左右不对称。

森林是零棵或多棵互不相交的树的集合（通常有序）。一棵树里，某个结点的全部子树组成一个森林；给这个森林加上一个公共根，就又变成一棵树。

树有多种画法。树形表示法是根在上的倒挂树；凹入表示法用长短线表示层次，像书的目录；文氏图用嵌套圆；嵌套括号把根写在左边、子树写在括号里，例如 $A(B(D(I,J),E), C(F,G(K,L),H))$ 。表示法多样，说明树在建模里用得广。

例如 A 的孩子是 B、C、D，B 的孩子是 E、F。用左孩子 / 右兄弟画出来：



任意度的树只需两个指针域。代价是不能 $O(1)$ 取得「第 k 个孩子」，必须沿兄弟链走过去。图 6.7 就是这张图。

6.1.2 森林与二叉树的等价转换

树和森林都可以一对一地变成二叉树，相关操作也就都能转到二叉树上做。形象的做法是「连线、切线、旋转」三步：

- 1. 连线：把兄弟结点用线连起来。
- 2. 切线：只保留父结点到第一个孩子的连线，砍掉到其余孩子的连线。
- 3. 旋转：以根为轴顺时针转一下，画面才像通常的二叉树。

转换后，一个结点的左孩子是它在原树（或森林）里的第一个孩子，右孩子是原来的下一个兄弟。左枝上是父子关系，右枝上是兄弟关系。单棵树的根没有兄弟，所以转成二叉树后根的右孩子一定为空。

形式地说：森林 $F = \{T_1, T_2, \dots, T_n\}$ 转成二叉树 $B(F)$ —— F 空则 $B(F)$ 空；否则 $B(F)$ 的根是 T_1 的根， $B(F)$ 的左子树是 T_1 的子树森林转成的二叉树， $B(F)$ 的右子树是 $\{T_2, \dots, T_n\}$ 转成的二叉树。

反过来是三步的逆：逆时针旋转；若 x 是 y 的左孩子，就把 x 以及 x 右侧整条右链上的结点都补连到 y ；再删掉所有到右孩子的边。二叉树 B 对应的森林 $F(B)$ ：空树对应空森林；否则 $F(B)$ 是一棵以 B 的根为根、以 $F(B_L)$ 为子树森林的树，再加上 $F(B_R)$ 。这两种转换互逆。

6.1.3 树的抽象数据类型

和二叉树一样，树的 ADT 分结点类和树类。对外运算是：用一个值建根、在某结点下插入第一个孩子、在某结点旁插入下一个兄弟、问父结点、删掉一棵子树、以及先根 / 后根 / 层次三种周游。原书【代码 6.1】【代码 6.2】只给出声明。本书把这些运算直接写在 GeneralTree 上，不再另设空基类。

6.1.4 树的周游

先根：先访问结点，再依次周游各孩子子树。后根：先周游各孩子子树，再访问结点。层次：按离根的距离一层一层走，需要队列。对本节的例子：

周游	顺序	本例
先根	结点，再孩子子树	A B E F C D
后根	孩子子树，再结点	E F B C D A
层次	按离根的距离	A B C D E F

递归周游和递归销毁在极深的退化树上会耗尽调用栈，和第 5 章是同一类风险。

6.2 树的链式存储结构

一般树的存储比二叉树麻烦，因为度不固定。常见有四种想法，本章主实现是第四种。

6.2.1 「子结点表」表示方法

每个结点保存一块孩子指针数组（或一张表）。取第 k 个孩子是 $O(1)$ ，按编号随机访问方便。度不固定时数组要扩容，在孩子序列中间插入也要搬指针。空间上，度为 d 的结点就要留 d 个指针槽，稀疏时浪费。本章不把它当主实现。

6.2.2 静态「左子/右兄」表示法

结点数事先已知时，不必每个结点 new 一次，用数组下标代替指针即可：child、sibling 存的是另一个下标，而不是地址。语义与动态版相同，只是存储从堆变成一块连续数组。适合并查集那种「一开始就知道有 n 个元素」的场合。

6.2.3 动态表示法

每个结点单独分配，用指针相连。树的大小事先不知道、会频繁长出新结点时，比静态数组合适。后面的 GeneralTree 就是动态分配。

6.2.4 动态「左子/右兄」表示

这是本章的主实现。任意度的树只需两个指针域：child 指向第一个孩子，sibling 指向下一个兄弟。再加一个 parent 便于向上走。代价是取第 k 个孩子必须沿兄弟链走 k 步。

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::GeneralTree<char> tree;
    tree.create_root('A');
    auto* b = tree.insert_first(tree.root(), 'B');
    auto* c = tree.insert_next(b, 'C');
    tree.insert_next(c, 'D');
    tree.insert_first(b, 'E');
    tree.insert_next(tree.root()->child->child, 'F');

    std::cout << " 先根: ";
    tree.preorder([](char value) { std::cout << value; });
    std::cout << "\n后根: ";
    tree.postorder([](char value) { std::cout << value; });
    std::cout << "\n层次: ";
    tree.breadth_first([](char value) { std::cout << value; });
    std::cout << '\n';

    dsa::DisjointSet sets(5);
    sets.unite(0, 1);
    sets.unite(1, 2);
    sets.unite(3, 4);
}
```

```
std::cout << "0 与 2 同集合: " << (sets.same(0, 2) ? " 是" : " 否") << '\n';
std::cout << "0 与 3 同集合: " << (sets.same(0, 3) ? " 是" : " 否") << '\n';
}
```

在仓库根目录运行:

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch06/general_tree \
    code/ch06/general_tree/demo.cpp -o /tmp/tree-demo
/tmp/tree-demo
```

输出是:

```
先根: ABEFCD
后根: EFBCDA
层次: ABCDEF
0 与 2 同集合: 是
0 与 3 同集合: 否
```

`insert_first(parent, value)` 把新结点插到孩子链的最前面;`insert_next(node, value)` 插到该结点的下一个兄弟。先插入 B, 再 `insert_next(B, C)`、`insert_next(C, D)`, 孩子从左到右就是 B、C、D。

`create_root` 清空旧树并新建根。`insert_first` 先让新结点的 `sibling` 指向原来的第一个孩子, 再把它接到 `parent->child` 上——所以后插入的孩子会出现在兄弟链前端。`delete_subtree` 沿着父的孩子链或森林的根兄弟链找到指向它的指针, 改写成下一个兄弟, 再递归销毁。

孩子-兄弟树:

```
template <typename T>
class GeneralTree {
public:
    struct Node {
        T value;
        Node* child{nullptr};
        Node* sibling{nullptr};
        Node* parent{nullptr};

        explicit Node(const T& value) : value(value) {}
    };

    GeneralTree() = default;

    GeneralTree(const GeneralTree& other) : root_(clone(other.root_, nullptr)) {}

    GeneralTree& operator=(const GeneralTree& other) {
```

```

        if (this != &other) {
            GeneralTree copy(other);
            swap(copy);
        }
        return *this;
    }

GeneralTree(GeneralTree&& other) noexcept : root_(other.release()) {}

GeneralTree& operator=(GeneralTree&& other) noexcept {
    if (this != &other) {
        clear();
        root_ = other.release();
    }
    return *this;
}

~GeneralTree() { clear(); }

void swap(GeneralTree& other) noexcept {
    using std::swap;
    swap(root_, other.root_);
}

[[nodiscard]] Node* root() noexcept { return root_; }
[[nodiscard]] const Node* root() const noexcept { return root_; }

void create_root(const T& value) {
    clear();
    root_ = new Node(value);
}

Node* insert_first(Node* parent, const T& value) {
    if (parent == nullptr) {
        throw std::invalid_argument("parent must not be null");
    }
    Node* node = new Node(value);
    node->sibling = parent->child;
    node->parent = parent;
    parent->child = node;
    return node;
}

Node* insert_next(Node* node, const T& value) {
    if (node == nullptr) {
        throw std::invalid_argument("node must not be null");
    }

```

```

    Node* next = new Node(value);
    next->sibling = node->sibling;
    next->parent = node->parent;
    node->sibling = next;
    return next;
}

[[nodiscard]] Node* parent_of(Node* node) const noexcept {
    return node == nullptr ? nullptr : node->parent;
}

void delete_subtree(Node* node) {
    if (node == nullptr) {
        return;
    }

    Node** link = node->parent == nullptr ? &root_ : &node->parent->child;
    while (*link != nullptr && *link != node) {
        link = &(*link)->sibling;
    }
    if (*link != node) {
        throw std::invalid_argument("node is not part of this tree");
    }

    *link = node->sibling;
    node->sibling = nullptr;
    destroy(node);
}

void clear() noexcept {
    destroy(root_);
    root_ = nullptr;
}

template <class Visitor>
void preorder(Visitor&& visitor) const {
    pre(root_, visitor);
}

template <class Visitor>
void postorder(Visitor&& visitor) const {
    post(root_, visitor);
}

template <class Visitor>
void breadth_first(Visitor&& visitor) const {
    std::vector<Node*> queue;

```

```

    for (Node* node = root_; node != nullptr; node = node->sibling) {
        queue.push_back(node);
    }
    for (std::size_t index = 0; index < queue.size(); ++index) {
        visitor(queue[index]->value);
        for (Node* child = queue[index]->child; child != nullptr;
            child = child->sibling) {
            queue.push_back(child);
        }
    }
}

```

private:

*// Recursive destruction and traversals preserve the textbook presentation.
 // They have a Stack Overflow Risk for a pathologically deep tree.*

```

static void destroy(Node* node) noexcept {
    if (node == nullptr) {
        return;
    }
    destroy(node->child);
    destroy(node->sibling);
    delete node;
}

static Node* clone(const Node* node, Node* parent) {
    if (node == nullptr) {
        return nullptr;
    }
    Node* copy = new Node(node->value);
    copy->parent = parent;
    try {
        copy->child = clone(node->child, copy);
        copy->sibling = clone(node->sibling, parent);
    } catch (...) {
        destroy(copy);
        throw;
    }
    return copy;
}

```

```

template <class Visitor>
static void pre(Node* node, Visitor& visitor) {
    for (; node != nullptr; node = node->sibling) {
        visitor(node->value);
        pre(node->child, visitor);
    }
}

```

```

template <class Visitor>
static void post(Node* node, Visitor& visitor) {
    for (; node != nullptr; node = node->sibling) {
        post(node->child, visitor);
        visitor(node->value);
    }
}

Node* release() noexcept {
    Node* result = root_;
    root_ = nullptr;
    return result;
}

Node* root_{nullptr};
};

```

6.2.5 父指针表示法和并查集

find 在返回根之前把沿途结点的父指针改成根，这就是路径压缩。unite 比较两棵树的秩，把较矮的挂到较高的下面；两棵一样高时，新根的秩加 1。

```

class DisjointSet {
public:
    explicit DisjointSet(std::size_t count) : parent_(count), rank_(count, 0) {
        for (std::size_t index = 0; index < count; ++index) {
            parent_[index] = index;
        }
    }

    std::size_t find(std::size_t index) {
        if (index >= parent_.size()) {
            throw std::out_of_range("disjoint-set index");
        }
        if (parent_[index] != index) {
            parent_[index] = find(parent_[index]);
        }
        return parent_[index];
    }

    bool unite(std::size_t left, std::size_t right) {
        left = find(left);
        right = find(right);
        if (left == right) {
            return false;
        }
    }
};

```

```

    }
    if (rank_[left] < rank_[right]) {
        std::swap(left, right);
    }
    parent_[right] = left;
    if (rank_[left] == rank_[right]) {
        ++rank_[left];
    }
    return true;
}

[[nodiscard]] bool same(std::size_t left, std::size_t right) {
    return find(left) == find(right);
}

private:
    std::vector<std::size_t> parent_;
    std::vector<std::size_t> rank_;
};

```

6.3 树的顺序存储结构

链式存储每个结点单独分配，指针占空间，也不利于整块读写。如果把结点按某一种周游次序排进数组，再用很少的附加信息记下「谁是下一个兄弟」或「有几个孩子」，就能在连续空间里还原整棵树。原书给了四种，思路都对；本章不另写一套未验证的实现，只把还原办法说清楚。

6.3.1 带右链的先根次序表示

按先根次序把结点排进数组。每个结点另存一个「右链」下标，指向它的下一个兄弟；没有下一个兄弟就存空。因为先根次序里，一个结点的孩子们正好排在它后面、直到右链所指的位置之前，所以顺着数组往下走、再靠右链跳过整棵子树，就能还原所有父子和兄弟关系。

6.3.2 带双标记的先根次序表示

右链要用一个整型下标。若改成两个布尔标记——「有没有孩子」「有没有下一个兄弟」——每位只占 1 bit，更省。扫描时用栈：遇到「有孩子」就下降；遇到「没有下一个兄弟」就上升到最近那个还没处理完兄弟的祖先。信息量和带右链相同，只是用栈换空间。

6.3.3 带度数的后根次序表示

按后根次序存放，每个结点记下自己的度数（孩子个数）。后根的特点是：一个结点出现时，它的全部孩子已经作为连续的一段排在它前面。扫描时用栈，每读到一个度数为 d 的结点，就从栈顶弹出 d 棵子树做它的孩子，再把这棵新树压回去。扫完

数组，栈里剩下的就是整棵树（或森林）。

6.3.4 带双标记的层次次序表示

按层次周游的次序存放，标记含义与 6.3.2 相同：有没有孩子、有没有下一个兄弟。因为同一层的兄弟本来就排在一起，一层扫完再扫下一层，用队列就能还原。适合需要按层处理的外部存储。

6.4 K 叉树

有些应用里每个结点的孩子数有固定上限 K ，例如三子棋的博弈树、某些 B 树的内存模拟。这时不必走「任意度 + 兄弟链」，可以规定每个结点至多 K 个孩子，叫做 K 叉树。

满 K 叉树、完全 K 叉树的定义与二叉树平行：满 K 叉树的每个结点要么是叶，要么恰好 K 个孩子；完全 K 叉树只有最下两层的度可以小于 K ，且最下层靠左对齐。按层从 0 编号时，结点 i 的孩子们是 $Ki+1, \dots, Ki+K$ ，父结点是 $\lfloor (i-1)/K \rfloor$ 。 $K=2$ 就回到二叉树。

本章主实现仍是任意度的左子 / 右兄，不单独做一份 K 叉数组。需要固定 K 、又想顺序存放时，用上面的编号公式即可。

本章小结

一般树允许任意多个孩子。森林是若干互不相交的树。树和森林与二叉树可以按「长子—左、兄弟—右」一一转换。链式存储里，左子/右兄用两个指针表示任意度；顺序存储则按某种周游次序排进数组，再靠右链、双标记或度数还原树形。并查集用父指针表示集合，路径压缩和按秩合并让大量操作接近常数。 K 叉树是度有上限的特例。

习题

1. 画出三棵树组成的森林转换成的二叉树，再转换回去，验证互逆。
2. 对图 6.7 那棵树写出先根、后根、层次序列。
3. 度为 2 的有序树和二叉树差在哪里？举一个「左空右不空」的例子。
4. 用带度数的后根次序表示一棵小树，并说明扫描时栈如何弹出孩子。
5. 对元素 0..6 依次 `unite(0,1)`、`unite(1,2)`、`unite(3,4)`，画出父指针，再 `find(0)` 后画出路径压缩的结果。
6. 完全 3 叉树按层编号，写出结点 4 的父和孩子们。

上机题

1. 实现森林与二叉树的互相转换，并用先根序列对拍。
2. 用并查集判断无向图是否连通，并统计连通分量个数。
3. 比较带路径压缩与不带路径压缩的 `find` 在退化链上的时间。

第 7 章 图

图由顶点和边组成。有向边表示单向关系，无向边表示双向关系，带权边表示代价。先弄清题目属于哪一种，再选算法。

源码：图与七个算法、可运行示例、交叉验证测试。

7.1 图的定义和基本术语

图比线性结构和树更一般：结点之间的关系可以是任意的。按第 1 章的分类——线性结构唯一前驱、唯一后继；树唯一前驱、多个后继；图对前驱和后继的个数都不加限制。线性结构和树都可以看成受限的图。

一个典型问题：要在几个城市之间建通信网，使每两个城市都能直接或间接通话，并且总造价尽量低。用顶点代表城市，边代表线路，边旁的数代表造价，问题就变成：在带权图里找一棵连通全部顶点、边权之和最小的树。这就是后面的最小生成树。

图记作 $G = \langle V, E \rangle$ 。 V 是顶点的有穷非空集合； E 是边的集合，每条边是一对顶点。边没有方向时叫无向图，无序对写成 (v_1, v_2) ， (v_1, v_2) 与 (v_2, v_1) 是同一条边。边有方向时叫有向图，有序对写成 $\langle v_1, v_2 \rangle$ ，也叫弧： v_1 是弧尾（起点）， v_2 是弧头（终点）； $\langle v_1, v_2 \rangle$ 与 $\langle v_2, v_1 \rangle$ 是两条不同的弧。

边或弧上可以附带权，表示距离、时间或代价。每条边都带权的图叫带权图。本书不考虑多重边（同一对顶点之间多于一条边）和自环（顶点到自己的边）。

用 n 表示顶点数， e 表示边数。无向图 e 的范围是 $0 \sim n(n-1)/2$ ，有向图是 $0 \sim n(n-1)$ 。边相对少的叫稀疏图，相对多的叫稠密图。任意两个顶点之间都有边的图叫完全图，它取到边数的上限。

若 V' 是 V 的子集， E' 是 E 的子集，且 E' 里的边只连 V' 里的顶点，则 $G' = \langle V', E' \rangle$ 是 G 的子图。一条边所连的两个顶点互为邻接点；这条边与这两个顶点相关联。有向图里还要分清「邻接到」和「邻接自」。

顶点的度是与它相关联的边数。有向图再分成入度（指进来的弧）和出度（指出去的弧）。度为 0 的顶点是孤立点。

从 u 出发、沿边走到 v 的顶点序列叫路径；边数是路径长度。有向图必须顺着弧走。起点和终点相同、且至少含一条边的路径叫回路（环）。除端点外没有重复顶点的路径叫简单路径。

无向图中，若任意两点之间都有路径，称图连通；极大的连通子图叫连通分量。有向图中，若任意两点 u 、 v 都存在 u 到 v 和 v 到 u 的有向路径，称强连通；只要把有向边看成无向边后连通，叫弱连通。

无向连通图的生成树，是包含全部顶点、有 $n-1$ 条边、因而没有环的连通子图。带权连通图里边权之和最小的生成树，就是最小生成树。最短路的「短」是边权总和最小，不一定是经过的边数最少。

选算法之前，先分清题目：

题目	前提	结果
从一个点能走到哪些点	任意图	DFS 或 BFS 的访问序列
课程或任务的可行顺序	有向无环图	拓扑序；有环则无解
一个源点到各点的最短路	边权非负	Dijkstra 的距离数组
任意两点最短路	顶点数较小	Floyd 的距离矩阵
连通所有点且总权最小	无向连通图	Prim 或 Kruskal 的边集

本单元用邻接矩阵。没有边记为 `infinity` (int 最大值的四分之一，给加法留余量)。环、非连通等正常失败用 `optional` 表达。DFS 仍是递归，深图有栈溢出风险。

7.2 图的抽象数据类型

图的对外运算是：指定顶点数构造、加一条边、问有多少个顶点，以及后面各节的周游、拓扑、最短路和最小生成树。原书【代码 7.1】【代码 7.2】的 ADT 声明残缺，标识符还被 OCR 空格切断。本书直接定义在 `Graph` 上。

7.3 图的存储结构

图常用两种存法。邻接矩阵用 $n \times n$ 的表， $A[i][j]$ 表示 i 到 j 有没有边、权是多少，适合稠密图和需要 $O(1)$ 问「两点之间有没有边」的算法 (Floyd、Prim)。邻接表为每个顶点挂一条出边链表，适合稀疏图和只沿出边走的算法 (DFS、BFS、Dijkstra、拓扑)。本章主实现是邻接矩阵；邻接表是同一组运算的另一种存储，不在这里另写一份未验证的实现。

7.3.1 相邻矩阵

邻接矩阵的第 `from` 行、第 `to` 列是边权。构造时对角线为 0 (到自己的距离是 0)，其余为 `infinity`。`add_edge(..., directed=false)` 同时写入对称位置，用来表示无向边。零权边可以表示——原书把 0 同时当成「无边」，那样权为 0 的边就存不进去。

7.4 图的周游

7.4.1 先跑一遍

下面这张有向图：0→1(2)、0→2(7)、1→2(1)、1→3(5)、2→3(1)、3→4(3)。从 0 到 4 的最短路是 0→1→2→3→4，总权 7，不是那条权为 7 的直达边 0→2 再往后走。

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::Graph graph(5);
```

```

graph.add_edge(0, 1, 2);
graph.add_edge(0, 2, 7);
graph.add_edge(1, 2, 1);
graph.add_edge(1, 3, 5);
graph.add_edge(2, 3, 1);
graph.add_edge(3, 4, 3);

std::cout << "DFS(0):";
for (std::size_t vertex : graph.dfs(0)) {
    std::cout << ' ' << vertex;
}
std::cout << "\nBFS(0):";
for (std::size_t vertex : graph.bfs(0)) {
    std::cout << ' ' << vertex;
}

const auto topo = graph.topological_sort();
std::cout << "\n拓扑序:";
if (topo) {
    for (std::size_t vertex : *topo) {
        std::cout << ' ' << vertex;
    }
} else {
    std::cout << " 无 (有环) ";
}

const auto distance = graph.dijkstra(0);
std::cout << "\nDijkstra(0->4) = " << distance[4] << '\n';
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch07/graph \
    code/ch07/graph/demo.cpp -o /tmp/graph-demo
/tmp/graph-demo

```

```

DFS(0): 0 1 2 3 4
BFS(0): 0 1 2 3 4
拓扑序: 0 1 2 3 4
Dijkstra(0->4) = 7

```

若再加上 4→0 形成环，topological_sort() 返回 nullptr。

7.4.2 深度优先与广度优先

DFS 进入一个顶点就立刻标记已访问，再沿编号从小到大的出边递归。BFS 用队列按层扩展，先到的顶点先出队。

```
[[nodiscard]] std::vector<std::size_t> dfs(std::size_t source) const {
    check_vertex(source);
    std::vector<bool> seen(vertices());
    std::vector<std::size_t> result;
    visit_depth_first(source, seen, result);
    return result;
}
```

```
[[nodiscard]] std::vector<std::size_t> bfs(std::size_t source) const {
    check_vertex(source);
    std::vector<bool> seen(vertices());
    std::queue<std::size_t> queue;
    std::vector<std::size_t> result;
    queue.push(source);
    seen[source] = true;
    while (!queue.empty()) {
        const std::size_t from = queue.front();
        queue.pop();
        result.push_back(from);
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity && !seen[to]) {
                seen[to] = true;
                queue.push(to);
            }
        }
    }
    return result;
}
```

7.4.3 拓扑排序

统计每个顶点的入度，把入度为 0 的点入队；每取出一个点，就把它指出的入度减 1。若最终取出的点数少于顶点数，图里有环。

```
[[nodiscard]] std::optional<std::vector<std::size_t>> topological_sort() const {
    std::vector<std::size_t> indegree(vertices());
    for (std::size_t from = 0; from < vertices(); ++from) {
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (from != to && adjacency_[from][to] < infinity) {
                ++indegree[to];
            }
        }
    }
    std::queue<std::size_t> queue;
    for (std::size_t vertex = 0; vertex < vertices(); ++vertex) {

```

```

        if (indegree[vertex] == 0) {
            queue.push(vertex);
        }
    }
    std::vector<std::size_t> result;
    while (!queue.empty()) {
        const std::size_t from = queue.front();
        queue.pop();
        result.push_back(from);
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (from != to && adjacency_[from][to] < infinity && --indegree[to] ==
                ↪ 0) {
                queue.push(to);
            }
        }
    }
    return result.size() == vertices() ?
        ↪ std::optional<std::vector<std::size_t>>(result)
           : std::nullopt;
}

```

7.5 最短路径

7.5.1 单源最短路径

Dijkstra 反复选出尚未确定的、当前距离最小的顶点，用它的出边松弛邻居。边权必须非负。

```

[[nodiscard]] std::vector<int> dijkstra(std::size_t source) const {
    check_vertex(source);
    std::vector<int> distance(vertices(), infinity);
    std::vector<bool> used(vertices());
    distance[source] = 0;
    for (std::size_t count = 0; count < vertices(); ++count) {
        const std::size_t from = nearest_unvisited(distance, used);
        if (from == vertices() || distance[from] == infinity) {
            break;
        }
        used[from] = true;
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity &&
                distance[to] > distance[from] + adjacency_[from][to]) {
                distance[to] = distance[from] + adjacency_[from][to];
            }
        }
    }
}

```

```

    return distance;
}

```

7.5.2 每对顶点之间的最短路径

Floyd 枚举中转点 *via*，比较 *from*→*to* 与 *from*→*via*→*to*。测试里五个源点的 Dijkstra 与 Floyd 逐项对拍。

```

[[nodiscard]] std::vector<std::vector<int>> floyd() const {
    auto distance = adjacency_;
    for (std::size_t via = 0; via < vertices(); ++via) {
        for (std::size_t from = 0; from < vertices(); ++from) {
            for (std::size_t to = 0; to < vertices(); ++to) {
                if (distance[from][via] < infinity && distance[via][to] < infinity) {
                    distance[from][to] = std::min(
                        distance[from][to], distance[from][via] + distance[via][to]);
                }
            }
        }
    }
    return distance;
}

```

7.6 最小生成树

目标不是任意两点最短，而是连通全部顶点且选中边的总权最小。

7.6.1 Prim 算法

从指定源点生长，每次把离当前树最近的顶点加进来。非连通则返回 `nullopt`。

```

[[nodiscard]] std::optional<std::vector<Edge>> prim(std::size_t source) const {
    check_vertex(source);
    std::vector<int> distance(vertices(), infinity);
    std::vector<std::size_t> predecessor(vertices());
    std::vector<bool> used(vertices());
    std::vector<Edge> result;
    distance[source] = 0;
    for (std::size_t count = 0; count < vertices(); ++count) {
        const std::size_t from = nearest_unvisited(distance, used);
        if (from == vertices() || distance[from] == infinity) {
            return std::nullopt;
        }
        used[from] = true;
        if (from != source) {
            result.push_back({predecessor[from], from, distance[from]});
        }
    }
    return result;
}

```

```

    }
    for (std::size_t to = 0; to < vertices(); ++to) {
        if (!used[to] && adjacency_[from][to] < distance[to]) {
            distance[to] = adjacency_[from][to];
            predecessor[to] = from;
        }
    }
}
}
return result;
}

```

7.6.2 Kruskal 算法

把边按权排序，用并查集跳过会形成环的边。连通图上与 Prim 得到相同总权、 $n-1$ 条边。

```

[[nodiscard]] std::optional<std::vector<Edge>> kruskal() const {
    std::vector<Edge> edges;
    for (std::size_t from = 0; from < vertices(); ++from) {
        for (std::size_t to = from + 1; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity) {
                edges.push_back({from, to, adjacency_[from][to]});
            }
        }
    }
    std::sort(edges.begin(), edges.end());
    std::vector<std::size_t> parent(vertices());
    for (std::size_t vertex = 0; vertex < vertices(); ++vertex) {
        parent[vertex] = vertex;
    }
    std::vector<Edge> result;
    for (const Edge& edge : edges) {
        const std::size_t from_root = find_root(parent, edge.from);
        const std::size_t to_root = find_root(parent, edge.to);
        if (from_root != to_root) {
            parent[from_root] = to_root;
            result.push_back(edge);
        }
    }
    return result.size() + 1 == vertices() ?
        ↪ std::optional<std::vector<Edge>>(result)
        : std::nullopt;
}

```

本章小结

图对前驱和后继的个数都不加限制。边可以无向或有向、可以带权。存储常用邻接矩阵（稠密、问边 $O(1)$ ）和邻接表（稀疏、沿出边走）。周游有深度优先和广度优先。有向无环图可以拓扑排序；有环则无解。非负权单源最短路用 Dijkstra，任意两点用 Floyd。无向连通图的最小生成树可用 Prim 或 Kruskal，二者总权相同。

习题

1. 分别画出 4 个顶点的无向完全图和有向完全图，并写出边数公式。
2. 对本章 demo 那张有向图，写出从 0 出发的一种 DFS 序列和一种 BFS 序列。
3. 给一个有环的有向图，说明拓扑排序如何发现环。
4. 在边权非负的图上用手算 Dijkstra，列出每一步选出的顶点和距离数组。
5. 说明为什么负权边不能直接用 Dijkstra。
6. 对同一张无向带权连通图分别做 Prim 和 Kruskal，验证边集可以不同、总权必须相同。
7. 邻接矩阵和邻接表各适合什么图、什么运算？给出空间代价。

上机题

1. 读入一个有向图，输出一种拓扑序；有环则报告。
2. 实现 Dijkstra，输出源点到各点的距离和一条最短路径。
3. 实现 Prim 与 Kruskal，对同一组随机连通图比较总权。
4. 用邻接表重做 DFS/BFS，并与邻接矩阵版对拍访问序列。

第 8 章 内部排序

排序把一个序列重排为非递减顺序。阅读时同时问四个问题：比较还是分配？是否稳定？额外空间多少？最好、平均、最坏时间是多少？

源码：全部手写排序、可运行示例、对拍测试。

8.1 排序问题的基本概念

排序把一个序列重排，使关键码按非递减（或非递增）次序排列。内部排序假定数据能全部放进内存；数据必须在外存上多趟进出的，是第 9 章的外排序。

读每一种方法时同时问四件事：它靠元素之间比较，还是靠关键码的数字结构做分配？相等的键排完之后相对次序变不变（稳定）？除了原序列还要多少额外空间？最好、平均、最坏各是什么时间？「稳定」不是装饰：按成绩排序时，两名同分学生若仍按原来的姓名顺序出现，后续按姓名再排才有意义。

比较排序在最坏情况下至少需要 $\Omega(n \log n)$ 次比较，这是判定树给出的下界。插入、选择、冒泡是 $O(n^2)$ ；堆和归并最坏也是 $O(n \log n)$ ；快排平均 $O(n \log n)$ ，已经有序时退化成 $O(n^2)$ 。计数和基数不是比较排序，代价取决于值域或位数。

本章所有实现接受有符号整数，测试含重复和负数。没有用 `std::sort / std::make_heap` 替代手写算法。

方法	平均时间	稳定性	主要特点
直接插入	$O(n^2)$	稳定	小规模、近乎有序时简单有效
冒泡	$O(n^2)$	稳定	教学直观，通常不作为通用选择
选择	$O(n^2)$	不稳定	交换次数少
堆排序	$O(n \log n)$	不稳定	最坏情况也有保证，原地排序
快速排序	$O(n \log n)$ 平均	不稳定	实务常用，坏划分时会退化
归并排序	$O(n \log n)$	稳定	需要 $O(n)$ 辅助空间
计数/基数	与值域或位数有关	可稳定	适合整数键，不是比较排序

「稳定」指相等键在排序前后的相对顺序不变。例如记录按成绩排序时，两名同分学生仍按原来的姓名顺序出现。本章所有实现接受有符号整数，测试含重复和负数。没有用 `std::sort / std::make_heap` 替代手写算法。

8.2 插入排序

8.2.1 先跑一遍

```
#include "modern.hpp"

#include <iostream>
```

```

#include <vector>

namespace {
void print(const char* name, const std::vector<int>& values) {
    std::cout << name;
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << '\n';
}
} // namespace

int main() {
    const std::vector<int> raw{3, -2, 7, 3, 0, -2, 9, 1};

    auto insertion = raw;
    dsa::sorting::insertion_sort(insertion);
    print(" 插入:", insertion);

    auto heap = raw;
    dsa::sorting::heap_sort(heap);
    print(" 堆排:", heap);

    auto quick = raw;
    dsa::sorting::quick_sort(quick);
    print(" 快排:", quick);

    auto radix = raw;
    dsa::sorting::radix_sort(radix);
    print(" 基数:", radix);
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch08/sorting \
    code/ch08/sorting/demo.cpp -o /tmp/sort-demo
/tmp/sort-demo

```

```

插入: -2 -2 0 1 3 3 7 9
堆排: -2 -2 0 1 3 3 7 9
快排: -2 -2 0 1 3 3 7 9
基数: -2 -2 0 1 3 3 7 9

```

四种算法交出同一条非递减序列。插入排序保持两个 -2、两个 3 的相对次序；堆排和快排不保证这一点。

直接插入把 `values[index]` 抽出来，向前挪动所有比它大的元素，把空位留给

它。相等元素不越过彼此，所以稳定。

```
// 算法 8.1: 直接插入排序。相等元素不越过彼此，故稳定。
inline void insertion_sort(std::vector<int>& values) {
    for (std::size_t index = 1; index < values.size(); ++index) {
        const int value = values[index];
        std::size_t hole = index;
        while (hole != 0 && value < values[hole - 1]) {
            values[hole] = values[hole - 1];
            --hole;
        }
        values[hole] = value;
    }
}
```

8.2.2 Shell 排序

增量每次减半的插入排序。不稳定，平均优于直接插入。实现见同一源文件中的 `shell_sort`。

8.3 选择排序

8.3.2 堆排序

先把数组建成最大堆，再反复把堆顶与堆尾交换并缩小堆。`sift_down` 必须比较左右两个孩子。

```
// 算法 8.4: 手写最大堆筛选与堆排序，不委托 std::make_heap/sort_heap。
inline void sift_down(std::vector<int>& values, std::size_t root, std::size_t
↪ count) {
    while (root * 2 + 1 < count) {
        std::size_t child = root * 2 + 1;
        if (child + 1 < count && values[child] < values[child + 1]) ++child;
        if (values[root] >= values[child]) return;
        using std::swap;
        swap(values[root], values[child]);
        root = child;
    }
}

inline void heap_sort(std::vector<int>& values) {
    for (std::size_t root = values.size() / 2; root != 0; --root) {
        sift_down(values, root - 1, values.size());
    }
    for (std::size_t end = values.size(); end > 1; --end) {
        using std::swap;
        swap(values[0], values[end - 1]);
    }
}
```

```

        sift_down(values, 0, end - 1);
    }
}

```

8.4 交换排序

8.4.2 快速排序

选区间末元素为枢轴，把更小的元素换到左侧，再递归两边。全相等的输入也必须也能结束。

```

inline std::size_t partition(std::vector<int>& values, std::size_t first,
    ↪ std::size_t last) {
    const int pivot = values[last - 1];
    std::size_t boundary = first;
    for (std::size_t index = first; index + 1 < last; ++index) {
        if (values[index] < pivot) {
            using std::swap;
            swap(values[boundary], values[index]);
            ++boundary;
        }
    }
    using std::swap;
    swap(values[boundary], values[last - 1]);
    return boundary;
}

```

```

inline void quick_sort_range(std::vector<int>& values, std::size_t first,
    ↪ std::size_t last) {
    if (last - first < 2) return;
    const std::size_t middle = partition(values, first, last);
    quick_sort_range(values, first, middle);
    quick_sort_range(values, middle + 1, last);
}

```

// 算法 8.6: 手写快排。

```

inline void quick_sort(std::vector<int>& values) { quick_sort_range(values, 0,
    ↪ values.size()); }

```

8.5 归并排序

稳定，需要 $O(n)$ 辅助空间。实现见 merge_sort。

8.6 分配排序和索引排序

8.6.2 基数排序

把有符号 `int` 的符号位翻转后，按字节做 4 趟计数收集。否则负数会按无符号序排列到最大。

```
// 算法 8.11: LSD 基数排序。翻转符号位使二补码有符号 int 按无符号序排序。
inline void radix_sort(std::vector<int>& values) {
    std::vector<int> buffer(values.size());
    for (unsigned shift = 0; shift < 32; shift += 8) {
        std::size_t counts[256]{};
        for (int value : values) {
            const auto key = static_cast<std::uint32_t>(value) ^ 0x80000000U;
            ++counts[(key >> shift) & 0xffU];
        }
        std::size_t offset = 0;
        for (std::size_t& count : counts) { const std::size_t old = count; count =
            ↪ offset; offset += old; }
        for (int value : values) {
            const auto key = static_cast<std::uint32_t>(value) ^ 0x80000000U;
            buffer[counts[(key >> shift) & 0xffU]++] = value;
        }
        values.swap(buffer);
    }
}
```

8.7 排序算法的时间代价

比较排序在最坏情况下至少 $\Omega(n \log n)$ 次比较。插入、选择、冒泡是 $O(n^2)$ ；堆、归并最坏也是 $O(n \log n)$ ；快排平均 $O(n \log n)$ 、最坏 $O(n^2)$ 。计数和基数不是比较排序，代价取决于值域或位数。

本章小结

内部排序假定数据能全部放进内存。比较排序有插入、选择、交换、归并、堆几条路线；分配排序（计数、基数）不靠元素之间比较。稳定与否、额外空间、最好/平均/最坏时间，是选择算法时要同时看的四件事。比较排序最坏至少 $\Omega(n \log n)$ 次比较。本章所有算法都是手写实现，不用标准库排序替代。

习题

1. 对序列 {49, 38, 65, 97, 76, 13, 27} 分别画出直接插入、冒泡、简单选择的第一趟结果。
2. 说明为什么直接插入稳定、堆排序不稳定。举一个使堆排序打乱相等键次序的例子。

3. 写出快速排序一次划分的过程，并指出最坏输入。
4. 对 8 个元素用手算堆排序：先建大顶堆，再写出每一趟交换堆顶之后的数组。
5. 二路归并需要多少额外空间？能否原地归并？代价是什么。
6. 基数排序为什么对有符号整数要先处理符号位。
7. 在什么情况下你会选插入而不是快排？选堆而不是归并？

上机题

1. 用同一组随机数据比较插入、堆、快排、归并的运行时间，画出 n 与时间的关系。
2. 构造使快排退化的输入，观察递归深度。
3. 实现带监视哨的插入排序，并与本章实现对拍。
4. 对含负数的整数序列验证基数排序与 `std::sort` 结果一致。

第 9 章 文件管理与外部排序

当数据大到内存装不下时，排序的瓶颈变成磁盘读写。外部排序先生成有序顺串，再进行多路归并。置换选择用最小堆尽可能延长当前顺串；竞赛树维护多路输入中当前最小的选手。

源码：置换选择与竞赛树、可运行示例、测试。

9.1 主存储器和外存储器

前面各章的结构基本上都在内存里，也叫内部数据结构。内存容量有限，程序一结束数据也就没了。大规模数据必须放到外存上，排序过程也就变成多次内存与外存之间的交换。

主存（RAM、cache、显存）在主板上，按单元连续编址，CPU 直接访问，一次存取时间可以看成很小的常数，单位是纳秒。它快、贵、容量相对小，断电就丢。外存（硬盘、磁带、U 盘）便宜、容量大，信息断电不丢，有的还能随身带走。但一次存取以毫秒甚至秒计，比内存慢几个数量级。

外存慢，一个原因是每次访问都要先定位再读写。磁盘要把磁头移到目标磁道，再等扇区转到磁头下，定位往往就要几毫秒到几十毫秒，远慢于真正把数据读出来。所以外存按固定大小的页块存取：一次定位读写一整页，减少定位次数。顺序扫描时再配合缓冲：一次读入一页或几页到内存，后面的访问尽量打在缓冲区里。

对外排序来说，目标首先是减少读写次数，而不是减少内存里的比较。一次磁盘 I/O 往往比一次比较贵几个数量级。

9.2 文件的组织和管理

文件是外存上的数据结构，由大量性质相同的记录组成。记录是有独立逻辑意义的一块数据，简单可以是一串字符，复杂则由若干字段组成。操作系统文件常常是连续字符流，结构不明显；数据库文件是有结构的记录集合，每条记录由若干不可再分的数据项组成。学生登记表——姓名、学号、性别、出生年月——就是后一种。

按记录长度，文件分定长和不定长：定长更好处理。按关键码个数，分单关键码和多关键码：多关键码文件除了主码还可以有若干次码。操作通常以记录为单位：顺序读、追加、按条件修改或删除。处理方式有实时（要求很快应答）和批量（允许较长反馈）。

用户看见的是逻辑文件：顺序定长、顺序变长、或按关键码存取。系统实现的是物理文件，常见几种：

1. 顺序文件：记录按逻辑次序放进连续物理块，物理顺序与逻辑顺序一致。顺序扫描很快，按关键码插入、删除要搬很多块。
2. 索引文件：主文件之外另造索引，先查索引再读记录。第 11 章专门讨论。
3. 散列文件：用散列函数把关键码映射到桶或块。第 10 章的闭散列思想可以搬到外存，但冲突处理要按块设计。

本章不实现页缓存和文件句柄，只抽出外排序里两件与文件组织无关、却决定 I/O 次数的事：如何生成更长的初始顺串，以及如何在 k 路归并里选出当前最小。

9.3 外排序

9.3.1 置换选择排序

顺串是磁盘上已经有序的一段记录。内存一次只能装 M 条时，朴素做法是每批排成一条长为 M 的顺串。置换选择可以做得更好：输出堆顶之后，若下一条记录不小于刚输出的值，它还可以进入当前顺串；否则冻结到下一趟。平均情况下第一趟长度约为 $2M$ 。

原书图 9.2 的输入是 50 49 35 45 30 25 15 60 16 27 1，工作区 $M = 7$ 。前 7 个建成最小堆后，堆顶是 15。接着读到 60——它比 15 大，可以进当前堆；再读到 16、27、1，它们都比当时的输出值小，被冻结。第一顺串因此是

```
15 25 30 35 45 49 50 60
```

长度 8，已经超过 M 。剩下的 1 16 27 构成第二顺串。

图 9.1 置换选择：工作区是最小堆。弹出堆顶后，下一条记录若不小于刚输出的值就入堆，否则冻结到下一趟。

若把整份输入推进一个堆再依次弹出，得到的是一条完全有序序列——那是堆排序，不是置换选择。旧实现曾经这样做，测试只断言 $\{3, 1, 2\} \rightarrow \{1, 2, 3\}$ ，堆排序也能过。现在的接口返回若干顺串，并用原书这组数据守门。

多路归并时，每次要在 k 路的队首里选出最小者。赢者树的内部结点保存胜者下标；败者树的内部结点保存败者，另用一个冠军槽记录全局最小。替换一名选手后，两者都只需沿叶到根重赛。

先跑一遍：

```
#include "modern.hpp"

#include <iostream>

int main() {
    const std::vector<int> input{50, 49, 35, 45, 30, 25, 15, 60, 16, 27, 1};
    const auto runs = dsa::external_sort::replacement_selection(input, 7);

    std::cout << " 工作区 M=7, 得到 " << runs.size() << " 个顺串\n";
    for (std::size_t index = 0; index < runs.size(); ++index) {
        std::cout << " 顺串 " << index + 1 << " (长度 " << runs[index].size() << " ) : ";
        for (int value : runs[index]) {
            std::cout << ' ' << value;
        }
        std::cout << '\n';
    }
}
```



```

dsa::external_sort::LoserTree tree({20, 6, 8, 9, 11});
std::cout << " 败者树当前冠军: " << *tree.winner() << '\n';
tree.replace(1, 15);
std::cout << " 替换后冠军: " << *tree.winner() << '\n';
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch09/external_sort \
    code/ch09/external_sort/demo.cpp -o /tmp/extsort-demo
/tmp/extsort-demo

```

工作区 M=7, 得到 2 个顺串
 顺串 1 (长度 8): 15 25 30 35 45 49 50 60
 顺串 2 (长度 3): 1 16 27
 败者树当前冠军: 6
 替换后冠军: 8

把 M 改成 11 (装得下全部输入), 会只得到一个顺串, 内容是整表有序——这时置换选择退化为堆排序, 正是「内存够用」的边界。

replacement_selection(input, memory) 先读入至多 memory 条建成最小堆。循环中弹出堆顶写入当前顺串; 若还有输入, 按「incoming >= emitted 则入堆, 否则冻结」分流。当前堆空了, 就把冻结区建成新堆, 开始下一趟。

```

// 算法 9.1: 置换选择。memory 是内存工作区能容纳的记录数 M。
// 返回若干顺串: 每个顺串内部有序, 第一趟的平均长度约为 2M, 而不是 M。
// 不属于当前顺串的新记录被冻结, 等当前堆耗尽后再建下一趟。
inline std::vector<std::vector<int>> replacement_selection(const std::vector<int>&
↳ input,
                                                                std::size_t memory) {
    if (memory == 0) {
        throw std::invalid_argument("replacement selection memory must be positive");
    }
    if (input.empty()) {
        return {};
    }

    std::size_t next = 0;
    std::vector<int> heap;
    heap.reserve(memory);
    while (next < input.size() && heap.size() < memory) {
        heap.push_back(input[next++]);
    }
    detail::heap_from(heap);
}

```

```

std::vector<std::vector<int>> runs;
std::vector<int> current_run;
std::vector<int> frozen;

while (!heap.empty() || next < input.size() || !frozen.empty()) {
    if (heap.empty()) {
        if (!current_run.empty()) {
            runs.push_back(std::move(current_run));
            current_run = {};
        }
        heap = std::move(frozen);
        frozen = {};
        while (next < input.size() && heap.size() < memory) {
            heap.push_back(input[next++]);
        }
        detail::heap_from(heap);
        if (heap.empty()) {
            break;
        }
    }

    const int emitted = detail::heap_pop(heap);
    current_run.push_back(emitted);

    if (next == input.size()) {
        continue;
    }
    const int incoming = input[next++];
    if (incoming >= emitted) {
        detail::heap_push(heap, incoming);
    } else {
        frozen.push_back(incoming);
    }
}
if (!current_run.empty()) {
    runs.push_back(std::move(current_run));
}
return runs;
}

```

9.3.2 二路外排序

对 m 个顺串两两归并，趟数是 $\lceil \log_2 m \rceil$ 。置换选择把初始顺串变长，就是为了减小 m 。

9.3.3 多路归并——选择树

赢者树用完全二叉数组：叶存放选手下标，内部结点写 better(左，右)。败者树在内部结点写败者，另用 champion_ 记全局胜者。替换后只沿叶到根重赛。

// 代码 9.2: 赢者树。内部结点保存两名选手比较后的胜者下标，根是全局最小。

```
class WinnerTree {
public:
    explicit WinnerTree(std::vector<int> players) : players_(std::move(players)) {
        if (players_.empty()) {
            return;
        }
        leaf_base_ = detail::TournamentOps::next_power_of_two(players_.size());
        tree_.assign(leaf_base_ * 2, detail::TournamentOps::no_player);
        for (std::size_t index = 0; index < players_.size(); ++index) {
            tree_[leaf_base_ + index] = index;
        }
        for (std::size_t node = leaf_base_ - 1; node > 0; --node) {
            tree_[node] = detail::TournamentOps::better(players_, tree_[node * 2],
                                                         tree_[node * 2 + 1]);
        }
    }

    [[nodiscard]] std::optional<std::size_t> winner_index() const {
        if (players_.empty() || tree_[1] == detail::TournamentOps::no_player) {
            return std::nullopt;
        }
        return tree_[1];
    }

    [[nodiscard]] std::optional<int> winner() const {
        const auto index = winner_index();
        return index ? std::optional<int>(players_[*index]) : std::nullopt;
    }

    void replace(std::size_t player, int value) {
        if (player >= players_.size()) {
            throw std::out_of_range("tournament player");
        }
        players_[player] = value;
        std::size_t node = leaf_base_ + player;
        tree_[node] = player;
        while (node > 1) {
            node /= 2;
            tree_[node] = detail::TournamentOps::better(players_, tree_[node * 2],
                                                         tree_[node * 2 + 1]);
        }
    }
}
```

```
private:
    std::vector<int> players_;
    std::vector<std::size_t> tree_;
    std::size_t leaf_base_{0};
};
```

// 代码 9.3: 败者树。内部结点保存败者下标, 另用 *champion_* 记录全局胜者。
 // 替换一名选手时只需沿叶到根重赛, 不必访问兄弟子树的内部结构。

```
class LoserTree {
public:
    explicit LoserTree(std::vector<int> players) : players_(std::move(players)) {
        if (players_.empty()) {
            return;
        }
        leaf_base_ = detail::TournamentOps::next_power_of_two(players_.size());
        loser_.assign(leaf_base_, detail::TournamentOps::no_player);
        subtree_winner_.assign(leaf_base_ * 2, detail::TournamentOps::no_player);
        for (std::size_t index = 0; index < players_.size(); ++index) {
            subtree_winner_[leaf_base_ + index] = index;
        }
        for (std::size_t node = leaf_base_ - 1; node > 0; --node) {
            replay_node(node);
        }
        champion_ = subtree_winner_[1];
    }

    [[nodiscard]] std::optional<std::size_t> winner_index() const {
        if (players_.empty() || champion_ == detail::TournamentOps::no_player) {
            return std::nullopt;
        }
        return champion_;
    }

    [[nodiscard]] std::optional<int> winner() const {
        const auto index = winner_index();
        return index ? std::optional<int>(players_[*index]) : std::nullopt;
    }

    [[nodiscard]] std::optional<std::size_t> loser_at(std::size_t node) const {
        if (node == 0 || node >= loser_.size() ||
            loser_[node] == detail::TournamentOps::no_player) {
            return std::nullopt;
        }
        return loser_[node];
    }
};
```

```

void replace(std::size_t player, int value) {
    if (player >= players_.size()) {
        throw std::out_of_range("tournament player");
    }
    players_[player] = value;
    subtree_winner_[leaf_base_ + player] = player;
    for (std::size_t node = (leaf_base_ + player) / 2; node > 0; node /= 2) {
        replay_node(node);
    }
    champion_ = subtree_winner_[1];
}

private:
void replay_node(std::size_t node) {
    const std::size_t left = subtree_winner_[node * 2];
    const std::size_t right = subtree_winner_[node * 2 + 1];
    loser_[node] = detail::TournamentOps::worse(players_, left, right);
    subtree_winner_[node] = detail::TournamentOps::better(players_, left,
        ⇐ right);
}

std::vector<int> players_;
std::vector<std::size_t> loser_;
std::vector<std::size_t> subtree_winner_;
std::size_t leaf_base_{0};
std::size_t champion_{detail::TournamentOps::no_player};
};

```

本章小结

外存按页存取，一次定位比一次比较贵几个数量级，所以外排序首先要减少读写次数。文件是外存上的记录集合，逻辑组织与物理组织（顺序、索引、散列）要分开看。外排序先生成初始顺串再多路归并。置换选择用堆把不属于当前顺串的记录冻结，第一趟平均长度约 $2M$ ，而不是 M 。 k 路归并用赢者树或败者树在 $O(\log k)$ 时间内选出当前最小。

习题

1. 说明为什么外存要按页读写，以及缓冲如何减少定位次数。
2. 对输入 50 49 35 45 30 25 15 60 16 27 1、 $M = 7$ ，写出置换选择的两个顺串，并指出哪些键被冻结。
3. 若 M 大到能装下全部输入，置换选择退化成什么。
4. 赢者树和败者树的内部结点各记什么？替换一名选手后为什么只需沿叶到根重赛。

5. m 个顺串做二路归并要多少趟? 置换选择怎样减少 m 。

上机题

1. 实现置换选择, 用原书图 9.2 的输入做守门测试。
2. 实现赢者树, 随机替换选手并与每次扫描 k 路的朴素选最小对拍。
3. 模拟 k 路归并: 输入是若干已排序向量, 用败者树输出完整有序序列。
4. 比较 $M = 4, 8, 16$ 时置换选择产生的顺串个数。

第 10 章 检索

检索的目标是在一组记录中找到目标键。顺序检索不要求有序；二分检索要求有序，靠每次排除一半区间加速；散列表用散列函数直接定位槽位，冲突时继续探测。

源码：检索、集合和散列表、可运行示例、墓碑探测测试。

检索是在一组记录里定位关键码等于给定值的那一条，或属性满足条件的那些条。成功就是至少找到一条，失败就是没有。精确匹配查单个值，范围查询查一个区间。

平均检索长度 $ASL = \sum P_i C_i$ ，其中 P_i 是查到第 i 个元素的概率， C_i 是比较次数。衡量算法还要看额外空间和实现复杂度。

可以把检索分成四类：基于线性表、按关键码直接访问（含散列）、树形索引、基于属性（倒排）。本章做前两类；树形索引见第 5、11、12 章，倒排见第 11 章。

10.1 基于线性表的检索

数据放在数组或链表里，按给定值 K 与表中元素比较，直到命中或能确定不在表中。这一节不要求散列函数，也不建树，只靠线性结构本身。

10.1.1 顺序检索

从表头逐个比到表尾。表可以无序，实现最简单，链表和数组都能用。最好情况目标在第一个位置，比较 1 次；最坏情况目标不在表里，比较 n 次；等概率时平均检索长度 $ASL = (n + 1)/2$ 。有时在表尾放一个与 K 相等的「监视哨」，循环里就不必每次都判断下标有没有出界，比较次数的常数会小一点，阶不变。

10.1.2 二分检索

表必须按关键码有序，并且能按下标随机访问，所以适合数组，不适合链表。每次取当前区间的中点：相等则停；目标更大则丢掉左半，否则丢掉右半。区间长度每次至少减半，比较次数是 $O(\log n)$ 。

实现时用半开区间 $[first, last)$ ：中点是 $first + (last - first) / 2$ ，走左半时 $last = middle$ ，走右半时 $first = middle + 1$ 。这样不必写 $middle - 1$ ，无符号下标不会在 $middle == 0$ 时下溢。循环条件是 $first < last$ ；区间空了还没找到，就返回空。

10.1.3 分块检索

把 n 个元素分成 b 块。块与块之间有序（后一块的最小关键码不小于前一块的最大关键码），块内部可以无序。另造一张块索引，每项记下该块的最大（或最小）关键码和块的起始位置。检索时先在索引上顺序或二分，确定目标可能在哪一块，再在那一块里顺序查。

块数选 $\sqrt{n}$ 量级时，索引和块内的平均比较次数比较均衡， ASL 大约是 $O(\sqrt{n})$ ，介于顺序的 $O(n)$ 和二分的 $O(\log n)$ 之间。它适合「主表很大、不能整表排序，但

可以按块组织」的场合。本章不另写未验证的分块实现。

```
// 算法 10.2: 无需修改输入容器的顺序检索。
inline std::optional<std::size_t> sequential_search(const std::vector<int>& values,
    ↪ int key) {
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (values[index] == key) return index;
    }
    return std::nullopt;
}

// 算法 10.3: 半开区间避免 `mid - 1` 在无符号下标下溢。
inline std::optional<std::size_t> binary_search(const std::vector<int>&
    ↪ sorted_values, int key) {
    std::size_t first = 0;
    std::size_t last = sorted_values.size();
    while (first < last) {
        const std::size_t middle = first + (last - first) / 2;
        if (sorted_values[middle] == key) return middle;
        if (sorted_values[middle] < key) first = middle + 1;
        else last = middle;
    }
    return std::nullopt;
}
```

10.2 集合的检索

检索也可以用来实现集合。集合只关心一个元素在不在里面，不保留重复，也不保证顺序。交、并、差、包含都可以建立在「是否属于」之上。

10.2.1 集合的数学特性

集合的元素互异：同一个值不能出现两次。「 x 属于 S 」是一个命题，不是一个位置。所以插入一个已经在里面的键、删除一个不在里面的键，都是正常、可预期的失败，应当返回「没做成」，而不是抛异常或打印一行。空集上的任何包含询问都是假。

10.2.2 计算机中的集合

同一组集合运算，底下可以用不同结构：线性表加顺序检索（实现简单，适合很小的集合）、有序表加二分、二叉搜索树、散列表。规模上去以后，线性表的 $O(n)$ 检索会拖垮交和并。本章的 IntSet 故意用线性表，把接口先钉死：insert / erase 返回 bool，contains 只回答在不在，intersection 和 includes 用检索组合出来。换成散列表时，调用方不用改。

```
// 代码 10.4、算法 10.5-10.7: 不重复整数集合。
class IntSet {
```



```

public:
    [[nodiscard]] bool insert(int value) {
        if (contains(value)) return false;
        values_.push_back(value);
        return true;
    }
    [[nodiscard]] bool erase(int value) {
        const auto found = sequential_search(values_, value);
        if (!found) return false;
        values_.erase(values_.begin() + static_cast<std::ptrdiff_t>(*found));
        return true;
    }
    [[nodiscard]] bool contains(int value) const { return sequential_search(values_,
        ↪ value).has_value(); }
    [[nodiscard]] IntSet intersection(const IntSet& other) const {
        IntSet result;
        for (int value : values_) if (other.contains(value))
            ↪ (void)result.insert(value);
        return result;
    }
    [[nodiscard]] bool includes(const IntSet& other) const {
        for (int value : other.values_) if (!contains(value)) return false;
        return true;
    }
    [[nodiscard]] std::size_t size() const noexcept { return values_.size(); }
private:
    std::vector<int> values_;
};

```

10.3 散列方法

前面的检索都要和表里的元素比较，至少看 $\log n$ 或 $\sqrt{n}$ 个。散列换一条路：用函数从关键码直接算出槽位下标，期望一次就能到。它不保持键的次序，不能按范围遍历；装载因子过高或散列不均匀时，冲突变多，会退化成接近线性。

10.3.1 散列函数

散列函数 h 把关键码映到 $\{0, 1, \dots, m-1\}$ 。它应当算得快，并且尽量把地址铺匀，否则很多键挤在同一段，再好的冲突处理也救不了。常见做法有：除留余数 ($h(k) = k \bmod m$, m 最好是素数)、折叠、平方取中、以及按字节处理的字符串散列。

本章实现了 ELFHash：每个字节先左移 4 位再加进去，高 4 位若非零就折回到低位并清掉。字节按 unsigned char 读，避免 char 在有的平台上是有符号的、高位 1 被当成负数。它不是密码学哈希，只求快和够匀。

```
// 算法 10.8: ELFhash, 逐字节处理, 不把 char 的符号性带入散列。
inline std::size_t elf_hash(const std::string& text) noexcept {
    std::size_t hash = 0;
    for (unsigned char character : text) {
        hash = (hash << 4U) + character;
        const std::size_t high_bits = hash & 0xF0000000U;
        if (high_bits != 0) hash ^= high_bits >> 24U;
        hash &= ~high_bits;
    }
    return hash;
}
```

10.3.2 开散列方法（拉链法）

开散列不在表内挤位置：每个槽挂一条链表（或动态数组），散列到同一地址的关键码都串在这条链上。插入是算地址再接到链头或链尾；查找、删除只在那一条链上走。删掉一个结点不会影响别的链，也不存在「探测序列被截断」的问题。

装载因子 $\alpha = n/m$ 可以大于 1，链只是变长。成功查找的期望比较次数大约是 $1 + \alpha/2$ 。实现简单，空间上每个结点多一个指针。本章主实现是闭散列，开散列不另写一份。

10.3.3 闭散列方法（开地址法）

闭散列把冲突消化在表内：回家地址被占了，就按某种规则探测下一个空槽。线性探测是最简单的一种——第 i 次看 $(h(k) + i) \bmod m$ 。二次探测、双重散列是为了减轻「挤成一团」的一次聚集。

表满了就插不进去，所以 α 必须小于 1，实务上常在 0.5 到 0.7 之间扩表。删除是闭散列的难点：不能把槽直接标成空。否则沿这条探测链查找后面的键时，会在空槽处误判「从来没插入过」而提前停止。

假设表长为 5，键 1、6、11 的散列地址都是 1，它们依次放在槽 1、2、3。删除 1 后若把槽 1 标空，再查 6 就会从槽 1 出发立刻停。正确做法是留下墓碑：查找时墓碑当作「继续往前」；插入时可以复用沿途第一个墓碑，但必须先走完、确认后面没有同一个键。

假设表长为 5，键 1、6、11 的散列地址都为 1，于是它们依次放在槽 1、2、3：

槽：	0	1	2	3	4
	空	1	6	11	空

删除键 1 后，若把槽 1 直接标为空，查找键 6 从槽 1 开始会误以为「从未插入过 6」并提前停止。墓碑表示「这里曾有元素，但探测链还要继续」。插入新键时可以复用第一个墓碑，但查重必须继续走到真正的空槽或找到同键为止。

图 10.5 就是上面这张槽位表。

散列表追求平均 $O(1)$ 查找，前提是装载因子不过高且散列分布均匀；它不保持键的有序关系，不适合按范围遍历。

10.3.4 闭散列表的算法

先跑一遍：

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::search::HashTable table(5);
    if (!table.insert(1) || !table.insert(6) || !table.insert(11)) {
        std::cout << " 插入 1、6、11 失败\n";
        return 1;
    }

    std::cout << " 插入 1、6、11 后:\n";
    for (std::size_t index = 0; index < table.capacity(); ++index) {
        const auto slot = table.slot_at(index);
        std::cout << " 槽 " << index << ": ";
        if (slot.state == dsa::search::HashTable::SlotState::used) {
            std::cout << slot.key << '\n';
        } else if (slot.state == dsa::search::HashTable::SlotState::tombstone) {
            std::cout << " 墓碑\n";
        } else {
            std::cout << " 空\n";
        }
    }

    if (!table.erase(1)) {
        std::cout << " 删除 1 失败\n";
        return 1;
    }
    std::cout << " 删除 1 后仍能找到 6? " << (table.contains(6) ? " 是" : " 否") <<
        '\n';
    if (!table.insert(16)) {
        std::cout << " 插入 16 失败\n";
        return 1;
    }
    std::cout << " 插入 16 后槽 1 是 "
        << table.slot_at(1).key
        << " (复用了墓碑) \n";
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch10/search_hash \
    code/ch10/search_hash/demo.cpp -o /tmp/hash-demo
/tmp/hash-demo
```

插入 1、6、11 后：

槽 0：空

槽 1：1

槽 2：6

槽 3：11

槽 4：空

删除 1 后仍能找到 6？ 是

插入 16 后槽 1 是 16（复用了墓碑）

若把删除写成「标空」而不是「标墓碑」，第二行会变成「否」——测试里有两条具名断言守住这件事。

HashTable::home 先取绝对值再取模，所以负键也能进表。find_slot 遇到 empty 就停，遇到 tombstone 继续；insertion_slot 记下沿途第一个墓碑，确认后方没有同键后复用它。按键删除返回 bool。

```
// 算法 10.9–10.13：线性探测闭散列表，显式区分空、占用和墓碑。
class HashTable {
public:
    enum class SlotState { empty, used, tombstone };
    struct SlotView { int key; SlotState state; };

    explicit HashTable(std::size_t capacity) : slots_(capacity) {
        if (capacity == 0) throw std::invalid_argument("hash table capacity must be
            ↪ positive");
    }

    [[nodiscard]] bool insert(int key) {
        const auto target = insertion_slot(key);
        if (!target) return false;
        Slot& slot = slots_[*target];
        if (slot.state == SlotState::used) return false;
        slot = Slot{key, SlotState::used};
        ++size_;
        return true;
    }

    [[nodiscard]] bool contains(int key) const { return find_slot(key).has_value();
        ↪ }

    [[nodiscard]] bool erase(int key) {
        const auto found = find_slot(key);
        if (!found) return false;
    }
```

```

        slots_[*found].state = SlotState::tombstone;
        --size_;
        return true;
    }
    [[nodiscard]] std::size_t size() const noexcept { return size_; }
    [[nodiscard]] std::size_t capacity() const noexcept { return slots_.size(); }
    [[nodiscard]] SlotView slot_at(std::size_t index) const {
        if (index >= slots_.size()) throw std::out_of_range("hash table slot");
        return SlotView{slots_[index].key, slots_[index].state};
    }
}

private:
    struct Slot { int key{0}; SlotState state{SlotState::empty}; };
    [[nodiscard]] std::size_t home(int key) const noexcept {
        const auto magnitude = key >= 0 ? static_cast<long long>(key) :
        ↪ -static_cast<long long>(key);
        return static_cast<std::size_t>(magnitude) % slots_.size();
    }
    [[nodiscard]] std::optional<std::size_t> find_slot(int key) const {
        for (std::size_t step = 0; step < slots_.size(); ++step) {
            const std::size_t index = (home(key) + step) % slots_.size();
            const Slot& slot = slots_[index];
            if (slot.state == SlotState::empty) return std::nullopt;
            if (slot.state == SlotState::used && slot.key == key) return index;
        }
        return std::nullopt;
    }
    [[nodiscard]] std::optional<std::size_t> insertion_slot(int key) const {
        std::optional<std::size_t> first_tombstone;
        for (std::size_t step = 0; step < slots_.size(); ++step) {
            const std::size_t index = (home(key) + step) % slots_.size();
            const Slot& slot = slots_[index];
            if (slot.state == SlotState::used && slot.key == key) return index;
            if (slot.state == SlotState::tombstone && !first_tombstone)
                ↪ first_tombstone = index;
            if (slot.state == SlotState::empty) return first_tombstone ?
                ↪ first_tombstone : index;
        }
        return first_tombstone;
    }
    std::vector<Slot> slots_;
    std::size_t size_{0};
};

```

10.3.5 散列方法的效率分析

成功检索的期望探测次数随装载因子 α 上升。线性探测大约是 $(1 + 1/(1 - \alpha))/2$ ；拉链法是 $1 + \alpha/2$ 。 α 接近 1 时闭散列急剧变差，应扩表或改开散列。

10.3.6 散列方法的应用

编译器符号表、缓存、去重都常用散列。需要范围查询或有序遍历时，应改用树或有序表。

本章小结

检索是按关键码或属性定位记录。线性表上有顺序、二分和分块：分别要求无序、有序可随机访问、块间有序。集合只关心「在不在」，插入已存在或删除不存在都是正常失败。散列用函数直接定位槽位，期望 $O(1)$ ；开散列把冲突挂到链上，闭散列在表内探测。闭散列删除必须留墓碑，否则会截断探测链。装载因子过高时要扩表。

习题

1. 写出顺序检索在最好、最坏、等概率平均时的 ASL。
2. 用半开区间手算二分查找 18 在有序表 {1,3,7,8,12,15,18,21} 中的过程。
3. 分块检索的块数取多少时 ASL 较均衡？为什么。
4. 说明集合的 insert 为什么返回 bool 而不是抛异常。
5. 表长 5，依次插入 1、6、11，画出线性探测的槽；删除 1 后若标空，查找 6 会怎样。
6. 比较开散列与闭散列的删除、装载因子上限和指针开销。
7. ELFHash 为什么按 unsigned char 读字节。

上机题

1. 对同一组随机键比较顺序、二分和散列的平均探测次数。
2. 实现闭散列的墓碑删除，并写测试：删掉头键后仍能查到其后的同址键。
3. 装载因子超过 0.7 时扩表再散列，观察聚集是否缓解。
4. 用 IntSet 的接口换上散列表实现，确认交、包含的测试不用改。

第 11 章 索引技术

索引不是保存完整记录的主文件，而是一张「关键码到记录位置」的查找表。它的目标是在海量外存记录中避免逐条扫描：先查较小的索引，再按位置读取目标记录。

原书本章没有独立的【算法】/【代码】清单，因此这里不提供未经验证的示意实现，也不伪造台账条目。下面是概念教程；实现练习留给需要持久化存储的上层项目。

11.1 线性索引

第 1 章和第 9 章已经说过：外存上逐条扫描主文件太贵，应该先查一张较小的表，再按位置读记录。线性索引就是这张表：项是 (key, location)，按 key 排序，可以在内存里二分。

稠密索引为每条记录建一项，主文件可以无序，查到索引项就直接得到记录位置。稀疏索引只为每个有序数据块建一项，通常记下该块的最小 key；查到块之后还要在块内再找一次。稀疏索引更省空间，但要求主文件按 key 分块有序。

第一层索引太大、内存放不下时，再为它建一层，成为多级索引。一次查询的路径是：内存里的顶层 → 读一个索引页 → 读数据页。这时的关键指标不是 CPU 比较次数，而是磁盘页访问次数。变长记录尤其适合「索引 + 位置」，因为主文件无法按下标直接算出地址。

11.2 静态多分树

磁盘按页读写，一个索引结点通常设计成恰好装满一页。二叉树每个结点只有两个孩子，同样多的 key 会把树撑得很高，查询就要读很多页。多分树每个结点有许多孩子，树高低，页访问少。

静态多分树在批量装入时按页填满，之后结构不再变。查找从根走到叶，每层读一页。插入、删除会破坏「一页正好满」的约定：多出来的记录只能进溢出区，少了的页会变稀，最终往往要重组整棵树。所以它适合「一次建好、很少改」的文件，不适合频繁更新的目录。

11.3 倒排索引

前面的索引都是从记录走到属性：先找到学号 0310，再看他是不是计算机系。倒排反过来：从属性值走到记录集合。例如

计算机系 -> [0310, 0330, 0341]
英语专长 -> [0310, 0421]

「计算机系且擅长英语」就是两个有序列表的交集，结果是 [0310]。或查询是并集，差查询是差集。全文检索用同一结构：词项映射到「哪些文档、出现在哪些位置」。短语查询还要用位置是否相邻来过滤。

倒排把查询变成集合运算，速度很快；代价是每次插入、删除、修改记录，都要同步维护所有相关列表。倒排表通常只存标识和位置，完整记录仍在主文件里。

11.4 B 树与 B+ 树

B 树是保持平衡的多路搜索树，专门为磁盘页设计。一个结点里有多个有序 key，key 之间是指向孩子的指针。查找从根开始，在页内确定下一条分支，再读孩子页。树高随阶数增大而降低，一次查找的页数大约是 $\log_m n$ 。

插入使结点超过上限（溢出）时，把它分裂成两个，并把分界 key 上推到父结点；父结点也可能接着分裂，最坏会一直裂到根，树增高一层。删除使结点低于下限（下溢）时，先向左右兄弟借 key；借不到就把两个结点合并，分界 key 从父结点拿下来。借和合并都可能向上传播。这些局部调整保证所有叶在同一层，所以 B 树始终平衡。

B+ 树把记录（或记录位置）全部放在叶上，内部结点只作路标，不存完整记录。叶按 key 从小到大串成一条链。点查询仍然从根走到叶；范围查询找到下限所在的叶之后，沿叶链顺序扫到上限即可，不必再回到内部结点。关系数据库的磁盘索引多用 B+ 树，正是因为范围扫描是常见操作。

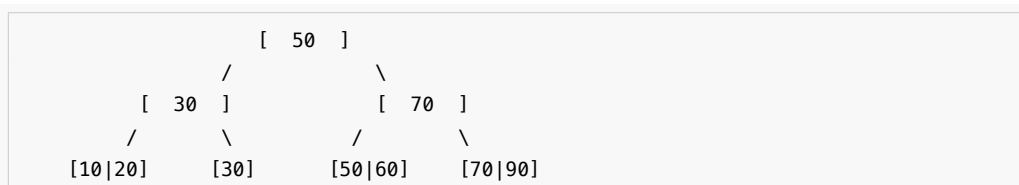
一棵 3 阶 B+ 树可以这样看（内部结点只导航，数据全在最下一层，叶之间有横链）：



本章的约定：3 阶指一个结点最多 3 个孩子，因此叶和内部结点都最多 2 个 key；内部结点的分界 key 取右子树最小 key 的复写。原书 11.4.2 用的是另一套约定——最大关键码复写，并把 m 阶 B+ 叶的容量定为 $\lceil m/2 \rceil$ 到 m 个 key（3 阶叶因此存 2 到 3 个 key）。两套约定都成立，对照原书读时注意此处差别。

查找 50：根上 $30 < 50 < 70$ ，走中间孩子；叶上直接取到 50。查找 35 到 80：先落到含 35 的叶，再沿 $\leftrightarrow$ 扫过 50、70，直到超过 80。

插入 60：中间叶暂成 $[30|50|60]$ ，超过 2 个 key 的上限。按原书 11.4.1 的分裂规则取中位数 50 作分界，叶裂成 $[30]$ 和 $[50|60]$ ，50 复写一份上推（B+ 的分界 key 在叶上仍保留）。上推之后根成了 $[30|50|70]$ ，要挂 4 个孩子，同样超限，于是根也分裂：中位数 50 上移，成为新根，树增高一层。



叶分裂时分界 key 是复写（叶上仍留一份），内部结点分裂时分界 key 是上移（原结点不再保留）——这是 B+ 树与 B 树最容易混的一处。

删除 70：先在叶上删，若叶下溢就向兄弟借或合并，再改内部结点的路标。

B 树与 B+ 树可以记成三句话：记录在不在内部结点；叶有没有横向链接；范围扫描要不要反复爬树。

11.5 位图、签名和红黑树

位图索引适合取值很少的属性，例如性别、省份、是否及格。每个属性值对应一个位串，第 i 位表示第 i 条记录有没有这个值。AND、OR、NOT 查询变成机器字上的按位运算，一次可以处理几十上百条记录。属性取值一多，位图会变得很宽，需要压缩（游程、字对齐混合等）。

签名文件把一篇文档的特征散列成较短的位串。查询时先用签名快速丢掉不可能匹配的文档，再回到原文确认。签名可能产生假阳性（签名说可能有、原文里没有），不能有假阴性。它适合「先粗筛、再精查」的全文检索，不替代倒排。

红黑树是内存里的近似平衡二叉搜索树：每个结点带一种颜色，通过几条局部规则保证从根到叶的黑结点数相同，最长路径不超过最短路径的两倍。查找、插入、删除都是 $O(\log n)$ ，旋转次数有常数上限。它适合进程地址空间里的有序映射，不替代按页组织的 B+ 树。

11.6 练习路径

1. 为变长记录实现二级稀疏索引，统计一次查询读取了几个页。
2. 为若干文档建立倒排表，完成 AND、OR、短语查询。
3. 用固定大小数组模拟 B+ 树页，测试分裂、借位、合并和范围扫描。
4. 为低基数列实现压缩位图，对比扫描主表和位图过滤的成本。

本章小结

索引是「关键码到记录位置」的表，用来少读磁盘。线性索引分稠密和稀疏，过大就做成多级。静态多分树把一个结点做成一页，树高低、页访问少，但不适合频繁更新。倒排从属性走到记录集合，查询变成交并差。B 树在页内分裂合并以保持平衡；B+ 树把记录放在叶上并用叶链做范围扫描。位图适合低基数属性，签名做粗筛，红黑树是内存有序映射。

习题

1. 稠密索引和稀疏索引各要求主文件怎样组织。
2. 多级索引一次查询大约读几页？关键指标为什么不是比较次数。
3. 静态多分树插入一条溢出记录会发生什么。

4. 用倒排表求「计算机系且擅长英语」的交集。
5. 在 11.4 插入 60 之后的那棵 3 阶 B+ 树上继续插入 65，写出叶和根，并说明这次为什么没有一直裂到根。
6. 位图、签名、红黑树、B+ 树各适合内存还是外存、点查询还是范围查询。

上机题

见 11.6 的四条练习路径。

第 12 章 高级数据结构

本章包含两个主题。可利用空间表复用固定数量的槽位：申请取得一个空闲槽，释放把它归还。最优二叉搜索树则把访问频率写成权重，用动态规划比较每个区间可能的根，得到总查找代价最小的树。

源码：空闲槽池与最优 BST、可运行示例、测试。

12.1 多维数组

多维数组是一维数组的扩充：数组的数组就是二维，二维数组再排成一列就是三维。结构上的特点是，元素本身还可以有结构，但同一数组里的元素类型相同。元素个数相对固定，一旦生成通常只改值、不改相对位置和个数，因此自然采用顺序存储。

12.1.1 多维数组的存储

C++ 里每一维的下界都是 0。二维数组既可以看成 m 个行向量组成的向量，也可以看成 n 个列向量组成的向量。每个元素 $a_{i,j}$ 同时属于第 i 行和第 j 列，最多有两个前驱、两个后继。推到 k 维，每个元素属于 k 个向量。

把数组按某种周游次序排成线性序列，就可以顺序存放。C++ 和 Pascal 按行优先：先排最右的下标，从右往左排。二维数组排出来是

$$a_{0,0}, a_{0,1}, \dots, a_{0,n-1}, a_{1,0}, \dots, a_{m-1,n-1}$$

$d_0 \times d_1 \times \dots \times d_{n-1}$ 数组中，元素 $A[j_0, \dots, j_{n-1}]$ 相对首地址的偏移是

$$d \cdot \left(\sum_{i=0}^{n-2} j_i \prod_{k=i+1}^{n-1} d_k + j_{n-1} \right)$$

其中 d 是一个元素所占单元数。每个元素的定位时间相同，所以这是随机存储结构。FORTRAN 按列优先，先排最左下标，公式左右对调。

12.1.2 特殊矩阵

矩阵常用二维数组表示，但有些矩阵里大量元素是 0 或同一个常数，不必存整块。

三角矩阵。 n 阶上三角（或下三角）里，对角线一侧全是 0 或常数 c 。只需存另一侧加上那个常数，一共 $(n^2 + n)/2$ 个单元。若用一维数组 `list[0 .. (n^2+n)/2-1]` 存下三角，元素 $a_{i,j}$ ($i \geq j$) 前面有 i 行、共 $(i^2 + i)/2$ 个非零元，再加本行的 j ，下标就是 $(i^2 + i)/2 + j$ 。

对称矩阵。 $a_{i,j} = a_{j,i}$ 。只存下三角（含对角线），另一半用对称关系映射。`list` 与 $a_{i,j}$ 的对应是： $i \geq j$ 时下标 $(i^2 + i)/2 + j$ ，否则 $(j^2 + j)/2 + i$ 。

12.1.3 稀疏矩阵

非零元很少、分布又不规则时，叫稀疏矩阵。 $m \times n$ 的矩阵里有 t 个非零元，稀疏因子 $\delta = t/(mn)$ ；通常 $\delta < 0.05$ 就按稀疏处理。这时不应再分配整块二维数组，而改存三元组（行，列，值）的线性表，或用十字链表：每个非零元同时挂在行链表和列链表上，便于按行、按列遍历。本章不另写未验证的十字链表实现。

12.2 广义表和存储管理

线性表的元素是不可再分的原子。广义表放宽这条：元素可以是原子，也可以是一个广义表。按是否共享、是否有环，分成三种：纯表对应树（每个子表只出现一次）；再入表对应向无环图（子表可以被共享）；循环表对应有环图。存储上常用带头尾指针的结点；表一旦共享，释放时就必须处理别名，不能只按树来递归 delete。

12.2.1 广义表的定义和存储结构

表头是第一个元素，表尾是去掉表头后剩下的那个表。空表既没有头也没有尾。任何一个非空广义表都可以唯一地拆成「头 + 尾」，所以递归算法写成「先处理头、再处理尾」就和定义对上了。

结点通常有一个标记位，区分原子和子表：原子结点存值，子表结点存指向另一个表的指针。再加表头结点，可以把空表和共享关系表示得更干净。原书图 12.7–12.9 画的就是无头结点、带头结点、以及带循环的几种。

12.2.2 可利用空间表

原书用重载 operator new/delete 和一条全局 avail 链实现结点复用。所有对象共享隐式全局状态，生命周期结束时还要 ::delete 整条链。现代实现改成显式的索引句柄池：acquire 从空闲栈弹出一个下标，release 把它推回去；耗尽返回 nullopt，重复/越界归还返回 false。释放后的下标失效，get 得到空指针。

普通二叉搜索树只要求中序遍历有序，树形可能很多。若键的查找频率不同，应把常查的键放得更靠近根。最优 BST 的输入包括成功查找权 $p[1..n]$ 与失败查找权 $q[0..n]$ ；动态规划对每个区间尝试每一个键作根，选择「左子树代价 + 右子树代价 + 本区间总权」最小的方案。

教材样例 $p = \{1, 5, 4, 3\}$ 、 $q = \{5, 4, 3, 2, 1\}$ 的总成本是 57，根是第 2 个键。 $\text{cost}[i][j]$ 是区间代价， $\text{root}[i][j]$ 记录取得最小值的根，因而还能按根表重建树形。朴素实现为 $O(n^3)$ 。

先跑一遍：

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::advanced::ReusableNodePool<int> pool(2);
```

```

const auto first = pool.acquire(11);
const auto second = pool.acquire(22);
std::cout << " 申请到槽 " << *first << " 和 " << *second
          << ", 剩余 " << pool.available() << '\n';
pool.release(*first);
const auto reused = pool.acquire(44);
std::cout << " 归还后再申请得到槽 " << *reused
          << ", 值为 " << *pool.get(*reused) << '\n';

const auto tree = dsa::advanced::optimal_bst({1, 5, 4, 3}, {5, 4, 3, 2, 1});
std::cout << " 最优 BST 总成本 " << tree.cost[0][4]
          << ", 根为键 " << tree.root[0][4] << '\n';
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch12/optimal_bst \
    code/ch12/optimal_bst/demo.cpp -o /tmp/bst-demo
/tmp/bst-demo

```

```

申请到槽 0 和 1, 剩余 0
归还后再申请得到槽 0, 值为 44
最优 BST 总成本 57, 根为键 2

```

把 `optimal_bst({1,2}, {3,4})` 这种长度对不上的输入送进去，会抛 `std::invalid_argument`。空树对应 `p = {}`、`q = {某个失败权}`，代价为 0。

池用 `vector<optional<T>>` 表示槽位占用，`vector<size_t>` 当空闲栈。构造时下标从大到小入栈，所以第一次 `acquire` 拿到 0。release 先确认该槽确实被占用，再 `reset` 并归还。

```

template <typename T>
class ReusableNodePool {
public:
    explicit ReusableNodePool(std::size_t capacity) : slots_(capacity) {
        for (std::size_t index = 0; index < capacity; ++index) {
            free_.push_back(capacity - index - 1);
        }
    }

    [[nodiscard]] std::optional<std::size_t> acquire(const T& value) {
        if (free_.empty()) {
            return std::nullopt;
        }
        const std::size_t index = free_.back();
        free_.pop_back();
        slots_[index] = value;
    }

```

```

        return index;
    }

    bool release(std::size_t index) {
        if (index >= slots_.size() || !slots_[index]) {
            return false;
        }
        slots_[index].reset();
        free_.push_back(index);
        return true;
    }

    [[nodiscard]] const T* get(std::size_t index) const noexcept {
        if (index >= slots_.size() || !slots_[index]) {
            return nullptr;
        }
        return &*slots_[index];
    }

    [[nodiscard]] std::size_t available() const noexcept { return free_.size(); }

private:
    std::vector<std::optional<T>> slots_;
    std::vector<std::size_t> free_;
};

```

12.2.3 存储的动态分配和回收

程序运行中向系统要一块、用完还回去。空闲块和已分配块穿插之后，会出现两种碎片：外部碎片是空闲区被切得太碎，哪一块都装不下新请求；内部碎片是分出去的块比实际需要大，多出来的字节浪费在块内。

选哪一块空闲区，有三种经典策略。首次适应从低地址扫到第一块够大的；最佳适应找刚好够用的最小块，留下的碎片更碎；最坏适应找当前最大的块，希望剩下的还大到能再用。无论哪种，相邻的空闲块都应当合并，否则外部碎片只会越来越多。边界标记（块头块尾记下大小和忙闲）让合并可以在常数时间完成。

12.2.4 失败处理策略和无用单元回收

分配失败时可以：直接拒绝；压缩（把已分配块搬到一起，挤出一整块空闲）；或者做垃圾回收，把程序已经够不着、却还占着的块收回来。

引用计数给每个对象记有多少指针指着它，减到零就释放。实现简单，但对象互相指形成环时，计数永远掉不到零，这块内存就漏了。标记-清除从一组根（栈上的指针、全局变量）出发走遍所有还能碰到的对象并打上标记，然后扫一遍堆，没标记的统统回收。它能处理环，但要求能从根集走到每一个活对象，并且要能区分「这是指针」和「这只是一个看起来像地址的整数」。

本章的句柄池把「失败」做成返回 `nullopt`，不劫持全局 `new`，也不假装自己是通用垃圾回收器。

12.3 Trie 结构和 Patricia 树

二叉搜索树按整个关键码比较，树的形状依赖插入次序。Trie 换一种切法：按关键码的字符（或二进制位）一层一层分支，第 i 层对应第 i 个字符。公共前缀在树里只存一次，所以它特别适合字典、IP 路由这种「按前缀分类」的问题。查找沿着字符走，时间与关键码长度成正比，与表里有多少个词关系不大。

把 `can`、`car`、`cat`、`do` 插进一棵字母 Trie，公共前缀 `ca` 只出现一次：

```
      .
     /\
    c  d
    |  |
    a  o*
   /\
  n* r* t*
```

带 `*` 的是词尾。查找 `car` 走 `c-a-r`，三步；查找 `cab` 在 `b` 处没有分支，失败。最长前缀匹配（路由）则走到不能再走为止，回退到最近的词尾。

纯 Trie 在「只有一个孩子」的内部结点上仍然分支，路径偏长。Patricia 树把这种单孩子结点压缩掉，边上记下「跳过几位再比」。查找时按记下的位位置取关键码的那一位，决定走左还是走右。路径更短，结点更少，仍然保持前缀共享。二者都是原书没错的结构；本仓库没有单独的可运行实现，用这张图就能把前缀共享说清楚。

12.4 改进的二叉搜索树

12.4.1 最佳二叉搜索树

最优 BST 先校验 `q.size() == p.size() + 1`。空区间的 `cost[i][i] = 0`，`weight[i][i] = q[i]`。长度从 1 扩到 n ，对每个区间 `[first, last]` 枚举根 r ，候选代价是 `cost[first][r-1] + cost[r][last] + weight[first][last]`。

```
struct OptimalBstResult {
    std::vector<std::vector<long long>> cost;
    std::vector<std::vector<std::size_t>> root;
};

inline OptimalBstResult optimal_bst(const std::vector<int>& successful,
                                     const std::vector<int>& unsuccessful) {
    if (unsuccessful.size() != successful.size() + 1) {
        throw std::invalid_argument("weight count");
    }
}
```

```

const std::size_t count = successful.size();
OptimalBstResult result{
    std::vector<std::vector<long long>>(count + 1,
                                        std::vector<long long>(count + 1, 0)),
    std::vector<std::vector<std::size_t>>(count + 1,
                                        std::vector<std::size_t>(count + 1, 0))};
std::vector<std::vector<long long>> weight(
    count + 1, std::vector<long long>(count + 1, 0));

for (std::size_t index = 0; index <= count; ++index) {
    // The book's c table measures internal-key comparison cost; an empty
    // interval has zero c cost while its unsuccessful weight remains in w.
    result.cost[index][index] = 0;
    weight[index][index] = unsuccessful[index];
}
for (std::size_t length = 1; length <= count; ++length) {
    for (std::size_t first = 0; first + length <= count; ++first) {
        const std::size_t last = first + length;
        weight[first][last] = weight[first][last - 1] + successful[last - 1] +
                               unsuccessful[last];
        result.cost[first][last] = std::numeric_limits<long long>::max() / 4;
        for (std::size_t root = first + 1; root <= last; ++root) {
            const long long candidate = result.cost[first][root - 1] +
                                       result.cost[root][last] + weight[first]
↪ [last];

            if (candidate < result.cost[first][last]) {
                result.cost[first][last] = candidate;
                result.root[first][last] = root;
            }
        }
    }
}
return result;
}

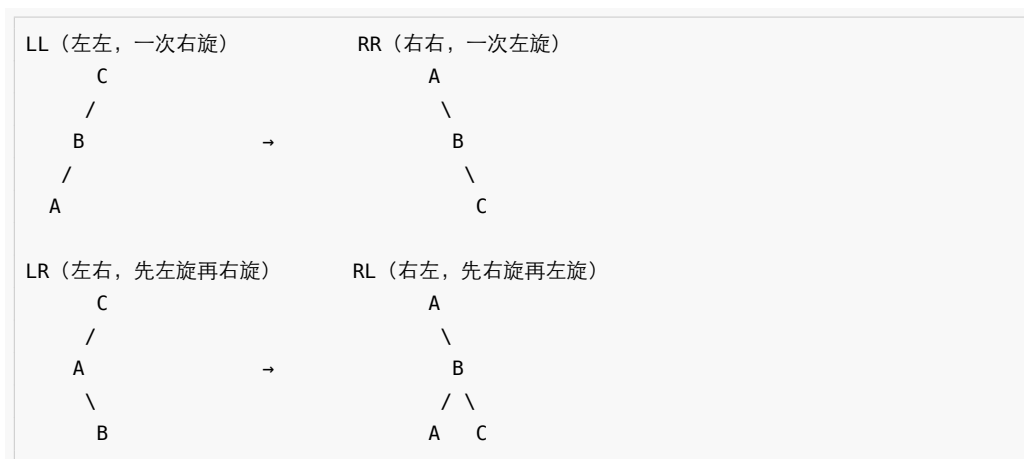
```

12.4.2 平衡的二叉搜索树

普通 BST 按插入次序生长，若键已经有序，会退化成一条链，查找变成 $O(n)$ 。平衡二叉搜索树在每次插入、删除之后做少量旋转，把左右子树的高度差限制住，从而保证树高 $O(\log n)$ 。

AVL 树给每个结点记一个平衡因子：右子树高度减左子树高度，只允许 -1 、 0 、 1 。插入或删除使某个祖先的因子变成 ± 2 时，按「哪边沉、沉在内侧还是外侧」分成四种情形。外侧失衡一次单旋转就能拉平；内侧失衡要先把孙子转到孩子、再把孩子转到祖先，即双旋转。旋转是局部的，不改变中序次序。查找、插入、删除最坏都是 $O(\log n)$ 。

四种情形用中序仍保持 $A < B < C$ 来记（圆括号里是子树）：



插入 1、2、3 会走出 RR：先得到右链 1-2-3，在 1 上左旋变成以 2 为根的平衡树。插入 3、2、1 是对称的 LL。插入 1、3、2 是 RL：先在 3 上右旋，再在 1 上左旋。删除一个结点后，从被删处向上检查，哪一层变成 ± 2 就在那一层做同样的旋转；有时旋转一次还不够，要继续向上走。

红黑树是另一种近似平衡，用颜色规则保证最长路径不超过最短路径的两倍，旋转次数有常数上限，见第 11.5 节。二者都是原书没错的结构；本章不另写未验证的 AVL 实现。

12.4.3 伸展树

伸展树不在结点上存平衡因子。每次访问（查找、插入、删除）一个结点之后，用旋转把它搬到根附近：最近被用到的键下次更容易先碰到。这是一种自调整，均摊 $O(\log n)$ 。

从被访问结点走到根，每次看最近两步的形状。之字形（先左后右，或先右后左）走两步只升一层；一字形（连续两次同向）走两步升两层。搬到根的孩子时，再做一次单旋转即可。半伸展比全伸展少转最后一轮，连续访问同一个结点时更省。

伸展树实现简单，不必维护额外的平衡域；代价是单次操作可能仍是 $O(n)$ ，只是摊还之后是对数。本章不另写未验证实现。

本章小结

多维数组按行优先或列优先顺序存放，按下标随机访问；三角、对称、稀疏矩阵可以压缩。广义表的元素可以是原子或子表，头尾分解对应递归。动态存储要处理碎片和回收：首次/最佳/最坏适应，引用计数与标记清除。Trie 按字符分支并共享前缀，Patricia 压缩单孩子路径。最优 BST 用动态规划选根；AVL 用平衡因子和四种旋转保证 $O(\log n)$ ；伸展树用访问后上移做均摊平衡。可利用空间表是定长结点的显式空闲栈，不劫持全局 new。

习题

1. 写出 3×4 数组按行优先时 $a_{2,1}$ 相对首地址的偏移（元素占 1 个单元）。
2. 把 4 阶下三角压成一维，给出 $a_{3,1}$ 的下标。
3. 说明纯表、再入表、循环表分别对应什么图，释放时要注意什么。
4. 对空闲块 900、1500、700，各举一个只有首次适应、只有最佳适应、只有最坏适应能满足的请求。
5. 把 can、car、cat 插入字母 Trie，画出结果，并写出查找 cab 的失败路径。
6. 教材样例 $p = \{1, 5, 4, 3\}$ 、 $q = \{5, 4, 3, 2, 1\}$ ，指出最优 BST 的根和总成本。
7. 从空 AVL 依次插入 1、2、3，画出每次旋转。再插入 0，需要旋转吗。
8. 伸展树中，之字形和一字形各升几层？为什么半伸展适合连续访问同一结点。

上机题

1. 实现下三角矩阵的压缩存储，并支持按下标读写。
2. 用句柄池管理固定个数的结点，测试耗尽、归还、复用和重复释放。
3. 实现一棵字母 Trie 的插入和查找，用一组单词对拍 `std::set`。
4. 按教材权重实现最优 BST，输出根表并重建树形。

原书插图

从只读底稿 `dsa_raw.md` 抽出的 292 张图。字节落在 `book/assets/`，alt 来自原书题注；少数图在底稿里没有独立题注，alt 会标明行号。正文各章只引用教学需要的几张；其余集中在本图册，避免把教程拆碎。

前言



前言插图（底稿第 115 行，原书无独立题注）

第 1 章 概论

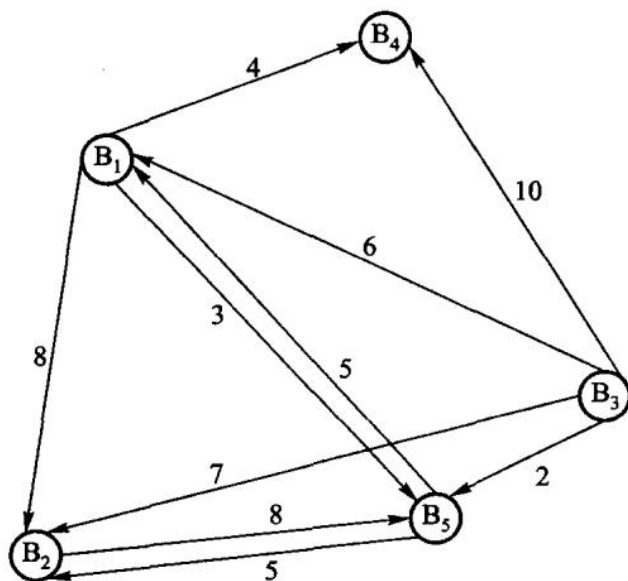
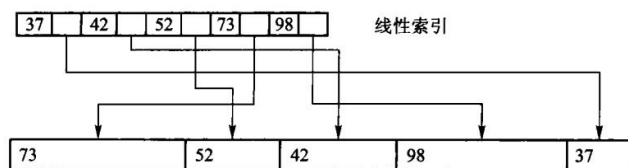


图 1.1 经纪人间的消息传递图

	B_1	B_2	B_3	B_4	B_5
B_1	0	8	∞	4	3
B_2	13	0	∞	17	8
B_3	6	7	0	10	2
B_4	∞	∞	∞	0	∞
B_5	5	5	∞	9	0

图 1.3 经纪人消息传递的相邻矩阵



第 1 章插图（底稿第 659 行，原书无独立题注）

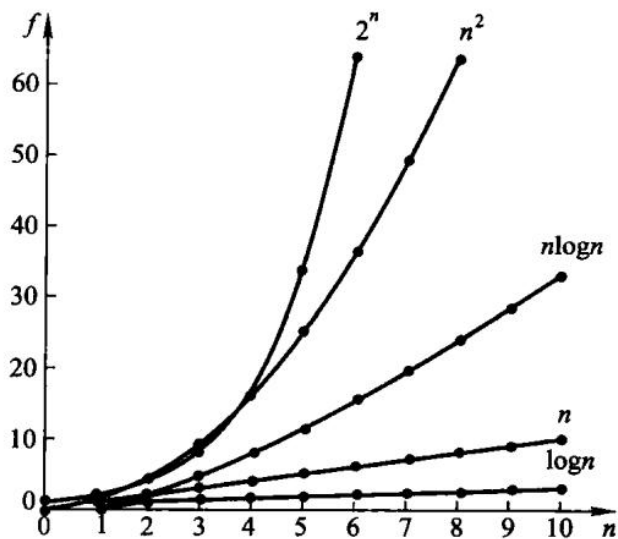


图 1.5 常用函数的增长趋势

第 2 章 线性表

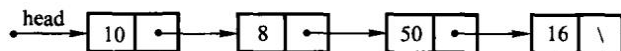


图 2.4 单链表示例

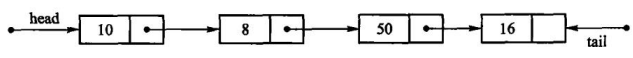
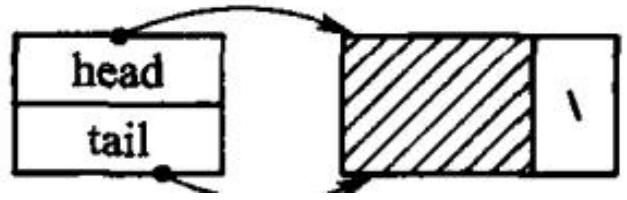
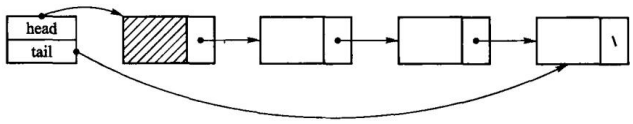


图 2.5 具有头、尾两指针的单链表



(a) 带有头结点的空表



(b) 典型的带有头结点的单链表；图 2.6 引入头结点的单链表

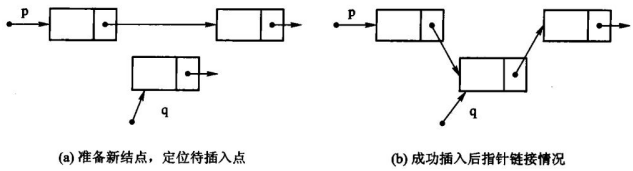
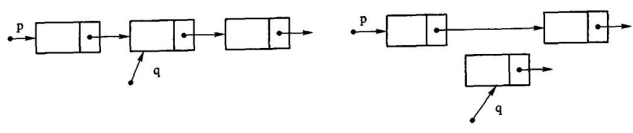


图 2.7 插入算法示意图



(a) 删除前指针情况；(b) 删除后指针情况；图 2.8 删除示意图

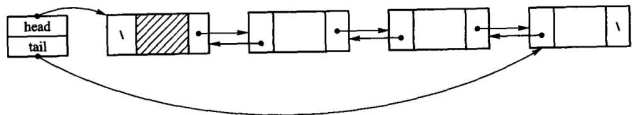


图 2.9 加入表头结点的双链表；图 2.9 中变量 head 用于指向表首结点，变量 tail 用于指向表尾结点。图 2.10 给出了相应的



图 2.10 双链表的结点

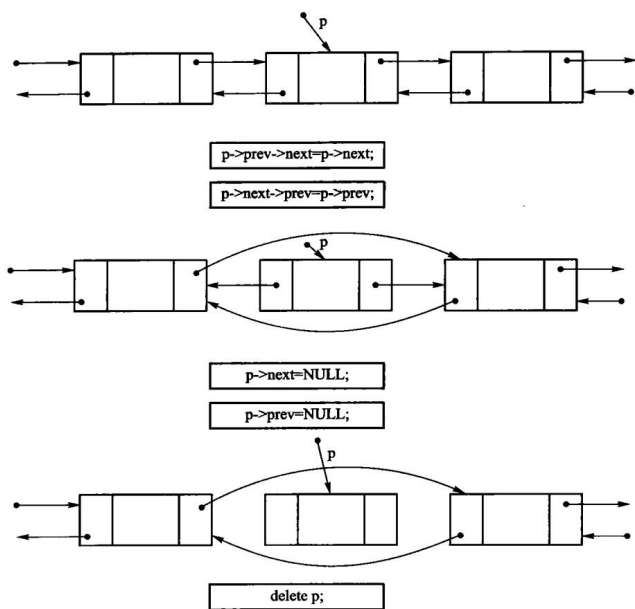


图 2.11 双链表的删除操作示意

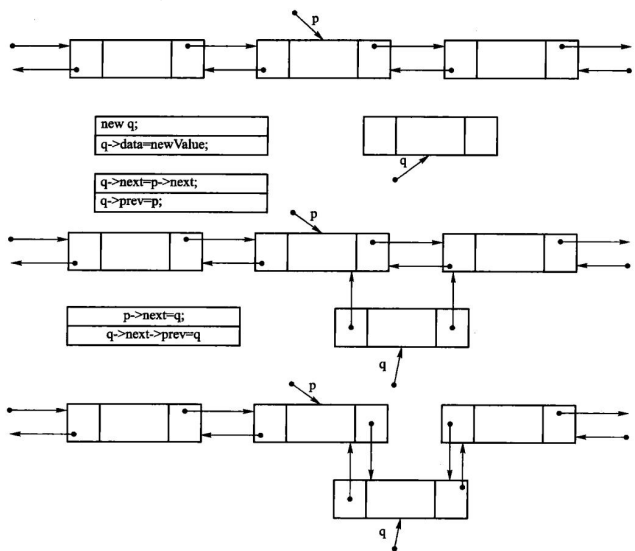


图 2.12 双链表插入操作示意

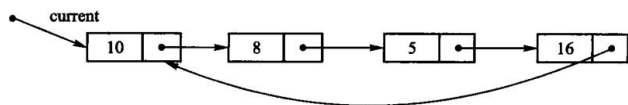


图 2.13 循环链表示意图

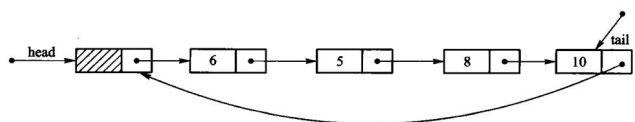


图 2.14 带头结点的循环链表示意图

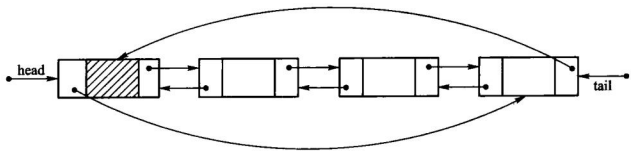


图 2.15 循环双链表示意图

第 3 章 栈与队列

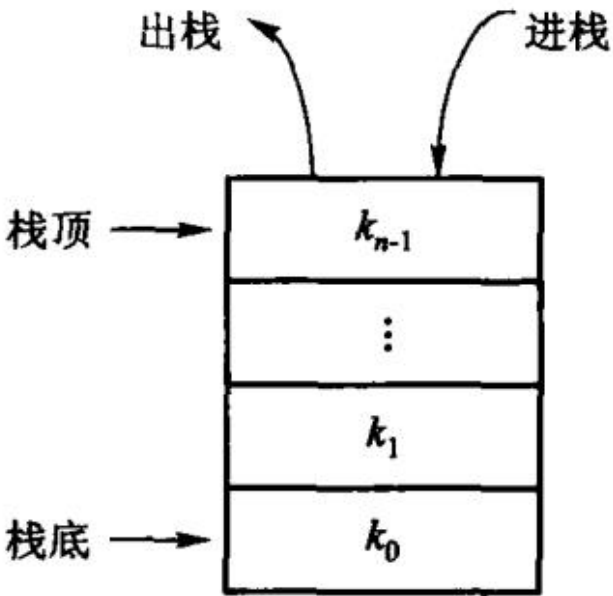


图 3.1 栈的示意图

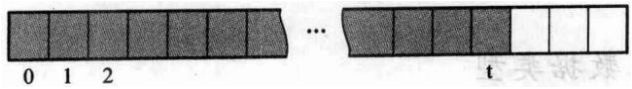
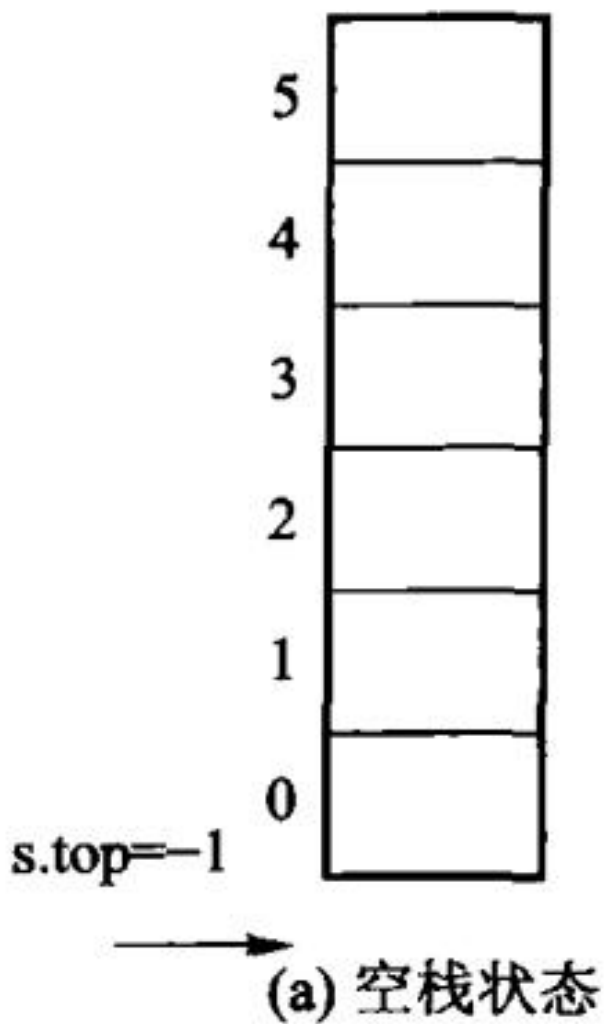


图 3.2 栈的顺序存储结构示意



第 3 章插图（底稿第 1898 行，原书无独立题注）

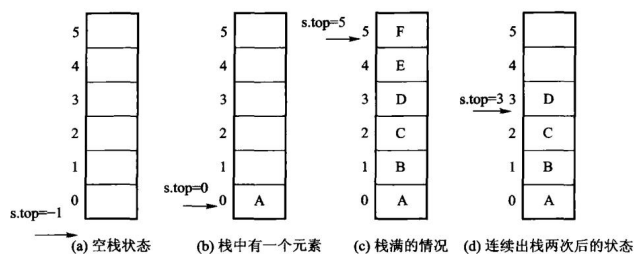


图 3.3 顺序栈的存储结构

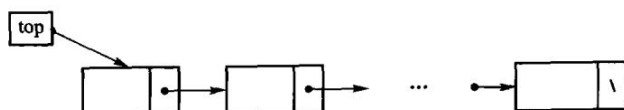


图 3.4 链式栈示意

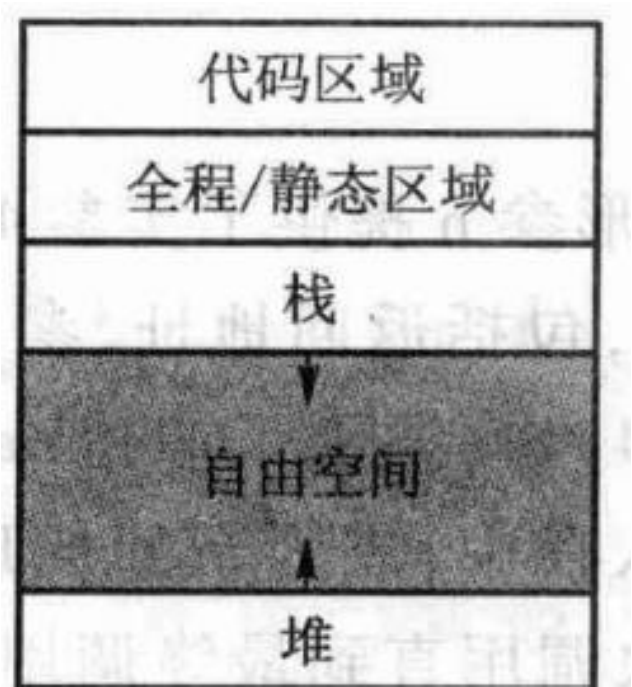


图 3.6 运行时存储器的组织形式

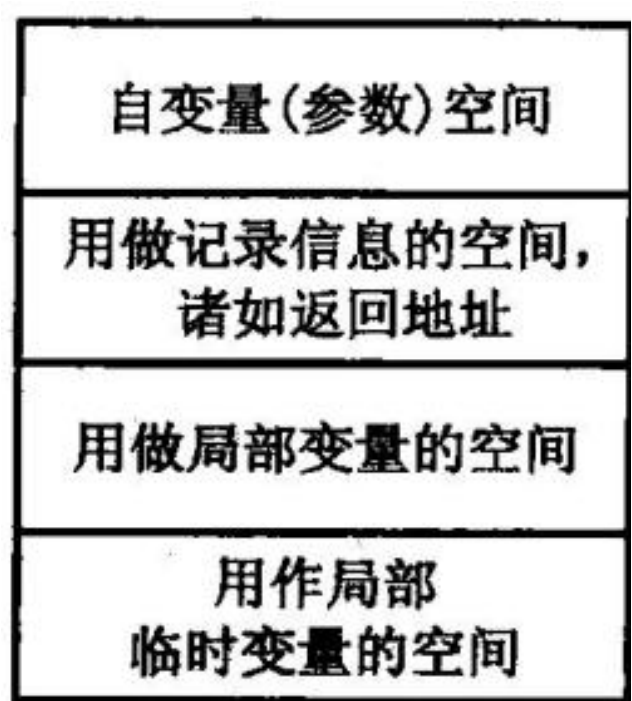


图 3.7 活动记录的内容

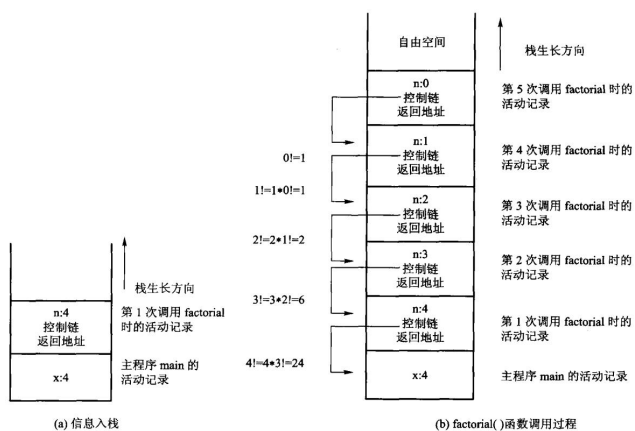


图 3.8 递归计算 factorial(4) 时，内部栈的变化情况示意

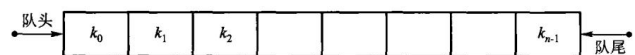


图 3.9 队列的示例

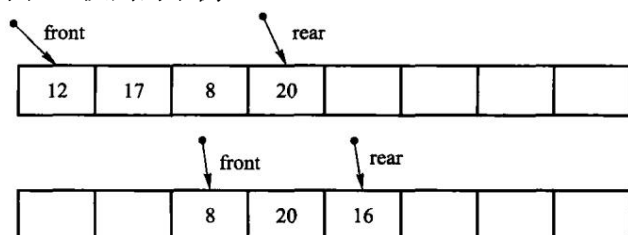
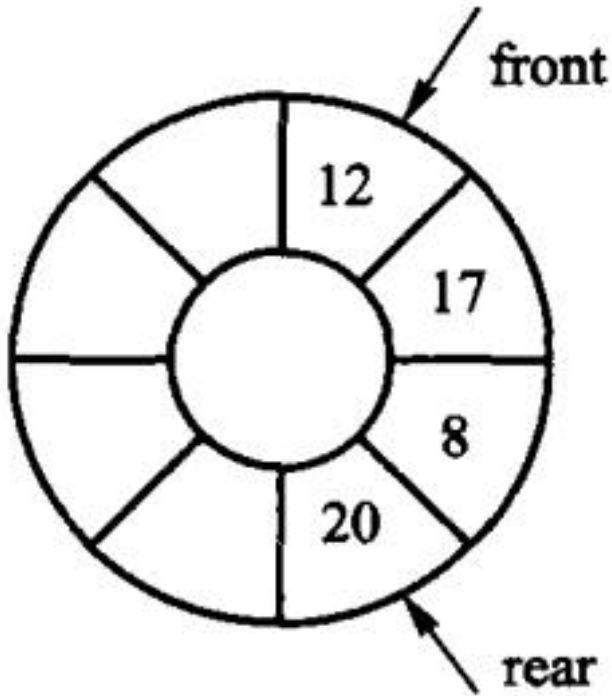
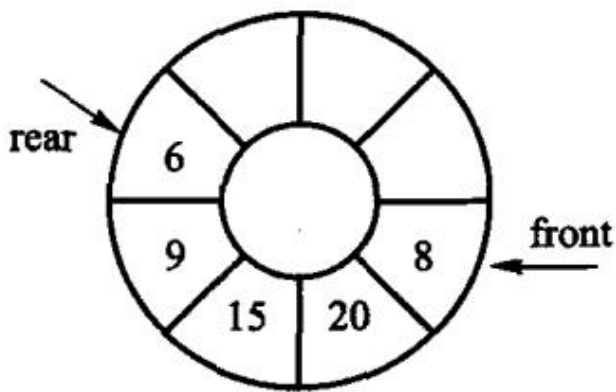


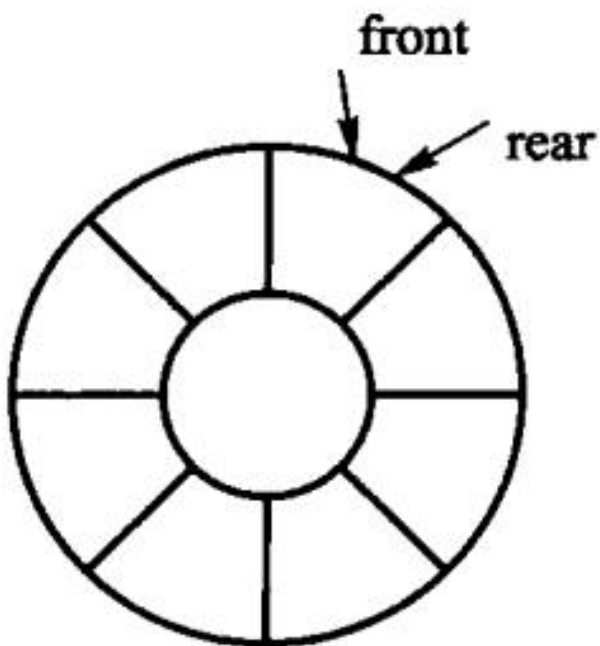
图 3.10 顺序队列的实现示意



(a) 初始队列

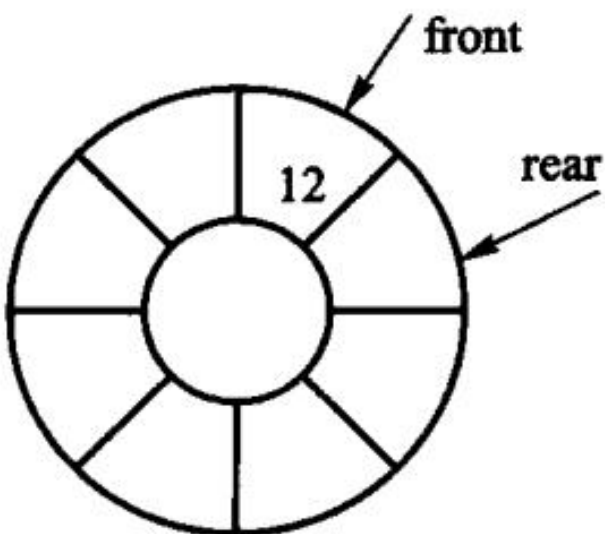


(b) 新队列；图 3.11 顺序队列的一种实现方式



(a) 队列为空的状态

第 3 章插图（底稿第 2621 行，原书无独立题注）



(b) 队列的一般状态

第 3 章插图（底稿第 2623 行，原书无独立题注）

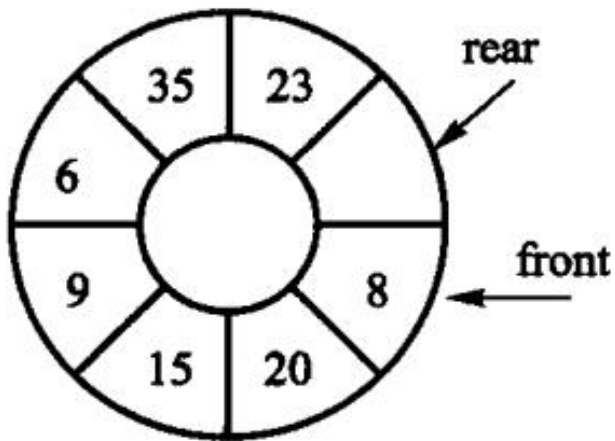


图 3.12 循环队列的实现示例



图 3.13 存储在同一个数组中的两个迎面增长的栈

第 4 章 字符串

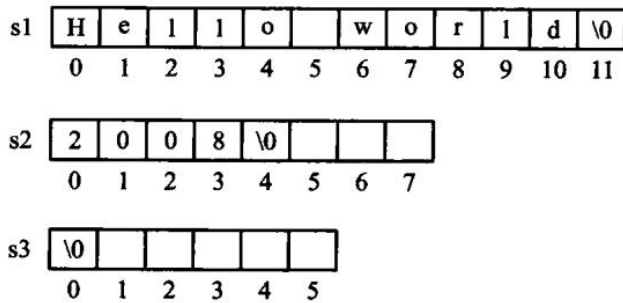


图 4.1 C++ 标准字符串的变量说明示意图

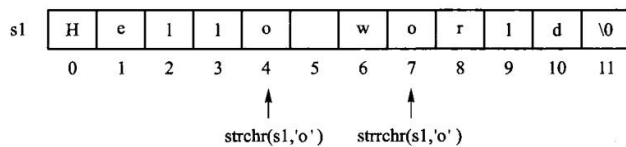


图 4.2 在字符串 s1 中寻找并定位给定字符 'o' 的位置

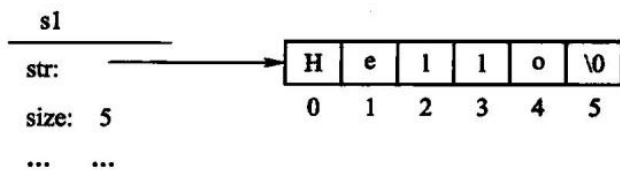


图 4.3 创建字符串的示意图

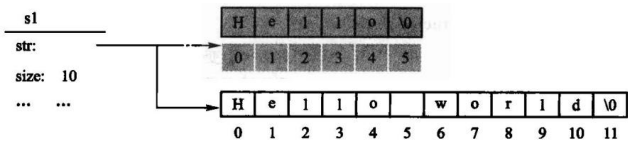


图 4.4 赋值运算示意图

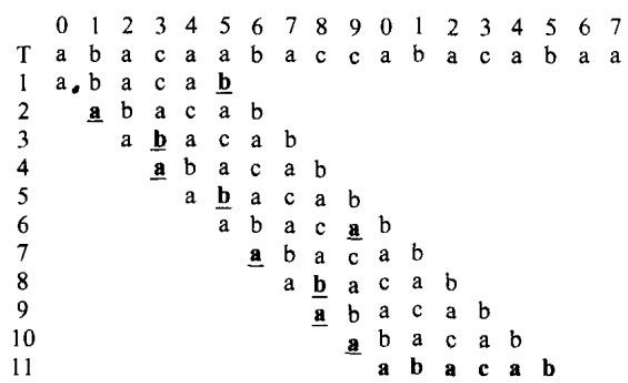


图 4.6 朴素匹配的示例



图 4.7 朴素匹配的最佳情况示例

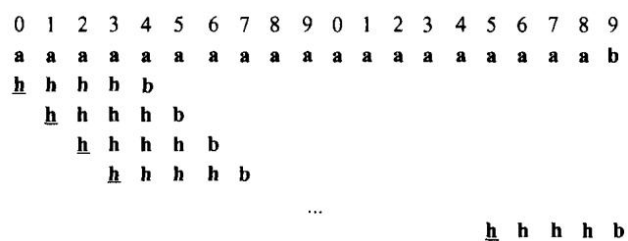
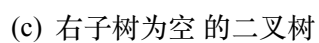


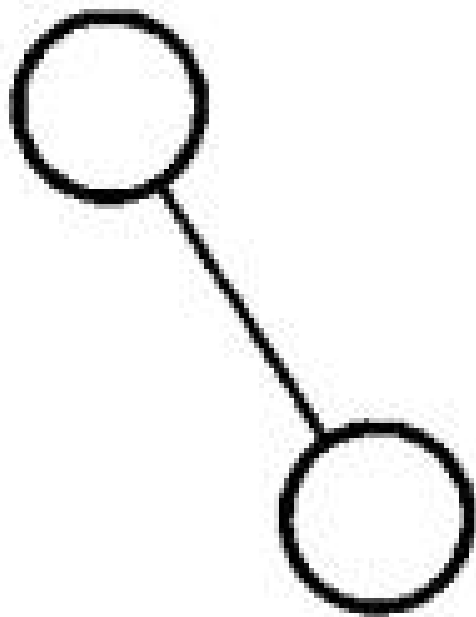
图 4.8 匹配失败的最佳情况示例

图 4.9 朴素匹配最差情况示例

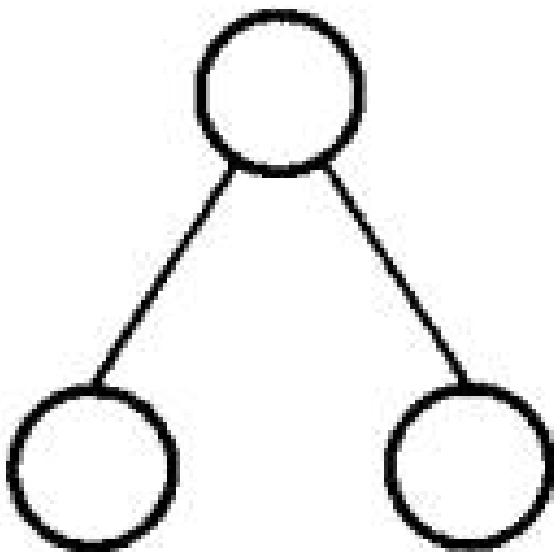
图 4.12 KMP 匹配示例

第 5 章 二叉树

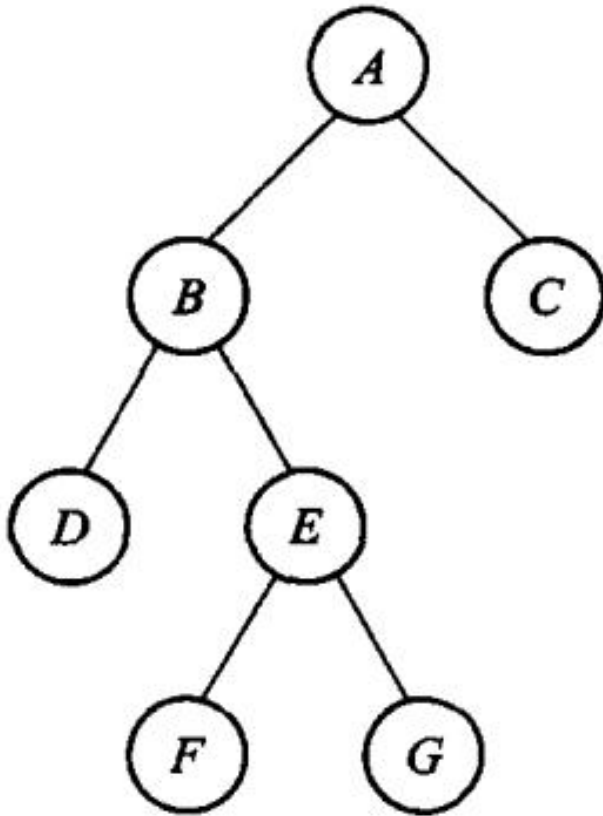




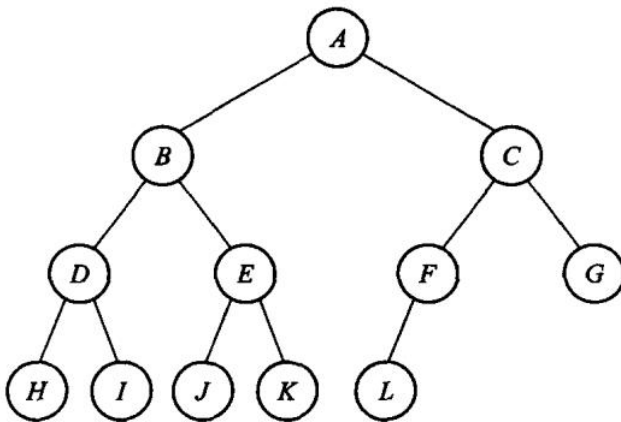
(d) 左子树为空 的二叉树



(e) 左、右子树均非空的二叉树；图 5.1 二叉树的 5 种基本形态



(a) 满二叉树



(b) 完全二叉树；图 5.2 满二叉树和完全二叉树示例

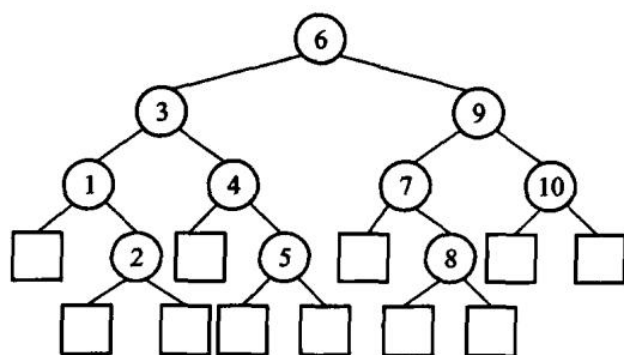


图 5.3 扩充二叉树

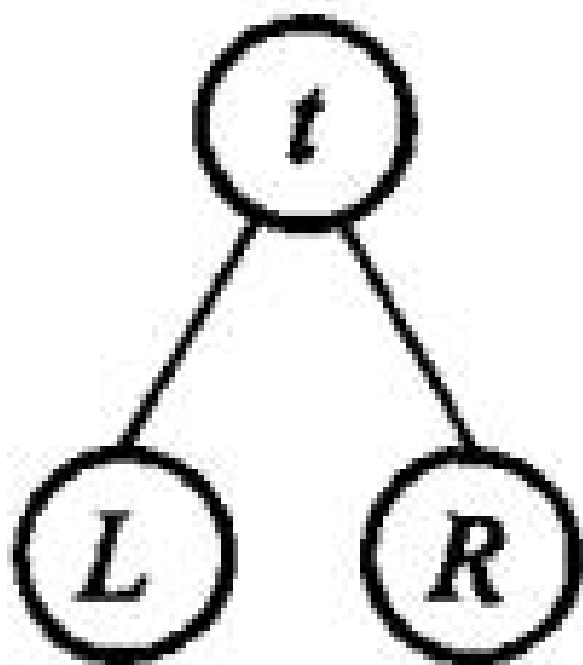


图 5.4 二叉树的基本结构

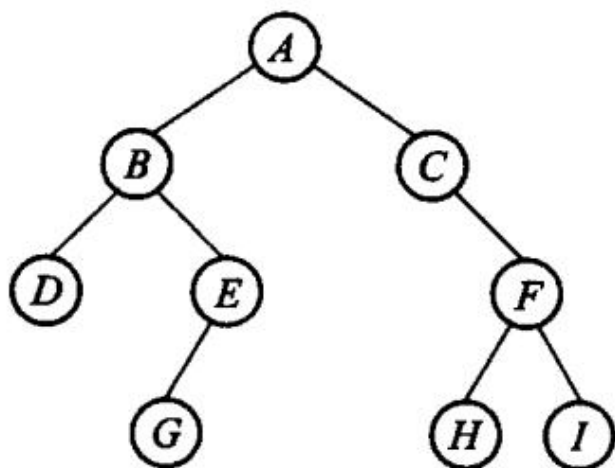


图 5.5 二叉树示例

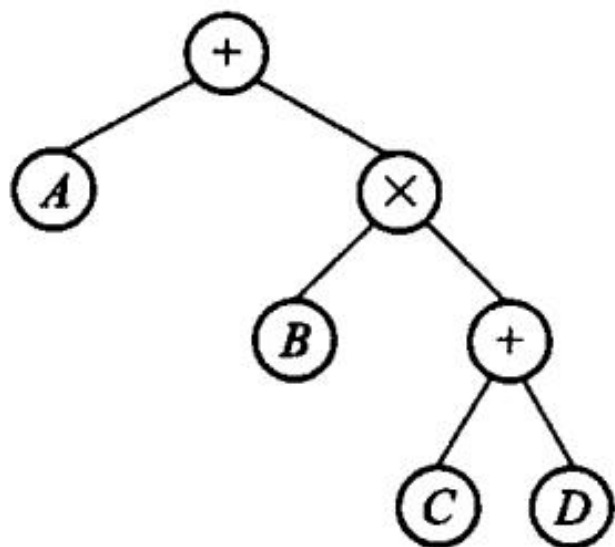


图 5.6 表达式树

left	info	right
------	------	-------

(a) 有 2 个指针域的结点结构

left	info	parent	right
------	------	--------	-------

(b) 有 3 个指针域的结点结构

图 5.7 二叉树结点的存储结构

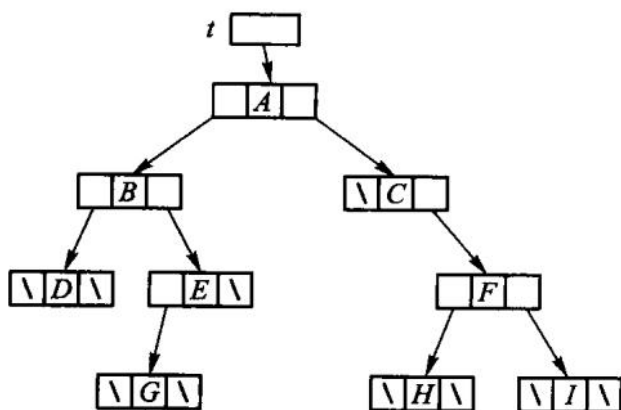


图 5.8 图 5.5 中二叉树的二叉链表存储结构

delete root;

}

}

【代码 5.8 结束】

第 5 章插图（底稿第 4103 行，原书无独立题注）

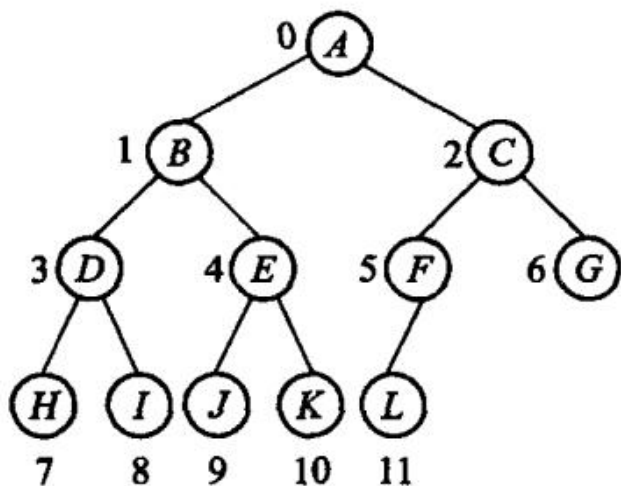


图 5.9 完全二叉树的结点编号

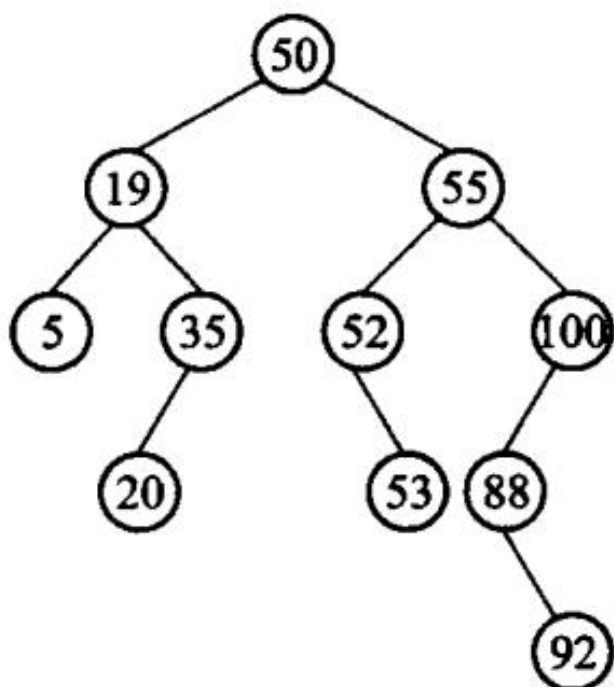
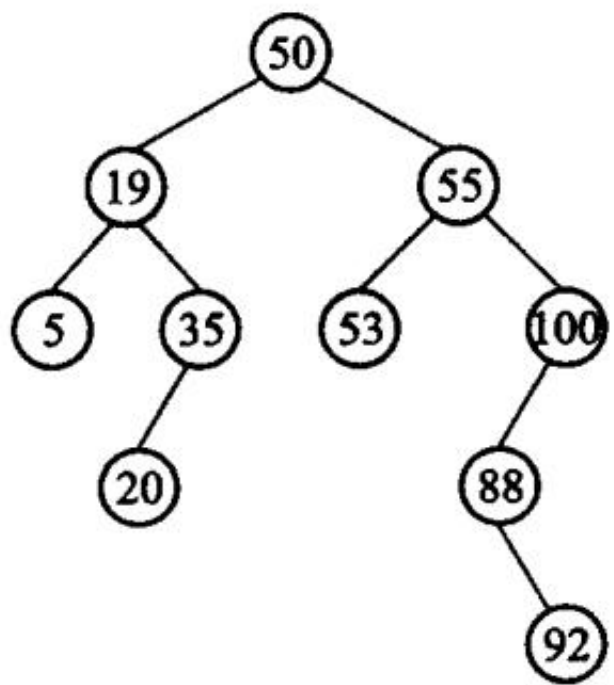


图 5.11 二叉搜索树



(a) 没有左子树的情况

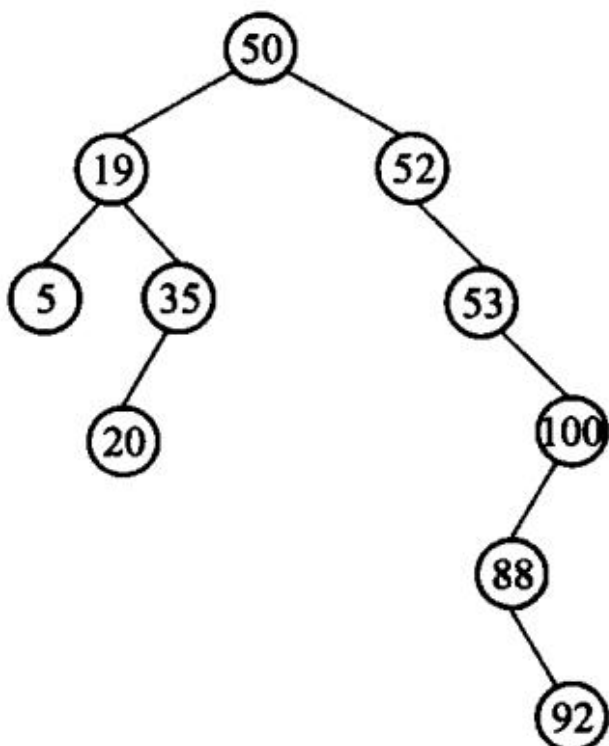


图 5.12 二叉搜索树的删除示例；(b) 有左子树的情况

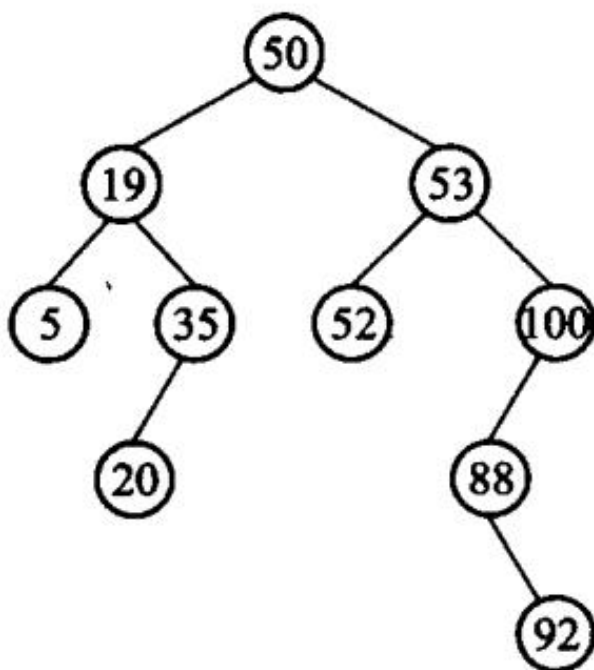


图 5.13 改进的二叉搜索树的删除

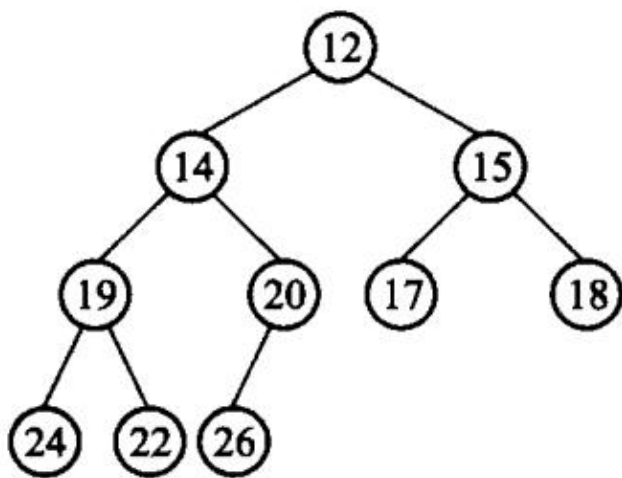


图 5.14 最小堆对应的完全二叉树

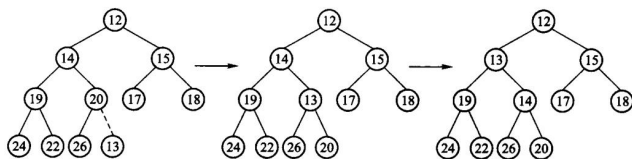


图 5.15 在最小堆 5.14 中插入元素 13

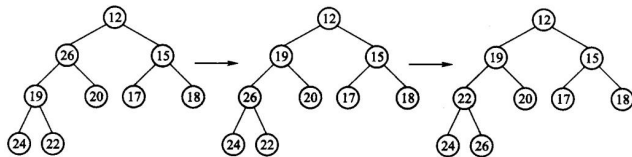


图 5.16 在最小堆 5.14 中删除元素 14

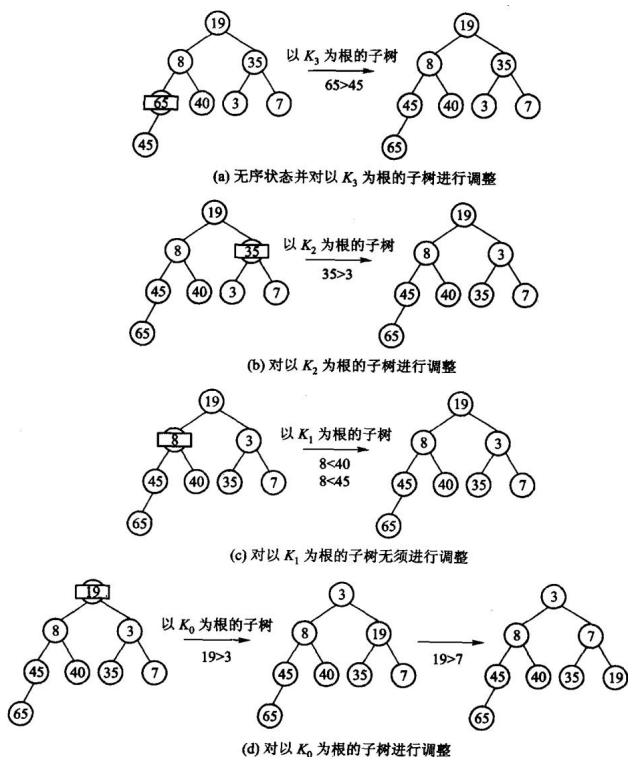
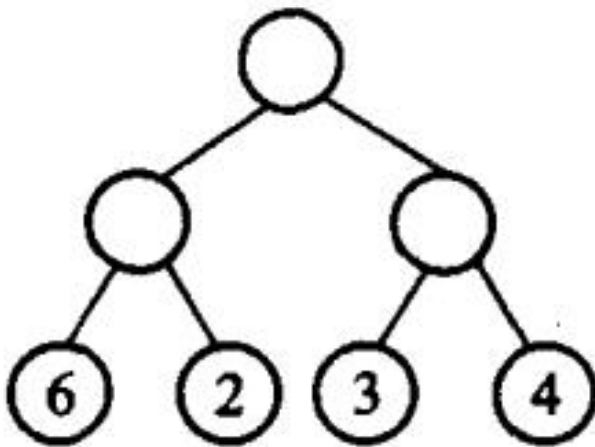
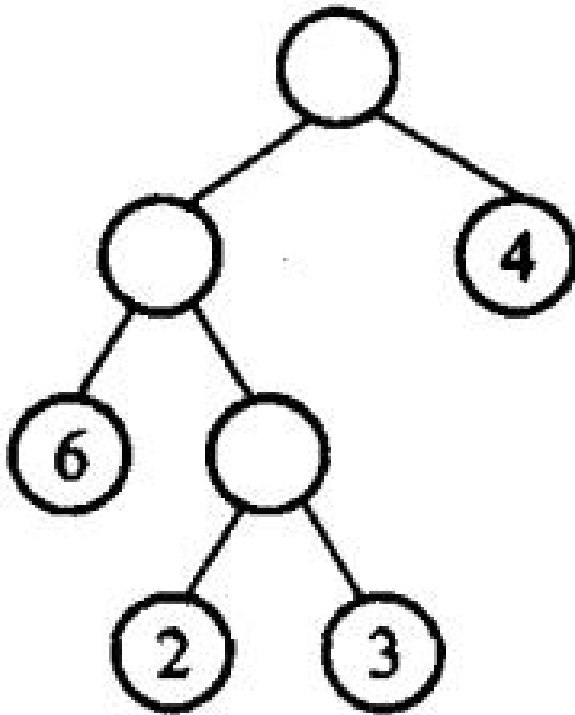


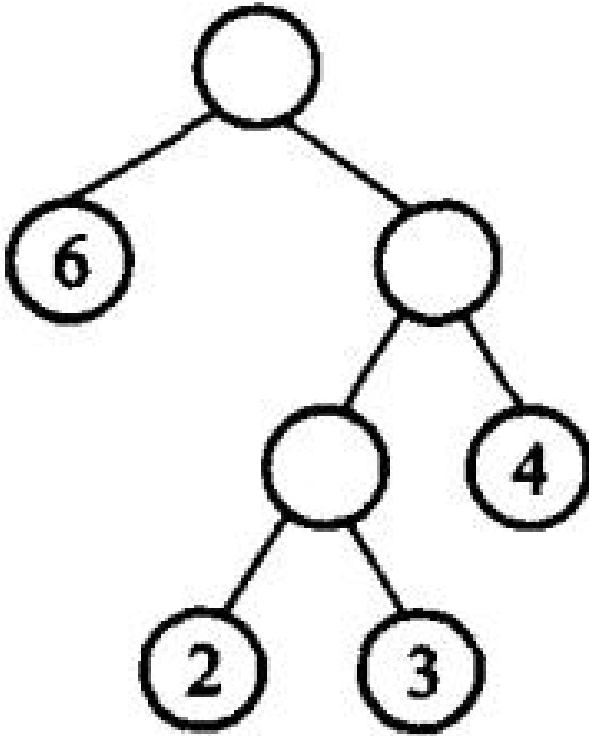
图 5.17 建堆过程示例



(a) 外部路径为 30



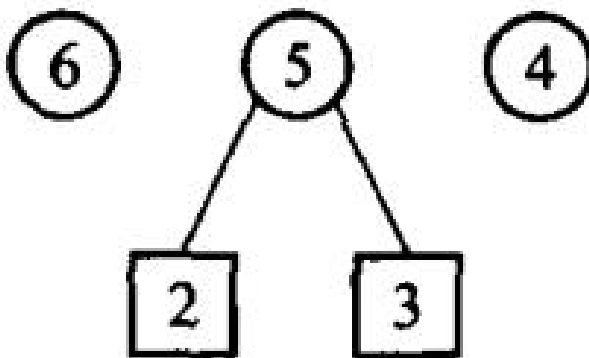
(b) 外部路径为 31



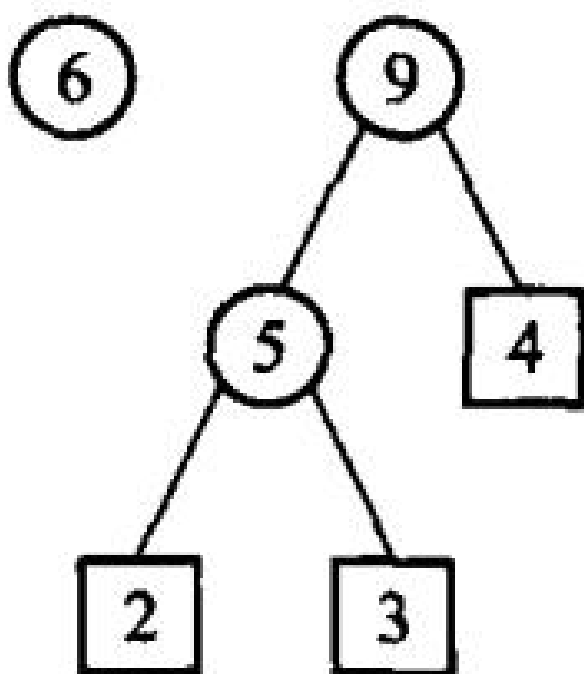
(c) 外部路径为 29；图 5.18 具有不同带权外部路径长度的二叉树



(a) 过程 1



(b) 过程 2



(c) 过程 3; (d) 过程 4

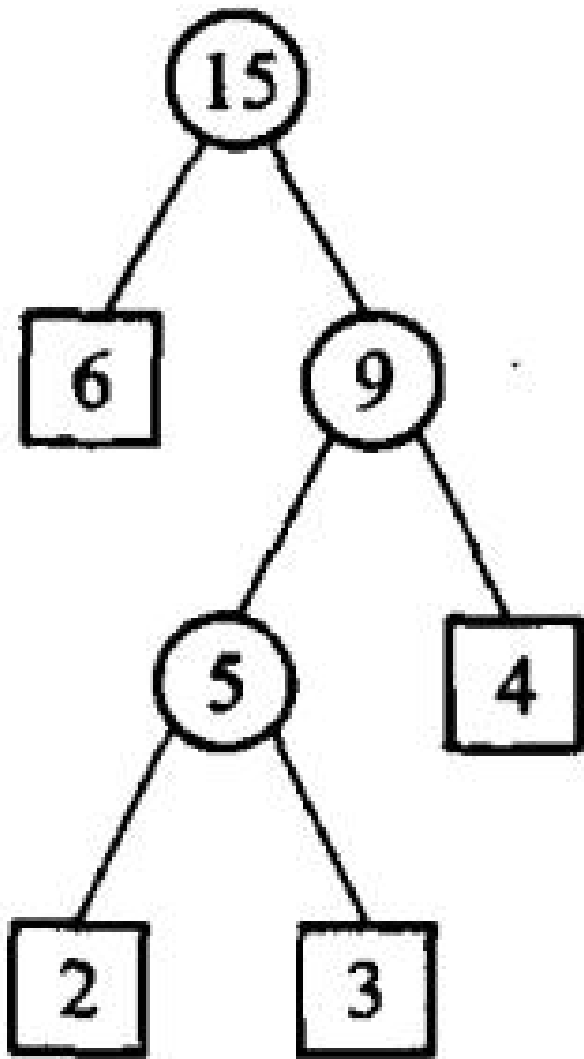


图 5.19 Huffman 树的构造过程

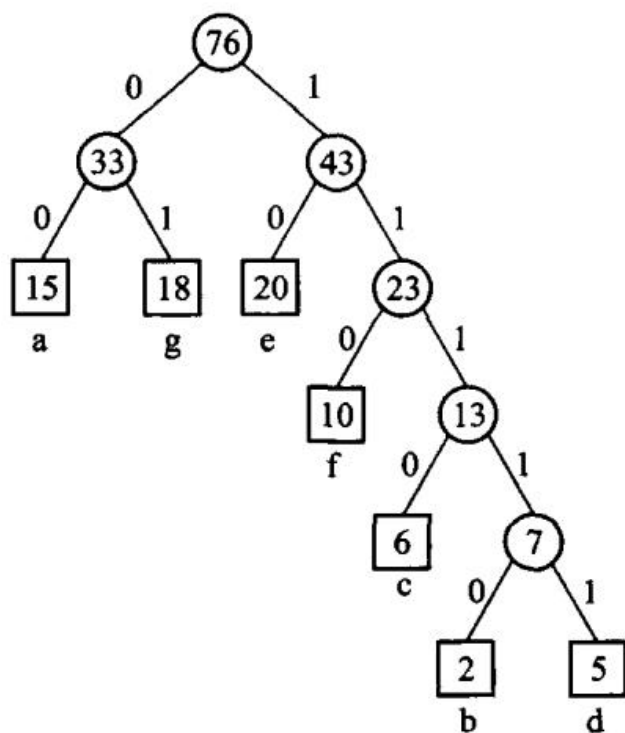


图 5.20 Huffman 编码示例

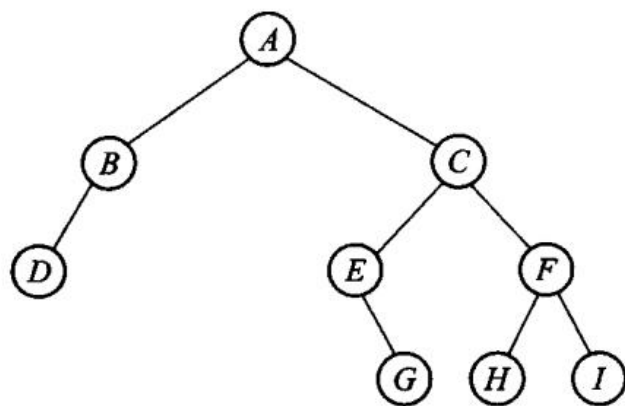
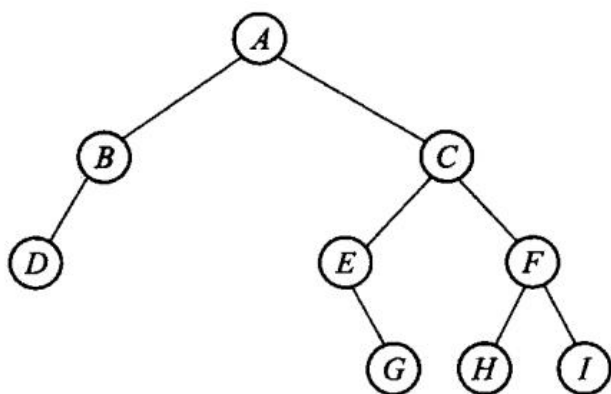
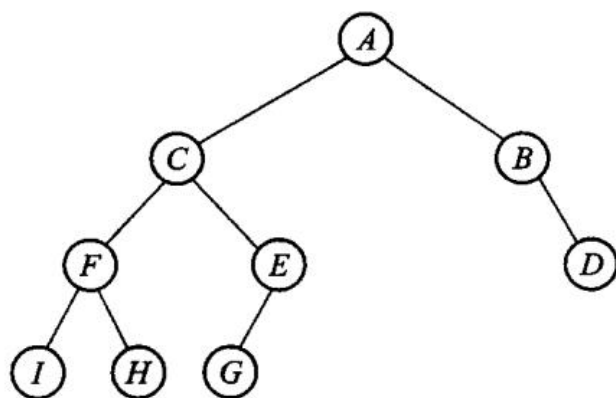


图 5.21 习题 2 的图例



(a) 原二叉树



(b) 镜面映射后的新二叉树；图 5.22 习题 7 的图例

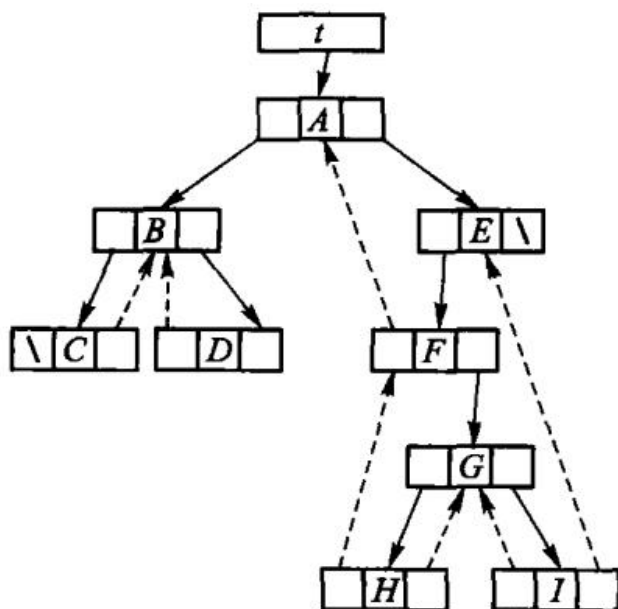


图 5.23 中序穿线二叉树

第 6 章 树

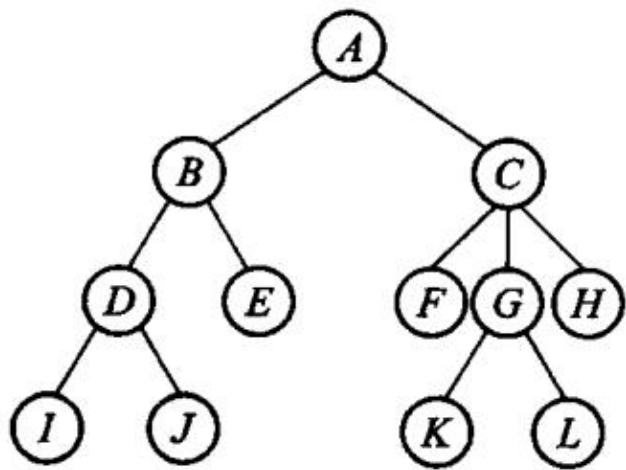
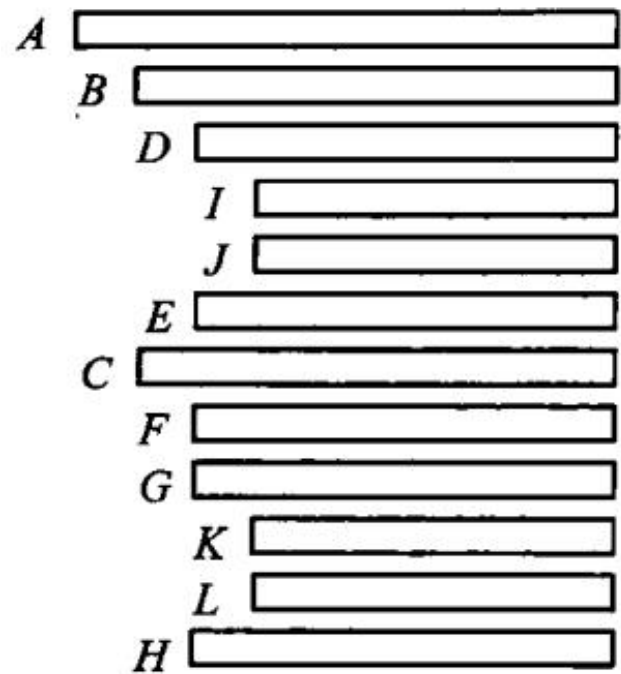
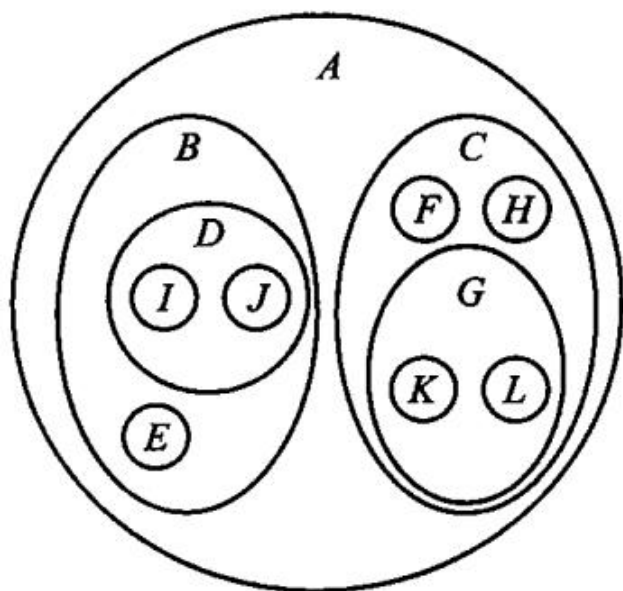


图 6.1 树形表示法

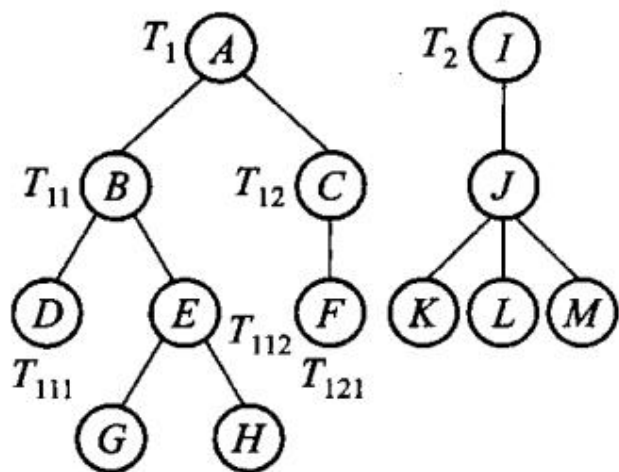


(a) 凹入表表示法

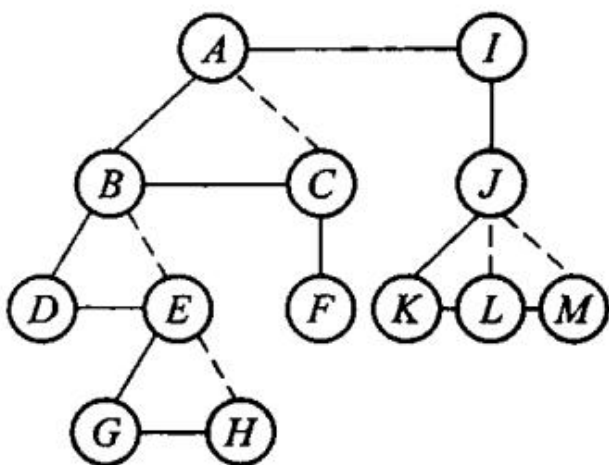
第 6 章插图（底稿第 4808 行，原书无独立题注）



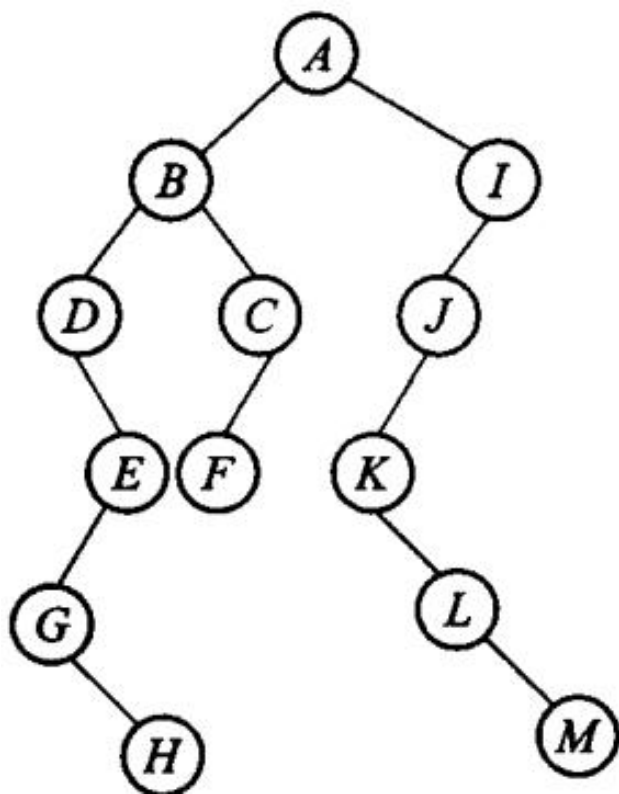
第 6 章插图（底稿第 4810 行，原书无独立题注）



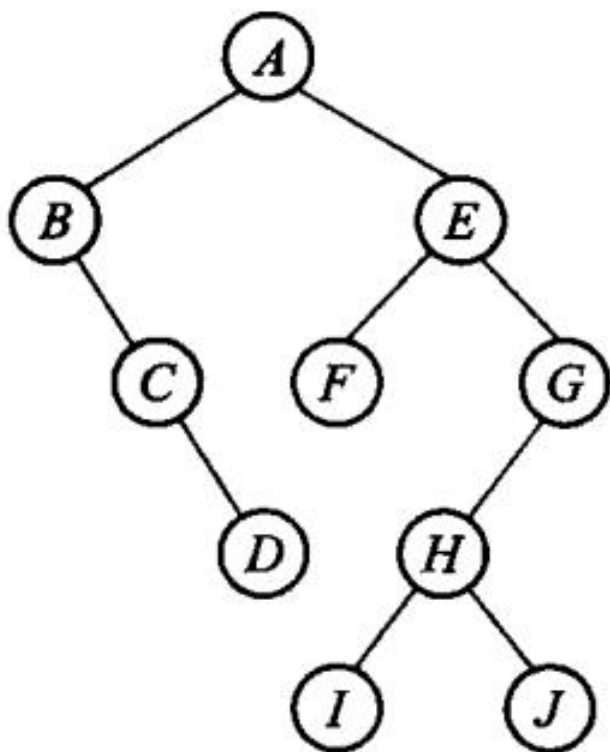
(a) 由 T_1 T_2 构成的森林



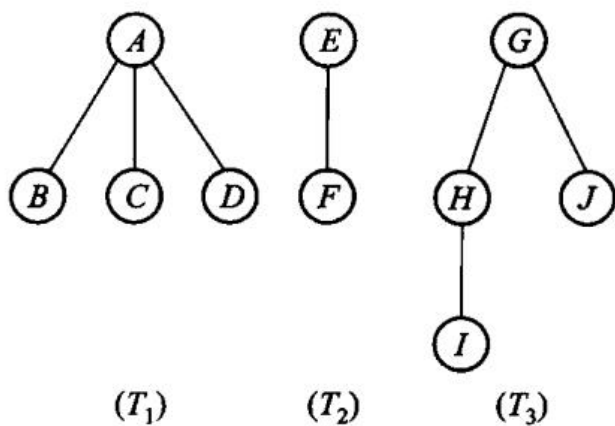
(b) 经连线、切线处理后的二叉树



(c) 进行旋转后的二叉树；图 6.3 森林转换为二叉树

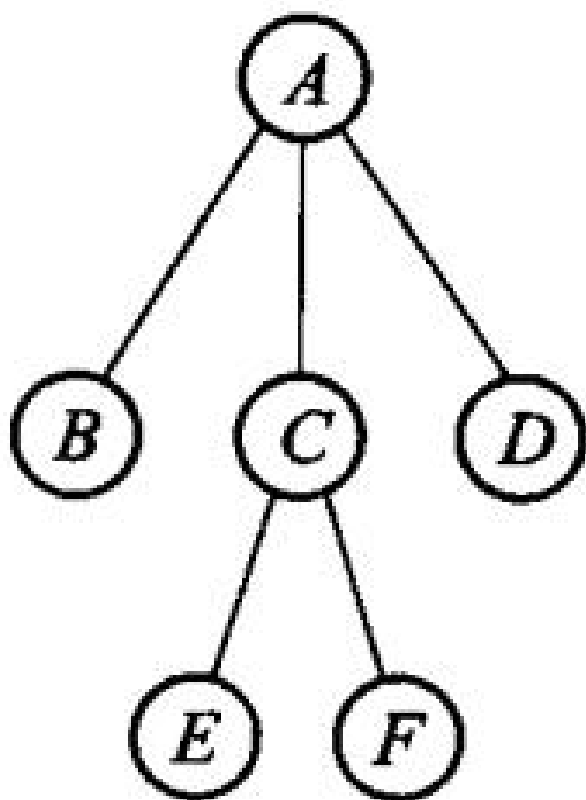


第 6 章插图（底稿第 4874 行，原书无独立题注）

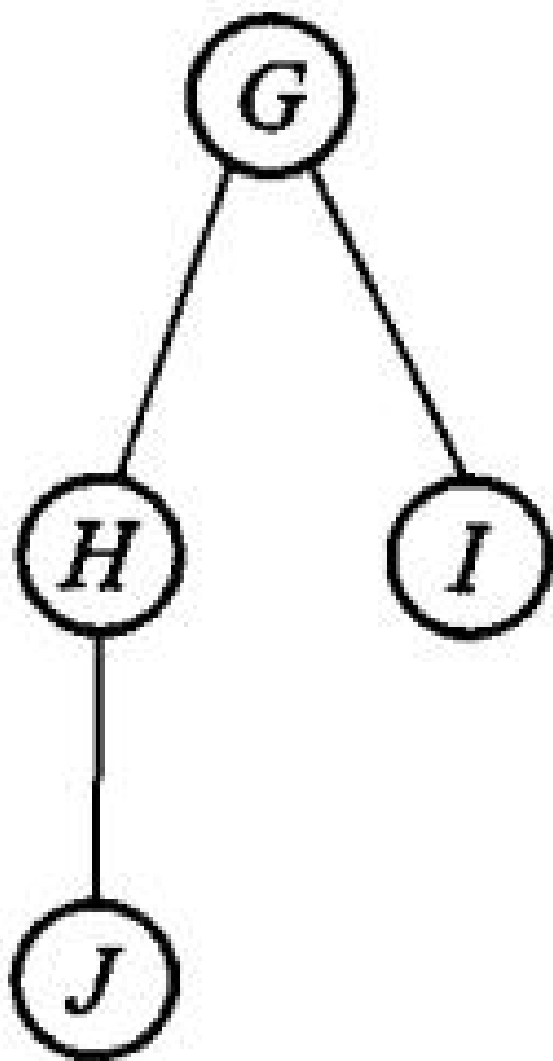


(b) 森林

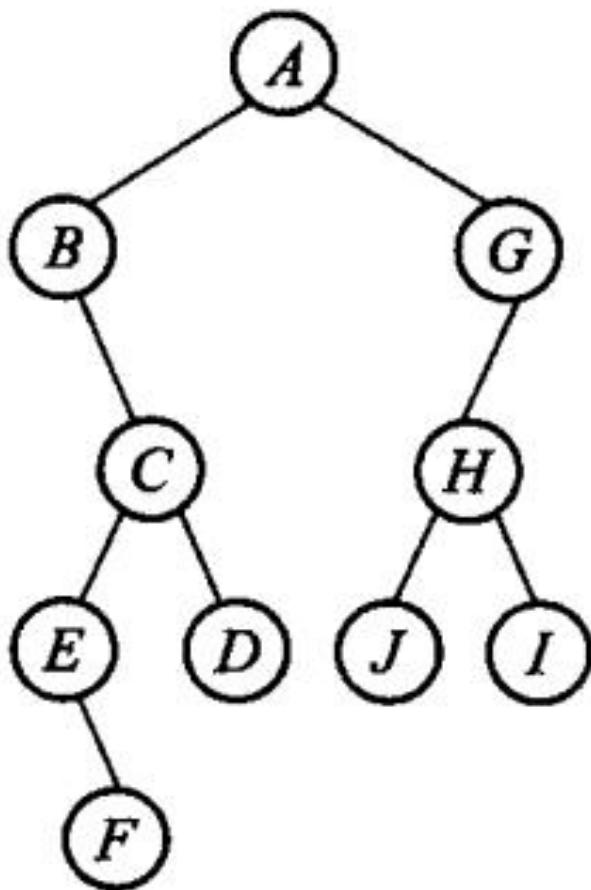
图 6.4 二叉树转换为森林



(a) 森林



第 6 章插图（底稿第 4959 行，原书无独立题注）



(b) 森林对应的二叉树；图 6.5 森林和对应的二叉树

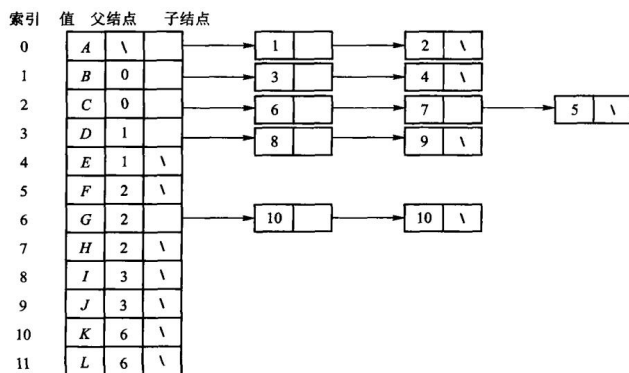


图 6.6 以“子结点表”表示法实现图 6.1 中的树

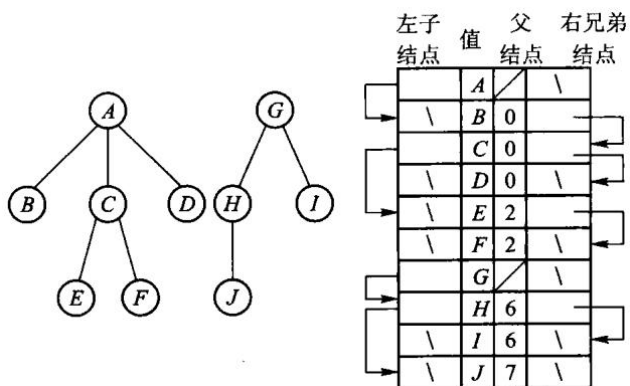


图 6.7“左子/右兄”表示两棵树

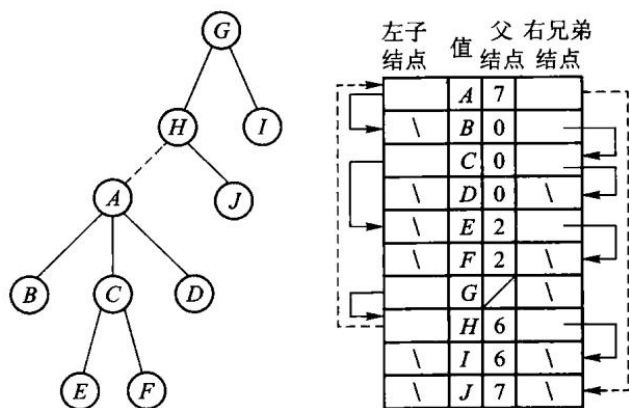
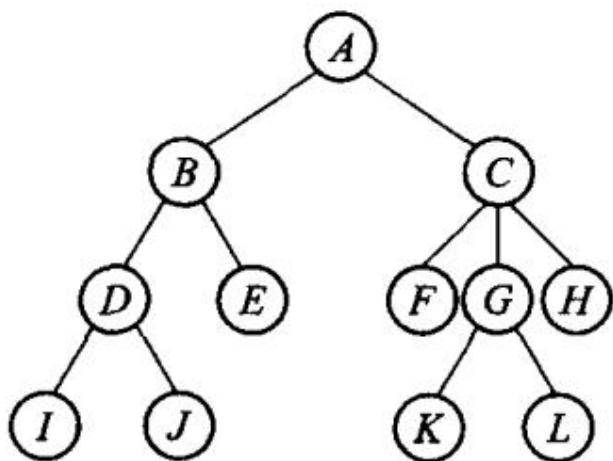
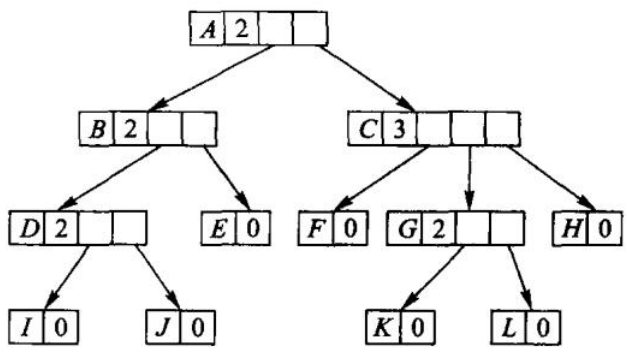


图 6.8 使用静态“左子/右兄”实现对树的归并



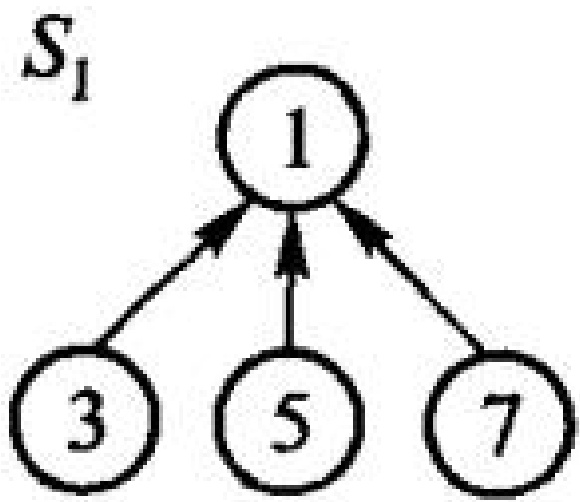
(a) 树



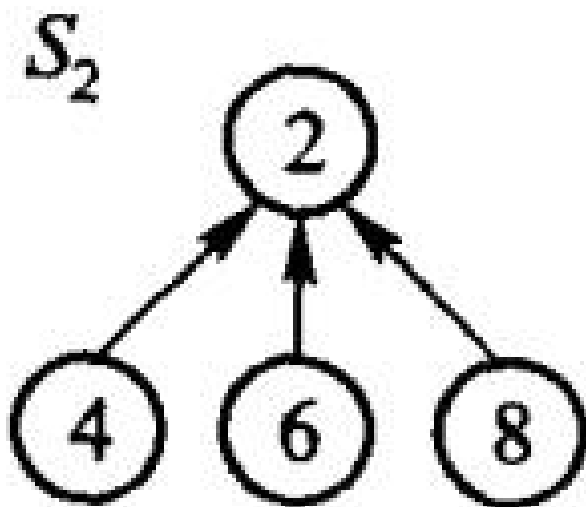
(b) 树的实现；图 6.9 树的动态表示法

结点索引	0	1	2	3	4	5	6	7	8	9	10	11
值	A	B	C	D	E	F	G	H	I	J	K	L
父结点索引	\	0	0	1	1	2	2	2	3	3	6	6

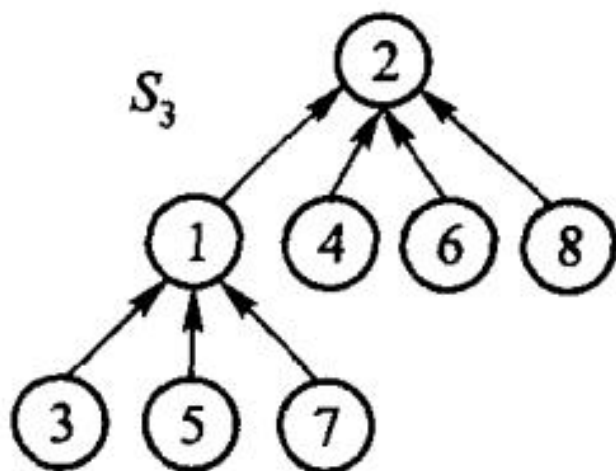
图 6.10 父指针表示法



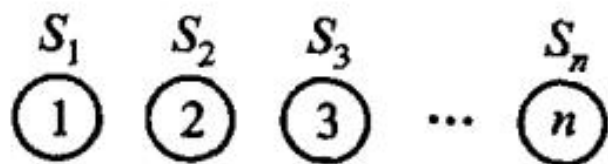
(a) 子集 s_1



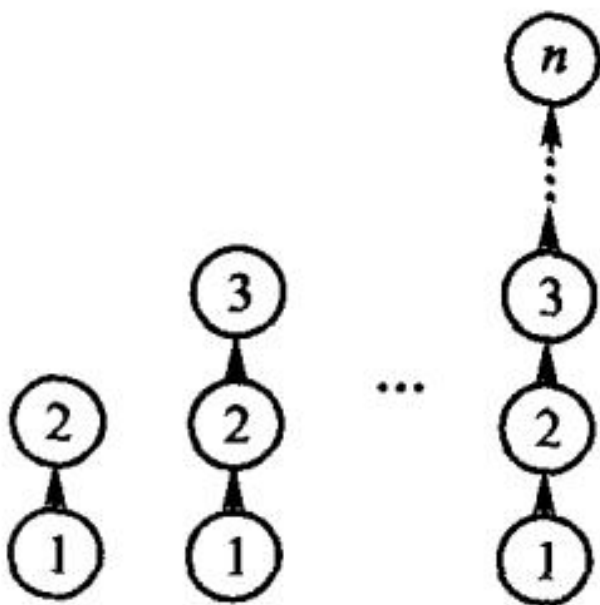
(b) 子集 s_2



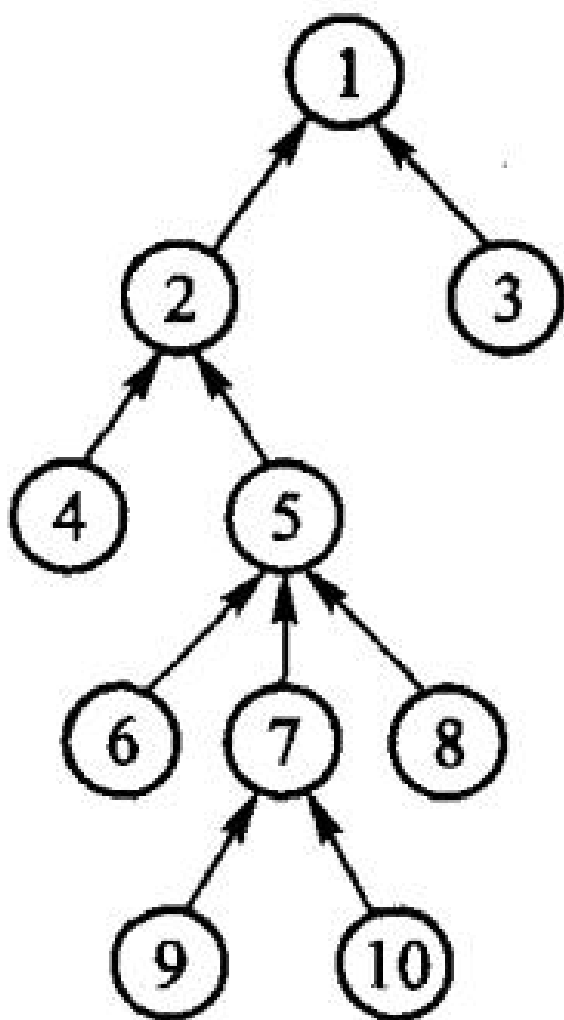
(c) 并集 S_3 ; 图 6.11 集合的表示方法



(a) n 个集合

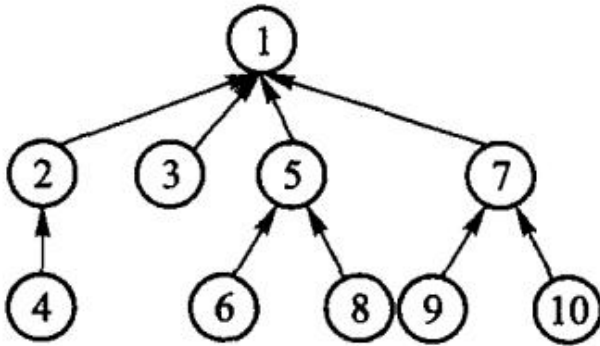


(b) 合并操作；图 6.12 合并操作的一个极端情况



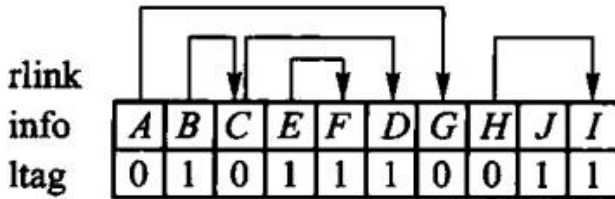
(a) 路径压缩之前

第 6 章插图（底稿第 5345 行，原书无独立题注）



(b) 路径压缩之后

图 6.13 路径压缩示例



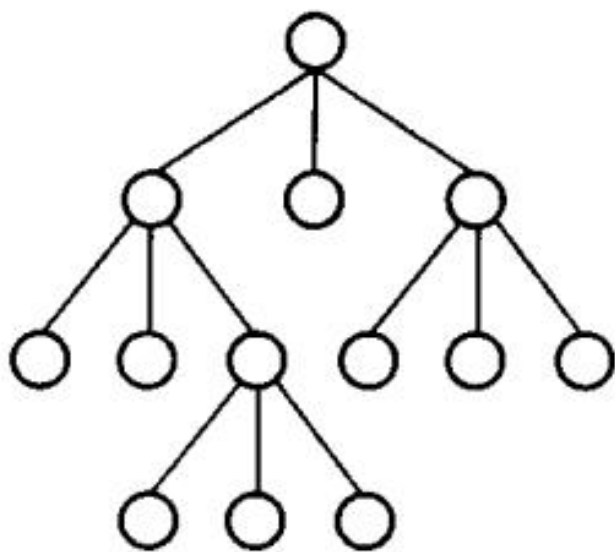
第 6 章插图（底稿第 5388 行，原书无独立题注）

info	B	E	F	C	D	A	J	H	I	G
degree	0	0	0	2	0	3	0	1	0	2

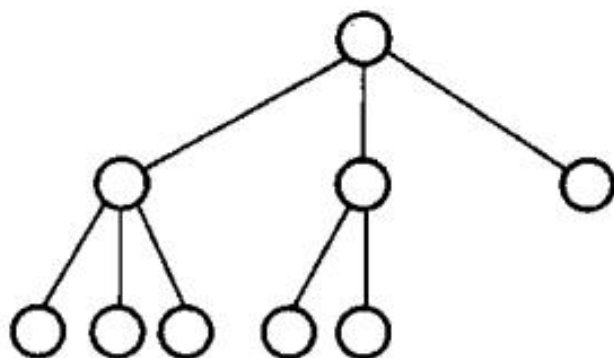
图 6.16 带度数的后根次序表示法

0	0	1	0	1	0	1	1	1	1
A	G	B	C	D	H	I	E	F	J
0	1	0	0	1	0	1	0	1	1

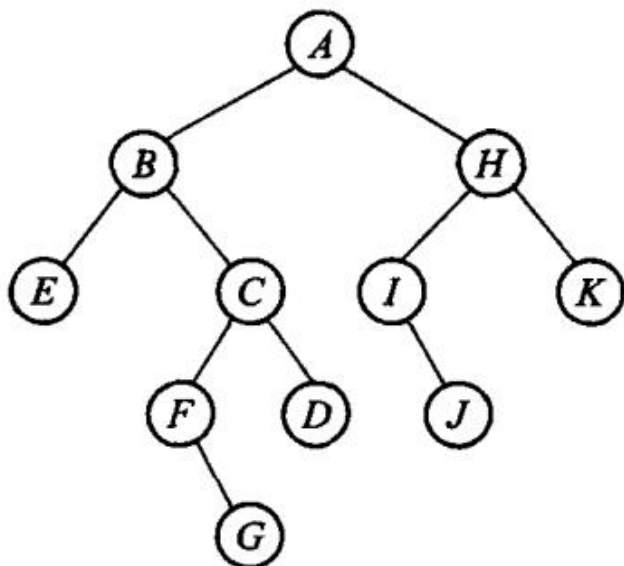
图 6.17 带双标记的层次次序表示法



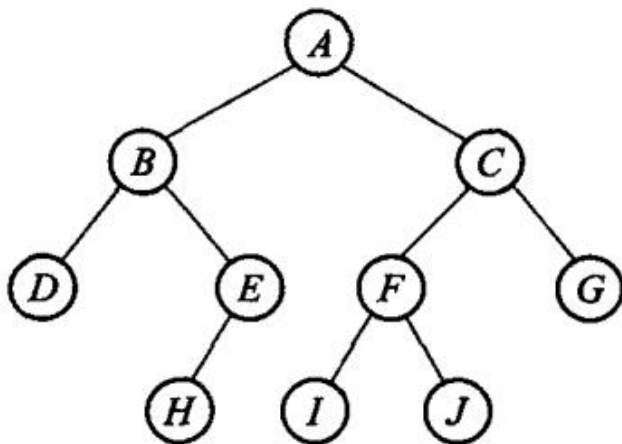
(a) 满 3 叉树



(b) 完全 3 叉树；图 6.18 满 3 叉树与完全 3 叉树



第 6 章插图（底稿第 5584 行，原书无独立题注）



第 6 章插图（底稿第 5604 行，原书无独立题注）

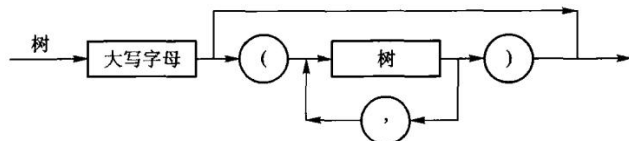
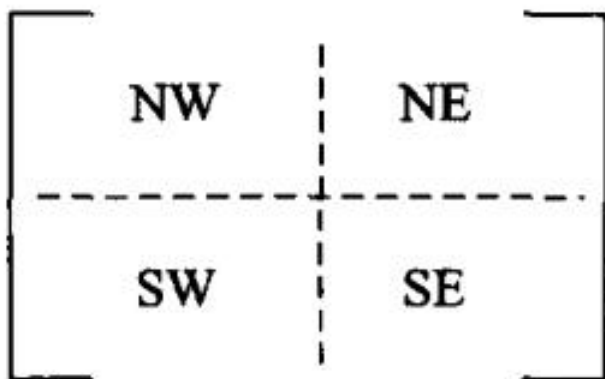
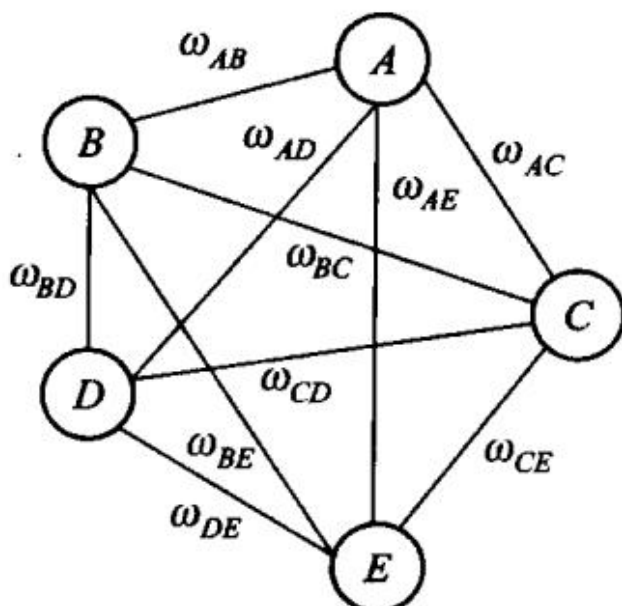


图 6.21 上机题 3 的图示

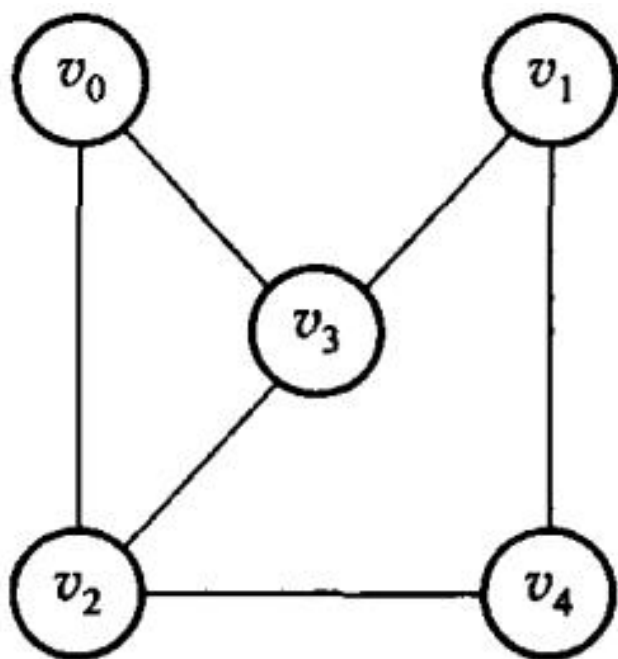


第 6 章插图（底稿第 5633 行，原书无独立题注）

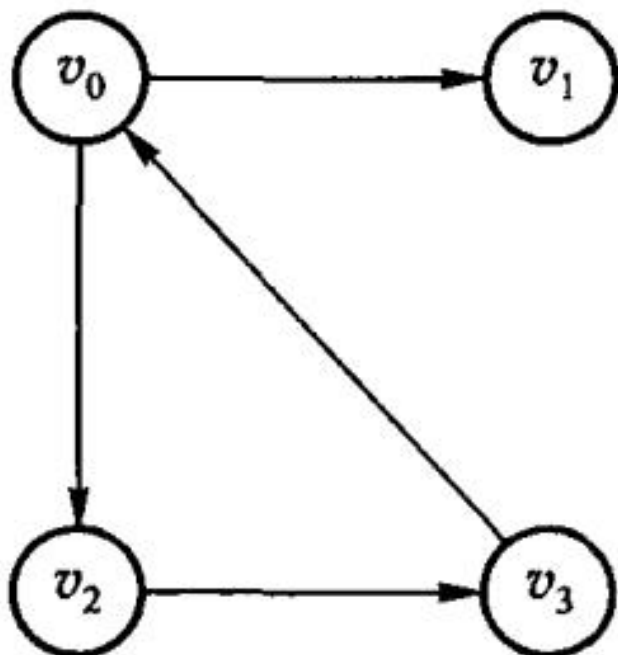
第 7 章 图



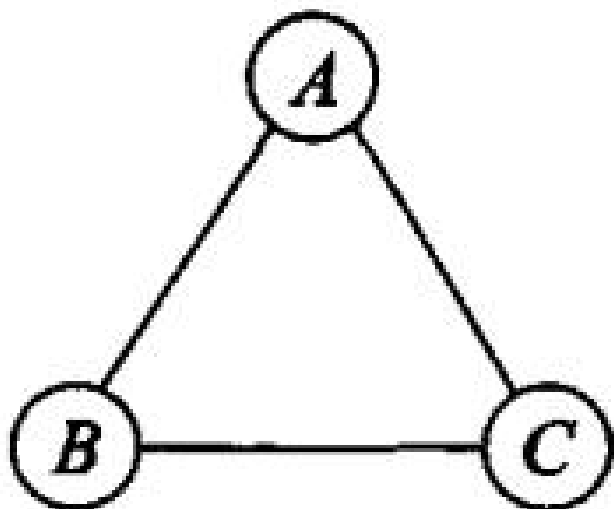
第 7 章插图（底稿第 5661 行，原书无独立题注）



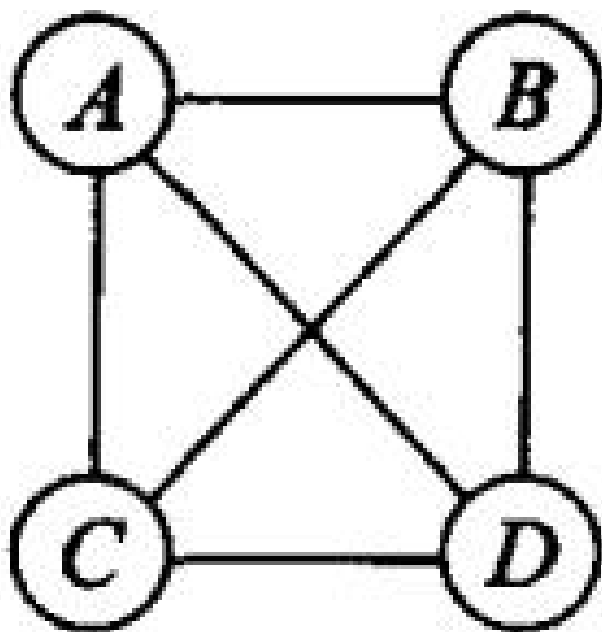
(a) 无向图 G_1



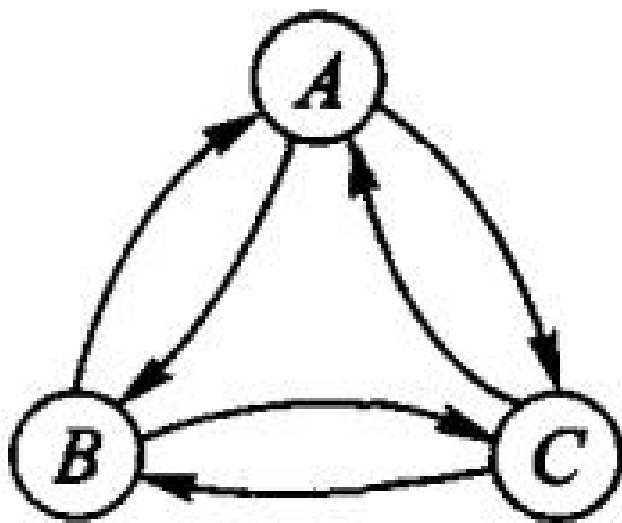
(b) 有向图 G_2 ; 图 7.2 图的示例



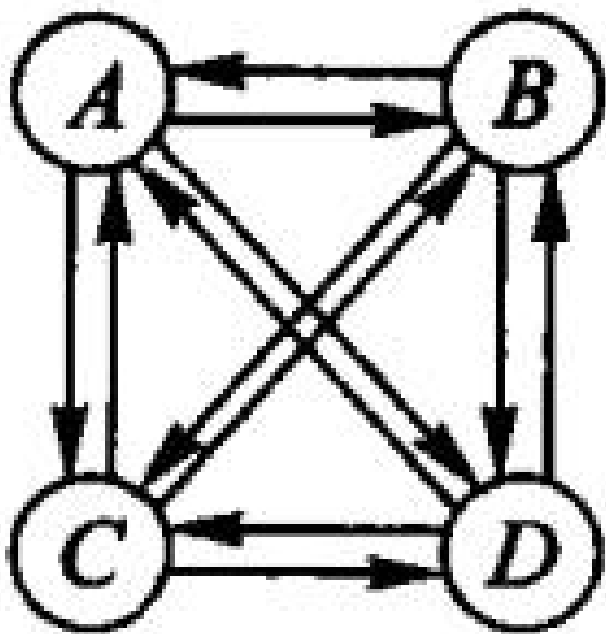
第 7 章插图（底稿第 5704 行，原书无独立题注）



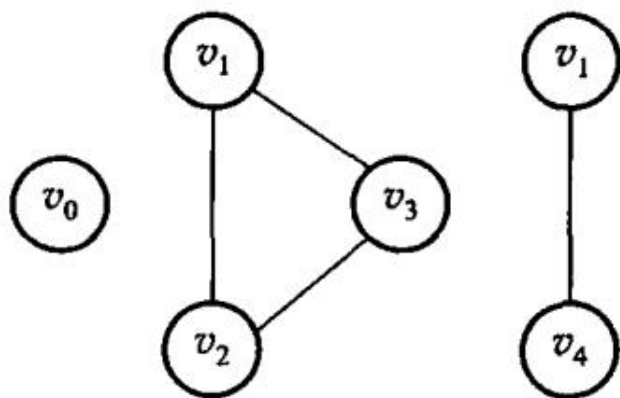
(a) 无向完全图



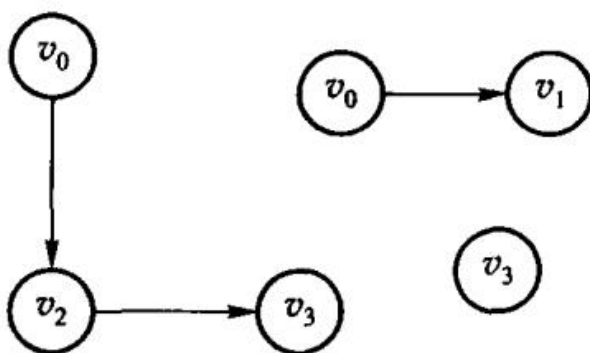
第 7 章插图（底稿第 5709 行，原书无独立题注）



(b) 有向完全图；图 7.3 完全图



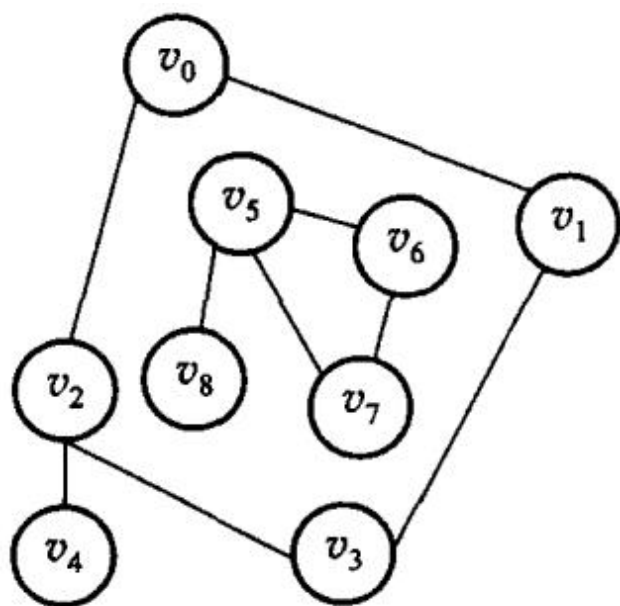
(a) 无向图 G_1 的若干子图



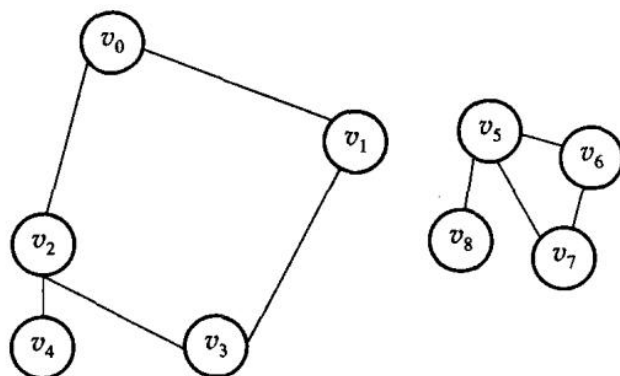
(b) 有向图 G_2 的若干子图；图 7.4 图 7.2 的若干子图



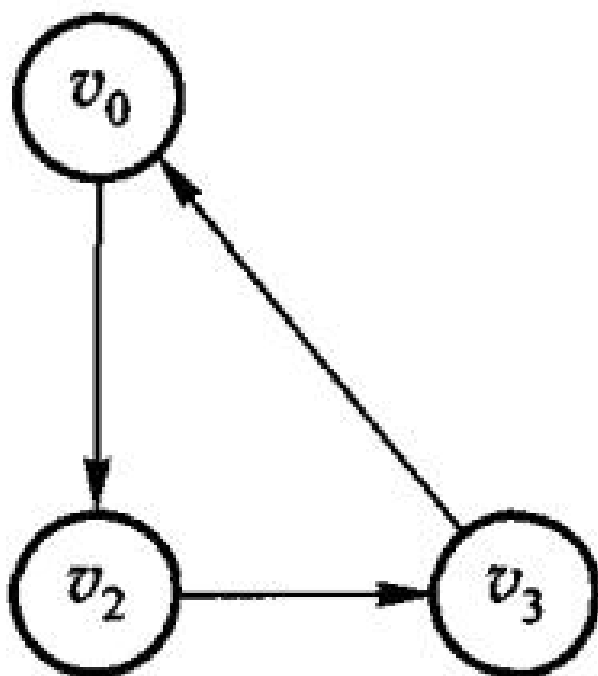
第 7 章插图（底稿第 5728 行，原书无独立题注）



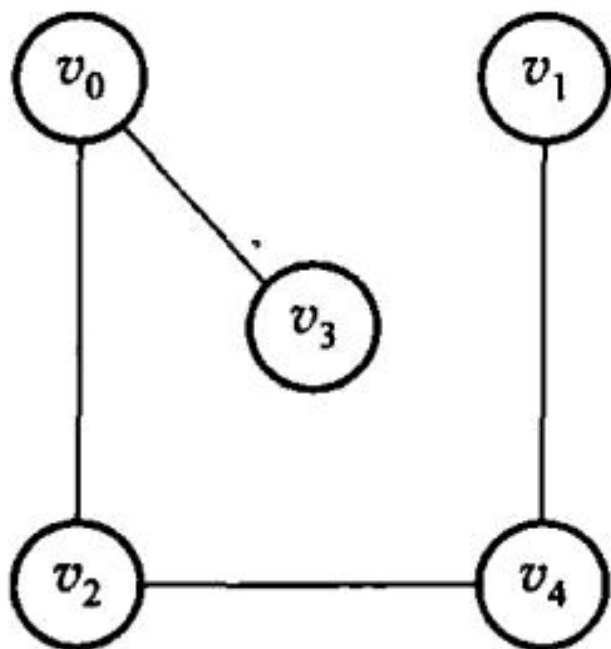
(a) 非连通的无向图 G_3



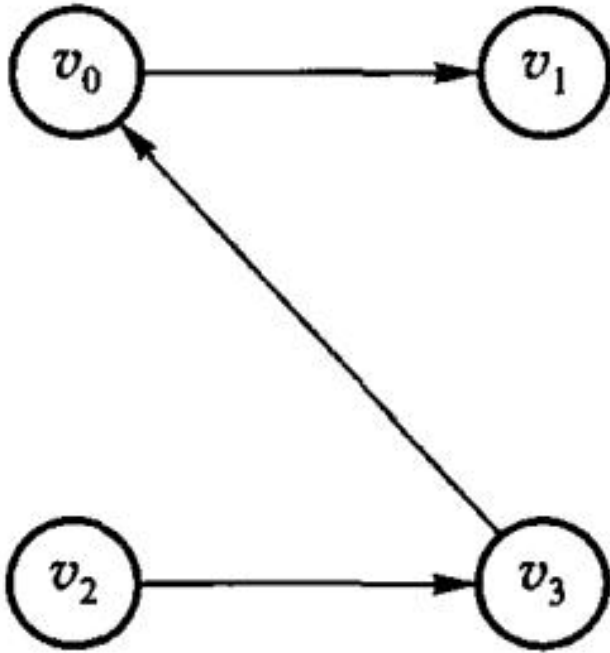
(b) 非连通无向图 G_3 的连通分量; 图 7.5 非连通无向图的连通分量示意图



第 7 章插图（底稿第 5743 行，原书无独立题注）



(a) 无向图 G_1 的生成树



(b) 有向图 G_2 的生成树；图 7.7 图 7.2 中无向图和有向图的生成树示例

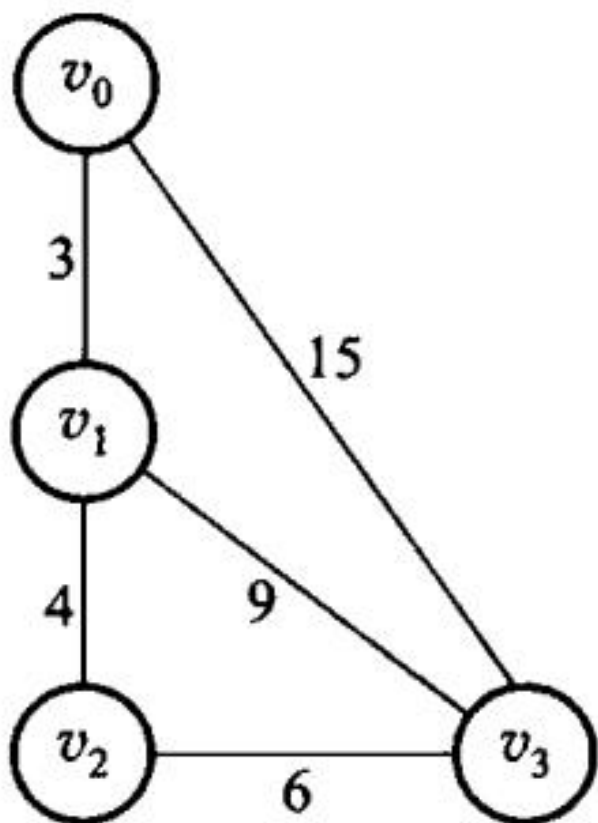
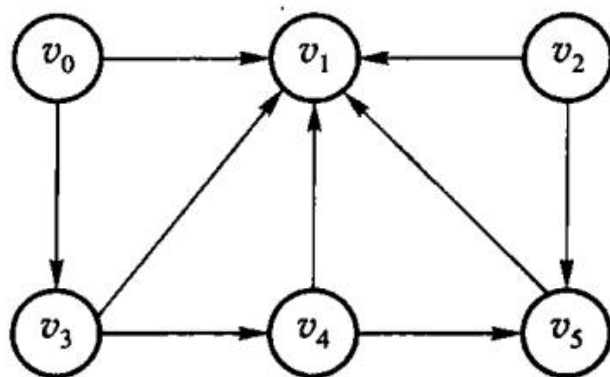


图 7.8 网络实例 G_a



(a) 有向图

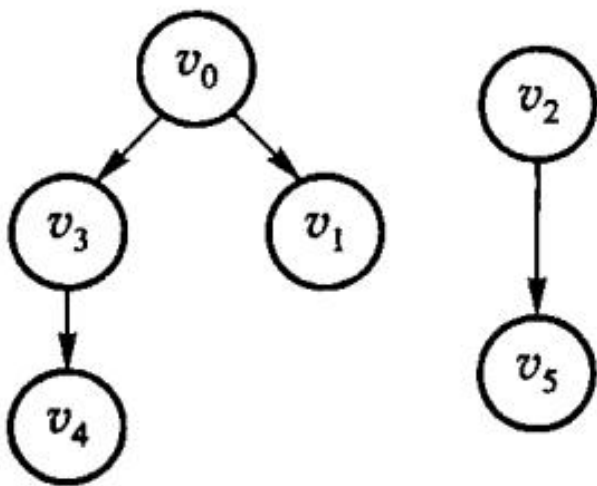


图 7.9 有向图及其生成森林; (b) 有向图的生成森林

顶点结点		边(或弧)结点		
data	firstarc	adjvex	nextarc	info

第 7 章插图 (底稿第 5978 行, 原书无独立题注)

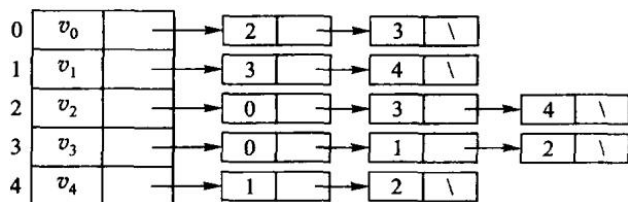


图 7.12 无向图 G_1 的邻接表表示; 图 7.13(a) 所示为有向图 G_2 的邻接表表示。对于有向图, 一条弧在邻接表中只出现一次。顶点 v_i 的边表的表目个数是该顶点的出度, 因此邻接表也称为出边表; 要知道顶点 v_i 的入度, 必须遍历整个邻接表, 查看有多少个边表目中的顶点 (即弧头顶点) 为 v_i 。为了便于确定顶点结点的入度, 可以建立有向图的逆邻接表, 如图 7.13(b) 所示。在有向图的逆邻接表中, 顶点 v_i 的边表中表示的是以该顶点为终点的边, 因此逆邻接表也称为入边表。逆邻接表中顶点 v_i 的边表的表目个数是该顶点的入度。

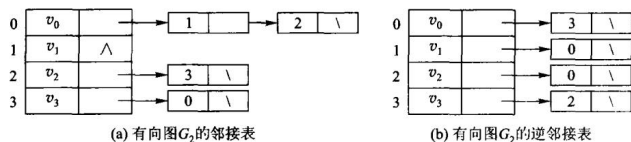


图 7.13 有向图 G_i 的邻接表和逆邻接表表示

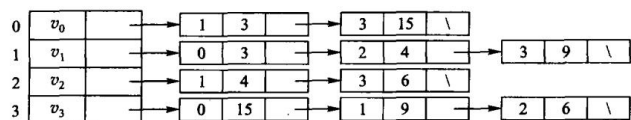


图 7.14 带权图 G_4 的邻接表表示

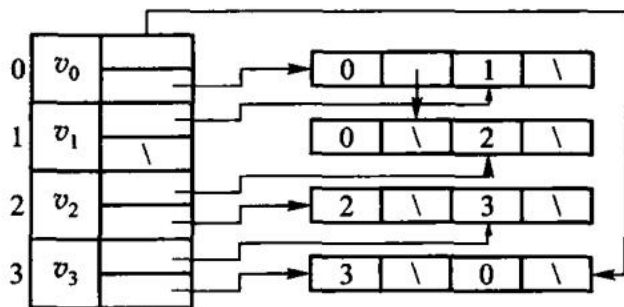


图 7.15 有向图的十字链表示例

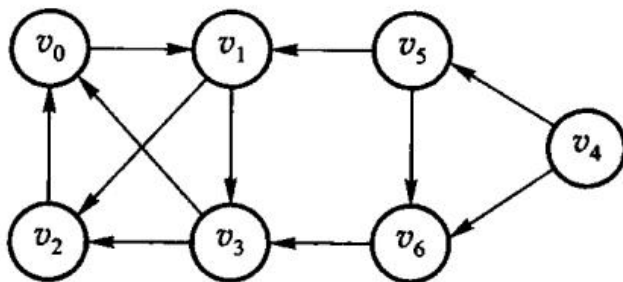


图 7.16 有向图 G

```
Visit(G, G.ToVertex(e));
G.Mark[G.ToVertex(e)] = VISITED;
Q.push(G.ToVertex(e));
```

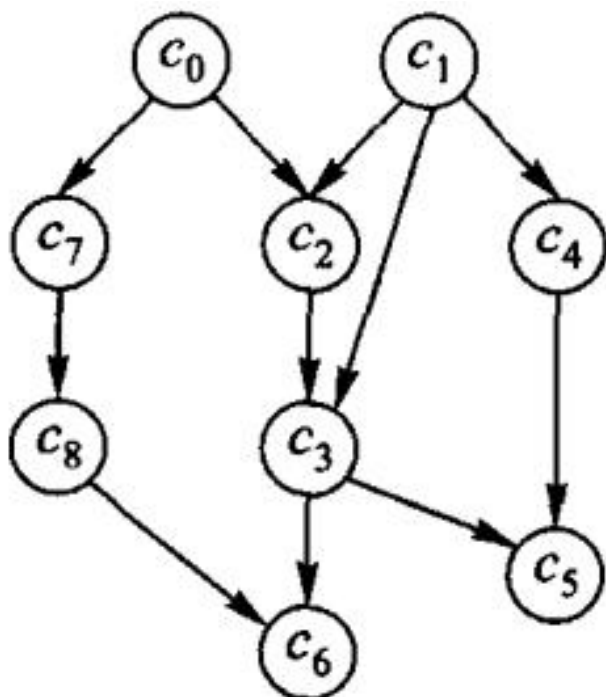
```
}
```

```
}
```

```
}
```

【算法 7.6 结束】

第 7 章插图（底稿第 6191 行，原书无独立题注）



第 7 章插图（底稿第 6214 行，原书无独立题注）

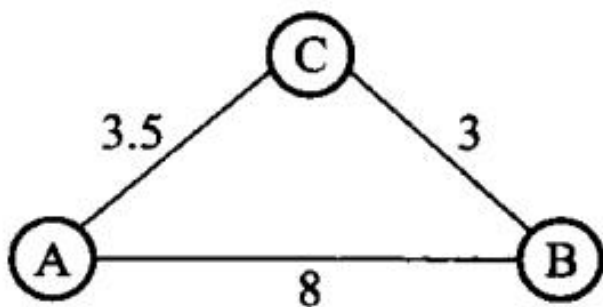


图 7.18 最短路径的示例

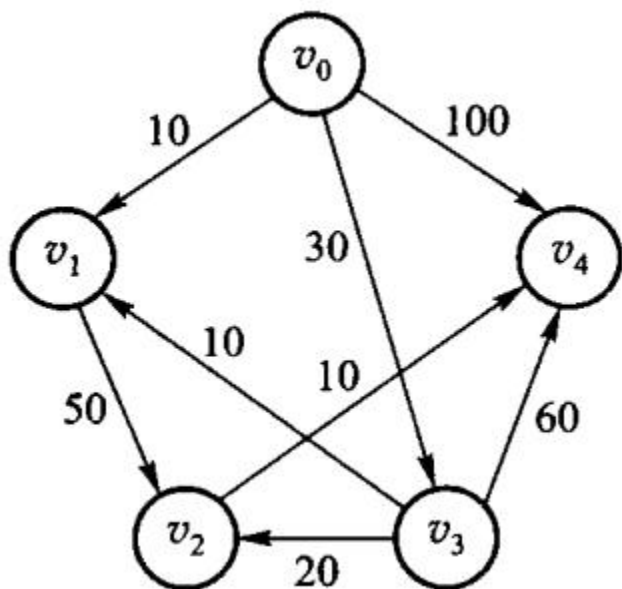


图 7.19 单源最短路径的示例

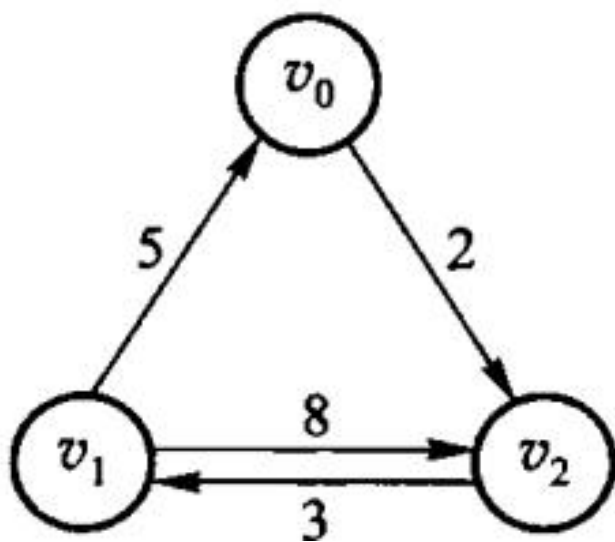


图 7.20 每对顶点间的最短路径的示例

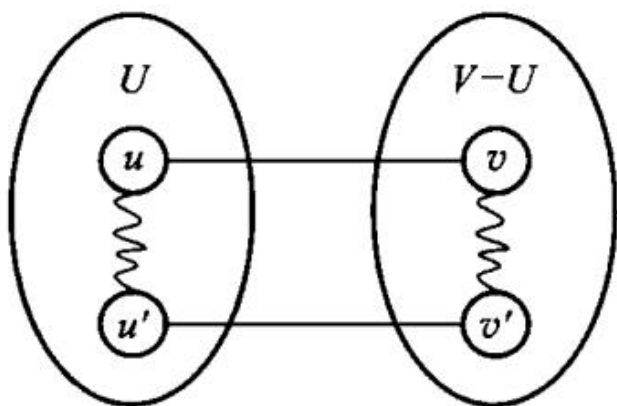
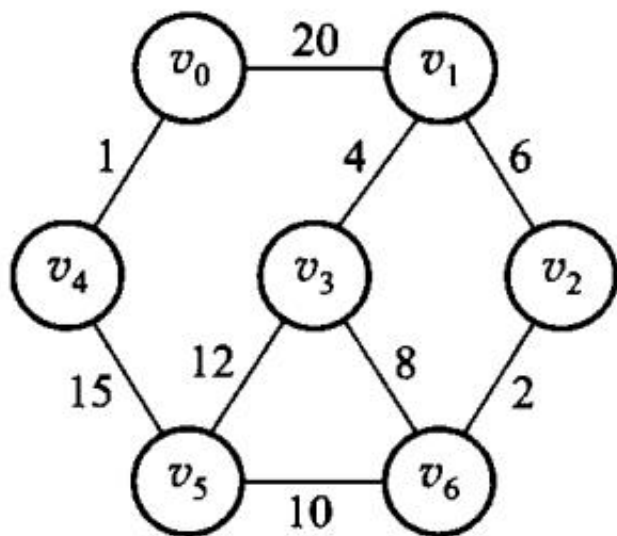
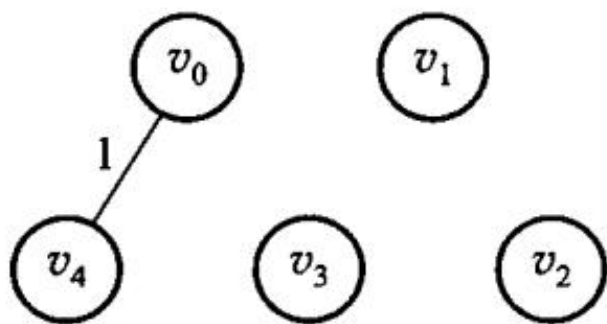


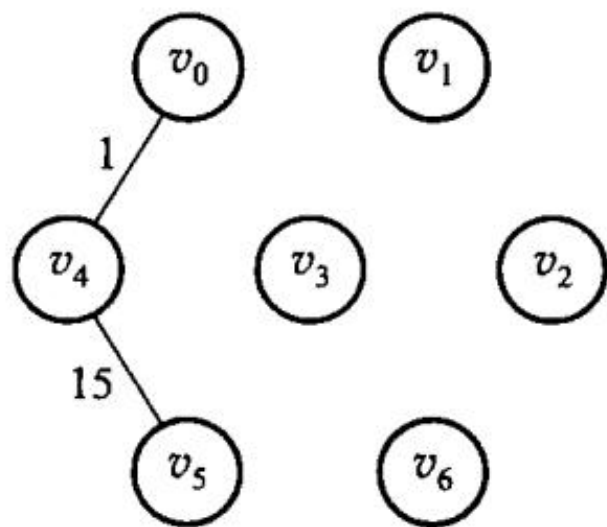
图 7.22 含 (u,v) 的回路



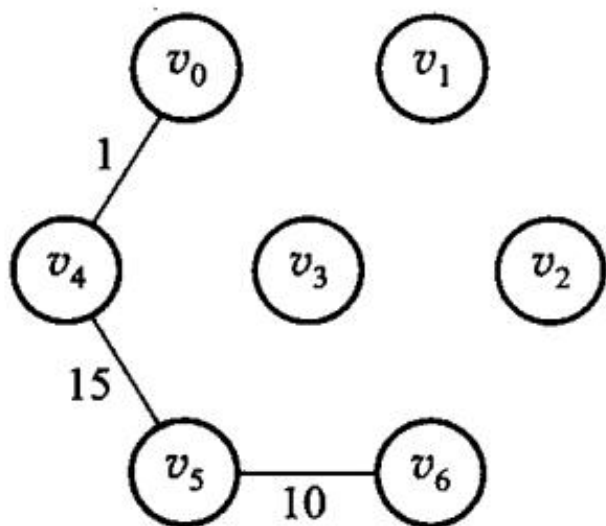
第 7 章插图（底稿第 6463 行，原书无独立题注）



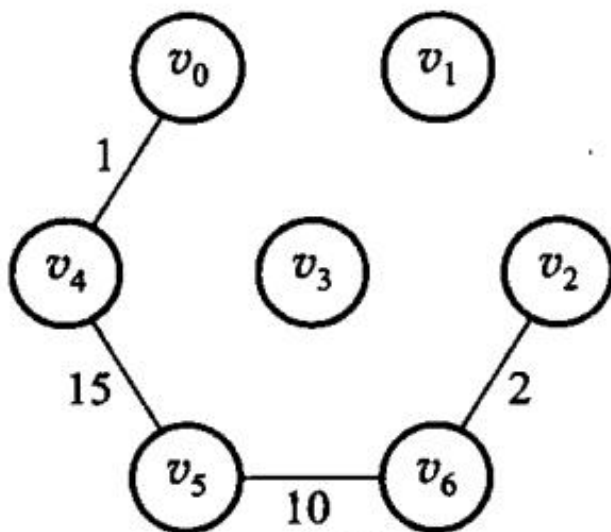
(a) 步骤 1



(b) 步骤 2

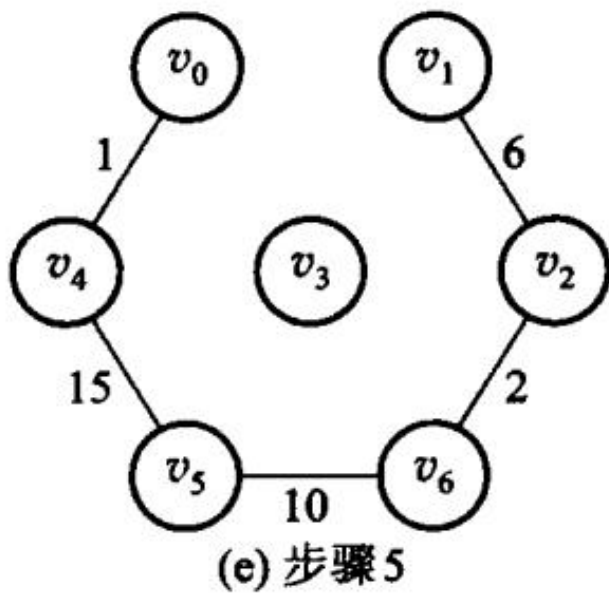


(c) 步骤 3



(d) 步骤 4

第 7 章插图（底稿第 6481 行，原书无独立题注）



第 7 章插图（底稿第 6483 行，原书无独立题注）

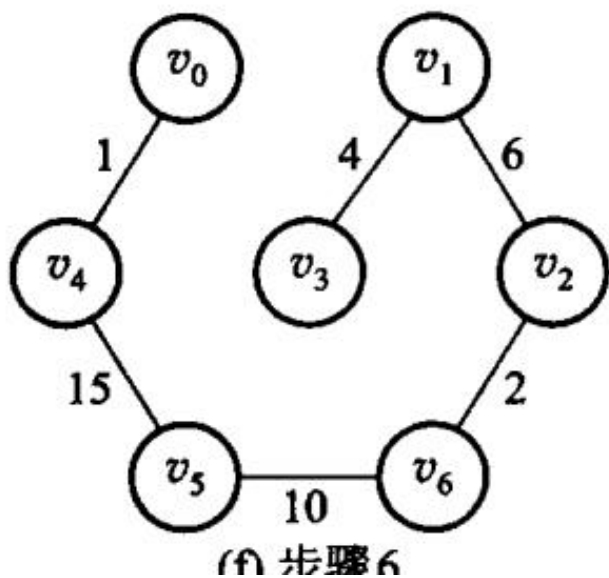
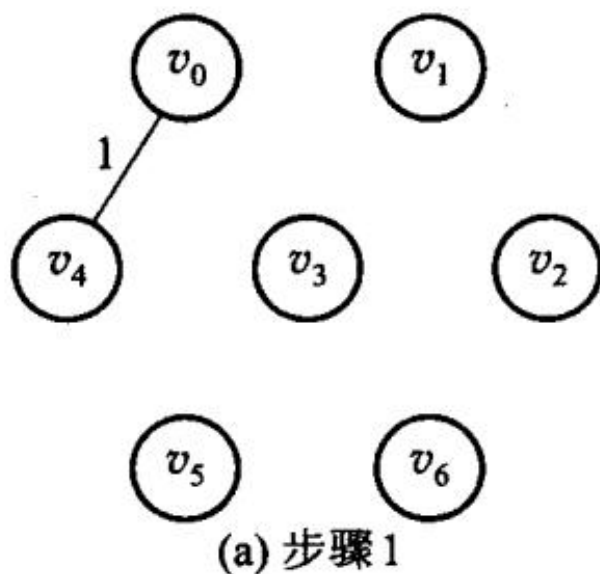
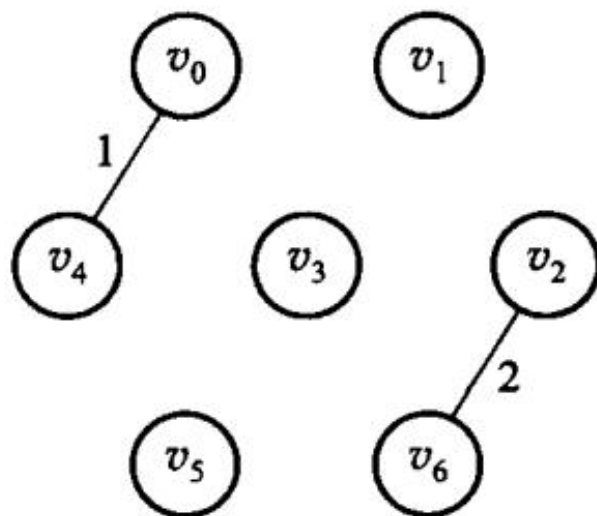


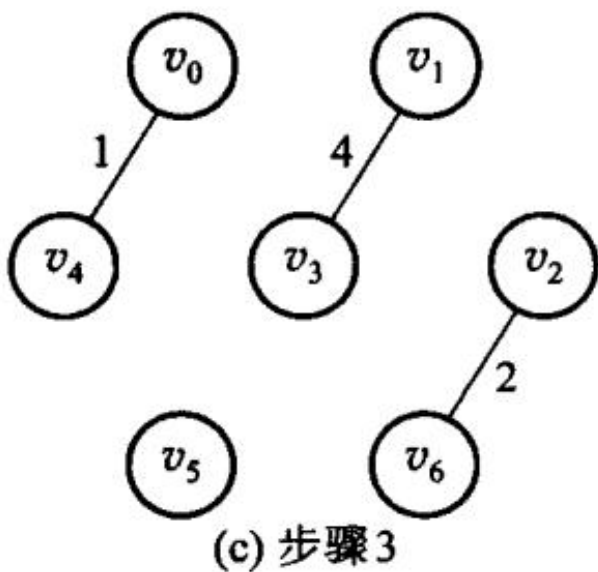
图 7.24 Prim 算法构造图 7.23 中带权图的最小生成树的步骤



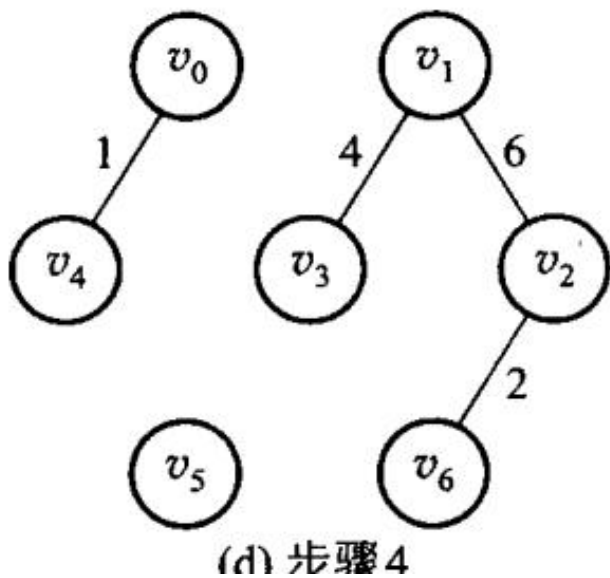
第 7 章插图（底稿第 6545 行，原书无独立题注）



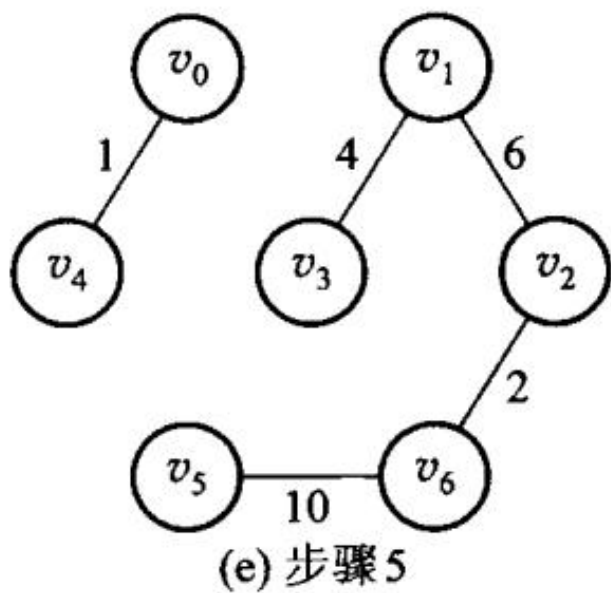
第 7 章插图（底稿第 6547 行，原书无独立题注）



第 7 章插图（底稿第 6549 行，原书无独立题注）



第 7 章插图（底稿第 6551 行，原书无独立题注）



第 7 章插图（底稿第 6553 行，原书无独立题注）

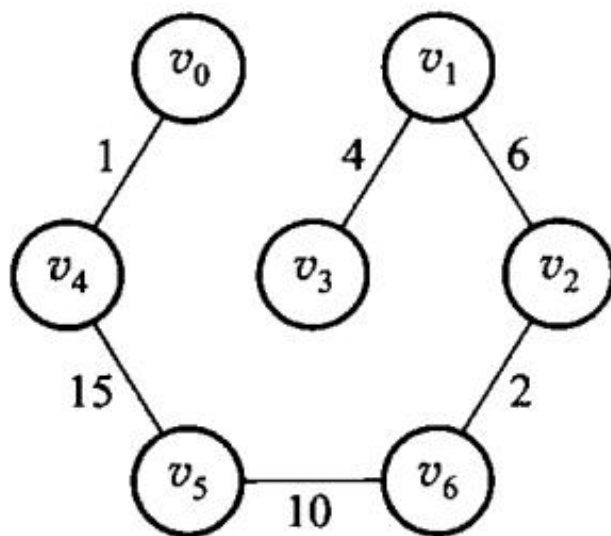


图 7.25 Kruskal 算法构造图 7.23 中带权图的最小生成树的步骤

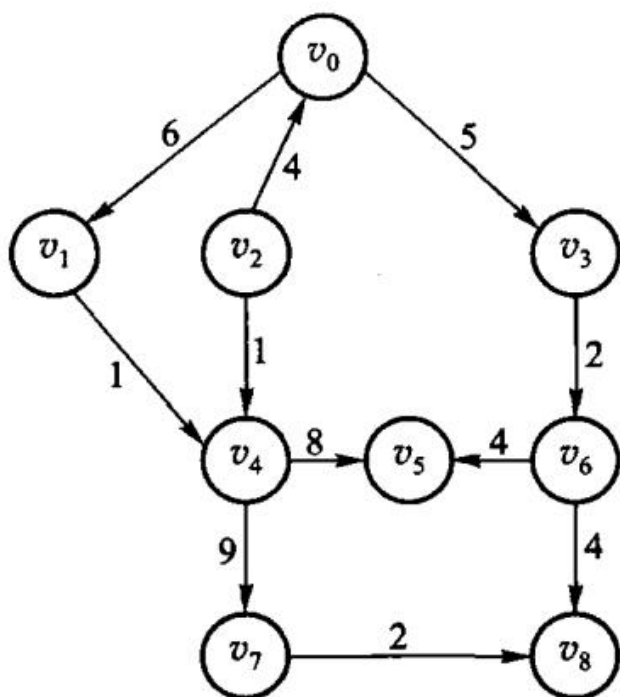


图 7.26 习题 1 的图例

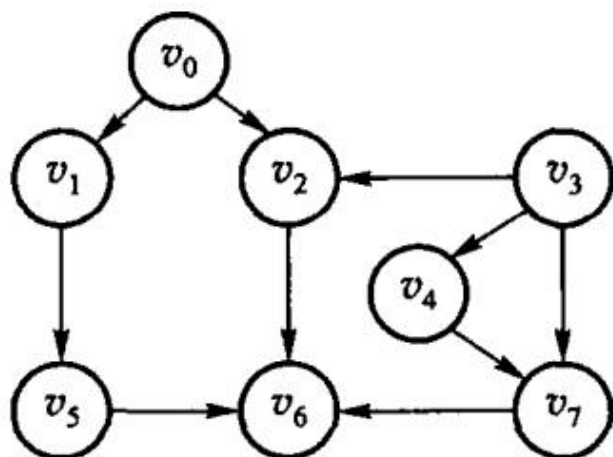


图 7.27 习题 2 的图例

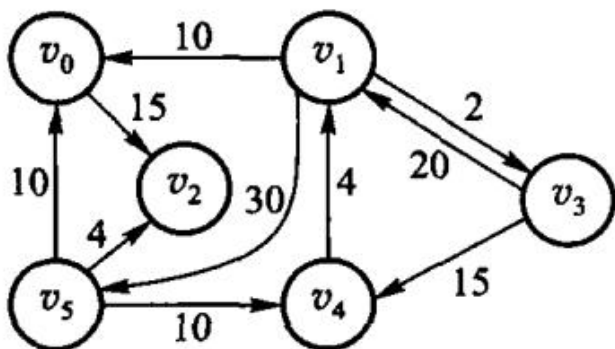


图 7.28 习题 3 的图例

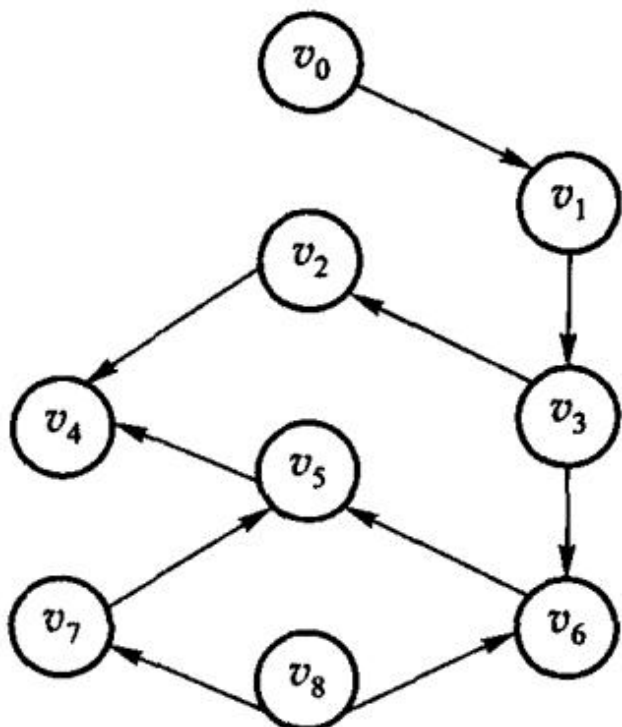


图 7.29 习题 4 的图例

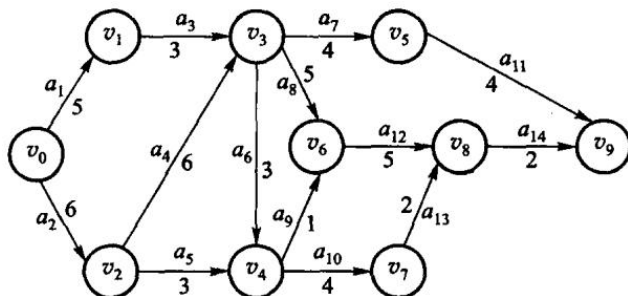


图 7.30 上机题 2 的图例

第 8 章 内排序

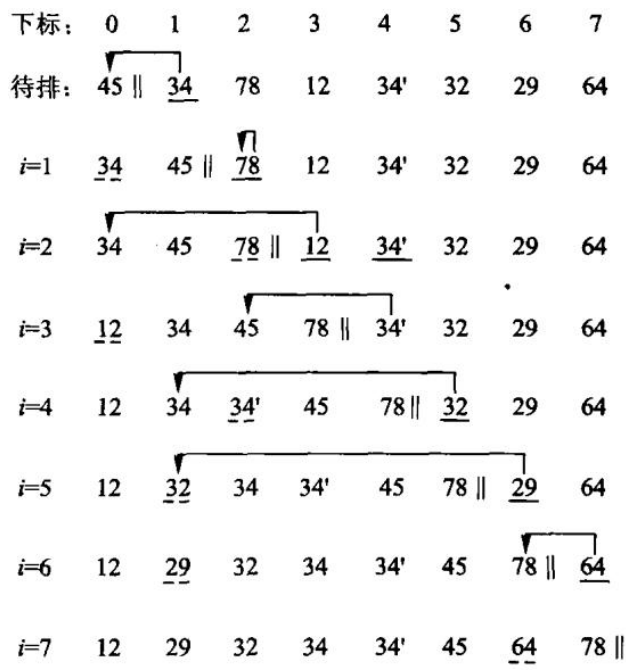


图 8.1 插入排序

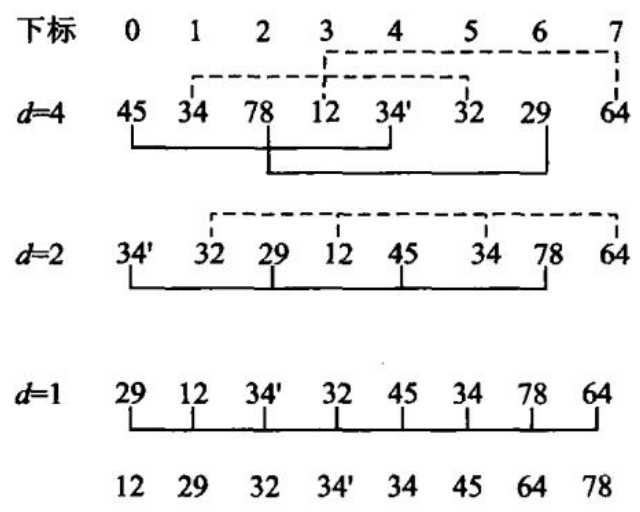
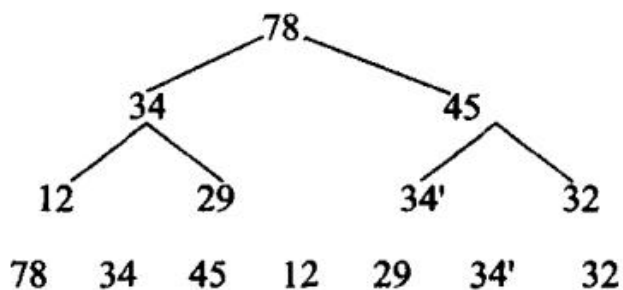
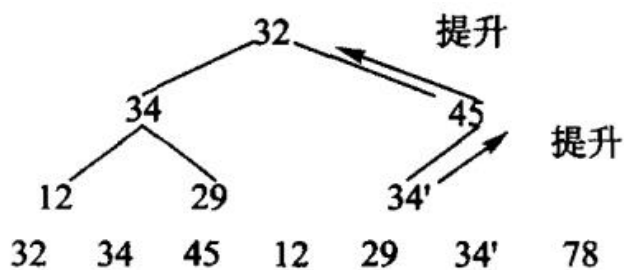


图 8.2 Shell 排序

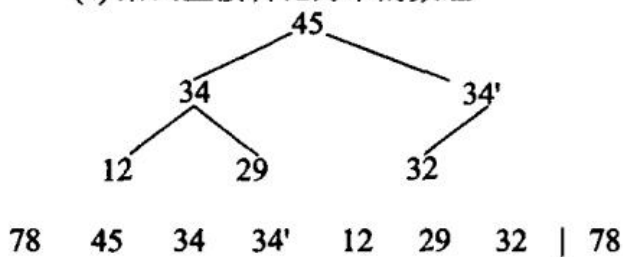


第 8 章插图（底稿第 6971 行，原书无独立题注）

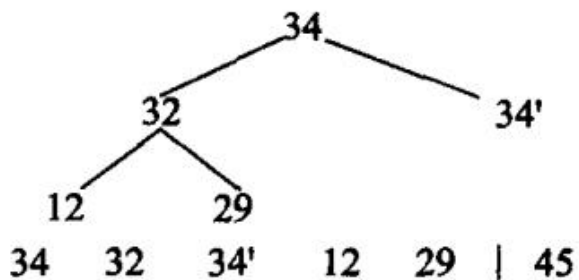


第 8 章插图（底稿第 6973 行，原书无独立题注）

(a) 给出直接转化为堆的数组



(b) 删除最大值 78，用 32 替代；(c) 重新建堆后得到的序列



(d) 得到新的序列；图 8.4 堆排序

下标: 0 1 2 3 4 5 6 7

$i=0$ 45 34 78 12 34' 32 29 64

$i=1$ 12 || 45 34 78 29 34' 32 64

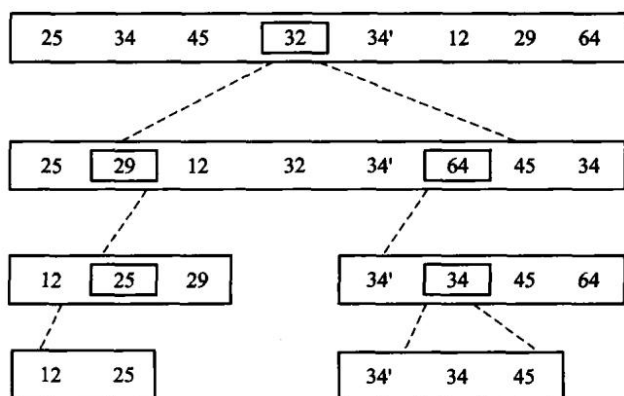
$i=2$ 12 29 || 45 34 78 32 34' 64

$i=3$ 12 29 32 || 45 34 78 34' 64

$i=4$ 12 29 32 34 || 45 34' 78 64

$i=5$ 12 29 32 34 34' || 45 64 78

图 8.5 冒泡排序



第 8 章插图（底稿第 7076 行，原书无独立题注）

下标	0	1	2	3	4	5	6	7
待分割	25	34	45	<u>32</u>	34'	12	29	64
第1行	25	34	45	64	34'	12	29	<u>32</u>
	<i>l</i>							<i>r</i>
第2行	25	34	45	64	34'	12	29	<u>32</u>
	<i>l</i>							<i>r</i>
第3行	25	<u>34</u>	45	64	34'	12	29	34
	<i>l</i>						<i>r</i>	
第4行	25	29	45	64	34'	12	<u>29</u>	34
		<i>l</i>					<i>r</i>	
第5行	25	29	<u>45</u>	64	34'	12	45	34
		<i>l</i>				<i>r</i>		
第6行	25	29	12	64	34'	<u>12</u>	45	34
			<i>l</i>			<i>r</i>		
第7行	25	29	12	<u>64</u>	34'	64	45	34
				<i>lr</i>				
结果	25	29	12	<u>32</u>	34'	64	45	34

图 8.7 分割过程

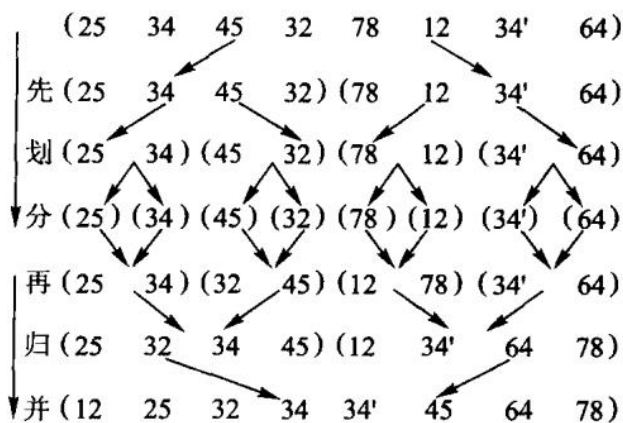


图 8.8 归并排序

count数组:

0	1	2	3	4	5	6	7	8	9
0	2	1	1	0	0	1	1	2	1

后继起始下标:

0	2	3	4	4	4	5	6	8	9
---	---	---	---	---	---	---	---	---	---

收集:

	2	3	6	7	8	8	9
--	---	---	---	---	---	---	---

(a) 高位优先

97 53 88 59 26 41 88' 31 22

0	1	2	3	4	5	6	7	8	9
		26 22	31	41	53 59			88 88'	

0 1 2 3 4 5 6 7 8 9

		22			26				
--	--	----	--	--	----	--	--	--	--

22 26 31 41 53 59 88 88' 97

(a) 高位优先

97 53 88 59 26 41 88' 31 22

0	1	2	3	4	5	6	7	8	9
	41 31	22	53			26	97	88 88'	59

41 31 22 53 26 97 88 88' 59

0 1 2 3 4 5 6 7 8 9

		22 26	31	41	53 59			88 88'	97
--	--	-------	----	----	-------	--	--	--------	----

22 26 31 41 53 59 88 88' 97

(b) 低位优先

204

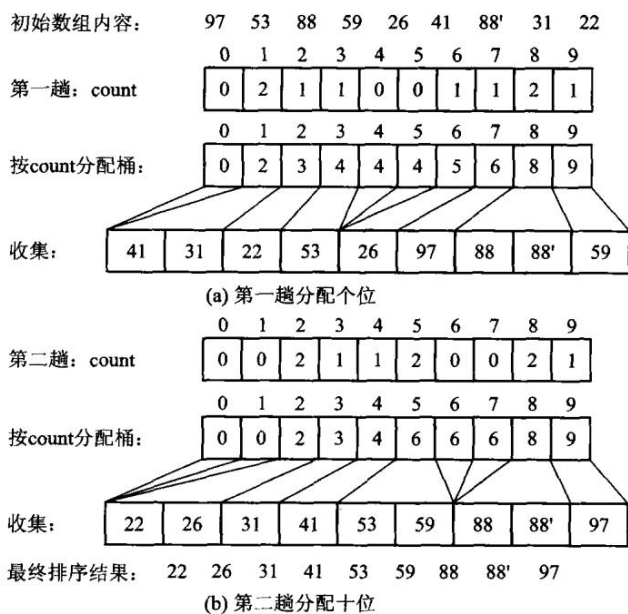
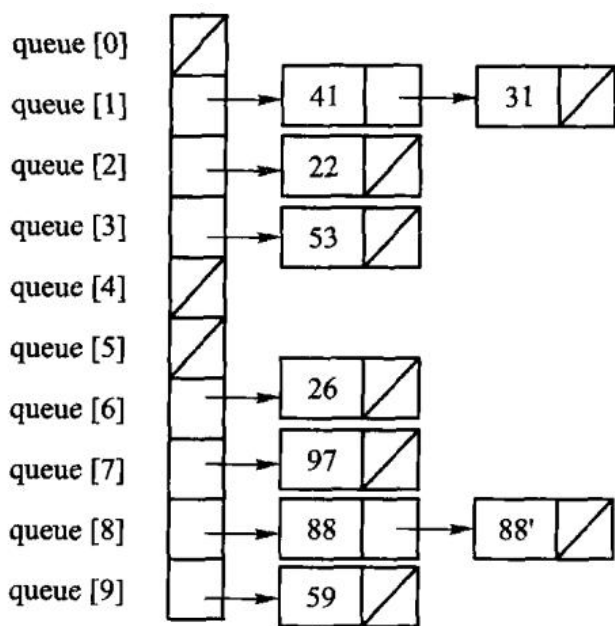
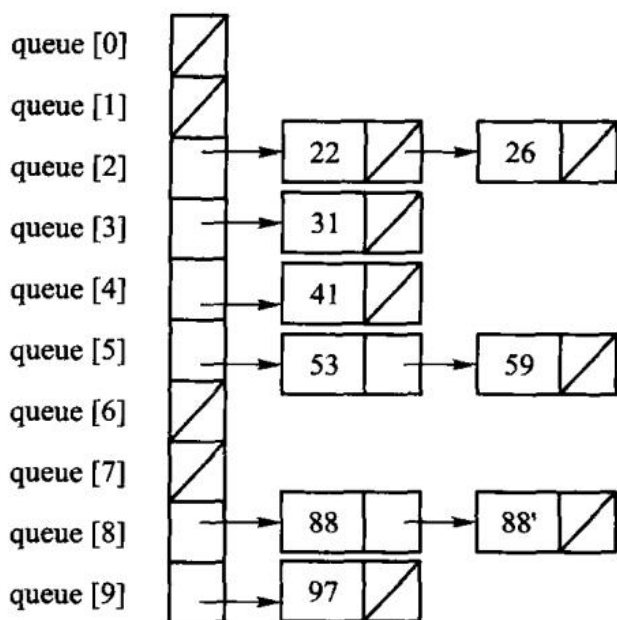


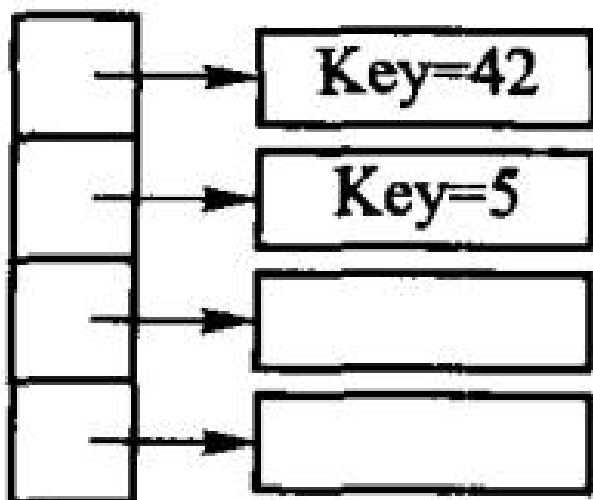
图 8.11 基于顺序存储和桶排序的基数排序



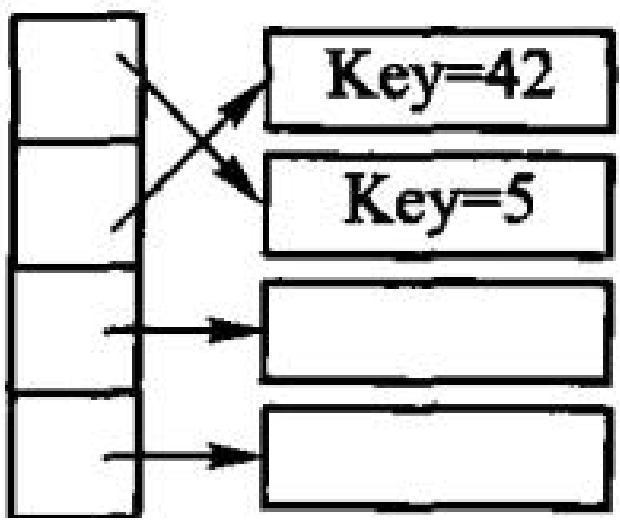
(b) 第一趟分配结果



(d) 第二趟分配结果



(a) 待排序记录及索引



(b) 排序后的记录；图 8.13 索引排序示意图

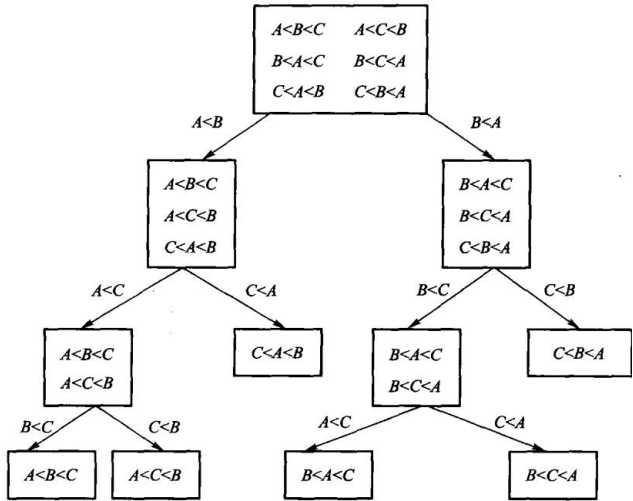


图 8.14 用判定树来模拟基于比较的排序

第 9 章 文件管理与外排序

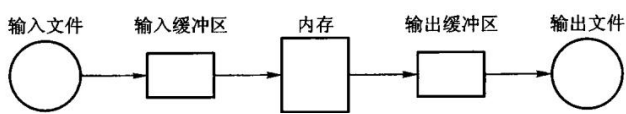


图 9.1 置换选择算法流程

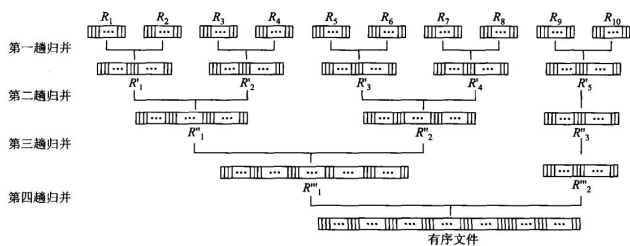
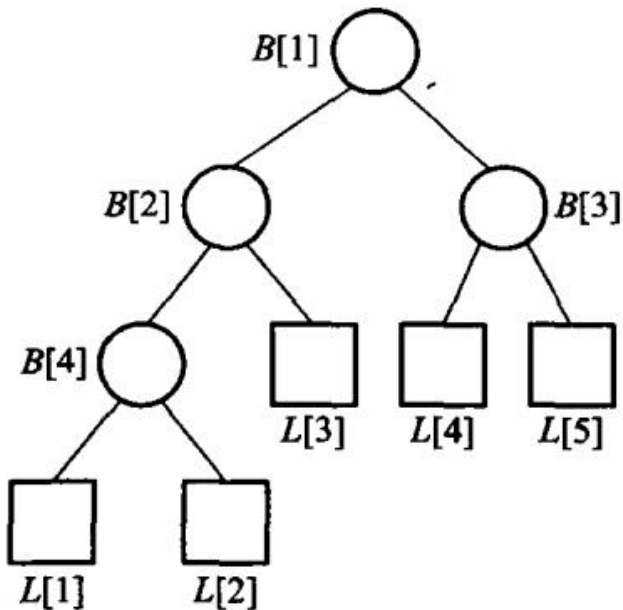


图 9.3 二路归并过程



第 9 章插图（底稿第 8396 行，原书无独立题注）

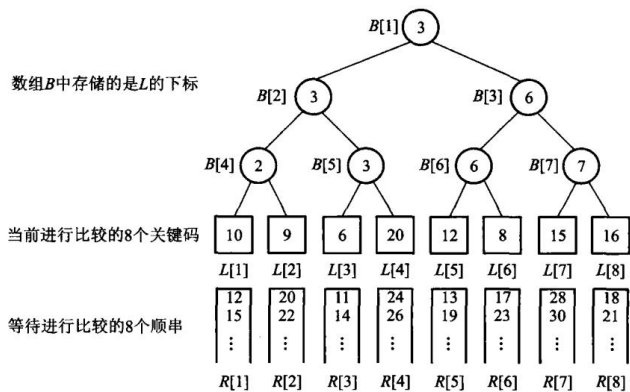


图 9.5 8 路归并的赢者树

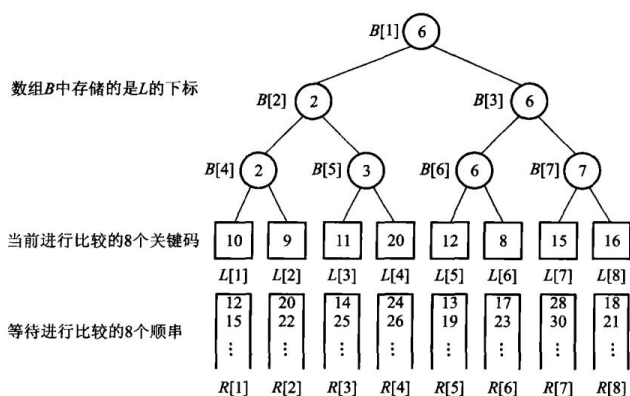


图 9.6 图 9.5 重构后的 8 路归并赢者树

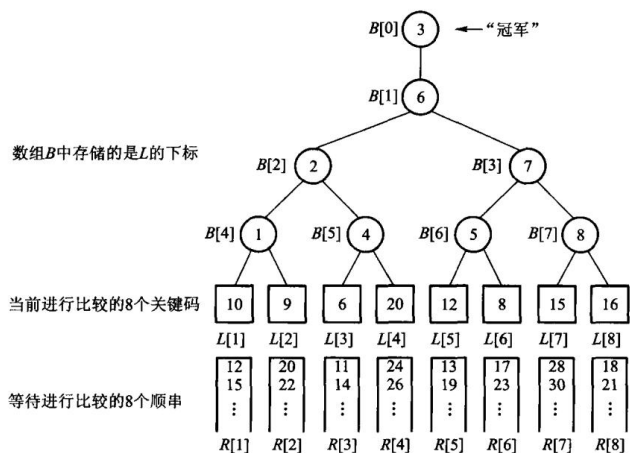


图 9.7 8 路归并的败者树示例

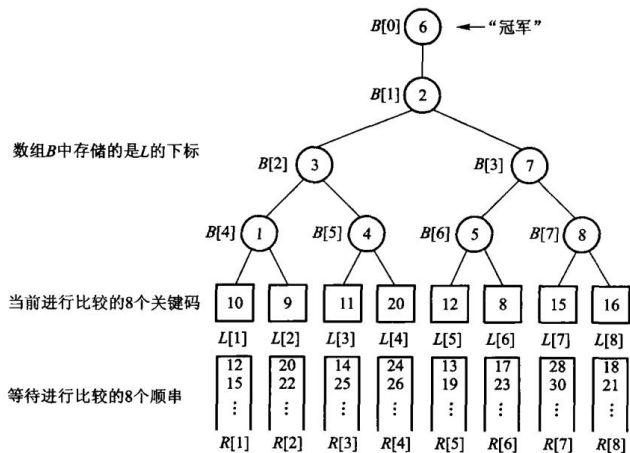


图 9.8 图 9.7 重构后的 8 路归并败者树

第 10 章 检索

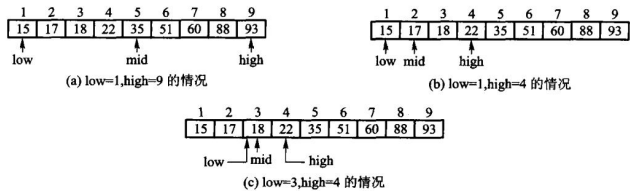


图 10.1 二分检索 K = 18 的过程

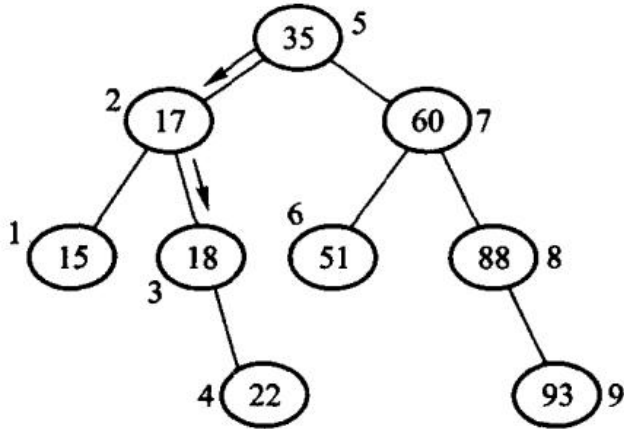


图 10.2 二分检索的决策树

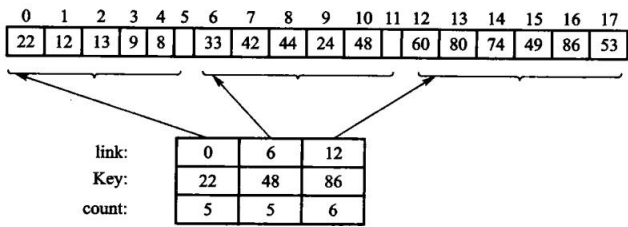


图 10.3 分块检索的存储表示

散列地址	关键码	散列地址	关键码
0		13	while
1	and	14	with
2		15	until
3	end	16	then
4	else	17	
5		18	repeat
6	if	19	
7	begin	20	
8	do	21	
9		22	
10	go	23	
11	for	24	
12	array	25	

图 10.5 例 10.2 中关键码对应的散列表

	5864	5864	
	4220	0224	
+	04	+	04
	[1]0088		6092
	$h(K) = 0088$		$h(K) = 6092$

(a) 移位叠加; (b) 分界叠加; 图 10.6 移位叠加和分界叠加

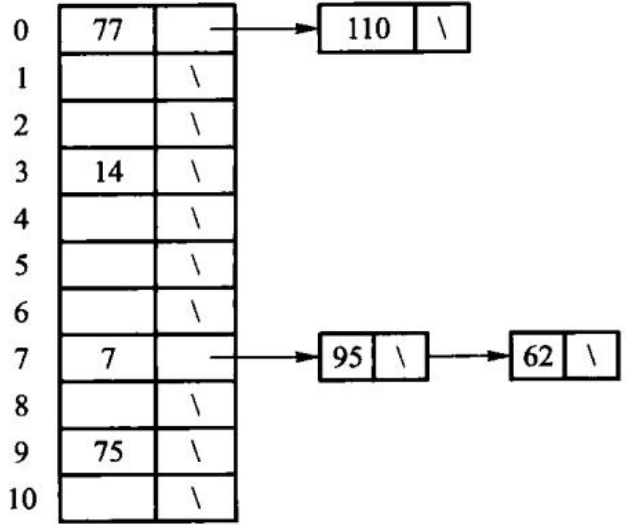


图 10.7 开散列方法的图示

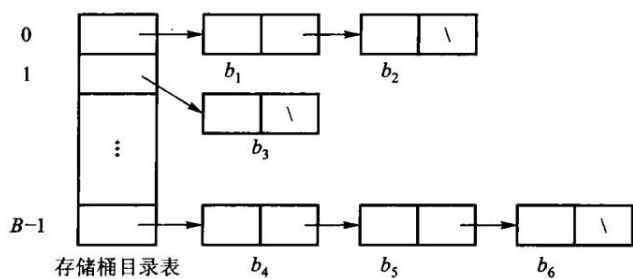


图 10.8 散列文件组织

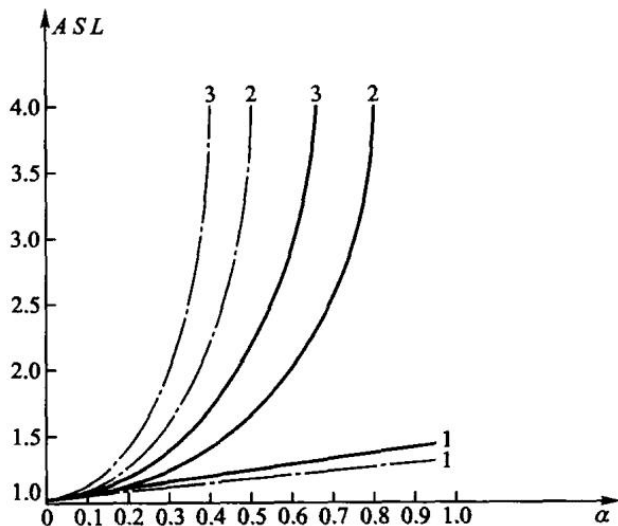


图 10.11 用几种不同方法解决碰撞时散列表的平均检索长度

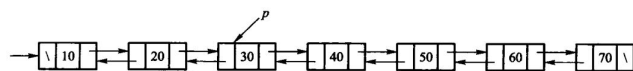


图 10.12 双向有序表

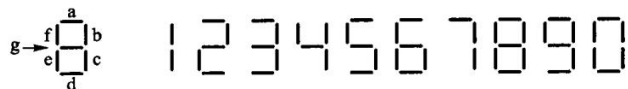


图 10.13 液晶七段数码管示意图

	S	X
S	OK	No
X	No	No

第 10 章插图（底稿第 9825 行，原书无独立题注）

散列表头

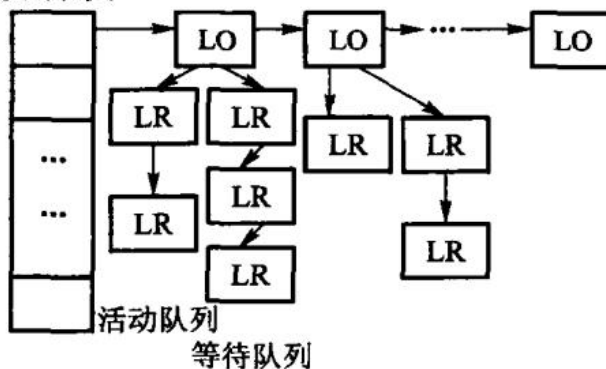


图 10.14 封锁方式的相容矩阵；图 10.15 封锁管理子系统示意图

第 11 章 索引技术

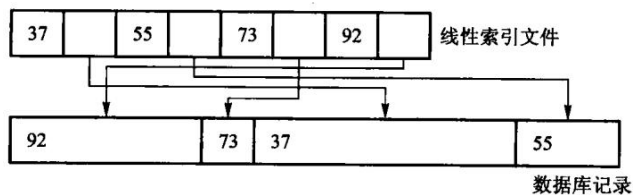


图 11.1 线性索引，图中的数据库记录不等长

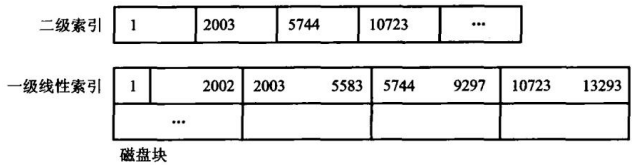


图 11.2 二级索引文件

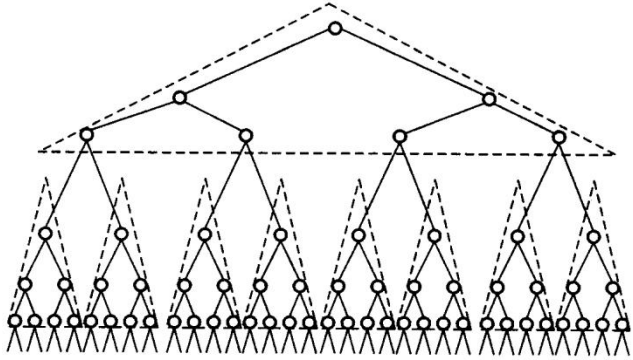


图 11.3 多分树

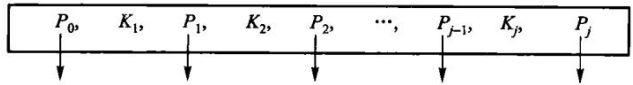
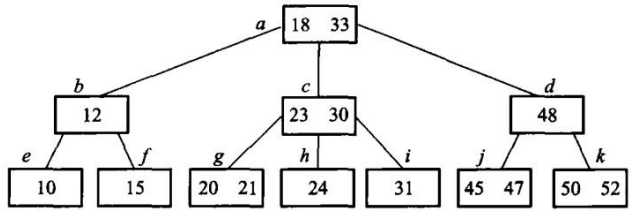
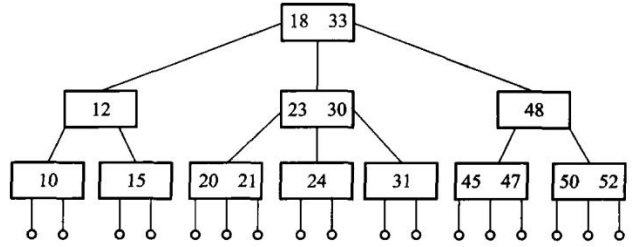


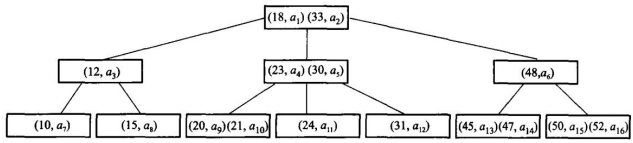
图 11.4 B 树结点的一般形式



(a) 3 阶 B 树



(b) 画出外部空结点的 3 阶 B 树



(c) B 树的隐含指针，其中 $a_1 \sim a_{16}$ 代表关键码所在主文件中具体磁盘块的索引地

址；图 11.5 3 阶 B 树

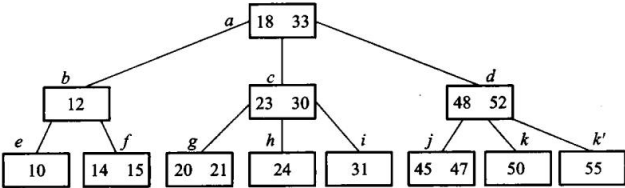


图 11.6 在图 11.5 的 3 阶 B 树中插入关键码 14、55 后的结果

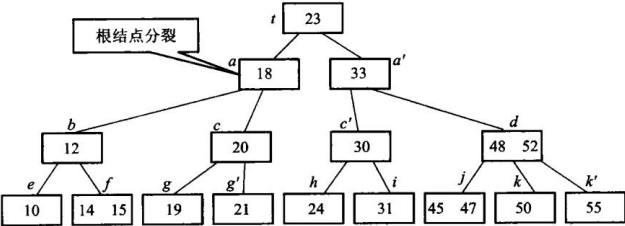
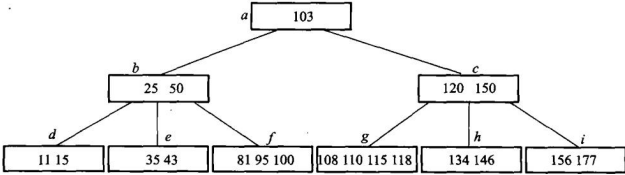
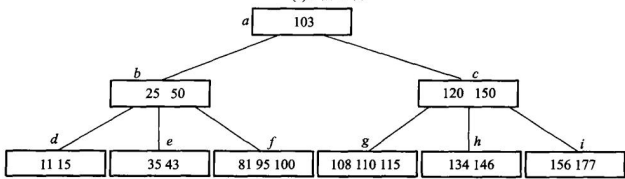


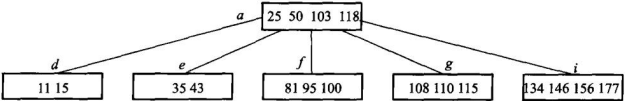
图 11.7 在图 11.5 的 3 阶 B 树中插入关键码 19 后导致根分裂



(a) 5 阶 B 树



(b) 删除 120, 先与后继交换, 删后下溢出, 向左邻借关键码



(c) 继续删 150, 先交换, 删后下溢出, 合并操作传递到根; 图 11.8 5 阶 B 树, 连续删除关键码 120、150 示意图 6

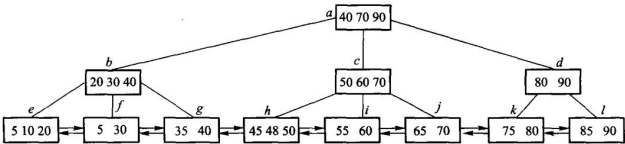


图 11.9 3 阶 B + 树

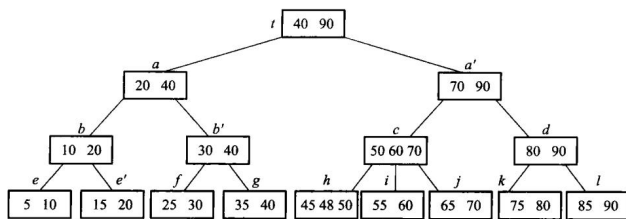


图 11.10 在图 11.9 的 3 阶 B+ 树中插入 15 后，树增高一层

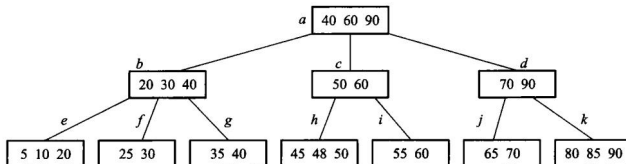


图 11.11 从图 11.9 的 3 阶 B+ 树中删除 75 后

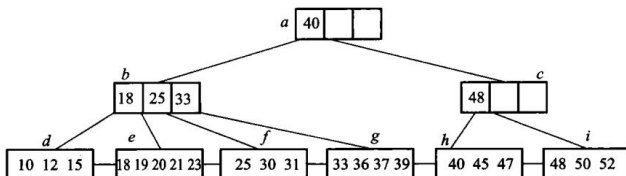


图 11.12 混合型 B+ 树，内部结点阶为 4, 叶结点阶 5

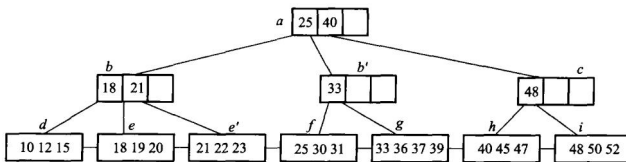


图 11.13 在图 11.12 混合型 B+ 树 (内部结点阶为 4, 叶结点阶 5) 中，插入 22 后的结果

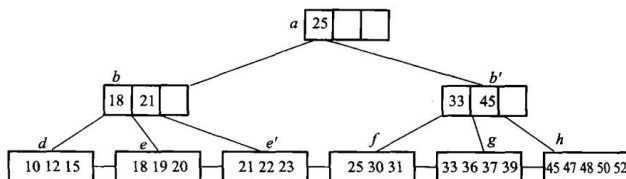


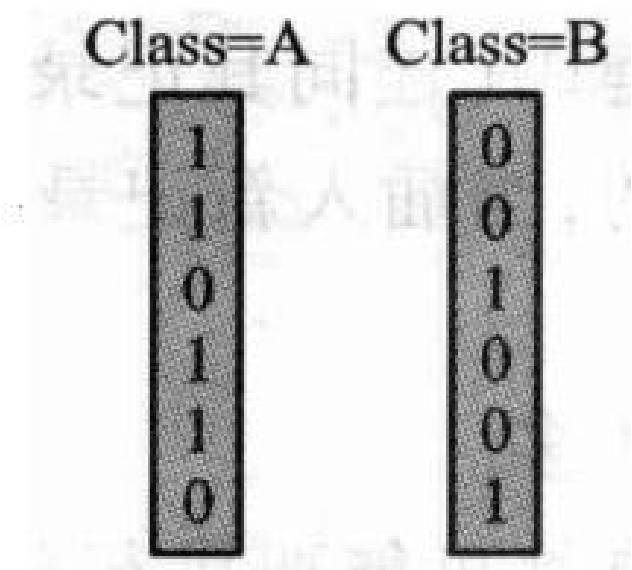
图 11.14 在图 11.13 混合型 B+ 树 (内部结点阶为 4, 叶结点阶 5) 中，删除 40 后的结果

Date	Store	State	Class	Sales
3/1	32	NY	A	6
3/1	36	AL	A	9
3/1	38	NY	B	5
3/1	41	AK	A	11
3/1	43	NY	A	9
3/1	46	AK	B	3

State=NY

1
0
1
0
1
1
0

(a) 百货销售数据库记录以及 State=NY(纽约州) 的位向量



(c) 数据域 Class 的位向量集合；图 11.15 百货销售数据库记录的位图索引示意

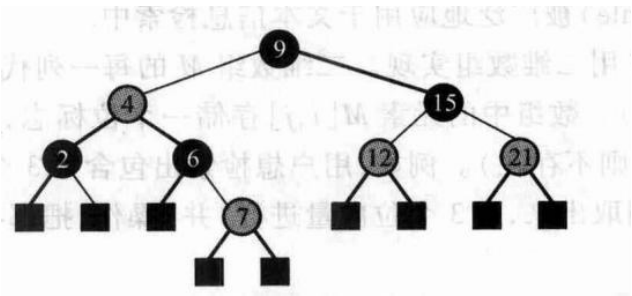


图 11.16 红黑树示意图

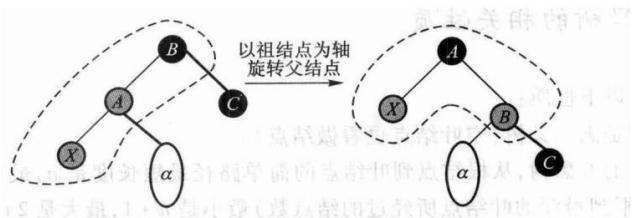


图 11.17 父结点是红色，叔父结点是黑色的情况，旋转解决红红冲突

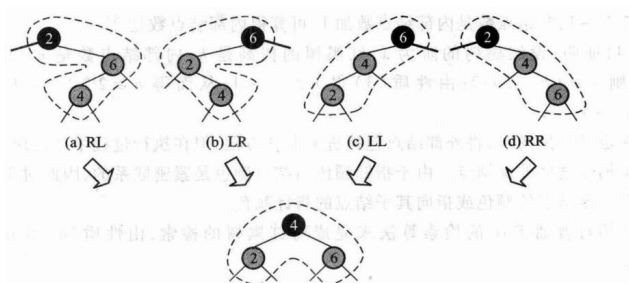


图 11.18 叔父结点为黑色，进行重构调整每个结点的阶都保持原值，调整完成

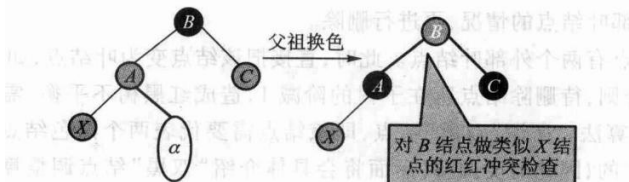


图 11.19 父结点、叔父结点均为红色，则父和祖换色，然后递归处理方法

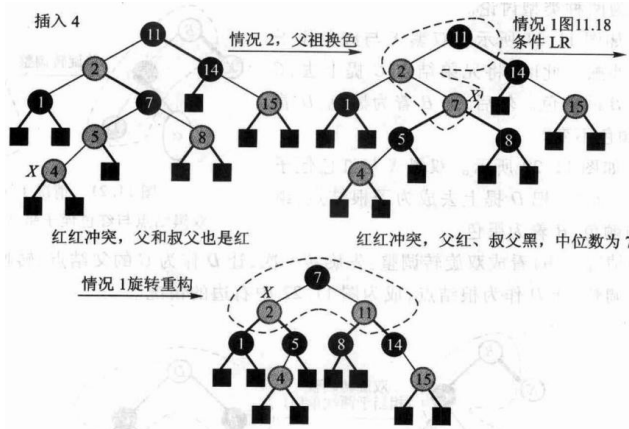


图 11.20 红黑树插入示例

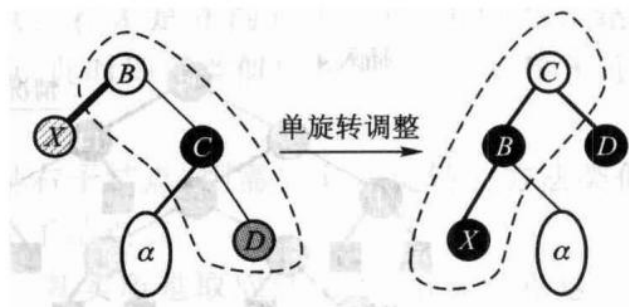


图 11.21 情况 1(a):

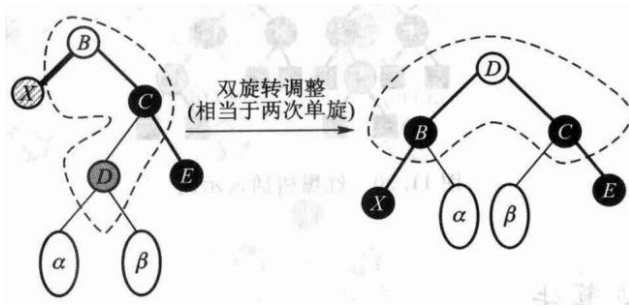


图 11.22 情况 1(b): 双黑结点与红色侄子同边顺

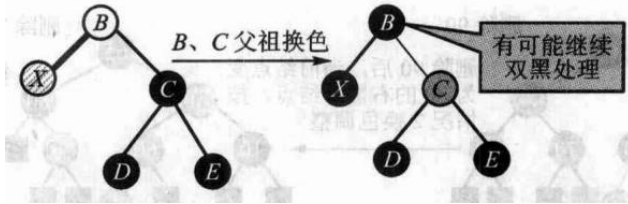


图 11.23 情况 2 双黑的兄弟结点为黑色，且兄弟结点有两个黑色子结点

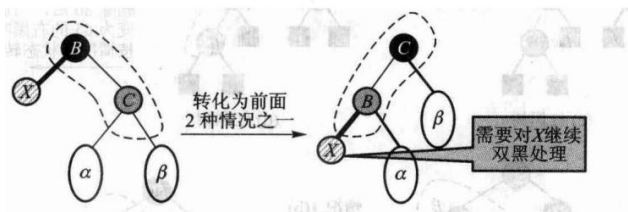


图 11.24 情况 3 双黑的兄弟结点为红色

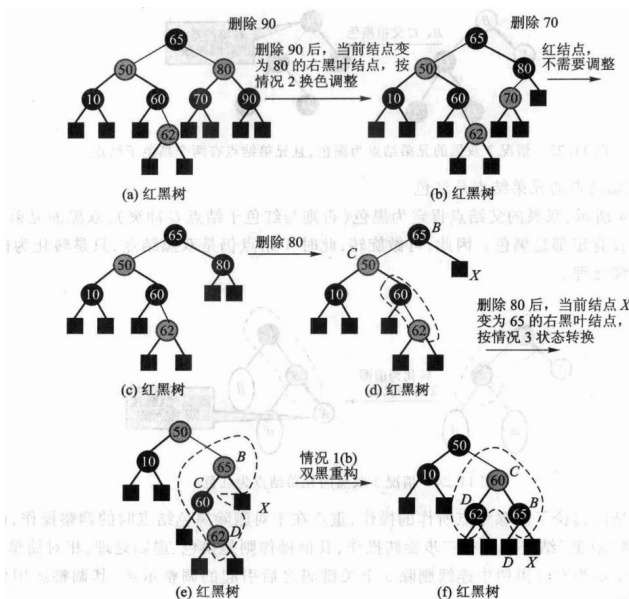


图 11.25 红黑树连续删除 90、70、80, 引起的双黑调整示例

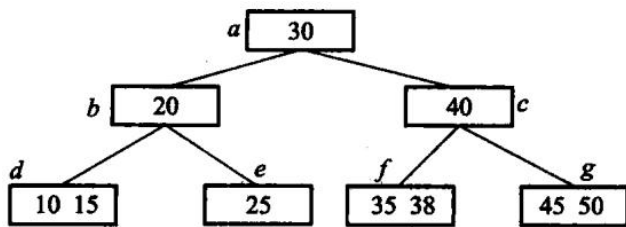


图 11.26 习题 8. 的 3 阶 B 树图示

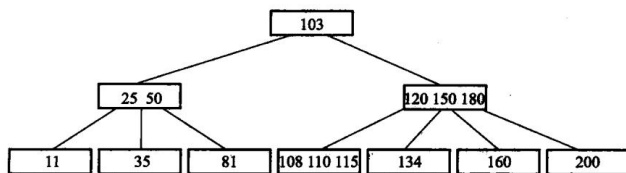


图 11.27 习题 9. 的 4 阶 B 树

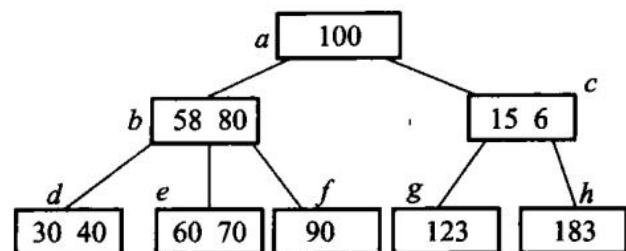


图 11.28 习题 10. 的 3 阶 B 树

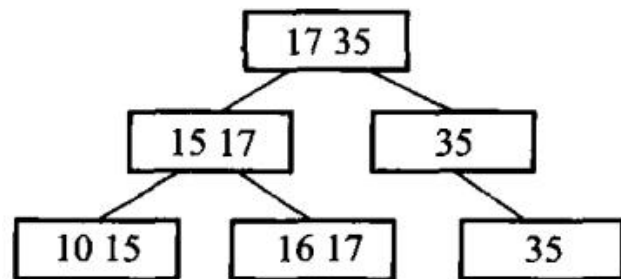


图 11.29 2 阶 B^+ 树图示

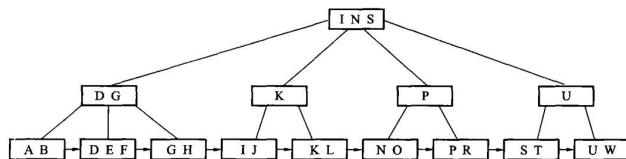


图 11.30 混合型 B^+ 树，内部结点阶为 4、叶结点阶为 3

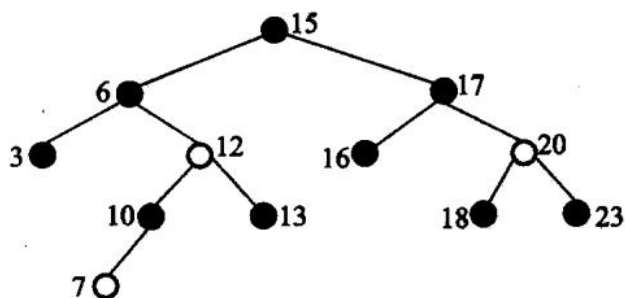


图 11.31 红黑树

第 12 章 高级数据结构

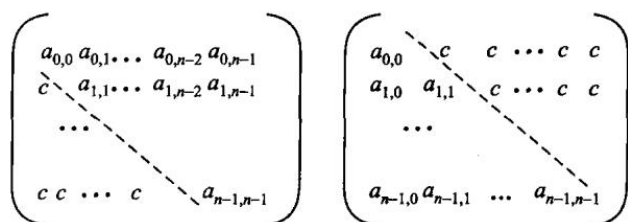


图 12.1 上三角矩阵和下三角矩阵，其中 c 为常数

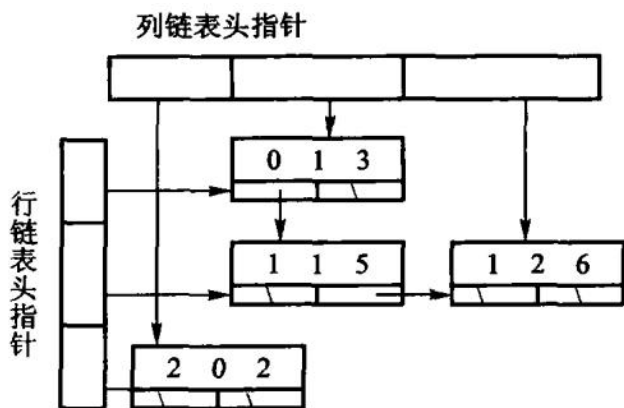


图 12.2 稀疏矩阵的十字链表

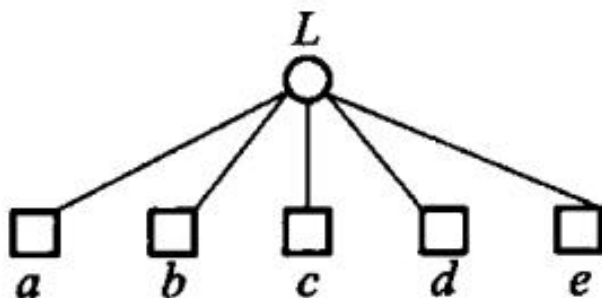


图 12.3 纯线性结构表

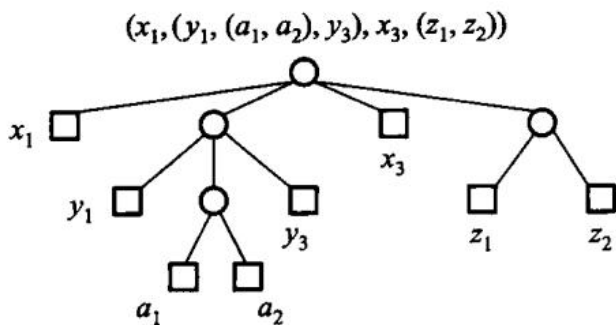


图 12.4 纯表的树表示

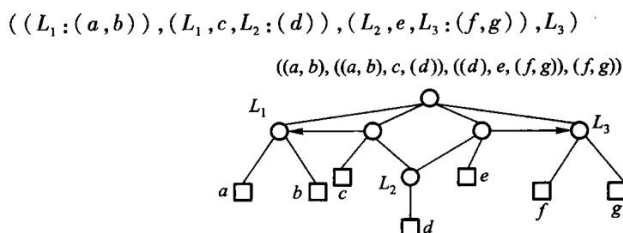


图 12.5 一个再入表的例子，其中没有标志箭头的边都是指向下方的

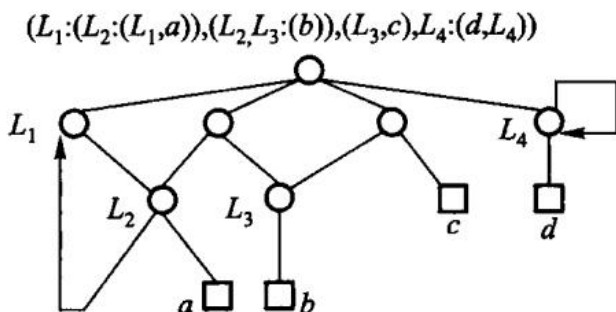


图 12.6 循环表可以看到这是一个有环图；图 12.6 中，因为 L_1 和 L_4 调用自身，导致了回路。其他没有箭头的边，方向都是从上至下的。循环表也可以算作一种存在回路的特殊再入表。

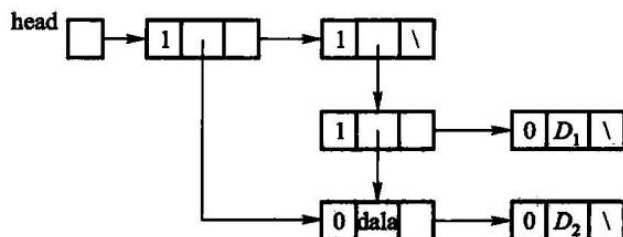


图 12.7 没有头结点的广义表

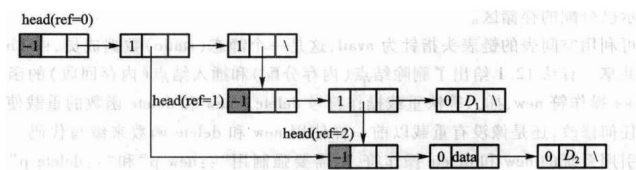


图 12.8 带头结点的广义表

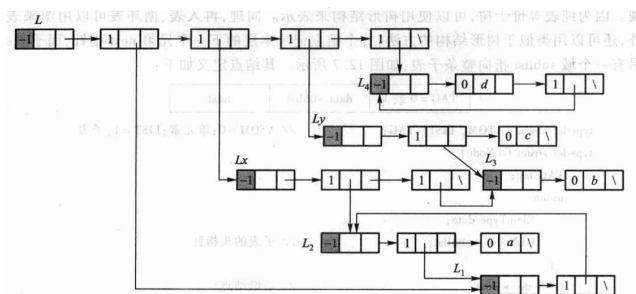
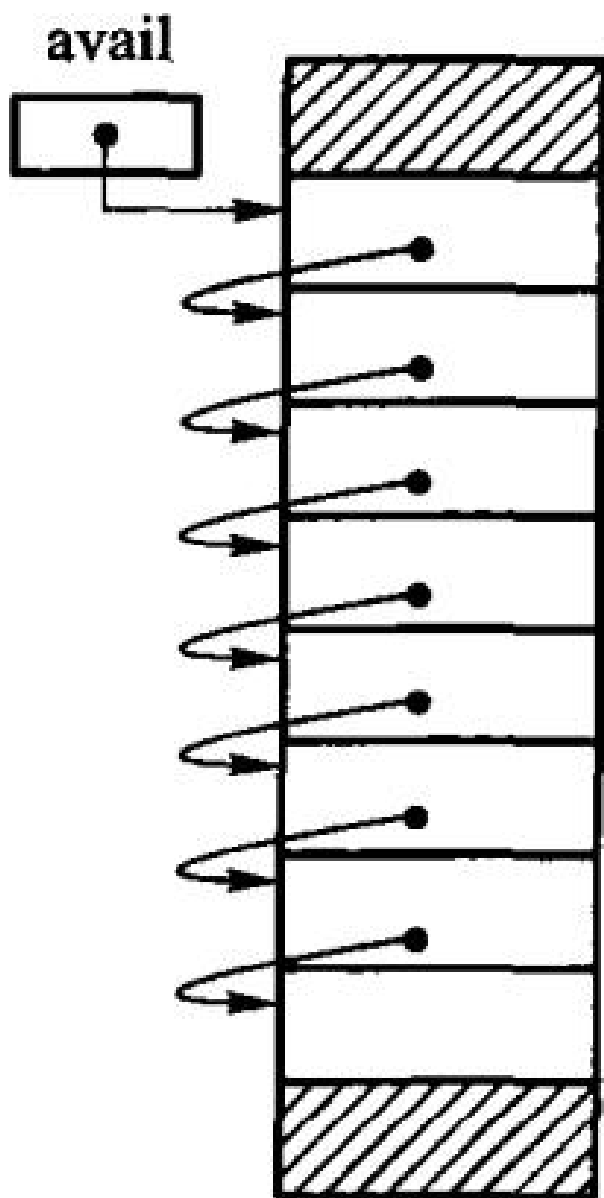
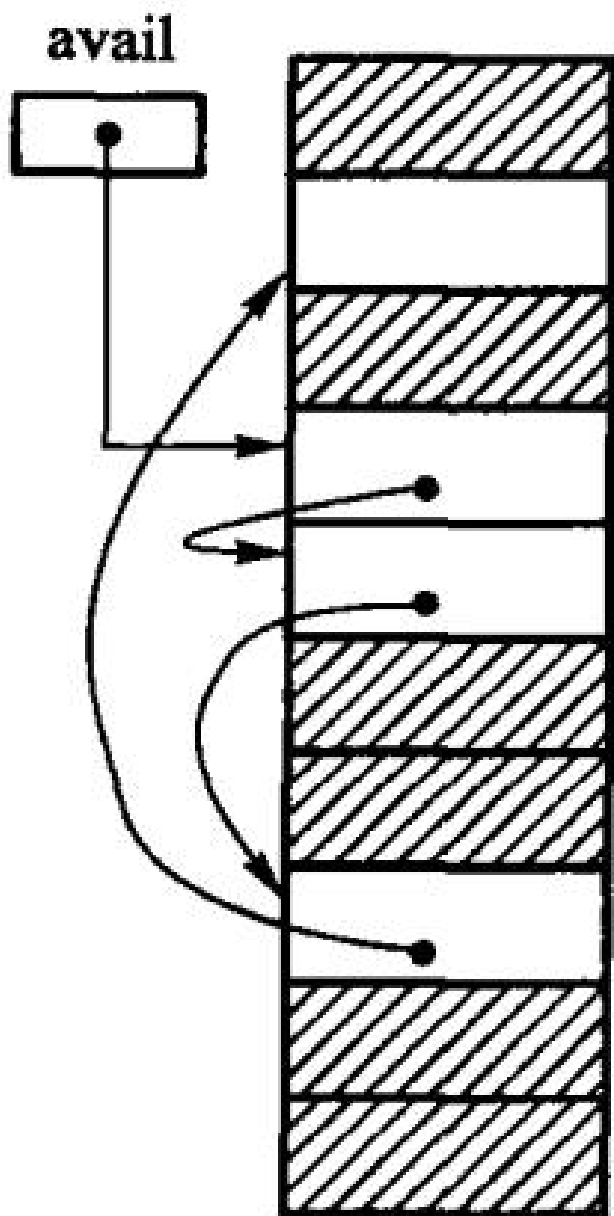


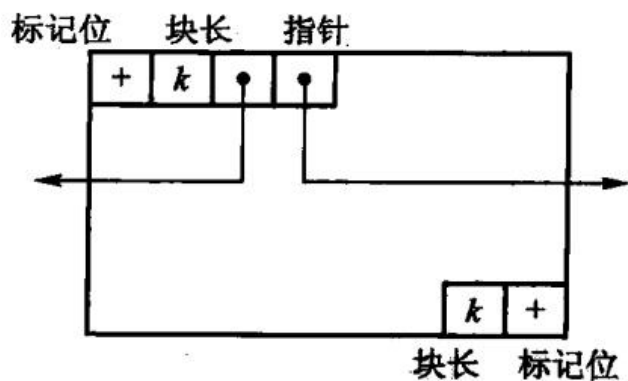
图 12.9 带头结点的循环广义表



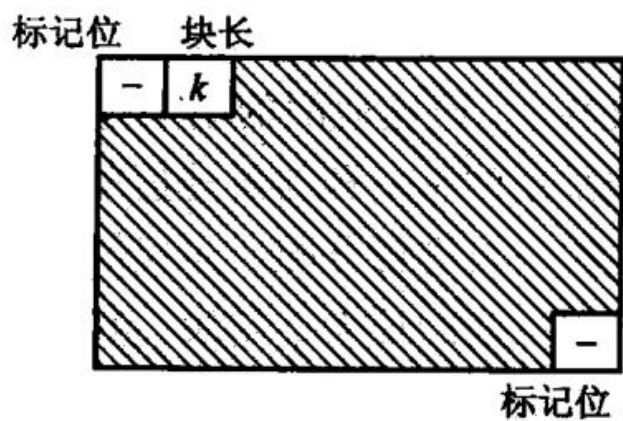
(a) 初始状态的可利用空间表



(b) 系统运行一段时间后的可利用空间表；图 12.10 结点等长的可利用空间表



(a) 空闲块的结构



(b) 已分配块的结构；图 12.11 存储器看到的块结构

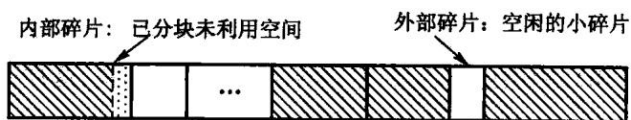


图 12.12 外部碎片和内部碎片

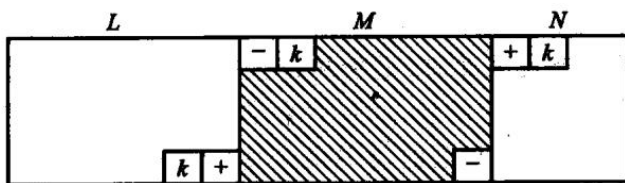


图 12.13 把块 M 释放回可利用空间表

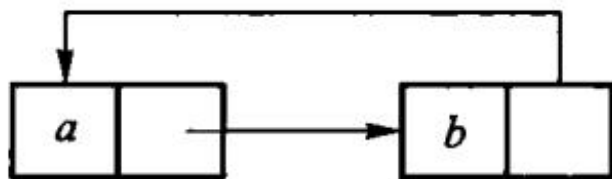
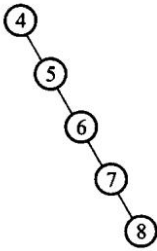


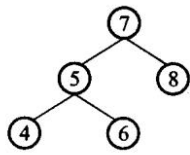
图 12.14 无用单元循环引用

输入顺序为 4、5、6、7、8



(a) 不平衡的 BST 树

输入顺序为 7、5、4、6、8



(b) 平衡的 BST 树

图 12.15 BST 的平衡性示意图

元素为 2、5、9、17、41、45、63

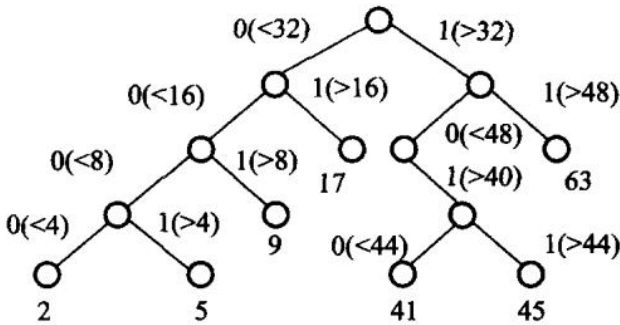
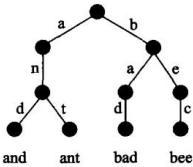


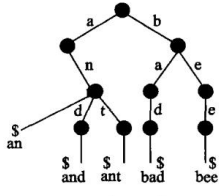
图 12.16 一棵二叉 Trie 树

存储单词 and、ant、bad、bee



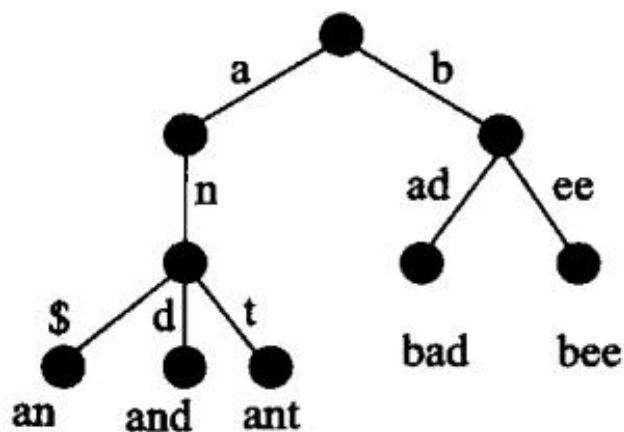
(a) 单词等长

存储单词 an、and、ant、bad、bee

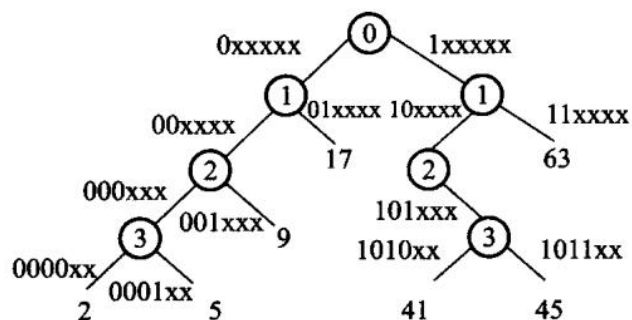


(b) 单词不等长

图 12.17 存储单词的 Trie 结构



第 12 章插图 (底稿第 11086 行, 原书无独立题注)



编码: 2: 000010 5: 000101 9: 001001
17: 010001 41: 101001 45: 101101 63: 111111

图 12.19 x 表示那个位可以为 1 或者 0

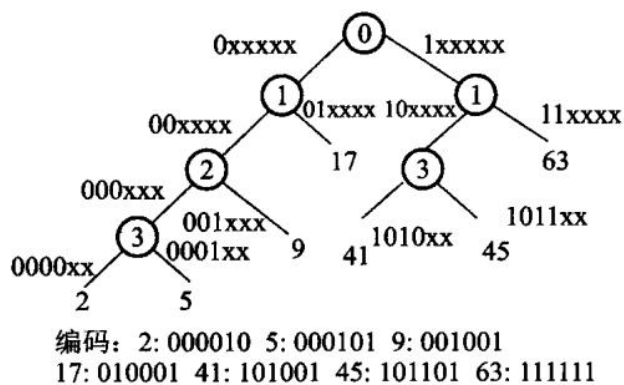


图 12.20 最后得到的 PATRICIA Trie 结构

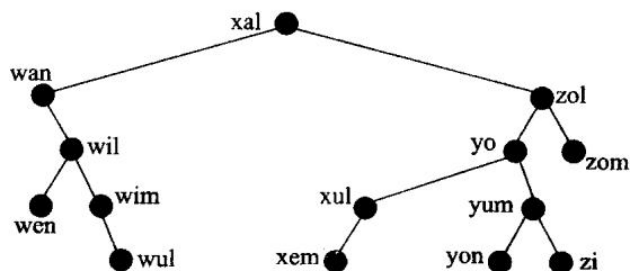


图 12.21 二叉搜索树

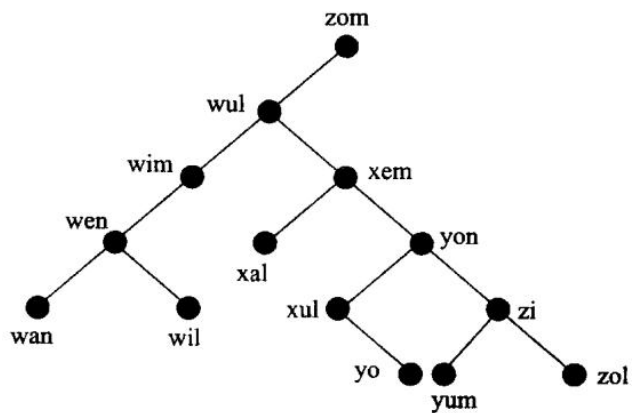


图 12.22 关键码与图 12.21 相同的是一棵 BST

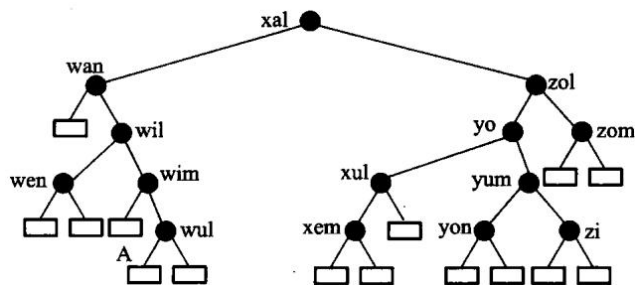


图 12.23 扩充二叉树

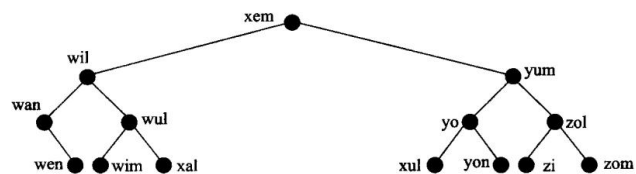


图 12.24 最佳二叉搜索树

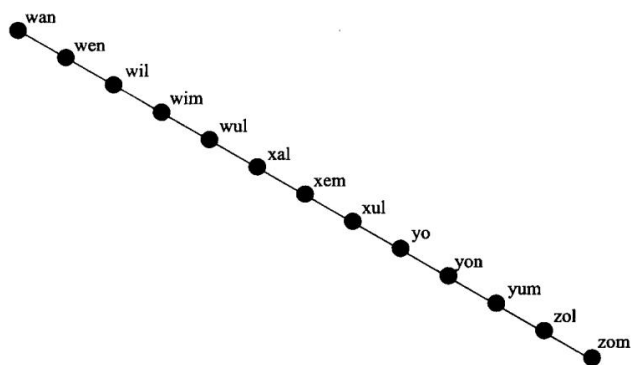


图 12.25 退化为线性的二叉搜索树

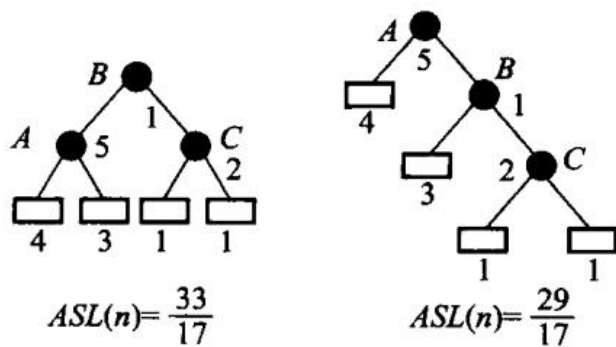


图 12.26 具有不等权结点的二叉搜索树

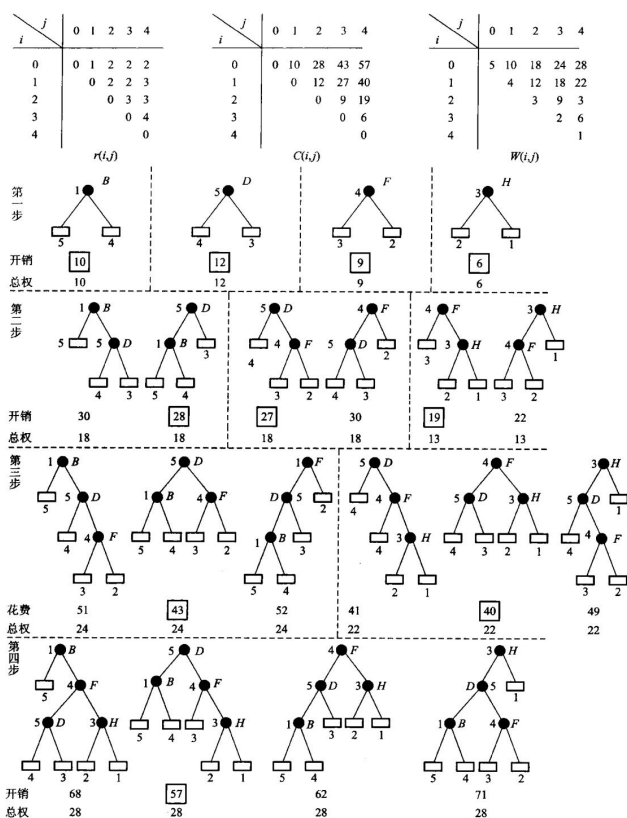
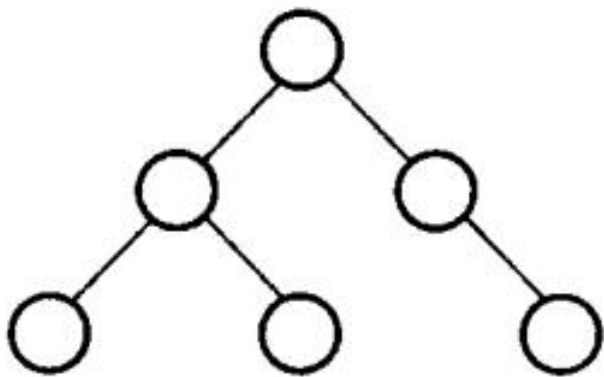
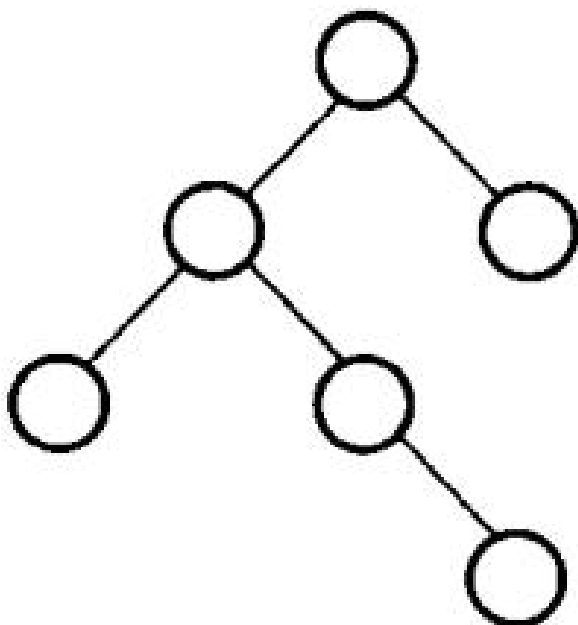


图 12.27 构造最佳二叉搜索树的过程



(a) 一棵 AVL 树



(b) 非 AVL 树的 BST；图 12.28 AVL 树与非 AVL 树

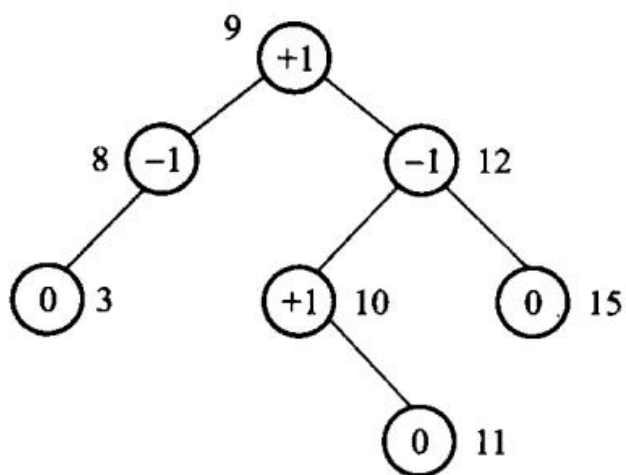
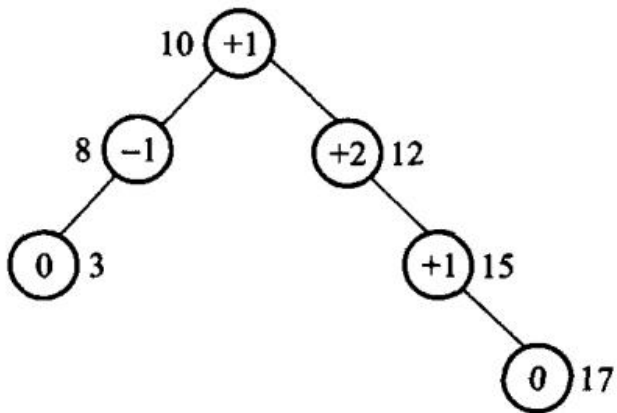
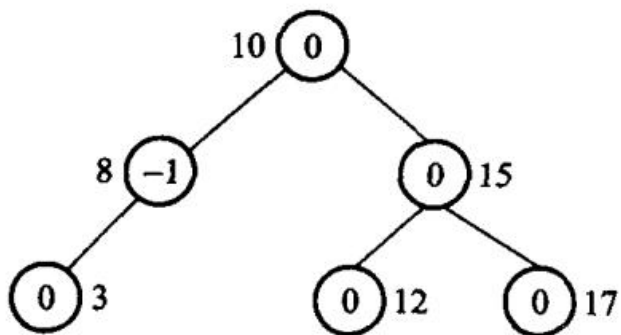


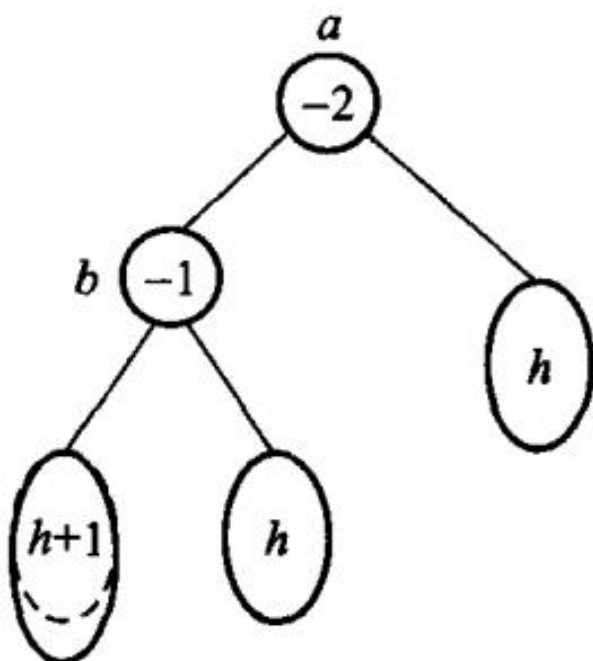
图 12.29 带平衡因子的 AVL 树



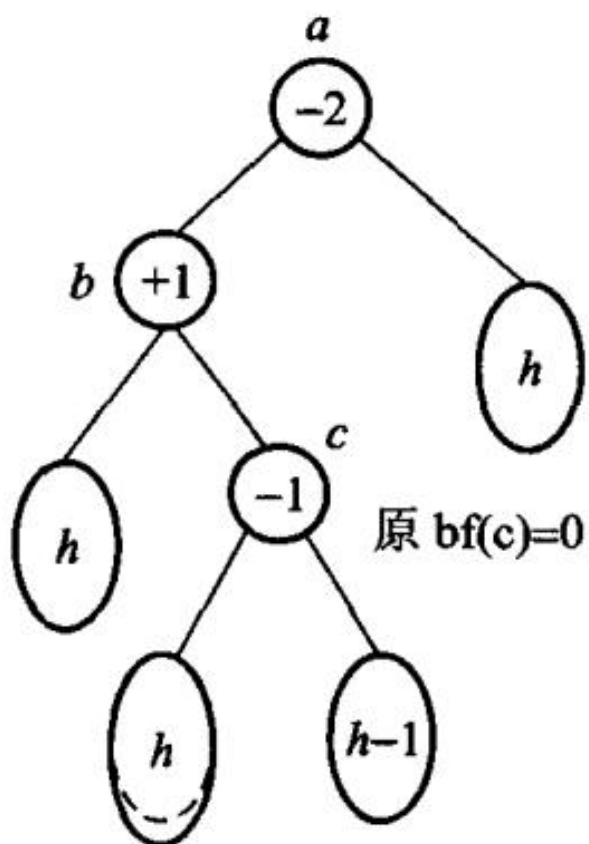
(a) 被破坏的 AVL 树



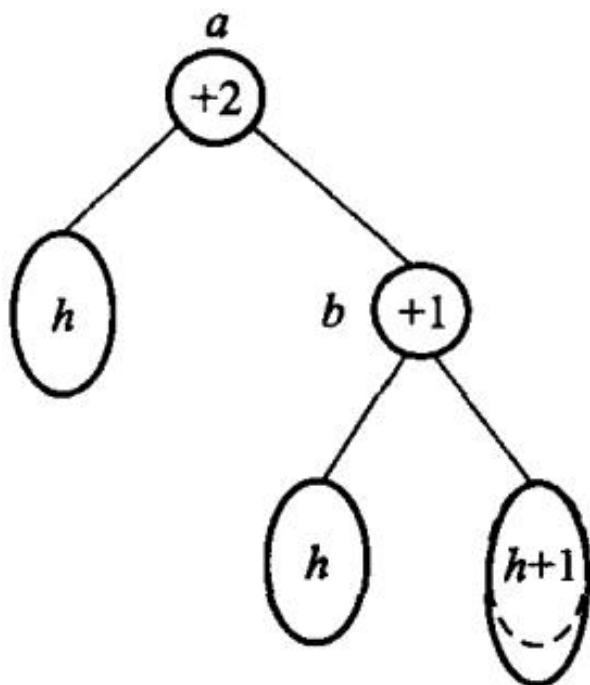
(b) 重新调整为 AVL 树；图 12.30 AVL 树的调整示意图



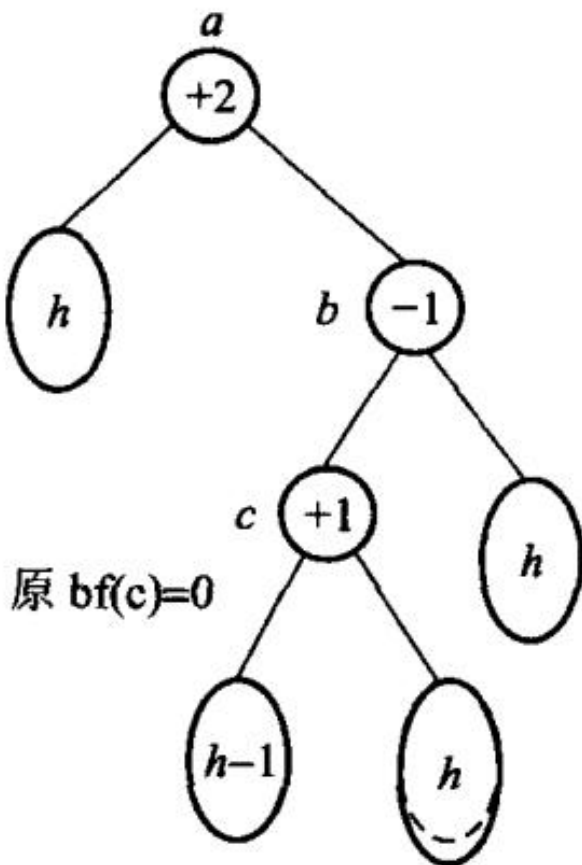
(a) LL 不平衡



(b) LR 不平衡



(c) RR 不平衡



(d) RL 不平衡；图 12.31 AVL 树的不平衡类型。圆圈表示结点；椭圆表示子树，内部符号表示其高度

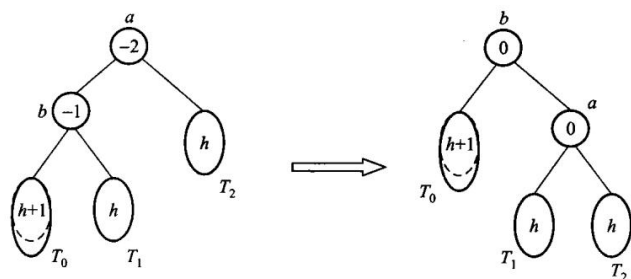
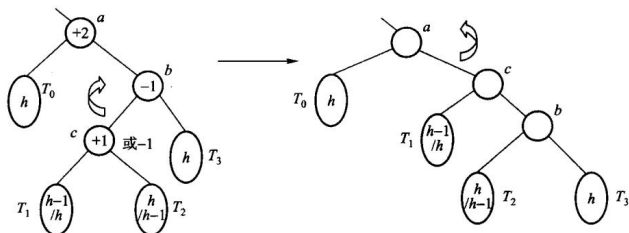
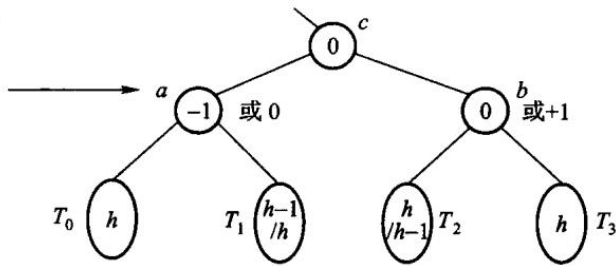


图 12.32 LL 型不平衡经过单旋转后重新变成平衡结构，RR 型单旋转类似



(b) 中间状态平衡因子无意义



(c) a 的平衡因子为-1 或 0, b 的平衡因子为 0 或 +1; 图 12.33 RL 型双旋转示意图, LR 型双旋转类似; 图 12.34 是从空 AVL 树开始, 顺序插入单词 {cup, cop, copy, hit, hi, his, hia} 后得到的 AVL 树, 单词之间按照字典顺序比较。

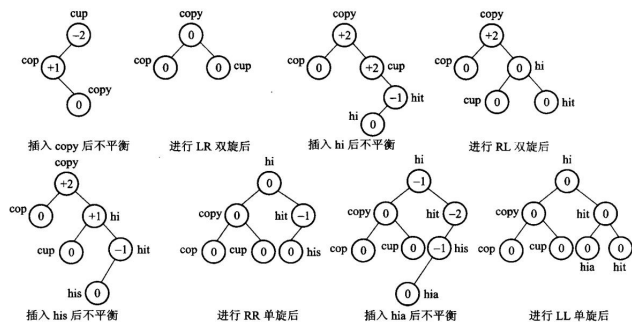


图 12.34 AVL 树的插入过程

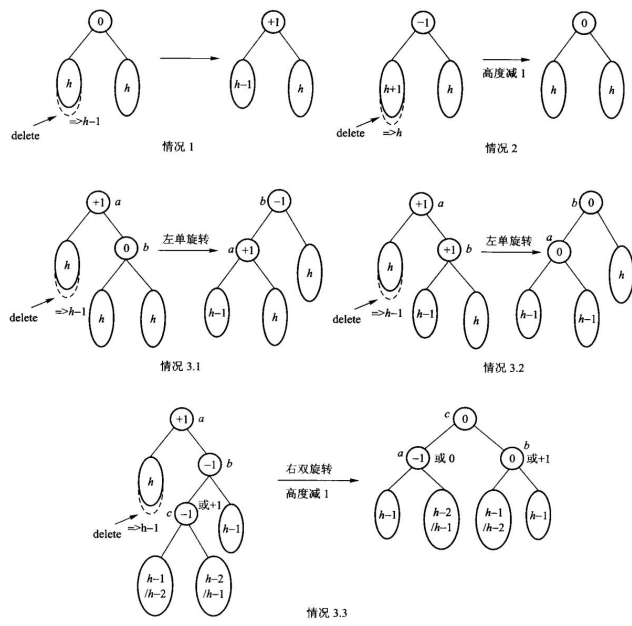
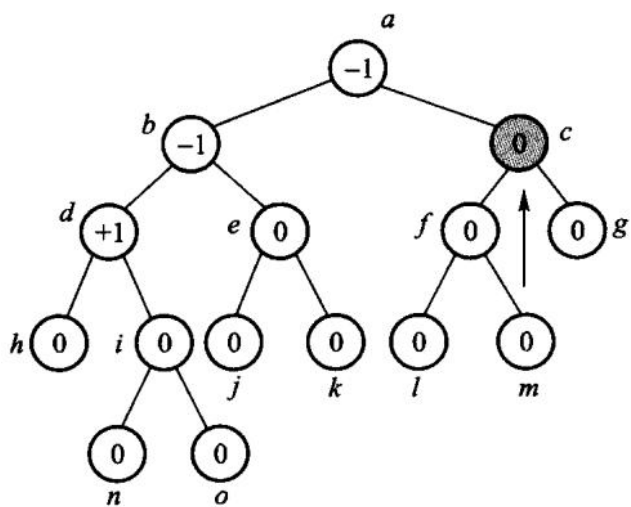
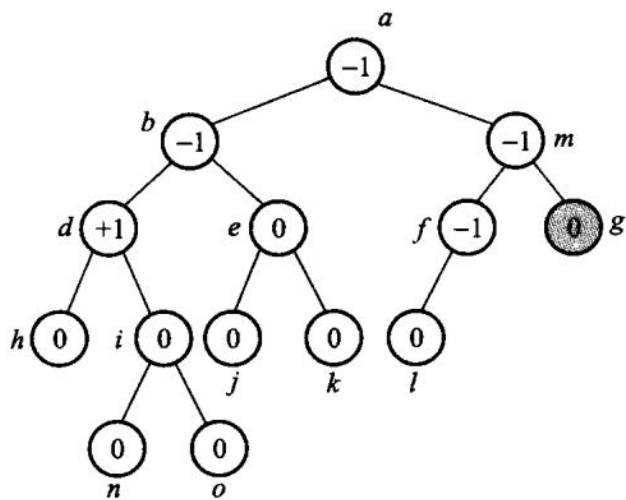


图 12.35 AVL 树删除之后的平衡操作



(a) 删除结点 c , 则需要使用其中序前驱 m



(b) 删除结点 g

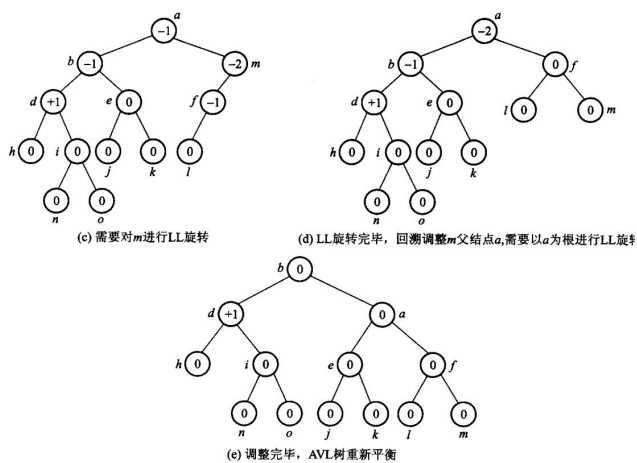


图 12.36 AVL 树的删除过程

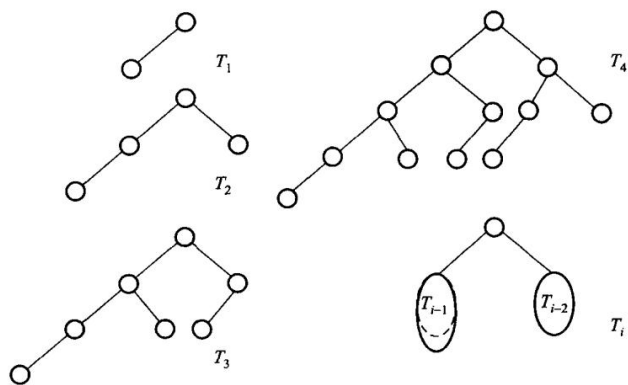
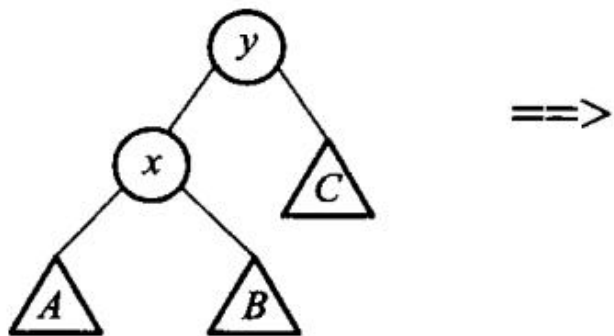
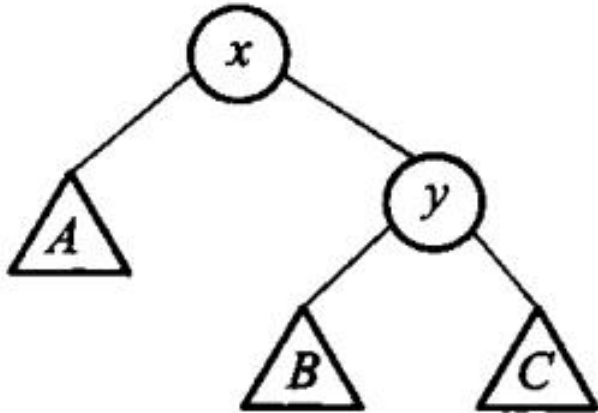


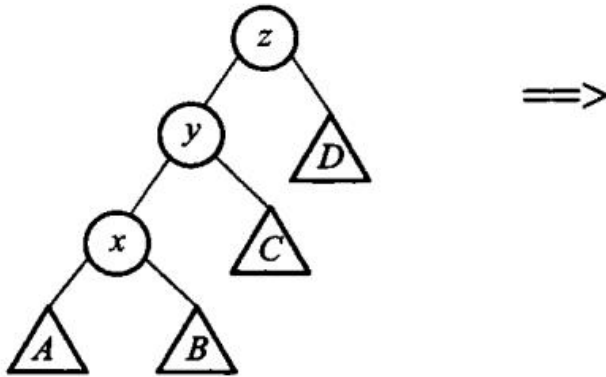
图 12.37 最接近于不平衡的 AVL 树



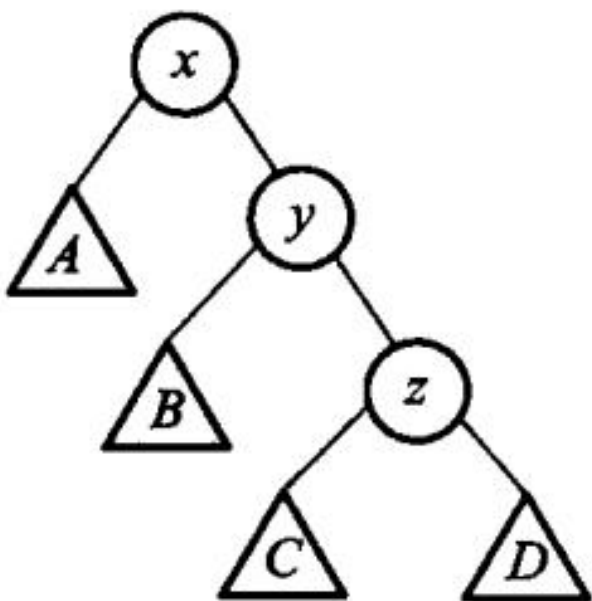
第 12 章插图（底稿第 11540 行，原书无独立题注）



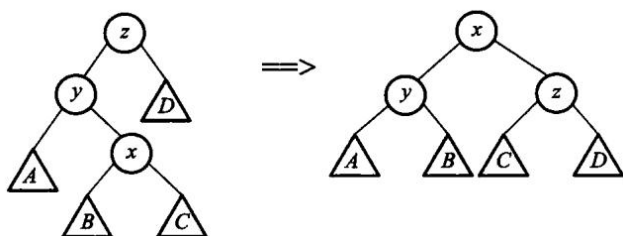
(b) 旋转发生以后的树；图 12.38 当被展开的结点是根结点的子结点时，才会发生伸展树的单旋转



(a) 结点 x、结点 y、结点 z 为一字形的树结构



(b) 旋转发生之后的树结构；图 12.39 伸展树的一字形旋转



(a) 结点 x、结点 y、结点 z 为之字形的树结构 (b) 旋转发生之后的树结构；图 12.40 伸展树的之字形旋转

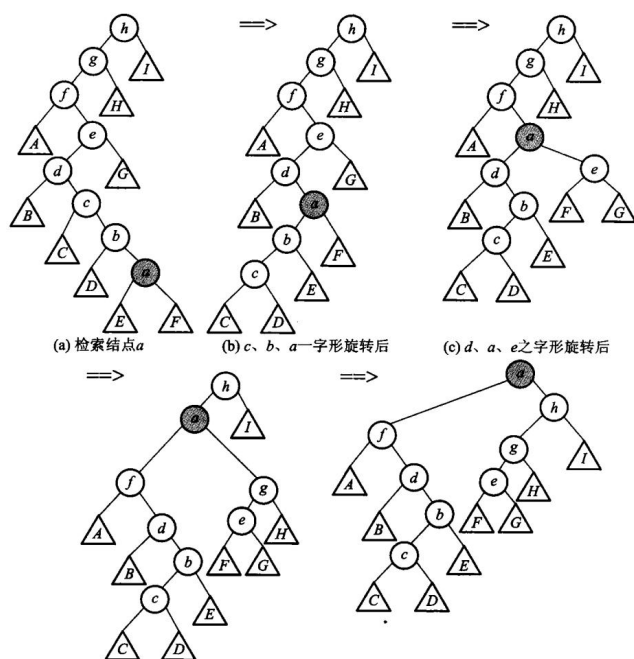


图 12.41 在伸展树中完成一次检索以后再展开的例子

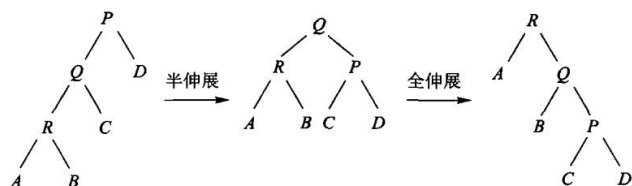


图 12.42 半伸展和全伸展的区别

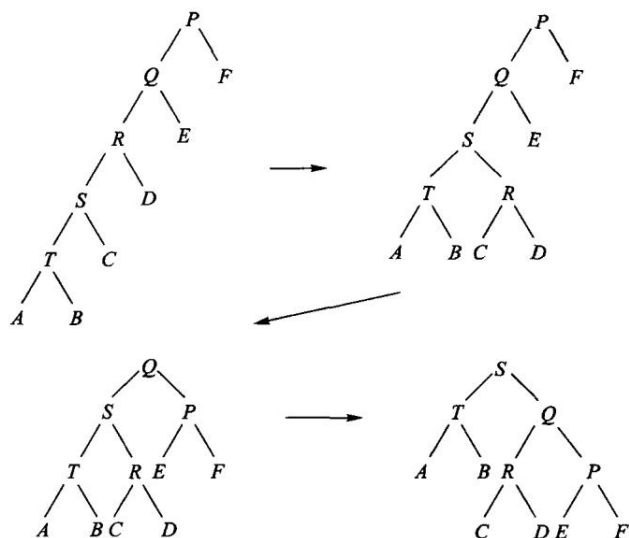


图 12.43 连续两次访问结点 T，半伸展树的变化示例

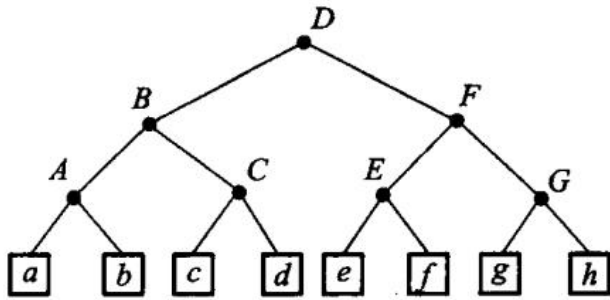


图 12.44 外部结点存储记录信息的半伸展树

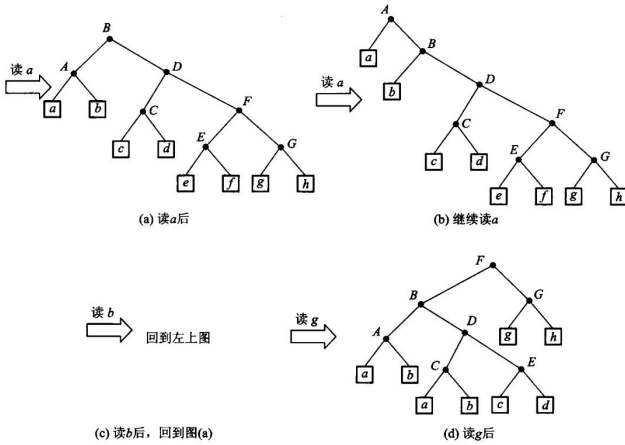
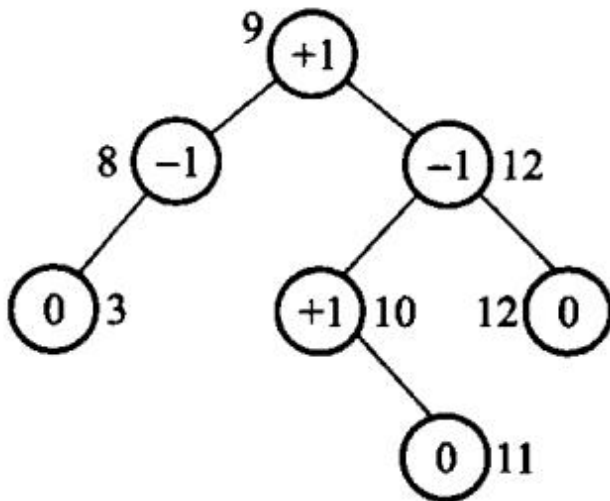


图 12.45 对图 12.44 连续读 aabg 后，半伸展树的变化过程



第 12 章插图（底稿第 11733 行，原书无独立题注）

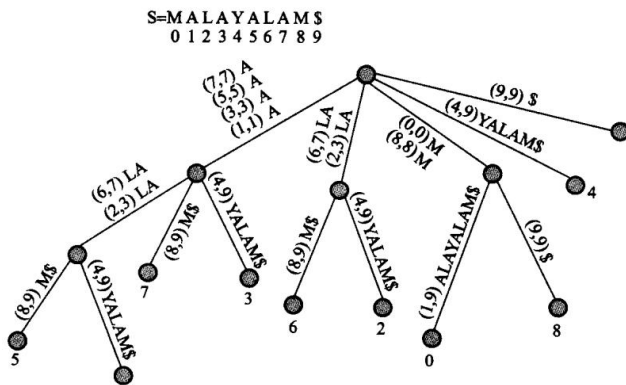
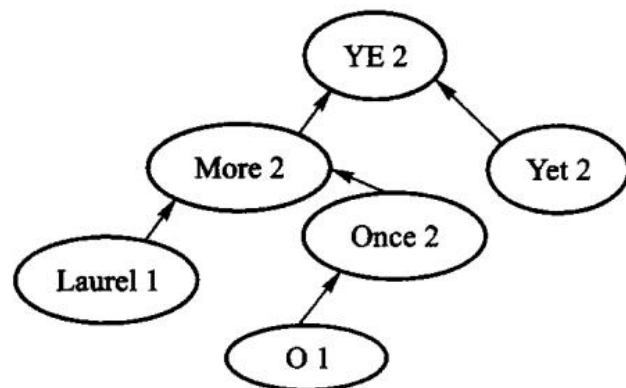


图 12.47 带路径压缩的后缀树



第 12 章插图（底稿第 11816 行，原书无独立题注）

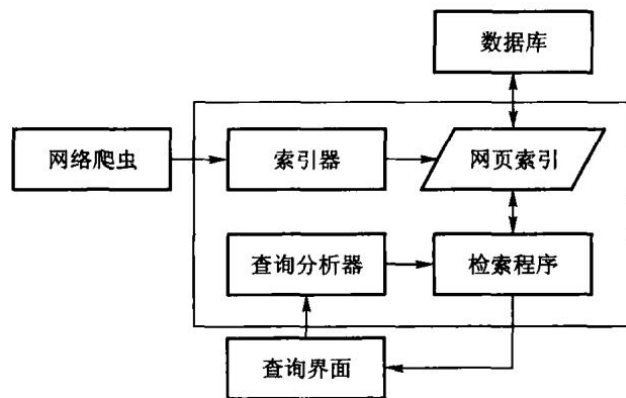


图 12.49 搜索引擎基本架构

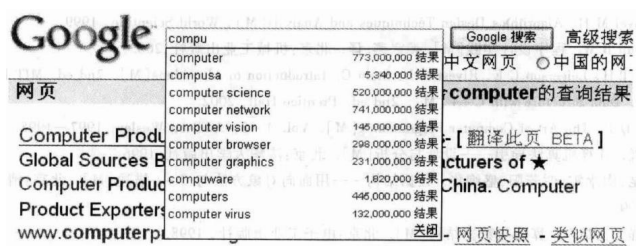


图 12.50 Google 的 Ajax 输入提示框

原书勘误

《数据结构与算法》（张铭、王腾蛟、赵海燕，高等教育出版社 2008）印刷清单里，按原样进编译器会失败，或对拍后算法结果确定是错的条目。每条都指向某个单元的 legacy.md，那里有命令和真实输出。

不收录：OCR 把 } 认成 1、把 ++ 拆成 + +、全角分号——那是扫描损伤，不是原书设计。也不收录「用了 cout」「没用 optional」这类现代化取舍。

底稿 dsa_raw.md 只读，本表是给对照纸书阅读的人用的。

怎么读

「印刷即错」指：把 OCR 噪声修回之后，逻辑或语法仍然不能通过现代 C++ 编译器，或与书中自己的约定对拍失败。现代实现写在 code/，书稿 book/ 印的是那份能跑的代码。

编译不过

清单	错在哪	证据
代码 2.1 / 代码 2.2 / 算法 2.5	成员函数名叫 delete，这是关键字	error: expected unqualified-id before 'delete' array_list
代码 2.1	class List 通篇没有 public:，ADT 的每个运算都是私有的	error: 'void List<T>::clear()' is private · 同上
算法 2.3	循环上界 n 从未声明	error: 'n' was not declared in this scope · 同上
代码 2.6	const Link* 赋给 Link* next，丢弃 const	error: assigning to 'Link<int> *' from 'const Link<int> *' · linked_list
代码 3.2	int top 与 bool top(T&) 重名，整类编译不过	error: 'bool arrStack<T>::top(T&)' conflicts with a previous declaration · array_stack
算法 3.3	循环变量 i 从未声明	error: 'i' was not declared in this scope · 同上

清单	错在哪	证据
代码 3.4	又是 <code>Link<T>* top</code> 与 <code>bool top(T&)</code> 重名	与代码 3.2 同一处错误， 两个存储结构都犯 · <code>linked_stack</code>
算法 3.5 / 算法 3.9	<code>s.pop(&tmp)</code> 传指针，本书自己的栈 ADT 要的是引用	<code>error: cannot convert 'double*' to 'double&'</code> <code>expression_eval</code>
算法 3.11	<code>enum rdType {0,1,2}</code> （枚举值不能是字面量）； <code>public class knapNode</code> （Java 语法，出现两次）	<code>error: expected identifier before numeric constant /</code> <code>error: expected unqualified-id before 'public'</code> <code>knapsack</code> <code>knapsack</code>
算法 3.12	同一个 <code>stack.top</code> 既当数据成员又当成员函数——依赖代码 3.2/3.4 的重名缺陷才可能存在	
算法 4.4	<code>assert(str != '\0')</code> 是指针与字符比较	<code>error: ISO C++ forbids comparison between pointer and integer ·</code> <code>string_class</code>
算法 4.4	<code>String(char* s)</code> 使书中自己的例子 <code>String s1 = "Hello"</code> 在 C++11 起非法	字符串字面量是 <code>const char*</code> · 同上
代码 10.1	成员初始化写 <code>key</code> ，数据成员却是 <code>Key</code>	<code>error: member initializer 'key' does not name a non-static data member ·</code> <code>search_hash</code>

能编译、跑起来是错的

清单	错在哪	证据
算法 3.6 / 3.8 / 3.9	阶乘不查溢出： <code>factorial(21)</code> 返回负数， <code>factorial(66)</code> 返回 0	UBSan: signed integer overflow: 21 * 2432902008176640000 · <code>recursion_and_stack</code>

清单	错在哪	证据
算法 3.6	负数静默返回 1	定义域错误，不是「空阶乘」· 同上
算法 4.5	越界 return NULL，随后 strlen(nullptr)	运行期 SEGV · string_class
算法 4.6 / 算法 4.8	0 起始下标下返回 j - pLen + 1，一律差 1	四面数据对拍 std::string::find，含图 4.12 自己的例子 · pattern_matching
算法 10.9	散列表析构写 delete HT，HT 是数组	应为 delete[] · search_hash

书内自相矛盾

位置	矛盾	说明
算法 3.5 正文 vs 代码	正文自己说 operand1 == 0.0 判断除零是错的，印出来的代码照旧	见第 3 章表达式一节
代码 3.1 vs 算法 3.5 / 3.9	栈 ADT 是 pop(T&)，算法写成 pop(&x)	两处清单从未放进同一个翻译单元
第 4.3 节 vs 算法 4.6 / 4.8	正文明写下标从 0 起，返回值却按 1 起始算	图 4.12 只画过程、没给返回值，所以印出来没人发现

反复出现、但单独一条通常还能「跑两下」

这些不是某一条清单独有，而是同一类所有权错误在多章重复。编译能过，拷贝一次就二次释放。

- 有析构、无拷贝构造 / 拷贝赋值：顺序表、链表、顺序栈、链式栈、字符串、最小堆、图的裸数组。
- 容器里 cout 报错、用 bool + 出参表达空状态：第 2、3、4 章通病。
- int 当下标与容量：有符号溢出是未定义行为。

现代实现按 D-001 处理：显式五法则、std::optional 表达空、真错误抛标准异常。

曾经误判、已撤回

第 1 章股市传言的印刷矩阵里，B1 / B5 的最大最短路一度被读成 8，并据此声称「原书应返回 B1 而非正文的 B3」。那些 8 是 OCR 把 ∞ 认错；选起点用的图 1.3 是图片，

从未被 OCR。按正文还原，原书这个样例是对的。详见 [adt/legacy.md](#)。

还没收进本表的

- 算法 2.11、代码 3.1、算法 7.6、算法 7.9 的结束标记被 OCR 吃掉，切片边界仍是手定 (T-008)。
- 第 6–12 章不少「标识符被空格切断」「全角分号」更像 OCR 而不是作者写错，故不列入上表。